
航芯 ACH512 用户手册

Version 2.3



上海爱信诺航芯电子科技有限公司

<http://www.aisinochip.com>

条款协议

本文档的所有部分，其著作权归上海爱信诺航芯电子科技有限公司（以下简称航芯科技）所有，未经航芯科技授权许可，任何个人及组织不得复制、转载、仿制本文档的全部或部分组件。本文档没有任何形式的担保、立场表达或其他暗示，若有任何因本文档或其中提及的产品所有资讯所引起的直接或间接损失，航芯公司及所属员工恕不为其担保任何责任。除此以外，本文档所提到的产品规格及资讯仅供参考，内容亦会随时更新，恕不另行通知。

版本修订

版本	日期	作者	描述
Rev 0.1	2016.3.4	Eric	
Rev 0.2	2016.3.18	Eric	1、系统概述章节：优化主要特点小节； 2、处理器章节修改部分图片和内容； 3、系统配置章节：修改部分描述； 4、EFC章节：增加eflash参数； 5、电气参数章节：修订部分描述和格式；
Rev 0.3	2016.3.28	Eric	1、RTC章节：RTV_CR的bit0定义保留，更改RTC_TRIM寄存器的默认值； 2、USB章节：在Tx_FIFO_ADDR/Rx_FIFO_ADDR处补充EP0 FIFO的地址空间描述以及FIFO分配空间的注意事项； 3、SCU章节：CLK_CTRLA的bit28描述错误； 4、ISO7816MS章节：三级标题中寄存器偏置20H/24H处有错误，SCISR的bit12改为保留位；
Rev 1.0	2016.5.25	eric/ bailing	1、系统概述章节：功能框图中去掉UARTC/TIMERB/DMA/PLL模块，时钟框图中修改UARTC为UARTB，去掉PLL时钟； 2、引脚描述章节：更新封装图和PINLIST 3、SCU章节：所有描述中删除TIMERB，删除UARTB，UARTC改为UARTB，删除PLL时钟选择及PLL寄存器，修改ANALOG_MISC_CSR中磁解码的部分定义； 4、删除DMA章节，删除各个模块中的DMA功能； 5、TIMER章节：去掉TIMERB这组定时器； 6、UART章节：删除UARTB，将UARTC修改为UARTB； 7、SPI章节：分开成SPIA和SPIB两个独立章节，其中SPIA支持主，多线，存储映射，SPIB支持主从，单线模式； 8、ADC章节：去掉DMA和温度传感器的相关信息，互换磁条通道2和磁条通道3的ADC通道位置,指定外发触发源具体定义 9、GPIO章节:修改GPIOB，GPIOC的基地址定义 10、RTC章节：改为只支持静态Tamper(0~2)
Rev 2.0	2017.7.15	eric/ bailing	基于V2版芯片特点进行更新
Rev 2.1	2017.8.16	eric/ bailing	更新部分文字描述和笔误，补充部分算法性能和电气参数
Rev 2.2	2017.8.17	eric/ bailing	更新电气参数中的功耗
Rev 2.3	2017.9.8	eric/ bailing	1、引脚描述章节：更新封装图管脚名称和信号描述 VDD33->VDDIO 2、更新电源框图 3、I2C章节：更新I2C_CLK_DI寄存器描述

目录

VERSION 2.3	I
1. 系统概述	17
1.1. 主要特点	17
1.1.1. 硬件功能	17
1.1.2. 安全功能	18
1.1.3. 软件支持	19
1.1.4. 封装形式	19
1.2. 功能框图	20
1.3. 电源框图	20
2. 引脚描述	22
2.1. 封装管脚分布	22
2.2. 信号描述	24
3. 处理器	32
3.1. 概述	32
3.2. 主要特性	32
3.3. 功能框图	32
3.4. 内核寄存器组	32
4. 系统配置(SCU)	35
4.1. 地址映射	35
4.2. 时钟框图	37
4.3. 时钟选择	37
4.4. PLL配置	38
4.5 复位源	38
4.5. 启动模式	39
4.6. 低功耗模式	39
4.7. 寄存器描述	40
4.7.1. 时钟控制寄存器CLK_CTRLA (偏移: 00h)	40
4.7.2. 时钟控制寄存器CLK_CTRLB (偏移: 04h)	43
4.7.3. 时钟分频寄存器CLK_DIVIDER (偏移: 08h)	45
4.7.4. PLL配置寄存器PLL_CSR (偏移: 0Ch)	46
4.7.5. 复位控制寄存器RESET_CTRLA (偏移: 10h)	47
4.7.6. 复位控制寄存器RESET_CTRLB (偏移: 14h)	48
4.7.7. 模拟参数和芯片控制寄存器ANALOG_MISC_CSR (偏移: 18h)	49
4.7.8. 蜂鸣器控制寄存器BUZZER_CTRL (偏移: 0x1C)	50
4.7.9. 系统重映射寄存器SYS_REMAP_CTRL (偏移: 0x20)	51
4.7.10. 系统唤醒控制寄存器WAKE_UP_CTRL (偏移: 0x24)	51
4.7.11. 管脚复用寄存器PAD_MUX_CTRL A(偏移: 2Ch)	52
4.7.12. 管脚复用寄存器PAD_MUX_CTRLB偏移: 30h)	55
4.7.13. 管脚复用寄存器PAD_MUX_CTRLC偏移: 34h)	57

4.7.14.	管脚复用寄存器PAD_MUX_CTRLD(偏移: 38h)	60
4.7.15.	管脚上拉控制寄存器PAD_PUCRA (偏移: 40h)	62
4.7.16.	管脚上拉控制寄存器PAD_PUCRB (偏移: 44h)	64
4.7.17.	传感器复位控制寄存器SENSOR_RESET_EN (偏移: 48h)	65
4.7.18.	SCI控制寄存器SCILDO_CTRL(偏移: 0x54).....	66
4.7.19.	USB PHY控制和状态寄存器USB_PHY_CTRL (偏移: 58h)	66
4.7.20.	Sensor 复位状态寄存器SENSOR_RST_STATUS (偏移: 5Ch)	67
4.7.21.	模拟模块控制寄存器ANACR (偏移: 60h)	68
5.	NVIC	70
5.1.	概述.....	70
5.2.	主要特性.....	70
5.3.	中断源.....	70
6.	MPU	72
6.1.	概述.....	72
6.2.	主要特性.....	72
7.	EFC	73
7.1.	概述.....	73
7.2.	主要特性.....	73
7.3.	寄存器描述.....	73
7.3.1.	控制寄存器EFC_CTRL (偏移: 00h).....	73
7.3.2.	写擦安全寄存器EFC_SEC (偏移: 04h).....	75
7.3.3.	地址校验寄存器EFC_ARCT(偏移: 08h).....	76
7.3.4.	擦除超时寄存器 (EFC_ERTO) (偏移: 0ch)	76
7.3.5.	写超时寄存器 (EFC_WRT0) (偏移: 10h)	76
7.3.6.	状态寄存器EFC_STATUS (偏移: 14h)	76
7.3.7.	中断状态寄存器EFC_INTSTATUS (偏移: 18h).....	77
7.3.8.	中断使能寄存器EFC_INEN (偏移: 1Ch).....	78
7.4.	功能描述.....	79
7.4.1.	Program Verify.....	79
7.4.2.	Page Erase Verify.....	79
7.4.3.	Double Read.....	79
7.4.4.	地址映射.....	80
7.5.	软件流程.....	80
7.5.1.	Read操作.....	80
7.5.2.	Write操作.....	80
7.5.3.	Erase操作.....	81
8.	DMAC	82
8.1.	概述.....	82
8.2.	主要特性.....	82
8.3.	寄存器描述.....	82
8.3.1.	中断状态寄存器DMACIntStatus (偏移: 00h)	83

8.3.2.	传输完成中断寄存器DMACIntTCStatus (偏移: 04h)	83
8.3.3.	传输完成中断清除寄存器DMACIntTCClr (偏移: 08h)	83
8.3.4.	传输错误中断寄存器DMACIntErrStatus (偏移: 0Ch)	84
8.3.5.	传输错误中断清除寄存器DMACIntErrClr (偏移: 10h)	84
8.3.6.	传输完成原始中断寄存器DMACRawIntTCStatus (偏移: 14h)	84
8.3.7.	传输错误原始中断寄存器DMACRawIntErrStatus (偏移: 18h)	84
8.3.8.	通道使能状态寄存器DMACEnChStatus (偏移: 1Ch)	85
8.3.9.	DMAC配置寄存器DMACConfig (偏移: 30h)	85
8.3.10.	同步寄存器DMACSync (偏移: 34h)	85
8.3.11.	源通道地址寄存器DMACCxSrcAddr (偏移: 100h, 120h, 140h, 160h)	86
8.3.12.	目标通道地址寄存器DMACCxDestAddr (偏移: 104h, 124h, 144h, 164h)	86
8.3.13.	通道链接表寄存器DMACCxLLI (偏移: 108h, 128h, 148h, 168h)	86
8.3.14.	通道控制寄存器DMACCxCtrl (偏移: 10Ch, 12Ch, 14Ch, 16Ch)	86
8.3.15.	通道配置寄存器DMACCxConfig (偏移: 110h, 130h, 150h, 170h)	88
8.4.	使用说明	90
8.4.1.	DMA优先级	90
8.4.2.	软件注意事项	90
8.4.3.	DMAC使用流程	90
8.4.4.	DMAC链表使用流程	90
9.	USB	92
9.1.	概述	92
9.2.	主要特性	92
9.3.	寄存器描述	92
9.3.1.	USB设备地址寄存器 FADDR (偏移: 00h)	93
9.3.2.	控制状态寄存器UCSR (偏移: 01h)	94
9.3.3.	发送中断状态寄存器IntrTx (偏移: 02h)	94
9.3.4.	接收中断状态寄存器IntrRx (偏移: 04h)	94
9.3.5.	发送中断使能寄存器INTRTXE (偏移: 06h)	95
9.3.6.	接收中断使能寄存器INTRRXE (偏移: 08h)	95
9.3.7.	USB中断状态寄存器IntrUSB (偏移: 0Ah)	95
9.3.8.	USB中断使能寄存器IntrUSBE (偏移: 0Bh)	95
9.3.9.	索引寄存器Eindex (偏移: 0Eh)	96
9.3.10.	USB测试模式寄存器Testmode (偏移: 0Fh)	96
9.3.11.	最大发送数据包设置寄存器TXPSZR (偏移: 10h)	96
9.3.12.	EP0 控制状态寄存器E0CSR (偏移: 12h)	96
9.3.13.	EPx发送控制状态寄存器TxCSR (偏移: 12h)	97
9.3.14.	最大接收数据包设置寄存器RXPSZR (偏移: 14h)	97
9.3.15.	EPx接收控制状态寄存器RxCSR (偏移: 16h)	98
9.3.16.	EP0 FIFO 计数寄存器E0COUNTR (偏移: 18h)	98
9.3.17.	EPx FIFO 计数寄存器RXCOUNT (偏移: 18h)	98
9.3.18.	EPx 发送FIFO大小TxFIFO_SIZE (偏移: 1Ah)	98
9.3.19.	EPx 接收FIFO大小RxFIFO_SIZE (偏移: 1Bh)	99
9.3.20.	EPx 发送FIFO地址TxFIFO_ADDR (偏移: 1Ch)	99
9.3.21.	Epx接收FIFO地址RxFIFO_ADDR (偏移: 1Eh)	99

9.3.22.	EP0 fifo通道Ep0FIFO (偏移: 20h)	99
9.3.23.	EP1 fifo通道Ep1FIFO (偏移: 24h)	99
9.3.24.	EP2 fifo通道Ep2FIFO (偏移: 28h)	100
9.3.25.	EP3 fifo通道Ep3FIFO (偏移: 2Ch)	100
9.3.26.	EP4 fifo通道Ep4FIFO (偏移: 30h)	100
9.3.27.	EPx发送数据长度寄存器 TX_PTR (偏移: 40h)	100
9.4.	使用流程	100
9.4.1.	控制传输	100
9.4.2.	大数据包传输 (split功能)	101
10.	NFM	102
10.1.	概述	102
10.2.	主要特性	102
10.3.	寄存器描述	102
10.3.1.	NFM控制寄存器NFMCTRL (偏移: 00h)	103
10.3.2.	NFM等待寄存器NFMWST(偏移: 04h)	103
10.3.3.	NFM状态寄存器NFMSTATUS (偏移: 08h)	104
10.3.4.	BCH配置寄存器BCH_CONFIG (偏移: 10h)	105
10.3.5.	BCH控制寄存器BCH_CTRL (偏移: 14h)	105
10.3.6.	BCH状态寄存器BCH_STATUS(偏移: 18h)	105
10.3.7.	BCH校验码寄存器BCH_CODE (偏移: 1Ch)	106
10.3.8.	BCH错误地址寄存器BCH_ERRADR (偏移: 20h)	106
10.3.9.	BCH错误向量寄存器BCH_ERRVEC (偏移: 24h)	106
10.3.10.	BCH纠错基地址寄存器BCH_BASEADDR(偏移: 28h)	107
10.3.11.	BCH校验码指针寄存器BCH_CODEPTR (偏移: 2Ch)	107
10.3.12.	BCH错误地址指针寄存器BCH_ERRADDRPTR (偏移: 30h)	107
10.3.13.	BCH错误向量指针寄存器BCH_ERRVECPTR (偏移: 34h)	107
10.3.14.	BCH页记数寄存器BCH_PAGENUM (偏移38h)	107
10.3.15.	BCH地址锁存寄存器BCH_ADDRLATCH (偏移: 3Ch)	107
10.3.16.	NFM命令通道寄存器NFMCMD (偏移: 40h)	107
10.3.17.	NFM地址通道寄存器NFMADDR (偏移: 44h)	107
10.3.18.	NFM ECC数据通道寄存器NFMECCDATA (偏移: 48h)	107
10.3.19.	NFM非ECC数据通道寄存器NFMNECCDATA (偏移: 4Ch)	108
10.4.	使用流程和注意事项	108
10.4.1.	写命令 (COMMAND) 时序	108
10.4.2.	写地址 (ADDRESS) 时序	108
10.4.3.	写数据 (DATA) 时序	109
10.4.4.	读数据 (DATA) 时序	109
10.4.5.	tWHR时序(WEN High to REN Low)	110
10.4.6.	tRHW时序(REN High to WEN Low)	110
10.4.7.	tADL时序(ALE to data out start)	110
10.4.8.	BCH编码流程	110
10.4.9.	BCH译码自动纠错模式流程	111
10.4.10.	BCH译码手动纠错模式流程	111
10.4.11.	数据通道访问配置	111

11. SDIO	113
11.1. 概述.....	113
11.2. 主要特性.....	113
11.3. 寄存器描述.....	113
11.3.1. 控制寄存器SDMMC_CTRL(偏移: 00h).....	114
11.3.2. 电源开关寄存器SDMMC_POWEREN(偏移: 04h).....	115
11.3.3. 时钟分频率寄存器SDMMC_CLKDIV(偏移: 08h).....	115
11.3.4. 时钟来源寄存器SDMMC_CLKSRC(偏移: 0Ch).....	116
11.3.5. 时钟使能寄存器SDMMC_CLKENA(偏移: 10h).....	116
11.3.6. 超时寄存器SDMMC_TIMEOUT(偏移: 14h).....	116
11.3.7. 位宽寄存器SDMMC_WIDTH(偏移: 18h).....	116
11.3.8. Block size 寄存器SDMMC_BLKSIZE(偏移: 1Ch).....	116
11.3.9. 字节个数寄存器SDMMC_BYTCNT(偏移: 20h).....	117
11.3.10. 中断屏蔽寄存器SDMMC_INTMASK(偏移: 24h).....	117
11.3.11. 命令参数寄存器SDMMC_CMDARG(偏移: 28h).....	117
11.3.12. 命令寄存器SDMMC_CMD(偏移: 2Ch).....	117
11.3.13. 应答0 寄存器SDMMC_RESP0(偏移: 30h).....	118
11.3.14. 应答1 寄存器SDMMC_RESP1(偏移: 34h).....	119
11.3.15. 应答2 寄存器SDMMC_RESP2(偏移: 38h).....	119
11.3.16. 应答3 寄存器SDMMC_RESP3(偏移: 3Ch).....	119
11.3.17. 可被屏蔽中断状态寄存器SDMMC_MINTSTS(偏移: 40h).....	119
11.3.18. 原始中断状态寄存器SDMMC_RINTSTS(偏移: 44h).....	119
11.3.19. 状态寄存器SDMMC_STATUS(偏移: 48h).....	120
11.3.20. FIFO比较寄存器SDMMC_FIFOTH(偏移: 4Ch).....	121
11.3.21. 卡检测寄存器SDMMC_CDETECT(偏移: 50h).....	122
11.3.22. 写保护寄存器SDMMC_WRTPRT(偏移: 54h).....	122
11.3.23. TCBCNT寄存器SDMMC_TCBCNT(偏移: 5Ch).....	122
11.3.24. TBBCNT寄存器SDMMC_TBBCNT(偏移: 60h).....	122
11.3.25. DEBOUNCE 寄存器SDMMC_DEBOUNCE(偏移: 64h).....	122
11.3.26. FIFO通道寄存器SDMMC_DATA(偏移: 100h).....	122
11.4. 使用说明.....	123
11.4.1. SDIO使用要点.....	123
11.4.2. SDIO使用流程.....	126
12. SPI.....	127
12.1. 概述.....	127
12.2. 主要特性.....	127
12.3. 寄存器描述.....	127
12.3.1. SPI发送数据寄存器SPI_TX_DAT(偏移00h).....	128
12.3.2. SI接收数据寄存器SPI_RX_DAT(偏移00h).....	128
12.3.3. SPI波特率设置寄存器SPI_BAUD(偏移: 04h).....	128
12.3.4. SPI控制寄存器SPI_CTL(偏移: 08h).....	128
12.3.5. SPI发送控制寄存器SPI_TX_CTL(偏移: 0Ch).....	129
12.3.6. SPI接收控制寄存器SPI_RX_CTL(偏移: 10h).....	130

12.3.7.	SPI 中断控制寄存器SPI_IE (偏移: 14h)	130
12.3.8.	SPI 状态寄存器SPI_STATUS (偏移: 18h)	132
12.3.9.	SPI 发送等待寄存器SPI_TXDelay(偏移: 1Ch).....	133
12.3.10.	SPI 批量传输数据个数寄存器SPI_BATCH (偏移: 20h).....	133
12.3.11.	SPI 从设备选择寄存器SPI_CS(偏移: 24h).....	133
12.3.12.	SPI 管脚输出方向SPI_OUT_EN(偏移: 28h)	133
12.3.13.	SPIA 存储映射控制寄存器SPI_MEMO_ACC(偏移: 2Ch).....	134
12.3.14.	SPIA 存储映射命令寄存器SPI_CMD(偏移: 30h)	135
12.3.15.	SPIA 存储映射参数寄存器SPI_PARA(偏移: 34h).....	135
12.4.	使用流程.....	136
12.4.1.	SPI 主模式发送.....	136
12.4.2.	SPI 主模式接收.....	136
12.4.3.	SPI 主模式接收时Dummy控制位.....	136
12.4.4.	SPI 从模式发送时Dummy控制位.....	136
12.4.5.	SPI 从模式接收时Dummy控制位.....	137
12.4.6.	SPIA 存储读映射.....	137
12.4.7.	SPIA 存储写映射.....	137
13.	CACHE.....	139
13.1.	概述.....	139
13.2.	主要特性.....	139
13.3.	寄存器描述.....	139
13.3.1.	控制寄存器CACHE_CR (偏移: d000h)	140
13.3.2.	Cache 数据空间CACHE_DATA (偏移: f000h~ffffh)	140
13.4.	使用流程.....	140
14.	UART.....	141
14.1.	概述.....	141
14.2.	主要特性.....	141
14.3.	寄存器描述.....	141
14.3.1.	数据寄存器UART_DR(偏移: 00h).....	141
14.3.2.	接收状态寄存器UART_RSR(偏移: 04h)	142
14.3.3.	标志位寄存器UART_FR(偏移: 18h).....	142
14.3.4.	整数分频因子寄存器UART_IBRD(偏移: 24h)	143
14.3.5.	小数分频因子寄存器UART_FBRD (偏移: 28h).....	144
14.3.6.	线控制器寄存器UART_LCRH (偏移: 2Ch)	144
14.3.7.	控制寄存器UART_CR (偏移: 30h).....	145
14.3.8.	FIFO 中断触发寄存器UART_IFLS(偏移: 34h)	145
14.3.9.	中断屏蔽使能清除寄存器UART_IMSC (偏移: 38h).....	146
14.3.10.	原始中断状态寄存器UART_RIS (偏移: 3Ch).....	147
14.3.11.	MASK 后的中断状态寄存器UART_MIS (偏移: 40h).....	147
14.3.12.	中断状态清除UART_ICR (偏移: 44h).....	148
14.4.	使用流程.....	149
14.4.1.	串口的发送和接收.....	149

15. TIMER	150
15.1. 概述	150
15.2. 主要特性	150
15.3. 寄存器描述	150
15.3.1. TIMER0 加载寄存器(T0_ARR偏移: 00h)	151
15.3.2. TIMER0 计数寄存器(T0_CNT偏移: 04h)	151
15.3.3. TIMER0 控制寄存器(T0_CR偏移: 08h)	152
15.3.4. TIMER0 中断状态寄存器(T0_IF偏移: 0Ch)	152
15.3.5. TIMER0 中断清除寄存器(T0_CIF偏移: 10h)	153
15.3.6. TIMER1 加载寄存器(T1_ARR偏移: 14h)	153
15.3.7. TIMER1 计数寄存器(T1_CNT偏移: 18h)	153
15.3.8. TIMER1 控制寄存器(T1_CR偏移: 1Ch)	153
15.3.9. TIMER1 中断状态寄存器(T1_IF偏移: 20h)	154
15.3.10. TIMER1 中断清除寄存器(T1_CIF偏移: 24h)	155
15.3.11. TIMER2 加载寄存器(T2_ARR偏移: 28h)	155
15.3.12. TIMER2 计数寄存器(T2_CNT偏移: 2Ch)	155
15.3.13. TIMER2 控制寄存器(T2_CR偏移: 30h)	155
15.3.14. TIMER2 中断状态寄存器(T2_IF偏移: 34h)	156
15.3.15. TIMER2 中断清除寄存器(T2_CIF偏移: 38h)	157
15.3.16. TIMER3 加载寄存器(T3_ARR偏移: 3Ch)	157
15.3.17. TIMER3 计数寄存器(T3_CNT偏移: 40h)	157
15.3.18. TIMER3 控制寄存器(T3_CR偏移: 44h)	157
15.3.19. TIMER3 中断状态寄存器(T3_IF偏移: 48h)	158
15.3.20. TIMER3 中断清除寄存器(T3_CIF偏移: 4Ch)	158
15.3.21. 定时器预分频寄存器(TIM_PSC偏移: 50h)	159
15.3.22. 工作模式控制寄存器ICMODE(偏移: 54h)	160
15.3.23. 输入捕获模式控制寄存器CCR(偏移: 58h)	160
15.3.24. 输入捕获模式中中断状态寄存器CCIF(偏移: 5Ch)	161
15.3.25. Channel0 输入捕获模式counter寄存器C0_CR(偏移: 60h)	161
15.3.26. Channel2 输入捕获模式counter寄存器C2_CR(偏移: 64h)	162
15.3.27. PWM控制寄存器PCR(偏移: 68h)	162
15.3.28. PWM中断状态寄存器CPIF(偏移: 6Ch)	163
15.3.29. PWM0 比较寄存器C0_PR(偏移: 70h)	163
15.3.30. PWM1 比较寄存器C1_PR(偏移: 74h)	163
15.3.31. PWM2 比较寄存器C2_PR(偏移: 78h)	163
15.3.32. PWM3 比较寄存器C3_PR(偏移: 7Ch)	163
15.4. 使用说明	164
15.4.1. Counter 工作模式	164
15.4.2. PWM模式	165
15.4.3. 输入捕获模式	166
16. WDT	167
16.1. 概述	167
16.2. 主要特性	167

16.3. 寄存器描述.....	167
16.3.1. 数据加载寄存器LOAD(偏移: 00h).....	167
16.3.2. 当前计数值(偏移: 04h).....	167
16.3.3. 控制寄存器(偏移: 08h).....	167
16.3.4. 喂狗寄存器(偏移: 0Ch).....	168
16.3.5. 中断清除时限寄存器(偏移: 10h).....	168
16.3.6. 原始中断状态寄存器(偏移: 14h).....	168
16.4. 使用流程.....	168
16.4.1. 定时器溢出产生中断.....	168
16.4.2. 定时器溢出产生复位.....	169
17. I2C	170
17.1. 概述.....	170
17.2. 主要特性.....	170
17.3. 寄存器描述.....	170
17.3.1. 控制状态寄存器I2C_CSR(偏移: 00h).....	170
17.3.2. 总线设备地址寄存器I2C_SLAVE_ADDR(偏移: 04h).....	171
17.3.3. SCL速率分频寄存器I2C_CLK_DIV(偏移: 08h).....	171
17.3.4. XFIFO控制寄存器I2C_FIFO_CTRL(偏移: 0Ch).....	172
17.3.5. 中断状态寄存器寄存器I2C_INT_STAT(偏移: 0xffe0).....	172
17.3.6. 中断原始状态寄存器I2C_INT_STAT_RAW(偏移: 0xffe4).....	173
17.3.7. 中断使能寄存器I2C_INT_EN(偏移: 0xffe8).....	173
17.3.8. 中断设置寄存器I2C_INT_SET(偏移: 0xffec).....	173
17.3.9. 中断清除寄存器I2C_INT_CLR(偏移: 0xffff0).....	174
17.3.10. FIFO接口寄存器I2C_FIFO(偏移: 0x100).....	174
17.4. 使用流程.....	174
17.4.1. I2C写.....	174
17.4.2. I2C读.....	175
18. ISO7816MS	176
18.1. 概述.....	176
18.2. 主要特性.....	176
18.3. 寄存器描述.....	176
18.3.1. 状态寄存器SCISR(偏移: 00h).....	177
18.3.2. 中断使能寄存器SCIIER(偏移: 04h).....	179
18.3.3. 控制寄存器SCICTRL(偏移: 08h).....	180
18.3.4. 主模式控制寄存器SCIMCTRL(偏移: 0Ch).....	182
18.3.5. 数据寄存器SCIDR(偏移: 10h).....	183
18.3.6. 复位时间寄存器SCIRSTT(偏移: 14h).....	183
18.3.7. 波特率寄存器SCIBPR(偏移: 18h).....	183
18.3.8. ETU计数寄存器SCIETU(偏移: 1Ch).....	183
18.3.9. 错误校验码寄存器SCIEDC(偏移: 20h).....	184
18.3.10. CCK计数寄存器SCICCKCNT(偏移: 24h).....	184
18.4. 使用流程.....	184
18.4.1. 主模式选择及上下电.....	184

18.4.2.	数据收发初始化.....	185
18.4.3.	发送字节步骤.....	185
18.4.4.	接收字节步骤.....	186
18.4.5.	接收转发送.....	186
18.4.6.	发送转接收.....	186
19.	SENSOR.....	187
19.1.	概述.....	187
19.2.	主要特性.....	187
19.3.	寄存器描述.....	188
19.3.1.	安全检测控制寄存器1SECR1(偏移: 00h)	188
19.3.2.	安全检测控制寄存器2 SECR2 (偏移: 04h)	189
19.3.3.	外部频率检测阈值寄存器EFDTH(偏移: 08h)	190
19.3.4.	安全检测中断使能寄存器SECINTEN(偏移: 10h)	190
19.3.5.	安全检测状态寄存器SESR(偏移: 14h)	191
19.3.6.	频率检测计数寄存器FDCNTR(偏移: 18h)	193
19.4.	使用流程.....	193
19.4.1.	模拟自检操作流程.....	193
19.4.2.	检测操作流程.....	193
19.4.3.	外部输入时钟频率检测.....	193
19.4.4.	温度检测TD	193
19.4.5.	电压检测VD	194
19.4.6.	电源毛刺检测PGD	194
19.4.7.	光检测LD	194
19.4.8.	总线异常检测.....	194
19.4.9.	Active Shielding 检测.....	194
19.4.10.	时钟毛刺检测CGD.....	195
19.5.	注意事项.....	195
20.	GPIO.....	196
20.1.	概述.....	196
20.2.	主要特性.....	196
20.3.	寄存器描述.....	196
20.3.1.	数据方向寄存器GPIO_DIR(偏移: 00h).....	196
20.3.2.	输出置位寄存器GPIO_SET(偏移: 08h).....	197
20.3.3.	输出清零寄存器GPIO_CLR(偏移: 0Ch).....	197
20.3.4.	GPIO输出引脚映射寄存器GPIO_ODATA(偏移: 10h).....	197
20.3.5.	GPIO输入引脚映射寄存器GPIO_IDATA(偏移: 14h)	197
20.3.6.	GPIO中断使能寄存器GPIO_IEN(偏移: 18h).....	198
20.3.7.	GPIO中断触发模式寄存器GPIO_IS(偏移: 1Ch)	198
20.3.8.	GPIO中断触发模式寄存器GPIO_IBE(偏移: 20h).....	198
20.3.9.	GPIO中断触发模式寄存器GPIO_IEV(偏移: 24h).....	198
20.3.10.	GPIO中断状态清除寄存器GPIO_IC(偏移: 28h).....	198
20.3.11.	GPIO原始中断状态寄存器GPIO_RIS(偏移: 2Ch).....	199
20.3.12.	GPIO屏蔽后中断状态寄存器GPIO_MIS(偏移: 30h)	199

20.4.	使用流程.....	199
20.4.1.	输入输出IO	199
20.4.2.	中断触发模式.....	199
20.4.3.	清除中断.....	199
21.	MIM.....	201
21.1.	概述.....	201
21.2.	主要特性.....	201
21.3.	寄存器描述.....	201
21.3.1.	控制寄存器MIM_CSCRx (偏移: 00h/04h/08h/0ch, x=0~3)	202
21.3.2.	TFT片选寄存器MIM_TFTS(偏移: 10h)	203
21.3.3.	TFT命令通道寄存器MIM_CMD (偏移: 14h).....	203
21.3.4.	TFT数据通道寄存器MIM_DATA (偏移: 18h).....	203
21.3.5.	区域划分寄存器MIM_SEG0 (偏移: 1ch)	204
21.3.6.	区域划分寄存器MIM_SEG1 (偏移: 20h)	204
21.3.7.	区域划分寄存器MIM_SEG2 (偏移: 24h)	204
21.3.8.	区域划分寄存器MIM_SEG3 (偏移: 28h)	205
21.4.	功能描述.....	205
21.4.1.	片选	205
21.4.2.	EBn信号时序.....	205
21.4.3.	字节使能管脚.....	206
21.4.4.	TFT-LCD控制.....	207
21.5.	使用方法.....	207
21.5.1.	片外存储器读操作.....	207
21.5.2.	片外存储器写操作.....	208
21.5.3.	TFT-LCD操作	208
22.	CRC16	209
22.1.	概述.....	209
22.2.	寄存器描述.....	209
22.2.1.	数据寄存器CRC16_DATA (偏移: 00H)	209
22.2.2.	初始值寄存器 CRC16_INIT (偏移: 04H)	209
22.2.3.	控制寄存器 CRC16_CTRL (偏移: 08H)	209
22.3.	操作方法.....	210
23.	算法库.....	211
23.1.	数据类型	211
23.2.	HRNG	211
23.2.1.	主要特性.....	211
23.2.2.	库文件说明.....	211
23.2.3.	函数说明.....	211
23.2.4.	注意	212
23.3.	DES	212
23.3.1.	主要特性.....	212
23.3.2.	库文件说明.....	212

23.3.3.	函数说明.....	212
23.3.4.	注意	213
23.4.	SM1.....	213
23.4.1.	主要特性.....	213
23.4.2.	库文件说明.....	213
23.4.3.	函数说明.....	213
23.4.4.	注意	214
23.5.	SM4.....	215
23.5.1.	主要特性.....	215
23.5.2.	库文件说明.....	215
23.5.3.	函数说明.....	215
23.5.4.	注意	216
23.6.	AES	216
23.6.1.	主要特性.....	216
23.6.2.	库文件说明.....	216
23.6.3.	函数说明.....	216
23.6.4.	注意	217
23.7.	SSF33.....	217
23.7.1.	主要特性.....	217
23.7.2.	库文件说明.....	217
23.7.3.	函数说明.....	217
23.7.4.	注意	218
23.8.	RSA.....	218
23.8.1.	主要特性.....	218
23.8.2.	库文件说明.....	219
23.8.3.	函数说明.....	219
23.8.4.	注意	220
23.9.	SM2_SM3.....	221
23.9.1.	主要特性.....	221
23.9.2.	库文件说明.....	221
23.9.3.	函数说明.....	221
23.9.4.	注意	224
23.10.	HASH.....	224
23.10.1.	主要特性	224
23.10.2.	库文件说明	224
23.10.3.	函数说明	225
23.10.4.	注意	225
23.11.	ECDSA.....	226
23.11.1.	主要特性	226
23.11.2.	库文件说明	226
23.11.3.	函数说明	226
23.11.4.	注意	227
23.12.	注意事项.....	227
24.	电气参数.....	228

24.1.	绝对最大额定值	228
24.2.	典型操作条件	228
24.3.	DC参数	228
24.4.	时钟参数	230
24.5.	功耗参数	230

保 密

图目录

图 1-1 功能框图.....	20
图 1-2 电源框图.....	21
图 2-1 LQFP100 封装管脚示意图.....	22
图 2-2LQFP64 封装管脚示意图.....	23
图 2-3 QFN40 封装管脚示意图.....	24
图 3-1 Cortex-M系列处理器功能框图.....	32
图 3-2Cortex-M系列的寄存器组.....	33
图 3-3Cortex-M系列的特殊功能寄存器.....	34
图 4-1 时钟模块框图.....	37
图 13-1 Cache在系统中位置.....	139
图 15-1 TIMER向上计数的PWM方式.....	165
图 15-2 TIMER向上下计数的PWM方式.....	166
图 19-1 SENSOR模块整体框图	187

表目录

表 2-1 引脚功能说明.....	24
表 4-1 模块地址划分.....	35
表 4-2 系统时钟选择.....	38
表 4-3 系统复位源.....	38
表 4-4 低功耗模式.....	39
表 5-1 中断源.....	70
表 8-1DBSIZE/SBSIZE对应的burst size.....	88
表 8-2 SWidth/DWidth对应的width.....	88
表 8-3FlowCtrl对应的传输方向和controller.....	89
表 8-4 目标外设和源外设请求号.....	89
表 10-1 数据通道访问配置.....	111
表 11-1 SDIO中断源描述	124
表 28-1 HRNG驱动函数说明	211
表 28-2 DES驱动函数说明	212
表 28-3 SM1 驱动函数说明	213
表 28-4 SM4 驱动函数说明	215
表 28-5 AES驱动函数说明	216
表 28-5 SSF33 驱动函数说明	217
表 28-6 RSA驱动函数说明.....	219
表 28-7 SM2_SM3 驱动函数说明	221
表 28-8 ECDSA驱动函数说明.....	226
表 24-1 绝对最大额定值.....	228
表 24-2 典型操作条件.....	228
表 24-3DC参数	228

1. 系统概述

ACH512 芯片是上海爱信诺航芯电子科技有限公司最新研制的一款基于安全算法的高性能 SOC 芯片，主要应用于 eMMC/SD/Nandflash 大容量存储设备、加密 U 盘、指纹识别等市场。芯片采用 32 位 ARM 高性能 Cortex™-M 系列内核，片内集成多种安全密码模块，包括国密 SM1、SM2、SM3、SM4、SSF33 算法以及 RSA/ECC、ECDSA、DES/3DES、AES128/192/256、SHA1/256/384/512 等安全算法，芯片提供了多种外围接口：USB2.0、UART、SDIO、ISO7816、I2C、SPI、NFM，MIM 等。

芯片包括 512K 字节的片内 eFlash，16K 字节的 ROM，128K 字节的片内 SRAM，4K 字节算法专用 SRAM（可开放做普通 SRAM），片内 ROM 提供各种算法接口程序供用户调用。

1.1. 主要特点

1.1.1. 硬件功能

- **CPU内核**
 - ARM 32 位 Cortex™-M 系列
 - 最高系统 110MHz 工作频率
 - 硬件单周期乘法器
 - Thumb-2 指令集
 - 三级流水线，指令和数据总线独立
 - 内置 MPU
 - NVIC 中断控制器
 - SysTick 定时器
- **CACHE**
 - 4K 字节指令加速（可作为通用 SRAM）
- **看门狗**
 - 32 位的递减计数器
 - 支持中断和复位模式配置
- **定时器**
 - 包含 4 个独立的 32 位递减/递增计数器
 - 支持三种工作模式：free-running, cyclic, single
 - 可配置为输入捕获模式和 PWM 输出比较
- **存储器**
 - 512K 字节的片内 Eflash 存储器
 - 128K 字节的片内 SRAM（两个独立的 64KB 空间）
 - 16K 字节的 ROM

- 通讯接口

- USB 接口：支持 USB2.0 高速和全速、1 个控制端点和 4 个通用双向端点、FIFO 地址可动态分配
- 7816MS 接口：支持 ISO7816-3 协议、支持 T0/T1 协议
- UART 接口：2 路 UART，支持波特率小数分频，支持 16 字节 FIFO，其中 UARTB 是高速 UART，且支持 CTS、RTS 功能
- NFM 控制器：自带 BCH 纠错引擎、支持 8bit/528B 纠错模式
- eMMC/SDIO 接口：支持 eMMC4.4.1，支持 1bit/4bit/8bit 模式
- I2C 主接口：支持 Standard/Fast/HS 模式、8 字节 FIFO
- GPIO 接口：最大支持 64 个 GPIO（包括复用）、支持边沿/电平中断
- MIM 存储器接口：支持 8/16 位片外 SRAM、Norflash 扩展，支持 8080 TFT-LCD 接口
- SPI 接口：2 路 SPI 接口、支持 Mode0/1/2/3 传输协议，支持主/从接口，其中 SPIA 支持外部存储器存储映射。

- 时钟

- 外部 12M 晶振
- 内部 RC48M 振荡器
- 内部 RC32K 振荡器
- 内部 PLL 时钟
- 内部 120M USB PHY 时钟(必须外接 12M 晶振)

- 低功耗

- 睡眠（SLEEP）、待机模式（STANDBY）、下电模式（POWEROFF）

- 调试模式

- JTAG-SWD 调试接口（仅开发版支持）

1.1.2. 安全功能

- 算法模块

- 64 位高速硬件公钥算法引擎（PKI），
- 64~4096bit RSA 算法单元
- ECC 素域、二元域算法单元
- DES/3DES 算法单元
- AES128/192/256 算法单元
- SM1 算法单元
- SM2 算法单元
- SM3/SHA1/256/384/512 算法单元
- SM4 算法单元

- SSF33 算法单元
- 真随机数发生器 HRNG
- CRC16-CCITT 校验单元
- **安全检测与防护**
 - 高低电压检测 2.5V~5.7V
 - 频率检测
 - 高低温检测, -40~85°C
 - 光检测
 - 电源/时钟毛刺检测
 - 主动金属屏蔽层
 - 总线加密串扰
 - 存储器奇偶校验
 - 128 位唯一芯片序列号

1.1.3. 软件支持

- 内置 ROM BOOTLOADER, 支持 USB/SPI/UART 接口下载
- Keil Realview MDK (至少 4.0 版本)

1.1.4. 封装形式

- LQFP100
- LQFP64
- QFN40

1.2. 功能框图

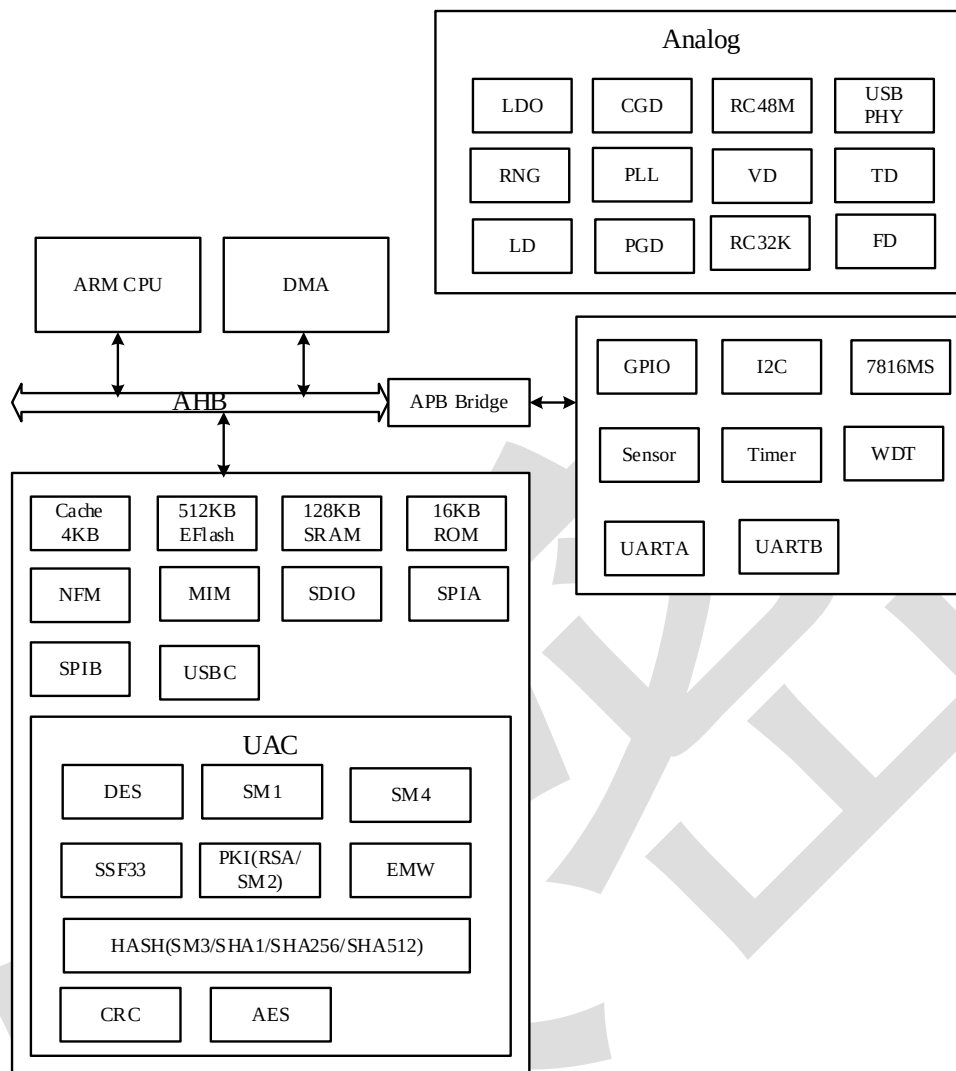


图 1-1 功能框图

1.3. 电源框图

芯片工作的电压范围是 1.68 到 5.5V。为了减小功耗，应用程序可以在性能、功耗中获得最佳折衷，电源控制提供了多种低功耗模式供用户选择。如下图所示，芯片的供电方式示意图。

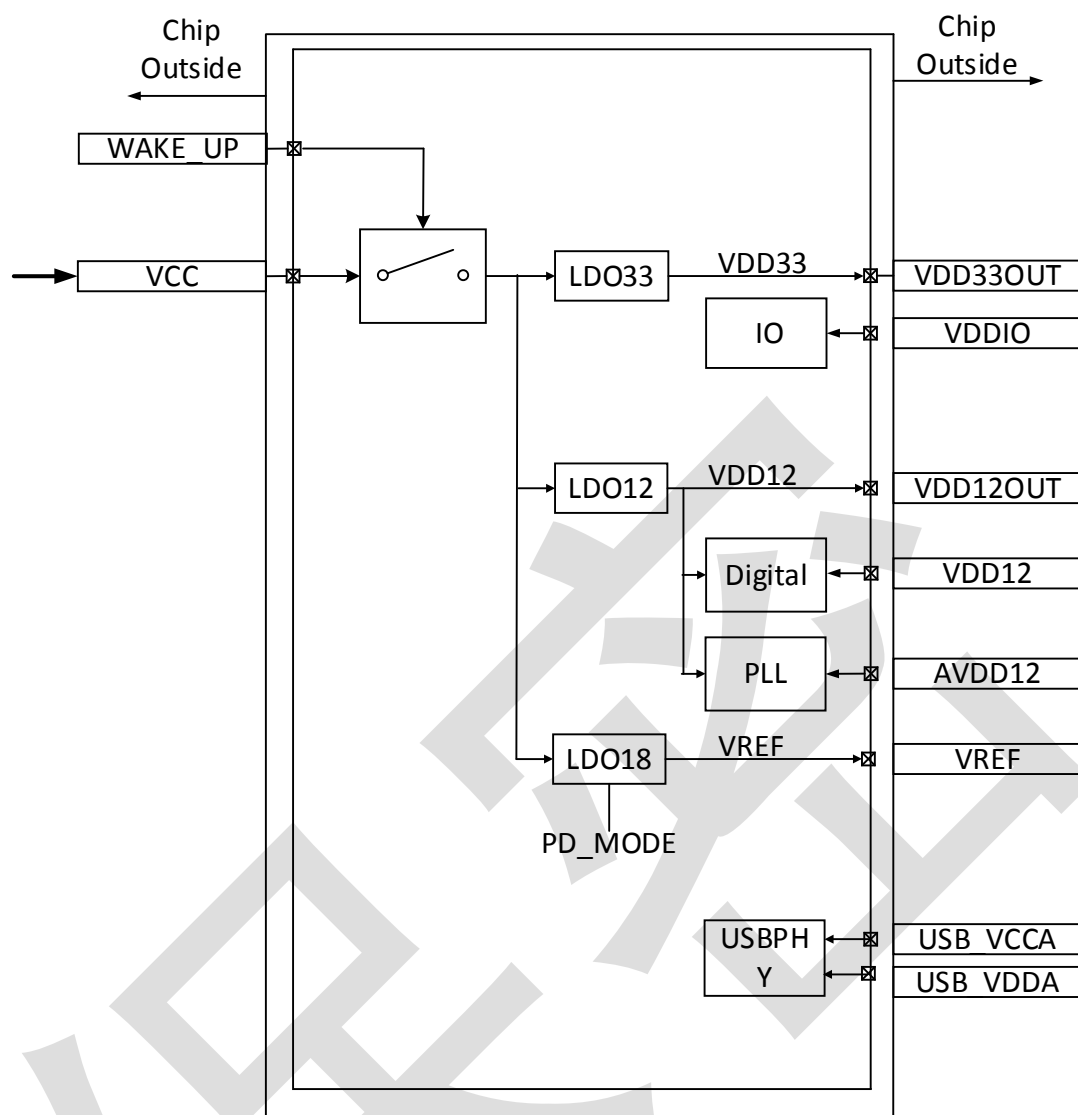


图 1-2 电源框图

2. 引脚描述

2.1. 封装管脚分布

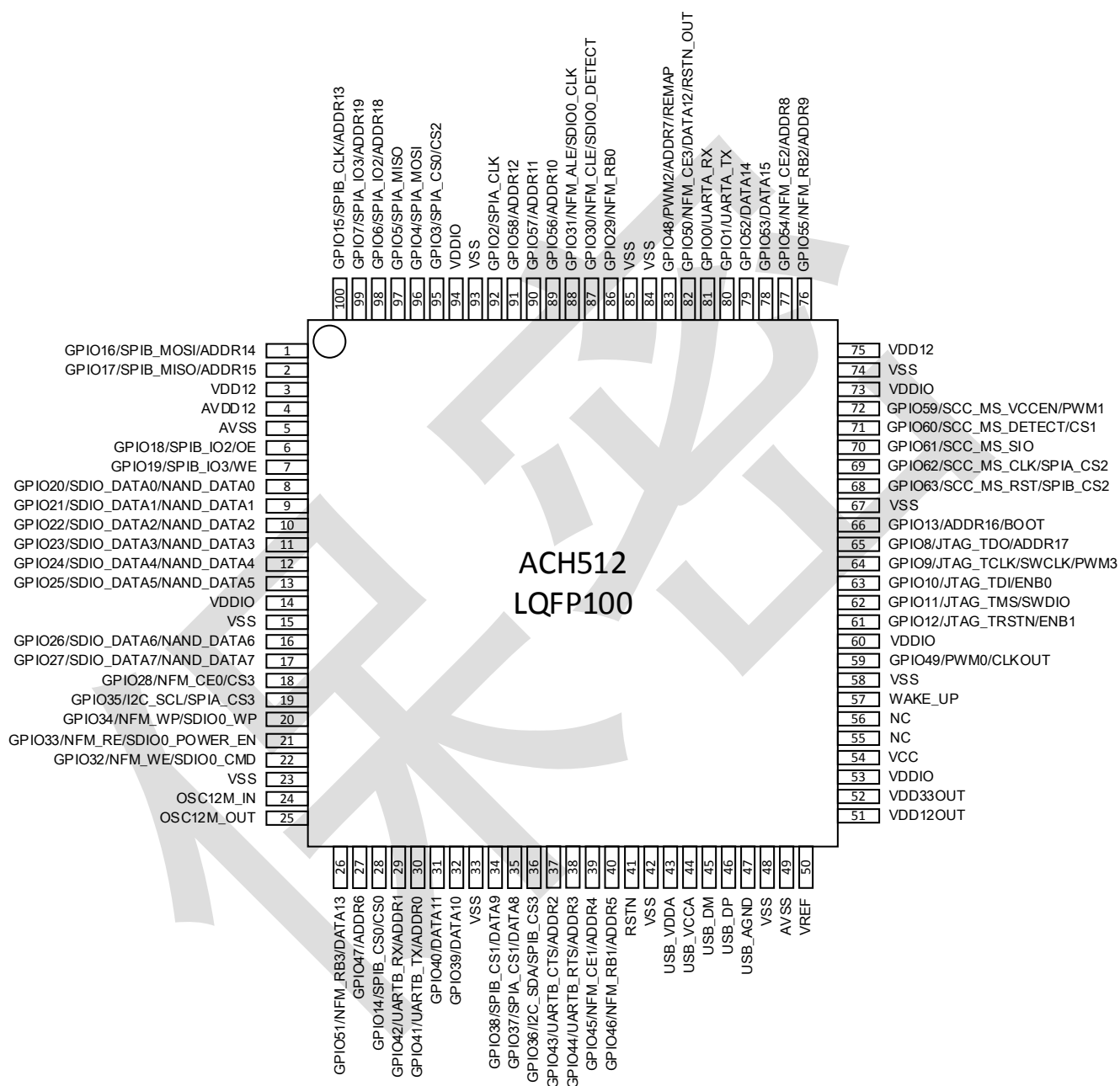


图 2-1 LQFP100 封装管脚示意图

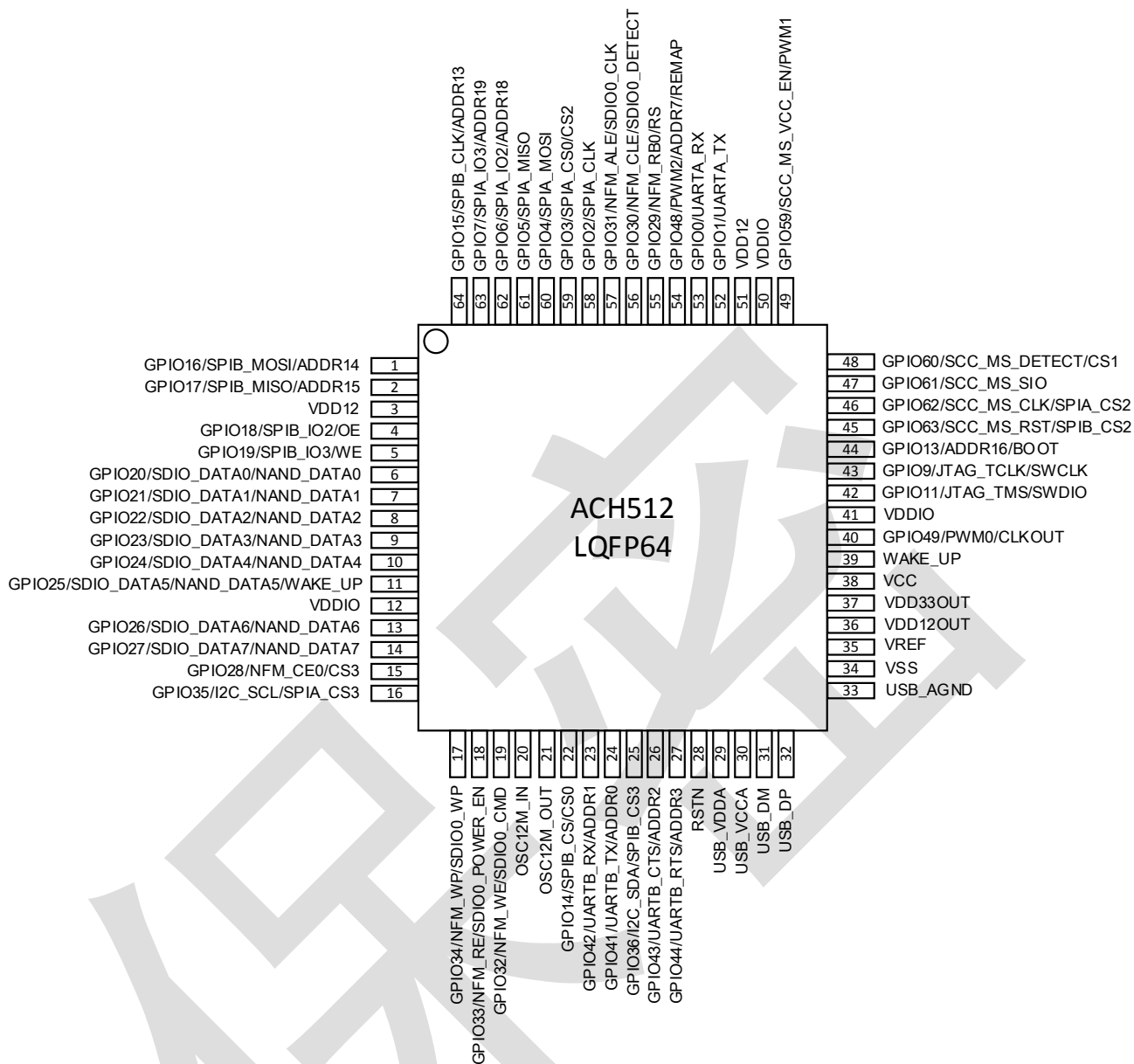


图 2-2LQFP64 封装管脚示意图

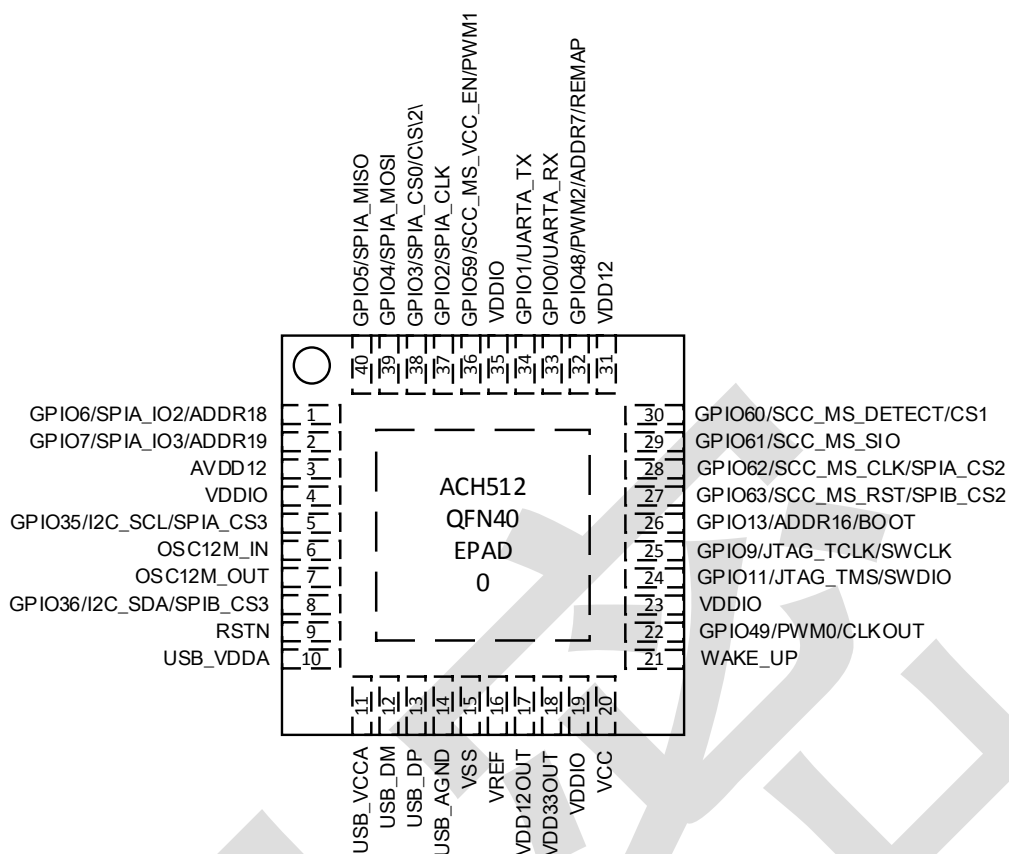


图 2-3 QFN40 封装管脚示意图

2.2. 信号描述

表 2-1 引脚功能说明

引脚功能			封装引脚编号			IO Type	复位状态		功能描述 (上电默认功能为功能 1)
功能 1	功能 2	功能 3	LQFP 100	LQFP 64	QFN 40		DIR	PUP D	
GPIO0	UARTA_RX	-	81	53	33	I/O	DI	PU	1. GPIO0 2. UARTA 接收信号
GPIO1	UARTA_TX	-	80	52	34	I/O	DI	PU	1. GPIO1 2. UARTA 发送信号
GPIO2	SPIA_CLK	-	92	58	37	I/O	DI	PU	1. GPIO2 2. SPIA 时钟信号
GPIO3	SPIA_CS0	CS2	95	59	38	I/O	DI	PU	1. GPIO3 2. SPIA 片选信号 0 3. 片外总线片选信号 2
GPIO4	SPIA_MOSI	-	96	60	39	I/O	DI	PU	1. GPIO4 2. SPIA IO0(MOSI)信号

GPIO5	SPIA_MISO	-	97	61	40	I/O	DI	PU	1. GPIO5 2. SPIA IO1(MISO)信号
GPIO6	SPIA_IO2	ADDR18	98	62	1	I/O	DI	PU	1. GPIO6 2. SPIA IO2(WP)信号 3. 片外总线地址信号 18
GPIO7	SPIA_IO3	ADDR19	99	63	2	I/O	DI	PU	1. GPIO7 2. SPIA IO3(HOLD)信号 3. 片外总线地址信号 19
GPIO8	JTAG_TDO	ADDR17	65	-	-	I/O	DI	PU	1. GPIO8 2. JTAG TDO 信号 3. 片外总线地址信号 17
GPIO9	JTAG_TCLK/ SWCLK	PWM3	64	43	25	I/O	DI	PU	1. GPIO9 2. JTAG TCLK 信号/JTAG SWCLK 信号 3. TIMER3 PWM 通道
GPIO10	JTAG_TDI	ENB0	63	-	-	I/O	DI	PU	1. GPIO10 2. JTAG TDI 信号 3. 片外总线字节使能信号 0
GPIO11	JTAG_TMS/ SWDIO	-	62	42	24	I/O	DI	PU	1. GPIO11 2. JTAG TMS 信号/JTAG SWDIO 信号
GPIO12	JTAG_TRST N	ENB1	61	-	-	I/O	DI	PU	1. GPIO12 2. JTAG TRSTN 信号 3. 片外总线字节使能信号 1
GPIO13	ADDR16	BOOT	66	44	26	I/O	DI	PU	1. GPIO13 2. 片外总线地址信号 16 3. boot 引脚, 芯片启动模式 选择 注: 上电时锁存该管脚电平, 作为启动模式选择管脚, 0: EFlash, 1: Rom
GPIO14	SPIB_CS0	CS0	28	22	-	I/O	DI	PU	1. GPIO14 2. SPIB 片选信号 0 3. 片外总线片选使能信号 0
GPIO15	SPIB_CLK	ADDR13	100	64	-	I/O	DI	PU	1. GPIO15 2. SPIB 时钟信号 3. 片外总线地址信号 13
GPIO16	SPIB_MOSI	ADDR14	1	1	-	I/O	DI	PU	1. GPIO16 2. SPIB IO0(MOSI)信号 3. 片外总线地址信号 14

GPIO17	SPIB_MISO	ADDR15	2	2	-	I/O	DI	PU	1. GPIO17 2. SPIB IO1(MISO)信号 3. 片外总线地址信号 15
GPIO18	SPIB_IO2	OE	6	4	-	I/O	DI	PU	1. GPIO18 2. SPIB IO2(WP)信号 3. 片外总线读使能信号
GPIO19	SPIB_IO3	WE	7	5	-	I/O	DI	PU	1. GPIO19 2. SPIB IO3(HOLD)信号 3. 片外总线写使能信号
GPIO20	SDIO_DATA 0	NAND_D ATA0	8	6	-	I/O	DI	PU	1. GPIO20 2. SDIO 数据信号 0 3. 片外总线数据信号 0(NFM 与 MIM 模块 DATA0 复用)
GPIO21	SDIO_DATA 1	NAND_D ATA1	9	7	-	I/O	DI	PU	1. GPIO21 2. SDIO 数据信号 1 3. 片外总线数据信号 1(NFM 与 MIM 模块 DATA1 复用)
GPIO22	SDIO_DATA 2	NAND_D ATA2	10	8	-	I/O	DI	PU	1. GPIO22 2. SDIO 数据信号 2 3. 片外总线数据信号 2(NFM 与 MIM 模块 DATA2 复用)
GPIO23	SDIO_DATA 3	NAND_D ATA3	11	9	-	I/O	DI	PU	1. GPIO23 2. SDIO 数据信号 3 3. 片外总线数据信号 3(NFM 与 MIM 模块 DATA3 复用)
GPIO24	SDIO_DATA 4	NAND_D ATA4	12	10	-	I/O	DI	PU	1. GPIO24 2. SDIO 数据信号 4 3. 片外总线数据信号 4(NFM 与 MIM 模块 DATA4 复用)
GPIO25	SDIO_DATA 5	NAND_D ATA5	13	11	-	I/O	DI	PU	1. GPIO25 2. SDIO 数据信号 5 3. 片外总线数据信号 5(NFM 与 MIM 模块 DATA5 复用)
GPIO26	SDIO_DATA 6	NAND_D ATA6	16	13	-	I/O	DI	PU	1. GPIO26 2. SDIO 数据信号 6 3. 片外总线数据信号 6(NFM 与 MIM 模块 DATA6 复用)

GPIO27	SDIO_DATA 7	NAND_D ATA7	17	14	-	I/O	DI	PU	1. GPIO27 2. SDIO 数据信号 7 3. 片外总线数据信号 7(NFM 与 MIM 模块 DATA7 复用)
GPIO28	NFM_CEO	CS3	18	15	-	I/O	DI	PU	1. GPIO28 2. NandFlash 片选信号 0 3. 片外总线片选信号 3
GPIO29	NFM_RB0	RS	86	55	-	I/O	DI	PU	1. GPIO29 2. NandFlash R/B 信号 0 3. RS 信号在 MIM 模块作为 8080 接口的 TFT 控制器时使 用, 低电平标志当前发送的 是命令, 高电平标志当前进 行的是数据操作
GPIO30	NFM_CLE	SDIO0_D TECT	87	56	-	I/O	DI	PU	1. GPIO30 2. NandFlash 命令锁存使能 信号 3. SDIO 卡到位检测信号
GPIO31	NFM_ALE	SDIO0_C LK	88	57	-	I/O	DI	PU	1. GPIO31 2. NandFlash 地址锁存使能 信号 3. SDIO 时钟信号
GPIO32	NFM_WE	SDIO0_C MD	22	19	-	I/O	DI	PU	1. GPIO32 2. NandFlash 写使能信号 3. SDIO 命令信号
GPIO33	NFM_RE	SDIO0_P OWER_E N	21	18	-	I/O	DI	PU	1. GPIO33 2. NandFlash 读使能信号 3. SDIO 电源使能信号
GPIO34	NFM_WP	SDIO0_W P	20	17	-	I/O	DI	PU	1. GPIO34 2. NandFlash 写保护信号 3. SDIO 写保护信号
GPIO35	I2C_SCL	SPIA_CS 3	19	16	5	I/O	DI	PU	1. GPIO35 2. I2C 时钟信号 3. SPIA 片选信号 3
GPIO36	I2C_SDA	SPIB_CS 3	36	25	8	I/O	DI	PU	1. GPIO36 2. I2C 数据信号 3. SPIB 片选信号 3
GPIO37	SPIA_CS1	DATA8	35	-	-	I/O	DI	PU	1. GPIO37 2. SPIA 片选信号 1 3. 片外总线数据信号 8(MIM 模块 DATA8)

GPIO38	SPIB_CS1	DATA9	34	-	-	I/O	DI	PU	1. GPIO38 2. SPIB 片选信号 1 3. 片外总线数据信号 9(MIM 模块 DATA9)
GPIO39	DATA10	-	32	-	-	I/O	DI	PU	1. GPIO39 2. 片外总线数据信号 10(MIM 模块 DATA10)
GPIO40	DATA11	-	31	-	-	I/O	DI	PU	1. GPIO40 2. 片外总线数据信号 11(MIM 模块 DATA11)
GPIO41	UARTB_TX	ADDR0	30	24	-	I/O	DI	PU	1. GPIO41 2. UARTB 发送信号 3. 片外总线地址信号 0
GPIO42	UARTB_RX	ADDR1	29	23	-	I/O	DI	PU	1. GPIO42 2. UARTB 接收信号 3. 片外总线地址信号 1
GPIO43	UARTB_CTS	ADDR2	37	26	-	I/O	DI	PU	1. GPIO43 2. UARTB CTS 信号 3. 片外总线地址信号 2
GPIO44	UARTB_RTS	ADDR3	38	27	-	I/O	DI	PU	1. GPIO44 2. UARTB RTS 信号 3. 片外总线地址信号 3
GPIO45	NFM_CE1	ADDR4	39	-	-	I/O	DI	PU	1. GPIO45 2. NandFlash 片选信号 1 3. 片外总线地址信号 4
GPIO46	NFM_RB1	ADDR5	40	-	-	I/O	DI	PU	1. GPIO46 2. NandFlash R/B 信号 1 3. 片外总线地址信号 5
GPIO47	ADDR6	-	27	-	-	I/O	DI	PU	1. GPIO47 2. 片外总线地址信号 6
GPIO48	PWM2	ADDR7	83	54	32	I/O	DO	PU	1. GPIO48 2. TIMER2 输入捕获/PWM 通道 3. 片外总线地址信号 7 4. REMAP: 芯片启动模式输出 (1: EFlash 启动, 0: ROM 启动)
GPIO49	PWM0	CLKOUT	59	40	22	I/O	DO	PU	1. GPIO49 2. TIMER0 输入捕获/PWM 通道 3. 芯片系统时钟信号输出

GPIO50	NFM_CE3	DATA12	82	-	-	I/O	DI	PU	1. GPIO50 2. NandFlash 片选信号 3 3. 片外总线数据信号 12(MIM 模块 DATA12) 4. RSTN_OUT: 芯片系统复位完成输出
GPIO51	NFM_RB3	DATA13	26	-	-	I/O	DI	PU	1. GPIO51 2. NandFlash R/B 信号 3 3. 片外总线数据信号 13(MIM 模块 DATA13)
GPIO52	DATA14	-	79	-	-	I/O	DI	PU	1. GPIO52 2. 片外总线数据信号 14(MIM 模块 DATA14)
GPIO53	DATA15	-	78	-	-	I/O	DI	PU	1. GPIO53 2. 片外总线数据信号 15(MIM 模块 DATA15)
GPIO54	NFM_CE2	ADDR8	77	-	-	I/O	DI	PU	1. GPIO54 2. NandFlash 片选信号 2 3. 片外总线地址信号 8
GPIO55	NFM_RB2	ADDR9	76	-	-	I/O	DI	PU	1. GPIO55 2. NandFlash R/B 信号 2 3. 片外总线地址信号 9
GPIO56	ADDR10	-	89	-	-	I/O	DI	PU	1. GPIO56 2. 片外总线地址信号 10
GPIO57	ADDR11	-	90	-	-	I/O	DI	PU	1. GPIO57 2. 片外总线地址信号 11
GPIO58	ADDR12	-	91	-	-	I/O	DI	PU	1. GPIO58 2. 片外总线地址信号 12
GPIO59	SCC_MS_VCC_EN	PWM1	72	49	36	I/O	DI	PU	1. GPIO59 2. 智能卡接口电源使能信号 3. TIMER1PWM 通道
GPIO60	SCC_MS_DETECT	CS1	71	48	30	I/O	DI	PU	1. GPIO60 2. 智能卡接口卡到位检测信号 3. 片外总线选择信号 1
GPIO61	SCC_MS_SIO	-	70	47	29	I/O	DI	PU	1. GPIO61 3. 智能卡接口数据信号
GPIO62	SCC_MS_CLOCK	SPIA_CS2	69	46	28	I/O	DI	PU	1. GPIO62 2. 智能卡接口时钟信号 3. SPIA 片选信号 2

GPIO63	SCC_MS_RST	SPIB_CS2	68	45	27	I/O	DI	PU	1. GPIO47 3. 智能卡接口复位信号 2. SPIB 片选信号 2
RSTN	-	-	41	28	9	I	DI	PU	芯片系统复位信号输入
WAKE_UP	-	-	57	39	21	I	DI	PU	芯片唤醒信号，上升沿唤醒(0V->VCC)，不使用芯片 wakeup 功能时，建议将 Wakeup 管脚接低
USB_DM	-	-	45	31	12	I/O	AI	-	芯片 USB 差分信号 DM
USB_DP	-	-	46	32	13	I/O	AI	-	芯片 USB 差分信号 DP
VCC	-	-	54	38	20	P	-	-	芯片电源 5V 输入，电压范围 1.68V-5.5V
VDD33OUT	-	-	52	37	18	P	-	-	芯片片内 LDO3.3V 输出，最大驱动电流 120mA，需接 4.7uF 电容
VREF	-	-	50	35	16	P	-	-	芯片片内参考电压 1.8V 输出，需接 1uF 电容
VDD12OUT	-	-	51	36	17	P	-	-	芯片片内 LDO1.2V 输出，最大驱动电流 80mA，需接 4.7uF 电容
VDDIO	-	-	14,53,60,73,94	12,41,50	4,19,23,35	P	-	-	芯片 IO 3.3V 输入，电压范围 1.68-3.63V
VDD12	-	-	3,75	51	31	P	-	-	芯片片内数字 1.2V 输入
AVDD12	-	-	4	3	3				芯片片内模拟 1.2V 输入
USB_VDDA	-	-	43	29	10	P	-	-	芯片 USB_PHY 模拟 1.2V 电源输入
USB_VCCA	-	-	44	30	11	P	-	-	芯片 USB_PHY 模拟 3.3V 电源输入
VSS	-	-	15,23,33,42,48,58,67,74,84,85,93	34	15	G	-	-	芯片数字地
AVSS	-	-	5,49	-	-	G	-	-	芯片模拟地(芯片内部模拟电路地)
USB_AGND	-	-	47	33	14	G	-	-	芯片 USB_PHY 模拟地
OSC12M_IN	-	-	24	20	6	A	AO	-	片外晶振 12MHz 输入
OSC12M_OUT	-	-	25	21	7	A	AI	-	片外晶振 12MHz 输出
NC	-	-	55,56	-	-	-	-	-	芯片 NC 管脚

EPAD	-	-	-	-	0	G	-	-	芯片地
------	---	---	---	---	---	---	---	---	-----

注：A–模拟信号； D–数字信号； I–Input； O–Output； G–Ground； P–Power； PU–上电默认 Pullup； PD–上电默认 Pulldown。

GPIO 驱动能力：GPIO20、GPIO21、GPIO22 和 GPIO35 为 16mA，其余的 GPIO 为 8mA。NC 脚必须悬空。

3. 处理器

3.1. 概述

Cortex-M 系列是一个 32 位高性能 ARM 处理器内核，采用哈佛结构，拥有独立的指令总线 and 数据总线，同时指令总线 and 数据总线共享同一个存储器空间。内核选择了适合于微控制器应用的三级流水线，增加了分支预测功能。

3.2. 主要特性

- 单指令乘法周期。
- 指令总线 and 数据总线被分开。
- Thumb-2 指令集架构，代码集成度更高。
- 取指都按 32 位处理。
- 先进的中断处理功能。
- 低功耗高性能。

3.3. 功能框图

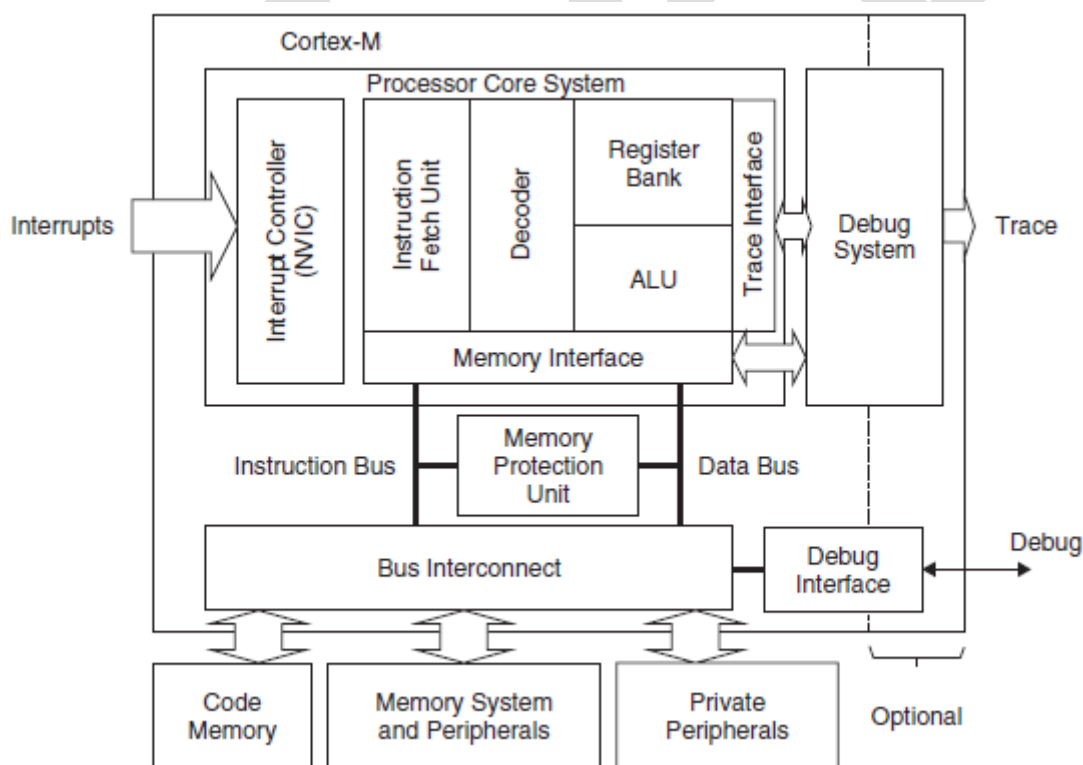


图 3-1 Cortex-M 系列处理器功能框图

3.4. 内核寄存器组

Cortex-M 系列处理器拥有 R0-R15 的寄存器组。其中 R13 作为堆栈指针 SP。SP 有两个，但在

同一时刻只能有一个可以被访问，这就是所谓的“banked”寄存器。

Name	Functions (and Banked Registers)
R0	General-Purpose Register
R1	General-Purpose Register
R2	General-Purpose Register
R3	General-Purpose Register
R4	General-Purpose Register
R5	General-Purpose Register
R6	General-Purpose Register
R7	General-Purpose Register
R8	General-Purpose Register
R9	General-Purpose Register
R10	General-Purpose Register
R11	General-Purpose Register
R12	General-Purpose Register
R13 (MSP)	Main Stack Pointer (MSP), Process Stack Pointer (PSP)
R13 (PSP)	
R14	Link Register (LR)
R15	Program Counter (PC)

图 3-2Cortex-M 系列的寄存器组

R0-R12: 通用寄存器:

R0 - R12 都是 32 位通用寄存器，用于数据操作。但是注意：绝大多数 16 位 Thumb 指令只能访问 R0 - R7，而 32 位 Thumb - 2 指令可以访问所有寄存器。

BankedR13: 两个堆栈指针:

拥有两个堆栈指针，然而它们是 banked 的，因此任一时刻只能使用其中的一个。

- 主堆栈指针（MSP）：复位后缺省使用的堆栈指针，用于操作系统内核以及异常处理例程（包括中断服务例程）
- 进程堆栈指针（PSP）：由用户的应用程序代码使用。

堆栈指针的最低两位永远是 0，这意味着堆栈总是 4 字节对齐的。

R14: 连接寄存器:

当系统调用一个子程序时，由 R14 存储返回程序地址

R15: 程序计数寄存器:

指向当前的程序地址。如果修改它的值，就能改变程序的执行流程。

特殊功能寄存器:

搭载了若干特殊功能寄存器，包括程序状态字寄存器组（PSRs），中断屏蔽寄存器组（PRIMASK, FAULTMASK, BASEPRI），控制寄存器（CONTROL）

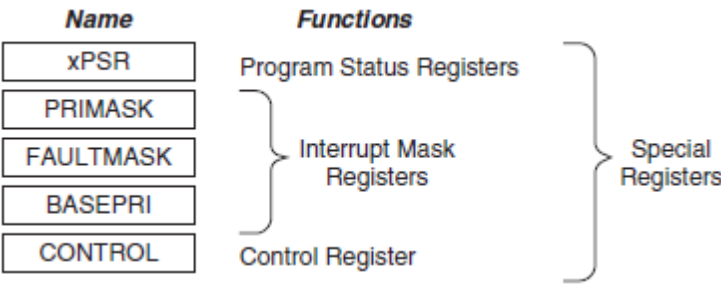


图 3-3Cortex-M 系列的特殊功能寄存器

4. 系统配置(SCU)

4.1. 地址映射

存储地址区域分为内部存储区域、外部存储区域和外设空间，各部分的地址划分如下

表 4-1 模块地址划分

模块名	ROM 启动	EFlash 启动
ROM	0x0000_0000——0x0000_3fff	0x1200_0000——0x1200_3fff
EFlash	0x1000_0000——0x1007_ffff	0x0000_0000——0x0007_ffff
EFlash Reg	0x1010_0000——0x11ff_ffff	0x0010_0000——0x0fff_ffff
SRAM	0x2000_0000——0x2001_ffff	
DMA	0x4000_0000——0x4000_ffff	
UAC Reg	0x4001_0000——0x4001_0fff	
Alg Sram	0x4001_1000——0x4001_1fff	
HRNG	0x4001_2000——0x4001_2fff	
CRC	0x4001_3000——0x4001_3fff	
DIV	0x4001_4000——0x4001_4fff	
AES	0x4001_5000——0x4001_5fff	
DES	0x4001_6000——0x4001_6fff	
SM1	0x4001_8000——0x4001_9fff	
SM3	0x4001_a000——0x4001_bfff	
SM4	0x4001_c000——0x4001_dfff	
SSF33	0x4001_e000——0x4001_ffff	
PKI	0x4002_0000——0x4002_1fff	
EMW	0x4002_e000——0x4002_ffff	
USB	0x4003_0000——0x4003_ffff	
NFM	0x4004_0000——0x4004_ffff	
SDIO	0x4005_0000——0x4005_ffff	
SPIA	0x4006_0000——0x4006_ffff	
SPIB	0x4007_0000——0x4007_ffff	
CACHE	0x4008_0000——0x4008_ffff	
External Mem0	0x6000_0000——0x607f_ffff	
External Mem1	0x6080_0000——0x60ff_ffff	
External Mem2	0x6100_0000——0x617f_ffff	
External Mem3	0x6180_0000——0x61ff_ffff	

MIM Reg	0x6200_0000——0x627f_ffff
SPIA MEM	0x6300_0000——0x63ff_ffff
UARTA	0x4801_0000——0x4801_ffff
TIMER	0x4803_0000——0x4803_ffff
WDT	0x4804_0000——0x4804_ffff
SCU	0x4806_0000——0x4806_ffff
I2C	0x4807_0000——0x4807_ffff
7816MS	0x4808_0000——0x4808_ffff
Sensor	0x4809_0000——0x4809_ffff
GPIOA (GPIO[31:0])	0x480a_0000——0x480a_ffff
GPIOB (GPIO[63:32])	0x480b_0000——0x480b_ffff
UARTB	0x480d_0000——0x480d_ffff

4.2. 时钟框图

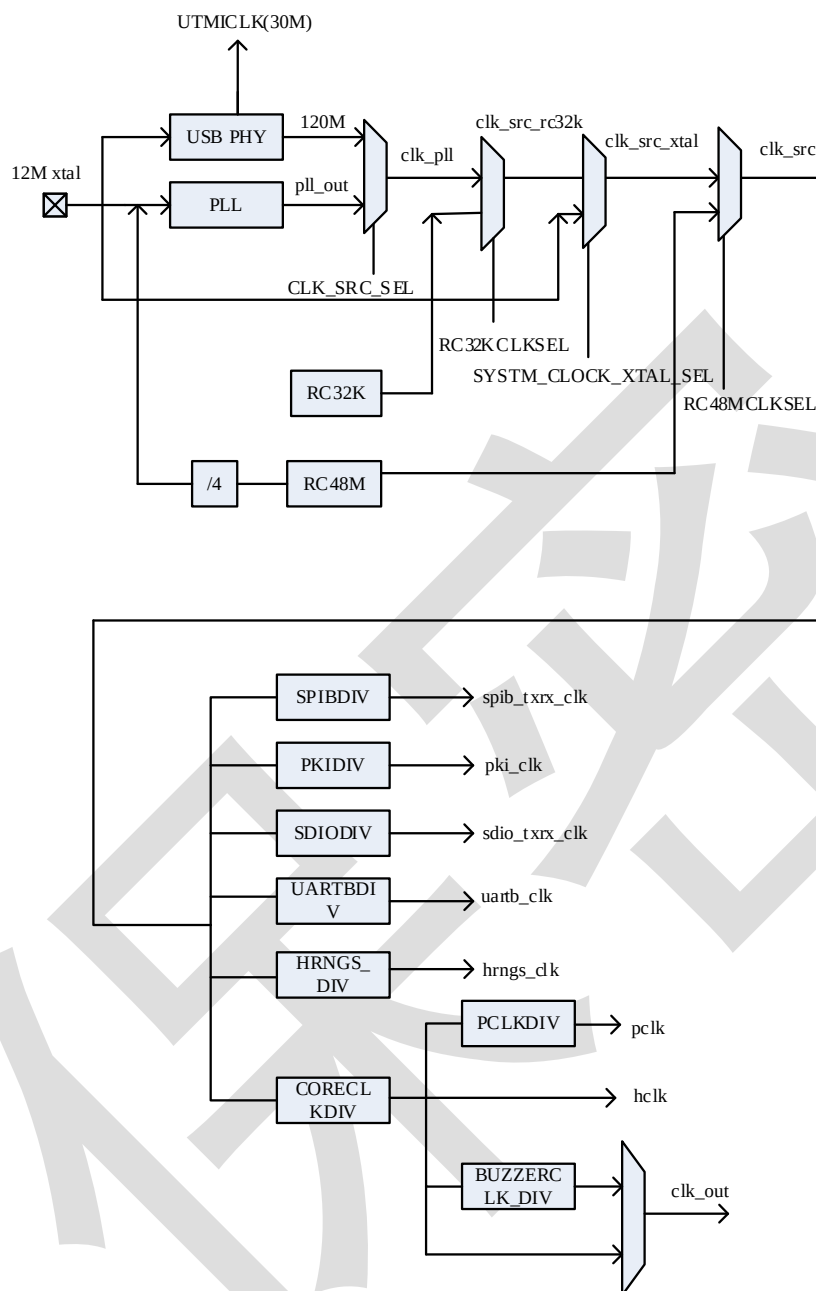


图 4-1 时钟模块框图

其中 HCLK 的模块包括：

CRC, EFC, SRAM, EMV, ROM, CACHE, USB, UAC, HRNG, DES, AES, SM1, SM3, SM4, SSF33, DMA, NFM, MIM, SPIA, SPI B (寄存器控制时钟)，SDIO (寄存器控制时钟)

PCLK 的模块包括：

WDT, TIMER, 7816MS, GPIO, UARTA, SENSOR, I2C, SCU

4.3. 时钟选择

系统时钟可以来自于 12M 片外晶振、PLL、120M USB PHY(需外接 12M 晶振)、RC48M 以及片

内 RC32K 震荡器，通过配置系统寄存器 CLK_CTRLB 来选择系统时钟的来源。关系如下表所示：

表 4-2 系统时钟选择

RC48MCLKSEL	SYSTEM_CLOCK_XTAL_SEL	RC32KCLKSEL	CLK_SRC_SEL	系统时钟来源
0	X	X	X	RC48M
1	0	X	X	XTAL
1	1	1	X	RC32K
1	1	0	0	PLL
1	1	0	1	USB PHY

4.4. PLL配置

内置 PLL，PLL 控制和状态寄存器参见详见 PLL_CSR 寄存器。

PLL 计算公式如下：

$$\text{CLK_OUT} = \text{XIN} \times \frac{M}{N} \times \frac{1}{\text{NO}}$$

$$\text{FOUTVCO} = \text{XIN} \times \frac{M}{N}$$

$$\text{NO} = 2^{\text{OD}}$$

PLL 的配置参数需要满足以下三条件：

$$1\text{MHz} \leq \frac{\text{XIN}}{N} \ll 50\text{MHz}$$

$$200\text{MHz} \leq \text{FOUTVCO} \leq 400\text{MHz}$$

$$M \geq 4; N \geq 1$$

4.5 复位源

芯片有多个复位源，包括 POR 上电复位、外部复位、安全检测 sensor 复位，看门狗复位，软复位以及模块自身的软件复位。复位源如下表：

表 4-3 系统复位源

复位源	描述
内部模拟 POR 上电复位	复位所有
外部 RSTN 管脚复位	复位所有
安全监测复位	复位系统(不影响 analog IP、EFC 控制器、SENSOR、USB_PHY、PLL、系统寄存器)，复位后，复位原因记录在系统寄存器中。
WDT_RST	
SOFT_RST	

模块 IP 复位	复位对应 IP 模块
----------	------------

4.5. 启动模式

芯片有两种启动模式：ROM 启动和 EFlash 启动。系统默认从 ROM 启动。

可通过下列两种方式实现 eflash 启动：

1. 上电复位期间系统会锁存 GPIO13 的状态，1 表示从 ROM 启动，0 表示为 EFLASH 启动。
2. 通过读 NVR 区的安全序列决定是否从 EFlash 启动。

优先级（1）<（2）

4.6. 低功耗模式

芯片工作模式有正常工作模式和低功耗工作模式。系统复位或电源复位后，大部分功能均开启且电源域全部处于供电状态。用户可通过降低系统时钟或关闭未使用的外设的时钟来实现较低的功耗。

此外，芯片拥有三种低功耗模：睡眠（sleep）、待机（standby）和下电（poweroff）模式。详细的描述如下表：

表 4-4 低功耗模式

模式	进入条件	退出条件	模式描述
睡眠 (sleep)	软件配置执行 WFI/WFE 指令	中断或事件	CPU 相关部分休眠，软件可配置关闭各数字模块时钟， 软件可配置关闭各模拟模块。
待机 (standby)	软 件 配 置 CHIP_STANDBY 控制位	USB RESET/RESUEM 信号唤醒 UARTA_RX 管脚低电平唤醒 SPIA_CS 管脚低电平唤醒 SPIB_CS 低电平信号 GPIO25 管脚低电平唤醒 SCC_DETECT 管脚卡到位有效唤醒 (高低电平与 SCC_DETECT 极性配置有关)	关闭全部数字逻辑时钟，软件可配置关闭各模拟模块。
下电 (poweroff)	软 件 配 置 POWEROFF 控制位	WAKE_UP 管脚上升沿唤醒	关闭电源模块

4.7. 寄存器描述

SCU 寄存器基地址：0x48060000

偏置	名称	描述
0x00	CLK_CTRLA	时钟控制寄存器 A
0x04	CLK_CTRLB	时钟控制寄存器 B
0x08	CLK_DIVIDER	时钟分频寄存器。
0x0C	PLL_CSR	PLL 配置寄存器
0x10	RESET_CTRLA	复位控制寄存器 A
0x14	RESET_CTRLB	复位控制寄存器 B
0x18	ANALOG_MISC_CSR	模拟参数和芯片控制寄存器
0x1C	BUZZER_CTRL	蜂鸣器控制寄存器
0x20	SYS_REMAP_CTRL	系统重映射控制寄存器
0x24	WAKE_UP_CTRL	系统唤醒控制寄存器
0x2C	PAD_MUX_CTRLA	管脚复用控制寄存器 A
0x30	PAD_MUX_CTRLB	管脚复用控制寄存器 B
0x34	PAD_MUX_CTRLC	管脚复用控制寄存器 C
0x38	PAD_MUX_CTRLD	管脚复用控制寄存器 D
0x40	PAD_PULLUP_PUCRA	管脚上拉控制寄存器 A
0x44	PAD_PULLUP_PUCRB	管脚上拉控制寄存器 B
0x48	SENSOR_RST_EN	传感器复位控制器器
0x54	SCILDO_CTRL	SCI 控制寄存器
0x58	USB_PHY_CSR	USB PHY 控制和状态寄存器
0x5C	SENSOR_RST_STATUS	传感器复位状态寄存器
0x60	ANACR	模拟控制寄存器

4.7.1. 时钟控制寄存器CLK_CTRLA（偏移：00h）

比特	名称	属性	复位值	描述
31	I2C_CLK_OFF	RW	0	I2C 模块时钟关断控制位。 0：使能时钟。 1：关断时钟。
30	WDT_CLK_OFF	RW	0	WDT 模块时钟关断控制位。 0：使能时钟。

				1: 关断时钟。
29	RSV	-	-	保留
28	UARTA_CLK_OFF	RW	0	UARTA 模块时钟关断控制位。 0: 使能时钟。 1: 关断时钟。
27	TIMER_CLK_OFF	RW	0	TIMER 模块时钟关断控制位。 0: 使能时钟。 1: 关断时钟。
26	SENSOR_CLK_OFF	RW	0	SENSOR 模块时钟关断控制位。 0: 使能时钟。 1: 关断时钟。
25	RSV	-	-	保留
24	7816MS_CLK_OFF	RW	0	7816MS 模块时钟关断控制位。 0: 使能时钟。 1: 关断时钟。
23	RSV	-	-	保留
22	GPIO_CLK_OFF	RW	0	GPIO 模块时钟关断控制位。 0: 使能时钟。 1: 关断时钟。
21: 20	RSV	-	-	保留
19	EMW_CLK_OFF	RW	1	EMW 模块时钟关断控制位。 0: 使能时钟。 1: 关断时钟。
18	USB_CLK_OFF	RW	0	USB 模块时钟关断控制位。 0: 使能时钟。 1: 关断时钟。
17	UAC_CLK_OFF	RW	1	UAC 模块时钟关断控制位。 0: 使能时钟。 1: 关断时钟。
16	SSF33_CLK_OFF	RW	1	SSF33 模块时钟关断控制位。 0: 使能时钟。 1: 关断时钟。
15	RSV	-	-	保留

14	SPIB_CLK_OFF	RW	1	SPIB 模块时钟关断控制位。 0: 使能时钟。 1: 关断时钟。
13	SPIA_CLK_OFF	RW	0	SPIA 模块时钟关断控制位。 0: 使能时钟。 1: 关断时钟。
12	SM4_CLK_OFF	RW	1	SM4 模块时钟关断控制位。 0: 使能时钟。 1: 关断时钟。
11	SM3_CLK_OFF	RW	1	SM3 模块时钟关断控制位。 0: 使能时钟。 1: 关断时钟。
10	SM1_CLK_OFF	RW	1	SM1 模块时钟关断控制位。 0: 使能时钟。 1: 关断时钟。
9	SDIO_CLK_OFF	RW	1	SDIO 模块时钟关断控制位。 0: 使能时钟。 1: 关断时钟。
8	ROM_CLK_OFF	RW	0	ROM 模块时钟关断控制位。 0: 使能时钟。 1: 关断时钟。
7	NFM_CLK_OFF	RW	1	NFM 模块时钟关断控制位。 0: 使能时钟。 1: 关断时钟。
6	MIM_CLK_OFF	RW	0	MIM 模块时钟关断控制位。 0: 使能时钟。 1: 关断时钟。
5	HRNG_CLK_OFF	RW	0	HRNG 模块时钟关断控制位。 0: 使能时钟。 1: 关断时钟。
4	EFC_CLK_OFF	RW	0	EFC 模块时钟关断控制位。 0: 使能时钟。 1: 关断时钟。

3	DMA_CLK_OFF	RW	1	DMA 模块时钟关断控制位。 0: 使能时钟。 1: 关断时钟。
2	DES_CLK_OFF	RW	1	DES 模块时钟关断控制位。 0: 使能时钟。 1: 关断时钟。
1	CACHE_CLK_OFF	RW	0	CACHE 模块时钟关断控制位。 0: 使能时钟。 1: 关断时钟。
0	RSV	-	-	保留

4.7.2. 时钟控制寄存器CLK_CTRLB（偏移：04h）

比特	名称	属性	复位值	描述
31	AES_CLK_OFF	RW	1	AES 模块时钟关断控制位。 0: 使能时钟。 1: 关断时钟。
30	RC48MCLKSEL	RW	0	系统时钟第四级选择。输出记为 clk_src 0: 来自 RC48M。 1: 来自 clk_src_xtal（第三级选择的结果）
29	CRC_CLK_OFF	RW	1	CRC 模块时钟关断控制位。 0: 使能时钟。 1: 关断时钟。
28:13	HRNGS_DIV	RW	0x30	随机源慢时钟分频控制位 0: 1 分频。 1: 2 分频。 48: 49 分频（默认）。 65535: 65536 分频。
12	RC32KCLKSEL	RW	0	系统时钟第二级选择。输出记为 clk_src_rc32k。 1: 来自 RC32K。 0: 来自 clk_pll（第一级选择的结果），
11	CLK_SRC_SEL	RW	0	系统时钟第一级选择。输出记为 clk_pll。

				0: 来自模拟 PLL（可配置频率）。 1: 来自 USB PHY（固定 120MHZ）
10: 9				保留位
8	SYSTEM_CLOCK_XTAL_SEL	RW	1	系统时钟第三级选择。输出记为 clk_src_xtal。 0: 来自外部 XTAL（12M）。 1: 来自 clk_src_rc32k（第二级选择的结果）
7	RSV	-	-	保留
6	UARTB_CLK_OFF	RW	1	UARTB 模块时钟关断控制位。（可以关断 UARTB 高速时钟和寄存器时钟） 0: 使能时钟。 1: 关断时钟。
5	PKI_CLK_OFF	RW	1	PKI 模块时钟关断控制位。 0: 使能时钟。 1: 关断时钟。
4	SPIB_TXRX_CLK_OFF	RW	1	SPIB 收发引擎时钟关断控制位。 0: 使能时钟。 1: 关断时钟。
3	RSV	-	-	保留
2	HRNG_SCLK_OFF	RW	1	随机源慢时钟关断控制位。 0: 使能时钟。 1: 关断时钟。
1	SDIO_TXCLK_OFF	RW	1	SDIO 发送逻辑时钟关断控制位。 0: 使能时钟。 1: 关断时钟。
0	SDIO_RXCLK_OFF	RW	1	SDIO 接收逻辑时钟关断控制位。 0: 使能时钟。 1: 关断时钟。

注:

1) 关闭 SDIO 模块的时钟，需要同时将 CLK_CTRLA 寄存器中 SDIO_CLK_OFF、CLK_CTRLB 寄存器中 SDIO_TXCLK_OFF、SDIO_RXCLK_OFF 三个比特全部置成 1。

2) CLK_CTRLA 寄存器中的 SPIB_CLK_OFF 控制 SPI 模块寄存器是否关闭，CLK_CTRLB 寄存器 SPIB_TXRX_CLK_OFF 控制 SPIA 收发引擎逻辑。关闭 SPIB 模块的时钟，需要将 CLK_CTRLA 寄

寄存器中的 SPIB_CLK_OFF 和 CLK_CTRLB 寄存器 SPIB_TXRX_CLK_OFF 全部置位。

3) CLK_CTRLA 中的 HRNG_CLK_OFF 比特位控制随机数模块的数字逻辑，CLK_CTRLB 中的 HRNG_SCLK_OFF 控制模拟随机源的时钟。

4.7.3. 时钟分频寄存器CLK_DIVIDER（偏移：08h）

比特	名称	属性	复位值	描述
31: 28	RSV	-	-	保留
27:24	UARTBDIV	RW	0x4	UARTB 收发引擎分频控制位。 0: 1 分频。 1: 2 分频。 2: 3 分频。 3: 4 分频。 4: 5 分频。（默认） 15: 16 分频。
23:20	SPIBDIV	RW	0x2	SPIB 收发引擎分频控制位。 0: 1 分频。 1: 2 分频。 2: 3 分频。（默认） 3: 4 分频。 15: 16 分频。
19:16				保留位
15:12	SDIODIV	RW	0x3	SDIO 收发引擎分频控制位。 0: 1 分频。 1: 2 分频。 2: 3 分频。 3: 4 分频。（默认） 15: 16 分频。
11:8	PKIDIV	RW	0x2	PKI 引擎分频控制位。 0: 1 分频。 1: 2 分频。 2: 3 分频。（默认） 3: 4 分频。

				 15: 16 分频。
7:4	PCLKDIV	RW	0x1	PCLK 分频控制位。 0: 1 分频。 1: 2 分频。（默认） 2: 3 分频。 3: 4 分频。 15: 16 分频。
3:0	CORECLKDIV	RW	0x1	HCLK 分频控制位。 0: 1 分频。 1: 2 分频。（默认） 15: 16 分频。

4.7.4. PLL配置寄存器PLL_CSR（偏移：0Ch）

比特	名称	属性	复位值	描述
31	PLL_SRC_SEL	RW	0	PLL 时钟源选择。 0: 选择片外 12M。 1: 选择片内 RC48M 的 4 分频。
30	PLL_FREE_RUN	RO	1	PLL 工作状态寄存器。该位即时地反映 PLL 的工作状态，当 PLL 进入 StandBy 模式或重新配置频率后，硬件自动清除该位；当 PLL 退出 StandBy 模式且配置生效后，硬件自动置位。 1: PLL 正常工作。 0: PLL 正常不正常。
29	PLL_LOCK	RO	1	PLL 锁定状态位。检测 PLL 锁定信号的上升沿。 1: PLL 已经锁定。 0: PLL 未锁定。
28:23	PLLLOCKDLY	RW	0x30	PLL 锁定等待时钟周期数。PLL 重新配置频率后，至少需要 500us 的时间才能锁定。该控制位设定的锁定时间计算如下： $PLLLOCKDLY * 512 * (1/PCLK)$ PCLK 为 24M 时，等待约 1ms。

22	PLL_UPDATE_EN	RW	0	PLL 配置更新控制位，下一个周期自动回 0。重新改写 PLL_OD、PLL_N 和 PLL_M、PLL_SRC_SEL 后，需要将该比特置位新的配置才能生效。复位时间 60* (1/PCLK) 1: PLL 配置更新。 0: PLL 配置不更新。
21	PLL_SLEEP	RW	0	PLL 休眠控制位 1: PLL 进入休眠。 0: PLL 不进入休眠。
20	RSV	-	-	保留
19:18	PLL_OD	RW	0x1	输出分频控制字段。PLL_OD 含有输出分配控制字段。
17:14	PLL_N	RW	0XC	降频因子分频器字段。PLL_N 含有 PLL 输入时钟的分频因子，该值的实际作用是参考频率的分频因子。该控制位不能小于 1。
13:0	PLL_M	RW	0xDC	增频因子分频器字段。PLL_M 含有 PLL 反馈回路中分频器的分频因子，该值的实际作用是参考频率的倍频因子。该控制位不能小于 4。

PLL 具体配置 (PLL_OD, PLL_N, PLL_M) 请参照 4.4 章节

4.7.5. 复位控制寄存器 RESET_CTRLA (偏移: 10h)

比特	名称	属性	复位值	描述
31	RSV	-	-	保留
30	I2C_RST	RW	0	I2C 模块软复位。写 1 复位，写 0 撤销。
29	WDT_RST	RW	0	WDT 模块软复位。写 1 复位，写 0 撤销。
28	RSV	-	-	保留
27	UARTA_RST	RW	0	UARTA 模块软复位。写 1 复位，写 0 撤销。
26	TIMER_RST	RW	0	TIMER 模块软复位。写 1 复位，写 0 撤销。
25	SENSOR_RST	RW	0	SENSOR 模块软复位。写 1 复位，写 0 撤销。
24	RSV	-	-	保留
23	7816MS_RST	RW	0	7816MS 模块软复位。写 1 复位，写 0 撤销。
22: 21	RSV	-	-	保留
20	GPIO_RST	RW	0	GPIO 模块软复位。写 1 复位，写 0 撤销。

19	RSV	-	-	保留
18	USB_RST	RW	0	USB 模块软复位。写 1 复位，写 0 撤销。
17	UAC_RST	RW	0	UAC 模块软复位。写 1 复位，写 0 撤销。
16	RSV	-	-	保留
15	SRAM_RST	RW	0	SRAM 模块软复位。写 1 复位，写 0 撤销。
14	SPIB_RST	RW	0	SPIB 模块软复位。写 1 复位，写 0 撤销。
13	SPIA_RST	RW	0	SPIA 模块软复位。写 1 复位，写 0 撤销。
12: 10	RSV	-	-	保留
9	SDIO_RST	RW	0	SDIO 模块软复位。写 1 复位，写 0 撤销。
8	RSV	-	-	保留
7	ROM_RST	RW	0	ROM 模块软复位。写 1 复位，写 0 撤销。
6	NFM_RST	RW	0	NFM 模块软复位。写 1 复位，写 0 撤销。
5	MIM_RST	RW	0	MIM 模块软复位。写 1 复位，写 0 撤销。
4	HRNG_RST	RW	0	HRNG 模块软复位。写 1 复位，写 0 撤销。
3	EFC_RST	RW	0	EFC 模块软复位。写 1 复位。下一个时钟自动回 0 启动重新预读，复位 SCU，Sensor，EFC 及其它数字模块，但是不复位 HRNG。
2	DMA_RST	RW	0	DMA 模块软复位。写 1 复位，写 0 撤销。
1: 0	RSV	-	-	保留

4.7.6. 复位控制寄存器RESET_CTRLB（偏移：14h）

比特	名称	属性	复位值	描述
31: 9	RSV	-	-	保留
8	PAD_RSTN_BYPASS	RW	0	外部复位管脚 BYPASS 功能。 1: 外部复位管脚无效。 0: 外部复位管脚有效。
7	WDTRST_EN	RW	1	WTD reset 使能，使能后允许看门狗信号复位系统 0: 不允许其复位系统逻辑 1: 允许看门狗 reset 复位系统逻辑
6	WDT_RST_OCCUR	RO	0	WDT 复位状态。（SOFT_RST_REMAP、SOFT_RESET_NO_REMAP、EFC_RST 和传

				感器复位将清除该标志位)。 1: 上一次复位是由 WDT_RST 引起。 0: 上一次复位不是由 WDT_RST 引起。
5: 4	RSV	-	-	保留
3	SOFT_RST_REMAP_OCCUR	RO	0	软复位状态。 1: 上一次复位是由 SOFT_RST_REMAP 引起。 0: 上一次复位不是由 SOFT_RST_REMAP 引起。
2	SOFT_RST_REMAP	WO	0	复位系统(不复位 analog IP、EFC 控制器、 SENSOR、USB_PHY、PLL、系统寄存器)。 写 1 产生系统复位，下一个时钟自动回 0。
1	RSV	-	-	保留
0	UARTB_RST	RW	0	UARTB 模块软复位。写 1 复位，写 0 撤销。

4.7.7. 模拟参数和芯片控制寄存器ANALOG_MISC_CSR（偏移：18h）

比特	名称	属性	复位值	描述
31: 30	RSV	-	-	保留
29	BOOT_PIN	RO	1	BootPin 状态（GPIO13）。用于指示芯片从 ROM 启动还是从 EFC 启动，优先级低于 NVR Remapping。 0:指示系统从 EFlash 启动，通过 SoftReset 实现。 1: 指示系统从 ROM 启动，优先级低于 NVR Remapping.
28	BOOT_MODE_STAT	RO	0	REMAP 标志（GPIO48），表示现在是 ROM 工作模式还是 EFlash 工作模式。 0: ROM 工作模式。 1: EFlash 工作模式。
27:25	RSV	-	-	保留
24	SRAM_SOFT_DESTROY	RW	0	SRAM 软件自毁功能启动。 1: 软件启动 SRAM 软件自毁功能。 0: 软件不启动 SRAM 软件自毁功能。
23	SRAM_SELF_DESTROY_EN	RW	0	SRAM 自毁功能使能。SRAM 自毁功能

				由软件触发和外部信号触发两种方式。 软件触发由 SRAM_SOFT_DESTROY 比特位实现；外部由 GPIO24 低电平触发。 1: 自毁功能使能。 0: 自毁功能禁止。
22	X12M_EN_SUSPEND_CTRL	RW	0	该比特位选择外接 12M 晶振是否使能是由该位控制。 1: 由系统 STANDBY 信号控制，当系统进入休眠后，自动关闭 12M 晶振；退出休眠时，自动打开 12M 晶振。 0: 由寄存器 X12M_EN 控制位控制。
21	X12M_EN	RW	1	外接 12M 晶振使能控制位。 1: 使能。 0: 禁止。
20: 0	RSV	-	-	保留

4.7.8. 蜂鸣器控制寄存器BUZZER_CTRL（偏移：0x1C）

比特	名称	属性	复位值	描述
31: 19	RSV	-	-	保留
18	CLK_OUT_MUX	RW	1	CLOCK OUT(GPIO[49])输出时钟选择。 1: 来自 BUZZER 分频器的输出。 0: 来自系统时钟 HCLK。
17	BUZZER_CLK_EN	RW	1	BUZZER 功能使能。 1: BUZZER 功能使能。 0: BUZZER 功能禁止。
16	BUZZER_CLK_POL	RW	0	BUZZER 时钟极性控制位 1: BUZZER 功能禁止时，输出高电平。 0: BUZZER 功能禁止时，输出低电平。
15: 0	BUZZER_CLK_DIV	RW	0xEF	BUZZER 时钟分频控制位 0: 1 分频。 1: 2 分频。

				239: 240 分频（默认）。 65535: 65536 分频。
--	--	--	--	---

4.7.9. 系统重映射寄存器SYS_REMAP_CTRL（偏移：0x20）

比特	名称	属性	复位值	描述
31: 10	RSV	-	-	保留
9	SOFT_RESET_NO_REMAP	WO	0	写 1 生效，下一个周期，自动清零。 SOFT_RESET_NO_REMAP 不会引起 RESET_CTRLB[3]置位。 1: 产生系统复位，不执行 REMAP 动作。 0: 无效。
8:0	RSV	-	-	保留

4.7.10. 系统唤醒控制寄存器WAKE_UP_CTRL（偏移：0x24）

比特	名称	属性	复位值	描述
31: 12	RSV	-	-	保留
11	RC48MPDEN	RW	0	待机（STANDBY）模式下，RC48M 是否自动关闭 0: RC48M 不自动关闭 1: RC48M 自动关闭
10	LDO12T10EN	RW	0	待机（STANDBY）模式下，LDO12 电压是否自动降低 0: LDO12 不自动降低 1: LDO12 自动降低
9	WAKEUP	RO	0	WAKEUP 管脚状态，该状态位即时地反映外部管脚 PAD_WAKEUP 的状态。
8	POWEROFF	RW	0	系统进入待机 PowerOff 模式。在进入 PowerOff 模式时，需要等待 PD_MODE 信号至少稳定 100us。 1: 进入 PowerOff 0: 重新上电为 0 WAKE_UP 管脚上升沿唤醒

7	PD_MODE	RW	0	PowrOff 模式下是否关闭 LDO18。 0: 不关闭。 1: 关闭。
6	CHIP_STANDBY	RW	0	系统进入 StandBy 模式。 1: 写 1 进入 StandBy 模式。 0: 再次进入 StandBy 模式需要先写 0。
5	USB_WAKEUP_EN	RW	1	USB 信号 StandBy 唤醒使能。(RESUME 和 RESET 唤醒) 0: 唤醒禁止 1: 唤醒使能
4	UART_RX_WAKEUP_EN	RW	0	UARTA RX 信号 StandBy 唤醒使能。低电平唤醒。 0: 唤醒禁止 1: 唤醒使能
3	SCI_DETECT_WAKEUP_EN	RW	0	SCI_DETECT_POL 为 1 时, 低电平 StandBy 唤醒; 为 0 时, 高电平 StandBy 唤醒。 0: 唤醒禁止 1: 唤醒使能
2	SPIB_CS_WAKEUP_EN	RW	0	SPIB 从模式 CS 信号 StandBy 唤醒使能。低电平唤醒。 0: 唤醒禁止 1: 唤醒使能
1	SPICA_CS_WAKEUP_EN	RW	0	SPIA 从模式 CS 信号 StandBy 唤醒使能。低电平唤醒。 0: 唤醒禁止 1: 唤醒使能
0	GPIO25_WAKEUP_EN	RW	1	GPIO25 信号 StandBy 唤醒使能。低电平唤醒。 0: 唤醒禁止 1: 唤醒使能

4.7.11. 管脚复用寄存器 PAD_MUX_CTRL A (偏移: 2Ch)

比特	名称	属性	复位值	描述
----	----	----	-----	----

31:30	GPIO15_SEL	RW	0	GPIO15 复用功能选择。 00: GPIO15 01: SPIB_CLK 10: ADDR13 11: 保留
29:28	GPIO14_SEL	RW	0	GPIO14 复用功能选择。 00: GPIO14 01: SPIB_CS0 10: CSN0 11: 保留
27:26	GPIO13_SEL	RW	0	GPIO13 复用功能选择。 00: GPIO13 01: 保留 10: ADDR16 11: 保留 芯片上电过程中，锁存该管脚的状态，作为启动的选择条件之一，即 BOOT_PIN 功能。
25:24	GPIO12_SEL	RW	01	GPIO12 复用功能选择。 00: GPIO12 01: JTAG_TRSTN 10: ENB1 11: 保留
23:22	GPIO11_SEL	RW	01	GPIO11 复用功能选择。 00: GPIO11 01: JTAG_TMS/SWDIO 1x: 保留
21:20	GPIO10_SEL	RW	01	GPIO10 复用功能选择。 00: GPIO10 01: JTAG_TDI 10: ENB0 11: 保留
19:18	GPIO9_SEL	RW	01	GPIO9 复用功能选择。

				00: GPIO9 01: JTAG_TCLK/SWDCLK 10: PWM3 11: 保留
17:16	GPIO8_SEL	RW	01	GPIO8 复用功能选择。 00: GPIO8 01: JTAG_TDO 10: ADDR17 11: 保留
15:14	GPIO7_SEL	RW	0	GPIO7 复用功能选择。 00: GPIO7 01: SPIA_IO3 (SPIA_HOLD) 10: ADDR19 11: 保留
13:12	GPIO6_SEL	RW	0	GPIO6 复用功能选择。 00: GPIO6 01: SPIA_IO2 (SPIA_WP) 10: ADDR18 11: 保留
11:10	GPIO5_SEL	RW	0	GPIO5 复用功能选择。 00: GPIO5 01: SPIA_MISO 1x: 保留
9:8	GPIO4_SEL	RW	0	GPIO4 复用功能选择。 00: GPIO4 01: SPIA_MOSI 1x: 保留
7:6	GPIO3_SEL	RW	0	GPIO3 复用功能选择。 00: GPIO3 01: SPIA_CS0 10: CSN2 11: 保留
5:4	GPIO2_SEL	RW	0	GPIO2 复用功能选择。

				00: GPIO2 01: SPIA_CLK 1x: 保留
3:2	GPIO1_SEL	RW	0	GPIO1 复用功能选择。 00: GPIO1 01: UARTA_TX 1x: 保留
1:0	GPIO0_SEL	RW	0	GPIO0 复用功能选择。 00: GPIO0 01: UARTA_RX 1x: 保留

4.7.12. 管脚复用寄存器PAD_MUX_CTRLB偏移: 30h)

比特	名称	属性	复位值	描述
31:30	GPIO31_SEL	RW	0	GPIO31 复用功能选择。 00: GPIO31 01: NFM_ALE 10: SDIO_CLK 11: 保留
29:28	GPIO30_SEL	RW	0	GPIO30 复用功能选择。 00: GPIO30 01: NFM_CLE 10: SDIO_DETECT 11: 保留
27:26	GPIO29_SEL	RW	0	GPIO29 复用功能选择。 00: GPIO29 01: NFM_RBN0 10: RS 11: 保留
25:24	GPIO28_SEL	RW	0	GPIO28 复用功能选择。 00: GPIO28 01: NFM_CEN0 10: CSN3

				11: 保留
23:22	GPIO27_SEL	RW	0	GPIO27 复用功能选择。 00: GPIO27 01: SDIO_DATA7 10: DATA7 (MIM 和 NFM 共用) 11:保留
21:20	GPIO26_SEL	RW	0	GPIO26 复用功能选择。 00: GPIO26 01: SDIO_DATA6 10: DATA6 (MIM 和 NFM 共用) 11:保留
19:18	GPIO25_SEL	RW	0	GPIO25 复用功能选择。 00: GPIO25 01: SDIO_DATA5 10: DATA5 (MIM 和 NFM 共用) 11:保留
17:16	GPIO24_SEL	RW	0	GPIO24 复用功能选择。 00: GPIO24 01: SDIO_DATA4 10: DATA4 (MIM 和 NFM 共用) 11:保留
15:14	GPIO23_SEL	RW	0	GPIO23 复用功能选择。 00: GPIO23 01: SDIO_DATA3 10: DATA3 (MIM 和 NFM 共用) 11:保留
13:12	GPIO22_SEL	RW	0	GPIO22 复用功能选择。 00: GPIO22 01: SDIO_DATA2 10: DATA2 (MIM 和 NFM 共用) 11:保留
11:10	GPIO21_SEL	RW	0	GPIO21 复用功能选择。 00: GPIO21

				01: SDIO_DATA1 10: DATA1 (MIM 和 NFM 共用) 11:保留
9:8	GPIO20_SEL	RW	0	GPIO20 复用功能选择。 00: GPIO20 01: SDIO_DATA0 10: DATA0 (MIM 和 NFM 共用) 11:保留
7:6	GPIO19_SEL	RW	0	GPIO19 复用功能选择。 00: GPIO19 01: SPIB_IO3(SPIB_HOLD) 10: WEN 11: 保留
5:4	GPIO18_SEL	RW	0	GPIO18 复用功能选择。 00: GPIO18 01: SPIB_IO2(SPIB_WP) 10: OEN 11: 保留
3:2	GPIO17_SEL	RW	0	GPIO17 复用功能选择。 00: GPIO17 01: SPIB_MISO 10: ADDR15 11: 保留
1:0	GPIO16_SEL	RW	0	GPIO16 复用功能选择。 00: GPIO16 01: SPIB_MOSI 10: ADDR14 11: 保留

4.7.13. 管脚复用寄存器PAD_MUX_CTRL0偏移：34h)

比特	名称	属性	复位值	描述
31:30	GPIO47_SEL	RW	0	GPIO47 复用功能选择。 00: GPIO47

				01: 保留 10: ADDR6 11: 保留
29:28	GPIO46_SEL	RW	0	GPIO46 复用功能选择。 00: GPIO46 01: NFM_RBN1 10: ADDR5 11: 保留
27:26	GPIO45_SEL	RW	0	GPIO45 复用功能选择。 00: GPIO45 01: NFM_CEN1 10: ADDR4 11: 保留
25:24	GPIO44_SEL	RW	0	GPIO44 复用功能选择。 00: GPIO44 01: UARTB_RTS 10: ADDR3 11: 保留
23:22	GPIO43_SEL	RW	0	GPIO43 复用功能选择。 00: GPIO43 01: UARTB_CTS 10: ADDR2 11: 保留
21:20	GPIO42_SEL	RW	0	GPIO42 复用功能选择。 00: GPIO42 01: UARTB_RX 10: ADDR1 11: 保留
19:18	GPIO41_SEL	RW	0	GPIO41 复用功能选择。 00: GPIO41 01: UARTB_TX 10: ADDR0 11: 保留

17:16	GPIO40_SEL	RW	0	GPIO40 复用功能选择。 00: GPIO40 01: 保留 10: DATA11 11: 保留
15:14	GPIO39_SEL	RW	0	GPIO39 复用功能选择。 00: GPIO39 01: 保留 10: DATA10 11: 保留
13:12	GPIO38_SEL	RW	0	GPIO38 复用功能选择。 00: GPIO38 01: SPIB_CS1 10: DATA9 11: 保留
11:10	GPIO37_SEL	RW	0	GPIO37 复用功能选择。 00: GPIO37 01: SPIA_CS1 10: DATA8 11: 保留
9:8	GPIO36_SEL	RW	0	GPIO36 复用功能选择。 00: GPIO36 01: I2C_SDA 10: SPIB_CS3 11: 保留
7:6	GPIO35_SEL	RW	0	GPIO35 复用功能选择。 00: GPIO35 01: I2C_SCL 10: SPIA_CS3 11: 保留
5:4	GPIO34_SEL	RW	0	GPIO34 复用功能选择。 00: GPIO34 01: NFM_WPN

				10: SDIO_WP 11: 保留
3:2	GPIO33_SEL	RW	0	GPIO33 复用功能选择。 00: GPIO33 01: NFM_REN 10: SDIO_POWER_EN 11: 保留
1:0	GPIO32_SEL	RW	0	GPIO32 复用功能选择。 00: GPIO32 01: NFM_WEN 10: SDIO_CMD 11: 保留

4.7.14. 管脚复用寄存器PAD_MUX_CTRLD偏移: 38h)

比特	名称	属性	复位值	描述
31:30	GPIO63_SEL	RW	00	GPIO63 复用功能选择。 00: GPIO63 01: SCC_MS_RST 10: SPIB_CS2 11: 保留
29:28	GPIO62_SEL	RW	00	GPIO62 复用功能选择。 00: GPIO62 01: SCC_MS_CLK 10: SPIA_CS2 11: 保留
27:26	GPIO61_SEL	RW	00	GPIO61 复用功能选择。 00: GPIO61 01: SCC_MS_SIO 10: 保留 11: 保留
25:24	GPIO60_SEL	RW	00	GPIO60 复用功能选择。 00: GPIO60 01: SCC_MS_DETECT

				10: CSN1 11: 保留
23:22	GPIO59_SEL	RW	00	GPIO59 复用功能选择。 00: GPIO59 01: SCC_MS_VCC_EN 10: PWM1 11: 保留
21:20	GPIO58_SEL	RW	00	GPIO58 复用功能选择。 00: GPIO58 01: 保留 10: ADDR12 11: 保留
19:18	GPIO57_SEL	RW	00	GPIO57 复用功能选择。 00: GPIO57 01: 保留 10: ADDR11 11: 保留
17:16	GPIO56_SEL	RW	00	GPIO56 复用功能选择。 00: GPIO56 01: 保留 10: ADDR10 11: 保留
15:14	GPIO55_SEL	RW	00	GPIO55 复用功能选择。 00: GPIO55 01: NFM_RBN2 10: ADDR9 11: 保留
13:12	GPIO54_SEL	RW	00	GPIO54 复用功能选择。 00: GPIO54 01: NFM_CEN2 10: ADDR8 11: 保留
11:10	GPIO53_SEL	RW	00	GPIO53 复用功能选择。

				00: GPIO53 01: 保留 10: DATA15 11: 保留
9:8	GPIO52_SEL	RW	00	GPIO52 复用功能选择。 00: GPIO52 01: 保留 10: DATA14 11: 保留
7:6	GPIO51_SEL	RW	00	GPIO51 复用功能选择。 00: GPIO51 01: NFM_RBN3 10: DATA13 11: 保留
5:4	GPIO50_SEL	RW	11	GPIO50 复用功能选择。 00: GPIO50 01: NFM_CEN3 10: DATA12 11: RSTN_OUT
3:2	GPIO49_SEL	RW	11	GPIO49 复用功能选择。 00: GPIO49 01: 保留 10: PWM0 11: CLOCKOUT
1:0	GPIO48_SEL	RW	11	GPIO48 复用功能选择。 00: GPIO48 01: PWM2 10: ADDR7 11: REMAP_FLAG

4.7.15. 管脚上拉控制寄存器PAD_PUCRA（偏移：40h）

比特	名称	属性	复位值	描述
31	GPIO31_PU_EN	RW	1	GPIO[31] 上拉使能，高有效

30	GPIO30_PU_EN	RW	1	GPIO[30] 上拉使能，高有效
29	GPIO29_PU_EN	RW	1	GPIO[29] 上拉使能，高有效
28	GPIO28_PU_EN	RW	1	GPIO[28] 上拉使能，高有效
27	GPIO27_PU_EN	RW	1	GPIO[27] 上拉使能，高有效
26	GPIO26_PU_EN	RW	1	GPIO[26] 上拉使能，高有效
25	GPIO25_PU_EN	RW	1	GPIO[25] 上拉使能，高有效
24	GPIO24_PU_EN	RW	1	GPIO[24] 上拉使能，高有效
23	GPIO23_PU_EN	RW	1	GPIO[23] 上拉使能，高有效
22	GPIO22_PU_EN	RW	1	GPIO[22] 上拉使能，高有效
21	GPIO21_PU_EN	RW	1	GPIO[21] 上拉使能，高有效
20	GPIO20_PU_EN	RW	1	GPIO[20] 上拉使能，高有效
19	GPIO19_PU_EN	RW	1	GPIO[19] 上拉使能，高有效
18	GPIO18_PU_EN	RW	1	GPIO[18] 上拉使能，高有效
17	GPIO17_PU_EN	RW	1	GPIO[17] 上拉使能，高有效
16	GPIO16_PU_EN	RW	1	GPIO[16] 上拉使能，高有效
15	GPIO15_PU_EN	RW	1	GPIO[15] 上拉使能，高有效
14	GPIO14_PU_EN	RW	1	GPIO[14] 上拉使能，高有效
13	GPIO13_PU_EN	RW	1	GPIO[13] 上拉使能，高有效
12	GPIO12_PU_EN	RW	1	GPIO[12] 上拉使能，高有效
11	GPIO11_PU_EN	RW	1	GPIO[11] 上拉使能，高有效
10	GPIO10_PU_EN	RW	1	GPIO[10] 上拉使能，高有效
9	GPIO9_PU_EN	RW	1	GPIO[9] 上拉使能，高有效
8	GPIO8_PU_EN	RW	1	GPIO[8] 上拉使能，高有效
7	GPIO7_PU_EN	RW	1	GPIO[7] 上拉使能，高有效
6	GPIO6_PU_EN	RW	1	GPIO[6] 上拉使能，高有效
5	GPIO5_PU_EN	RW	1	GPIO[5] 上拉使能，高有效
4	GPIO4_PU_EN	RW	1	GPIO[4] 上拉使能，高有效
3	GPIO3_PU_EN	RW	1	GPIO[3] 上拉使能，高有效
2	GPIO2_PU_EN	RW	1	GPIO[2] 上拉使能，高有效
1	GPIO1_PU_EN	RW	1	GPIO[1] 上拉使能，高有效
0	GPIO0_PU_EN	RW	1	GPIO[0] 上拉使能，高有效

4.7.16. 管脚上拉控制寄存器PAD_PUCRB（偏移：44h）

比特	名称	属性	复位值	描述
31	GPIO63_PU_EN	RW	1	GPIO[63] 上拉使能，高有效
30	GPIO62_PU_EN	RW	1	GPIO[62] 上拉使能，高有效
29	GPIO61_PU_EN	RW	1	GPIO[61] 上拉使能，高有效
28	GPIO60_PU_EN	RW	1	GPIO[60] 上拉使能，高有效
27	GPIO59_PU_EN	RW	1	GPIO[59] 上拉使能，高有效
26	GPIO58_PU_EN	RW	1	GPIO[58] 上拉使能，高有效
25	GPIO57_PU_EN	RW	1	GPIO[57] 上拉使能，高有效
24	GPIO56_PU_EN	RW	1	GPIO[56] 上拉使能，高有效
23	GPIO55_PU_EN	RW	1	GPIO[55] 上拉使能，高有效
22	GPIO54_PU_EN	RW	1	GPIO[54] 上拉使能，高有效
21	GPIO53_PU_EN	RW	1	GPIO[53] 上拉使能，高有效
20	GPIO52_PU_EN	RW	1	GPIO[52] 上拉使能，高有效
19	GPIO51_PU_EN	RW	1	GPIO[51] 上拉使能，高有效
18	GPIO50_PU_EN	RW	1	GPIO[50] 上拉使能，高有效
17	GPIO49_PU_EN	RW	1	GPIO[49] 上拉使能，高有效
16	GPIO48_PU_EN	RW	1	GPIO[48] 上拉使能，高有效
15	GPIO47_PU_EN	RW	1	GPIO[47] 上拉使能，高有效
14	GPIO46_PU_EN	RW	1	GPIO[46] 上拉使能，高有效
13	GPIO45_PU_EN	RW	1	GPIO[45] 上拉使能，高有效
12	GPIO44_PU_EN	RW	1	GPIO[44] 上拉使能，高有效
11	GPIO43_PU_EN	RW	1	GPIO[43] 上拉使能，高有效
10	GPIO42_PU_EN	RW	1	GPIO[42] 上拉使能，高有效
9	GPIO41_PU_EN	RW	1	GPIO[41] 上拉使能，高有效
8	GPIO40_PU_EN	RW	1	GPIO[40] 上拉使能，高有效
7	GPIO39_PU_EN	RW	1	GPIO[39] 上拉使能，高有效
6	GPIO38_PU_EN	RW	1	GPIO[38] 上拉使能，高有效
5	GPIO37_PU_EN	RW	1	GPIO[37] 上拉使能，高有效
4	GPIO36_PU_EN	RW	1	GPIO[36] 上拉使能，高有效
3	GPIO35_PU_EN	RW	1	GPIO[35] 上拉使能，高有效
2	GPIO34_PU_EN	RW	1	GPIO[34] 上拉使能，高有效
1	GPIO33_PU_EN	RW	1	GPIO[33] 上拉使能，高有效

0	GPIO32_PU_EN	RW	1	GPIO[32] 上拉使能，高有效
---	--------------	----	---	-------------------

4.7.17. 传感器复位控制寄存器SENSOR_RESET_EN（偏移：48h）

比特	名称	属性	复位值	描述
31:10	RSV	-	-	保留
9	CGD_ABNORMAL_RESET_EN	RW	0	外部时钟毛刺异常是否复位系统。 0：不复位； 1：复位；
8	BUS_ABNORMAL_RESET_EN	RW	0	总线校验异常是否复位系统。 0：不复位； 1：复位；
7	LD_ABNORMAL_RESET_EN	RW	0	光检测异常是否复位系统。 0：不复位； 1：复位；
6	AS_ABNORMAL_RESET_EN	RW	0	Active Shielding 异常是否复位系统。 0：不复位； 1：复位；
5	PGD_ABNORMAL_RESET_EN	RW	0	电源毛刺检异常是否复位系统。 0：不复位； 1：复位；
4	FD_ABNORMAL_RESET_EN	RW	0	频率检测异常是否复位系统。 0：不复位； 1：复位；
3	TD_LOW_RESET_EN	RW	0	低温检测异常是否复位系统。 0：不复位； 1：复位；
2	TD_HIGH_RESET_EN	RW	0	高温检测异常是否复位系统。 0：不复位； 1：复位；
1	VD_LOW_RESET_EN	RW	0	低压检测异常是否复位系统。 0：不复位； 1：复位；
0	VD_HIGH_RESET_EN	RW	0	高压检测异常是否复位系统。

				0: 不复位; 1: 复位;
--	--	--	--	-------------------

4.7.18. SCI控制寄存器SCILDO_CTRL(偏移: 0x54)

比特	名称	属性	复位值	描述
31: 5	RSV	-	-	保留
4	ISO7816_RSTN	RO	1	ISO 7816 RST (GPIO63/RST) 管脚状态, 反应芯片的 ISO 7816 RST 的当前状态 0: 低电平 1: 高电平。
3	Reader_VCC_EN_POL	RW	0	7616M 电源使能极性选择。 1: 低电平使能。 0: 高电平使能。
2	Reader_DETECT_POL	RW	1	7816MS 卡到位极性选择 0: 高电平卡到位。 1: 低电平卡到位。
1:0	RSV	-	-	保留

4.7.19. USB PHY控制和状态寄存器USB_PHY_CTRL (偏移: 58h)

比特	名称	属性	复位值	描述
31:21	RSV	-	-	保留
20	REFCLK_EN_STATUS	RO	0	该状态位表明 USB PHY 时钟已经停止, 不再需要外部的参考时钟, 软件可以根据此状态位选择是否关闭外部 12M 时钟。 0: 不需要外部 12M 时钟。 1: 需要外部 12M 时钟。
19	SUSPEND_MUX	RW	0	SUSPEND 控制选择位。 0: 由 USB 控制器控制。 1: 由系统寄存器控制。
18	SUSPEND_EN	RW	0	手动进入 SUSPEND 模式。 1: 进入 SUSPEND 模式。 0: 不进入 SUSPEND 模式。
17:16	CLK_MODE	RW	0	时钟 Gate 控制信号。

				1X: 普通模式。 00: 一级低功耗模式。 01: 二级低功耗模式。
15:13	SQ_MODE	RW	0	Adjust squelch voltage threshold
12:9	RREF_CTRL	RW	0	Adjust the internal rref resistor
8	USB_PHY_RESET	RW	0	USB PHY 软复位。写 1 复位，写 0 撤销。
7:6	USB_TF_SELECT	RW	0	调整 USB 眼图。 TF_SELECT DP/DM Tf and Tr 00 Low 01 Type 10 Type 1 Fast
5:4	USB_IBIAS_SELECT	RW	01	Overlap bias 电路 trimming 00: 3.3mA 01: 4.4mA 10: 4.4mA 11: 6.6mA
3: 1	USB_MATCH_RES	RW	0	FSDRV 输出电阻修调。 000: 45ohm 001: 54ohm 010: 51ohm 011: 48ohm 100: 42ohm 101: 39ohm 110: 36 ohm 111: 36 ohm
0	USB_PLL_EN	RW	1	USB PHY PLL 使能 1: 使能 0: 禁止

4.7.20. Sensor复位状态寄存器SENSOR_RST_STATUS（偏移：5Ch）

比特	名称	属性	复位值	描述
31:10	RSV	-	-	保留
9	CGD_ABNORMAL	RO	0	总线校验异常状态。

				0: 未发生异常; 1: 发生异常;
8	BUS_ABNORMAL	RO	0	总线校验异常状态。 0: 未发生异常; 1: 发生异常;
7	LD_ABNORMAL	RO	0	LD 异常状态。 0: 未发生异常; 1: 发生异常;
6	AS_ABNORMAL	RO	0	Active Shielding 异常状态。 0: 未发生异常; 1: 发生异常;
5	PGD_ABNORMAL	RO	0	PGD 异常状态。 0: 未发生异常; 1: 发生异常;
4	FD_ABNORMAL	RO	0	FD 异常状态。 0: 未发生异常; 1: 发生异常;
3	TD_LOW	RO	0	TD LOW 异常状态。 0: 未发生异常; 1: 发生异常;
2	TD_HIGH	RO	0	TD HIGH 异常状态。 0: 未发生异常; 1: 发生异常;
1	VD_LOW	RO	0	VD LOW 异常状态。 0: 未发生异常; 1: 发生异常;
0	VD_HIGH	RO	0	VD HIGH 异常状态。 0: 未发生异常; 1: 发生异常;

4.7.21. 模拟模块控制寄存器ANACR（偏移：60h）

比特	名称	属性	复位值	描述
31:28	RSV	-	-	保留

27: 26	TRIM_VD18	RW	0x01	VD18 修调信号
25	RC32K_EN	RW	0x01	RC32K 模块使能
24:20	RC48M_TUNE	RW	0x0c	RC48M 时钟 Trim 值 Trim 越高频率越高
19	RC48M_EN	RW	0x01	RC48M 模块使能
18:16	LDO18_TRIM	RW	0x04	修调 LDO18 的数字信号
15	LDO18_EN	RW	0x01	LDO18 使能信号
14:12	ANATEST_SEL	RW	0x00	模拟 ANATEST 信号选择
11	LDO12T10	RW	0x00	LDO12 电压降低强制使能 0: LDO12 电压不变 1: LDO12 电压降低
10: 7	LDO12_TRIM	RW	0x08	修调 VDD12_OUT 输出电压
6: 4	LDO33_TRIM	RW	0x04	修调 VDD33_OUT 输出电压
3: 0	BG_TRIM	RW	0x08	电源 bandgap 修调

5. NVIC

5.1. 概述

内嵌套向量中断控制器(NVIC) 是 Cortex-M 系列的一个重要组成部分。它与 CPU 处理器内核紧密耦合，实现低中断延迟以及对新到中断的有效处理，外部中断信号连接到 NVIC，NVIC 将对这些中断进行优先级排序。

所有的 NVIC 寄存器只能采用字传输。

NVIC 寄存器都是小端格式。访问处理器要正确处理处理器的大小端配置。

(关于 NVIC 更详细的内容可查看 Cortex-M 系列内核的相关官方文档)

5.2. 主要特性

- 支持 32 路向量中断
- 一个外部不可屏蔽中断 NMI
- 4 个带硬件优先级屏蔽的可编程中断优先级。
- 可嵌套中断支持
- 中断可屏蔽
- 电平触发和边沿触发

5.3. 中断源

表 5-1 中断源

中断号	中断源	备注
Int0	WDT	
Int1	Timer 计时中断	
Int2	保留	
Int3	UARTA	
Int4	SPIA	
Int5	SPIB	
Int6	GPIOA	
Int7	USB	
Int8	保留	
Int9	SM1	
Int10	DES	
Int11	PKI	
Int12	EFC	
Int13	保留	

Int14	I2C	
Int15	7816MS RST	
Int16	SM4	
Int17	GPIOB	
Int18	DMA	
Int19	Timer 输入捕获和 PWM 中断。	
Int20	SDIO	
Int21	UARTB	
Int22	BCH	
Int23	NFM	
Int24	EMW	
Int25	AES	
Int26	SENSOR	
Int27	7816MS	
Int28	保留	
Int29	保留	
Int30	保留	
Int31	WAKE_UP 管脚边沿	
NMI		

6. MPU

6.1. 概述

芯片具有一个存储器保护单元（MPU），可以实现对存储器（主要是内存和外设寄存器）的访问保护，从而使软件更加健壮和可靠。如果启用 MPU，则在使用前，必须根据应用需要分配区间和访问权限。MPU 在执行其功能时，是以“region”为单位的。一个 region 是一段由用户制定的连续地址空间。MPU 共支持 8 个 regions。它允许把每个 region 进一步划分成更小的子“region”。

6.2. 主要特性

- 阻止用户应用程序破坏操作系统使用的数据
- 阻止一个任务访问其它任务的数据区，从而把任务隔开。
- 可以把关键数据区设置为只读，从根本上消除了被破坏的可能
- 检测意外的存储访问，如，堆栈溢出，数组越界

(关于 MPU 更详细的内容可查看 Cortex-M 系列内核的相关官方文档)

7. EFC

7.1. 概述

芯片上集成了 512Kbyte 的 Eflash 存储器，用于保存芯片所有的关键脱机信息和数据。EFC 为 Eflash 控制器，在 Cpu 的配置下，完成 Flash 读、写、擦除等操作。

7.2. 主要特性

- 支持 Eflash 的读（8/16/32bit）、写（32bit）、擦除等操作流程；
- 读等待时间可以配置；
- 主区有 1024 个 page，每个 page 512 字节；
- NVR 区有 2 个 page，每个 page 512 字节；
- 支持对 NVR1 区域擦/写保护功能；
- 支持 Program Verify、Page Erase Verify 功能；
- 支持 Double Read 功能；
- 支持 Eflash 地址串扰功能；
- 支持 Eflash 写超时与擦除超时检测功能，超时时间可配置；
- chip 擦除时间 10ms，page 擦除时间 2ms，word 写 20us（max），读时间 52ns（max）；

7.3. 寄存器描述

寄存器基地址：ROM 启动为 0x10100000；EFlash 启动为 0x00100000；

偏置	名称	描述
0x00	EFC_CTRL	控制寄存器。
0x04	EFC_SEC	写擦除操作安全寄存器。
0x08	EFC_ADCT	写擦除地址校验寄存器。
0x0c	EFC_ERTO	擦除超时寄存器。
0x10	EFC_WRTO	写超时寄存器。
0x14	EFC_STATUS	状态寄存器。
0x18	EFC_INTSTATUS	中断状态寄存器。
0x1c	EFC_INEN	中断使能寄存器。

7.3.1. 控制寄存器EFC_CTRL (偏移：00h)

比特	名称	属性	复位值	描述
31:17	RSV	-	-	保留
16	EWCntEn	RW	0	Flash 擦/写完成标志选择位。 1: Flash 在擦写操作时，将不再依靠 TBIT

				信号判断擦写是否完成，而是由 EFC_ERTO 和 EFC_WRTO 来判断擦写是否完成。此位为 1 时，Fto_En 位也需要置 1，否则控制器将不会判断擦写是否完成，控制器将无法退出擦写操作。 0：使用 TBIT 信号作为擦写完成的判断。当此位为 0 时，如果 Fto_En 位为 1，则当 EFC_ERTO 和 EFC_WRTO 定时超时或者 TBIT 信号有一个条件成立时，系统即退出擦写操作。
15	Resetb	RW	1	写和擦除禁止位。 0：写/擦除禁止； 1：写/擦除使能；
14	Sleep1	RW	0	Eflash1 Sleep 模式启动位。（后 256Kbyte 空间） 0：Eflash1 退出 Sleep 模式； 1：Eflash1 进入 Sleep 模式； 退出 Sleep 模式后需要等待 10us 后,才可以操作 Eflash.
13	Sleep0	RW	0	Eflash0 Sleep 模式启动位。（前 256Kbyte 空间） 0：Eflash0 退出 Sleep 模式； 1：Eflash0 进入 Sleep 模式； 退出 Sleep 模式后需要等待 10us 后,才可以操作 Eflash.
12:8	Rd_Wait	RW	8	读等待时间设置位。其为读操作 EFlash 端口等待的时钟周期数为 Rd_Wait + 1。EFlash 要求读等待时间至少为 52ns，以满足 Eflash 读的最大延迟。 0：等待 1 个系统时钟周期； 1：等待 2 个系统时钟周期；
7	Fto_En	RW	0	写/擦除操作超时使能；

				1: 写/擦除超时功能使能; 0: 写/擦除超时功能禁止;
6	Ac_En	RW	0	写/擦除操作地址校验功能使能: 1: 写擦除地址校验功能使能; 0: 写擦除地址校验功能禁止; 使能此功能, 写或者擦除操作的地址将与 EFC_ARCT 中的数值作比对, 如果不相等, 则状态寄存器中的位将置 1。
5	Pev_Start	W	0	使能此功能, 将对 EFC_ARCT 中地址所在的 Sector 发起 Page Erase Verify 操作: 1: Page Erase Verify 功能启动; 0: Page Erase Verify 功能禁止;
4	Wrv_En	RW	0	使能此功能, 在写操作完成后将自动发起一次读验证: 1: Program Verify 功能使能; 0: Program Verify 功能禁止;
3	Dbr_En	RW	0	Double Read 功能使能: 1: Double Read 功能使能; 0: Double Read 功能禁止; 使能此功能, Eflash 读操作自动对地址随机读 2 遍, 并比对两次读取的值是否相同。
2	Chip_Erase_Mode	RW	0	Chip Erase Mode 模式设置位。 1: Chip Erase Mode 模式使能; 0: Chip Erase Mode 模式禁止。
1	Page_Erase_Mode	RW	0	Page Erase Mode 模式设置位。 1: Page Erase Mode 模式使能; 0: Page Erase Mode 模式禁止。
0	Write_Mode	RW	0	Write 模式设置位。 1: 写操作模式使能; 0: 写操作模式禁止。

7.3.2. 写擦安全寄存器EFC_SEC (偏移: 04h)

比特	名称	属性	复位值	描述
31:0	Write_Lock_Ser	W	0	向 Eflash 写数据/擦除前, 须向此寄存器内写

				0x55aaaa55 值, 否则控制其忽略此次写操作。
--	--	--	--	-----------------------------

7.3.3. 地址校验寄存器EFC_ARCT(偏移: 08h)

比特	名称	属性	复位值	描述
31:17	RSV	-	-	保留
16:0	Ad_Ct	RW	0	写/擦除地址校验地址寄存器; 写擦除操作时, 校验 EFlash 端口的[16:0]位 (按 Word 地址校验); 在 Page Erase Verify 操作中, 此寄存器用来保存比对的地址信息。

7.3.4. 擦除超时寄存器 (EFC_ERTO) (偏移: 0ch)

比特	名称	属性	复位值	描述
31:17	RSV	-	-	保留
16:0	ErSd	RW	0x1ffff	擦除TimeOut设置位。每一个单位代表256个系统时钟周期。当Eflash擦除操作在此时间断内未结束时, 控制器将认为此操作有误, 产生TimeOut状态信息, 并复位内部状态机。 注: Chip erase 时等待时间需大于 10ms, 否则擦除可能会出现问题; Pageerase 时需等待至少 2ms, 否则擦除可能会出现问题。

7.3.5. 写超时寄存器 (EFC_WRTO) (偏移: 10h)

比特	名称	属性	复位值	描述
31:17	RSV	-	-	保留
16:0	WrCycle	RW	0x1ffff	写操作TimeOut设置位。每一个单位代表256个系统时钟周期。当Eflash写操作在此时间断内未结束时, 控制器将认为此操作有误, 产生TimeOut状态信息, 并复位内部状态机。 注: 写操作时需等待至少20us, 否则写操作可能会出现问题。

7.3.6. 状态寄存器EFC_STATUS (偏移: 14h)

比特	名称	属性	复位值	描述
31:7	RSV	-	-	保留
6	Vdd18_Low	RO	0	0:Vdd18 电源电压正常; 1:Vdd18 电源电压过低。

				此位反应当前 Vdd18 的实时状态。
5	TBIT1	RO	0	此位反映 Eflash TBIT1 信号真实状态。
4	TBIT0	RO	0	此位反映 Eflash TBIT0 信号真实状态。
3	RSV	-	-	保留
2	Nvr2_Lock	RO	1	Nvr2 区是否锁住。 1: Nvr2 区已经锁住; 0: Nvr2 区没有锁住。
1	Nvr1_Lock	RO	0/1	Nvr1 区是否锁住。 1: Nvr1 区已经锁住; 0: Nvr1 区没有锁住。
0	EFlash_Ready	RO	1	EFlash 状态指示位。该反映 EFlash 工作的状态。 1: EFlash 状态空闲; 0: EFlash 状态忙。

7.3.7. 中断状态寄存器EFC_INTSTATUS (偏移: 18h)

比特	名称	属性	复位值	描述
31:9	RSV	-	-	保留
8	Vdd18_Low_Status	RW	0	Vdd18 电压过低状态中断位。 1: 发生过 Vdd18 电压过低情况; 0: 正常状态; 写 1 清 0。
7	Nvr_Err	RW	0	1: Nvr 区发生操作错误。当对应的 Nvr 区 Lock 住时, 对此 Nvr 区域进行写擦操作, 或者对 Nvr 区域进行 Chip Erase 操作时, 此位均会置 1; 0: 正常状态。 写 1 清 0。
6	PWr_Vf_Err	RW	0	Program Verify 中断状态位。 1: Program Verify 有误; 0: Program Verify 正确。 写 1 清 0。
5	Adct_Err	RW	0	地址校验错误中断状态位。 1: 地址校验有误; 0: 地址校验正确。 写 1 清 0。

4	Per_Vf_Done	RW	0	Page Erase Verify 或者 Program Verify 完成中断状态位。 1: Page Erase Verify 或者 Program Verify 完成; 0: Page Erase Verify 或者 Program Verify 未完成。 写 1 清 0。
3	Per_Vf_Err	RW	0	Page Erase Verify 中断状态位。 1: Page Erase Verify 有误; 0: Page Erase Verify 正确。 写 1 清 0。
2	ErWr_To_Err	RW	0	擦除/写操作发生 TimeOut 中断状态位。 1: 发生擦除/写操作 TimeOut; 0: 未发生擦除/写操作 TimeOut。 写 1 清 0。
1	Dr_Err	RW	0	Double Read 错误中断状态位。 1: 发生 Double Read 错误; 0: 未发生 Double Read 错误。 写 1 清 0。
0	ErWr_done	RW	0	写/擦除完成中断状态位。 1: 写/擦除完成, 写 1 清除该位。如果中断允许, 则产生中断; 0: 写/擦除未完成。

7.3.8. 中断使能寄存器EFC_INEN (偏移: 1Ch)

比特	名称	属性	复位值	描述
31:9	RSV	-	-	保留
8	Vdd18_Low_Status_En	RW	0	Vdd18_Low_Status 中断使能。 1: Vdd18_Low_Status 中断使能; 0: Vdd18_Low_Status 中断不使能。
7	Nvr_Err_En	RW	0	Nvr_Err 中断使能。 1: Nvr_Err 中断使能; 0: Nvr_Err 中断不使能。
6	PWr_Vf_En	RW	0	Program Verify 中断使能。

				1: Program Verify 中断使能; 0: Program Verify 中断不使能。
5	Adct_En	RW	0	地址校验错误中断使能。 1: 地址校验错误中断使能; 0: 地址校验错误中断不使能。
4	Per_Vf_Done_En	RW	0	Page Erase Verify 完成中断使能。 1: Page Erase Verify 完成中断使能; 0: Page Erase Verify 完成中断不使能。
3	Per_Vf_En	RW	0	Page Erase Verify 结果中断使能。 1: Page Erase Verify 结果中断使能; 0: Page Erase Verify 结果中断不使能。
2	ErWr_To_En	RW	0	擦除/写操作发生 TimeOut 中断使能。 1: 擦除/写操作 TimeOut 中断使能; 0: 擦除/写操作 TimeOut 中断不使能。
1	Dr_En	RW	0	Double Read 错误中断使能。 1: Double Read 错误中断使能; 0: Double Read 错误中断不使能。
0	Er_done_En	RW	0	擦除完成中断使能。 1: 写/擦除中断使能; 0: 写/擦除中断不使能;

7.4. 功能描述

7.4.1. Program Verify

EFC_CTRL 寄存器中的 Wrv_En 位为 1, 将使能 Program Verify 功能。Program Verify 功能是在 Program 操作后, 将对应的 Flash 地址中的数据读出, 并与写入的数据作比对。如果相同, 则 verify 成功, 否则 EFC_INTSTATUS 寄存器中的 PWr_Vf_Err 位将置 1。

7.4.2. Page Erase Verify

向 EFC_CTRL 寄存器中的 Pev_Start 位写 1, 将发起 Page Erase Verify 操作。Page Erase Verify 操作将 EFC_ARCT 寄存器中对应的 Flash Page 地址中的数据依次读出, 并与 0xffffffff 作比对。EFC_INTSTATUS 寄存器中的 Per_Vf_Done 为 1 表示比对完成。如果 Page 读出的数据与 0xffffffff 全部相同, 则 verify 成功, 否则 EFC_INTSTATUS 寄存器中的 Per_Vf_Err 位将置 1。

7.4.3. Double Read

使能此功能, 硬件自动对同一地址采样两次读取判断结果是否一致。两次地址采样之间的时间间隔由随机数决定。

注：使用此功能时需要同时使能随机数模块。

7.4.4. 地址映射

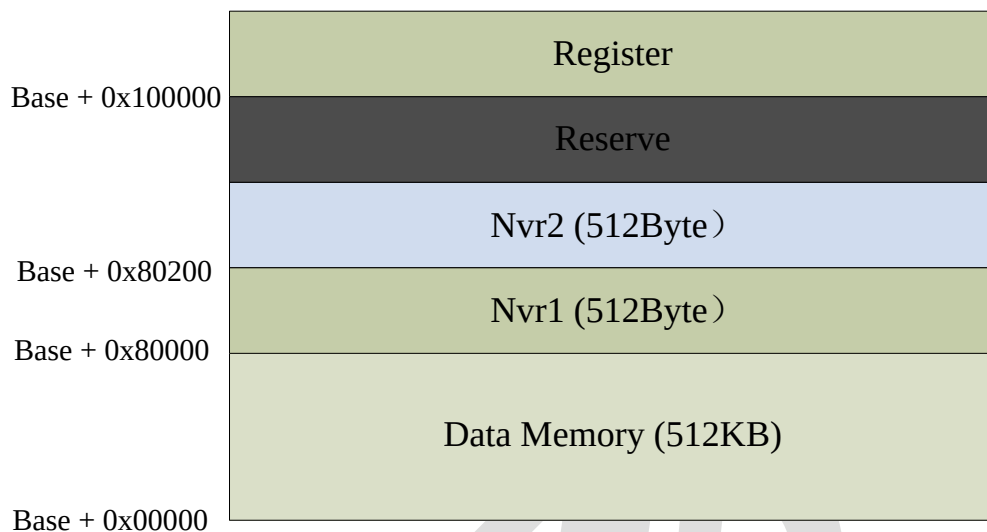


图 7-1Eflash 地址映射

注：Base：ROM 启动为 0x10000000；EFlash 启动为 0x00000000；

7.5. 软件流程

7.5.1. Read操作

EFlash 上电稳定后可以执行读操作。读操作注意配置读等待时间 RD_WAIT，最小值为 52ns。

7.5.2. Write操作

EFlash 上电稳定后可以执行写操作。写操作之前需要向 EFC_SEC 寄存器内写 0x55aaaa55，否则 EFC 忽略此次写操作。

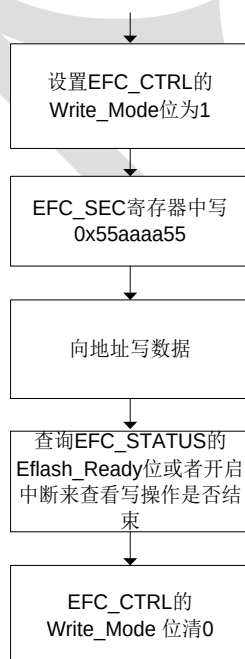


图 7-2 写操作流程

7.5.3. Erase操作

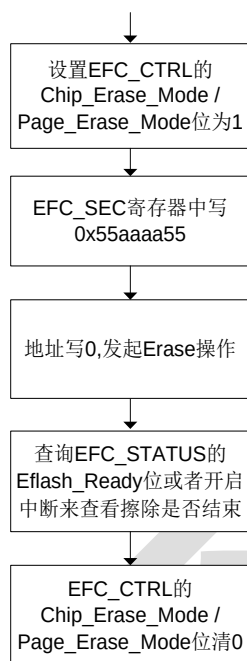


图 7-3 擦除操作流程

Chip 擦除操作以 256kbyte 为单位分别对 EFlash 进行擦除。当需要擦除整个 EFlash(512kbyte)，则需要进行两次擦除操作（前 256Kbyte 擦除选择地址为 0x0，后 256Kbyte 为 0x40000）。

8. DMAC

8.1. 概述

DMAC 即 DMA controller，可以有效提高系统数据传输效率。

8.2. 主要特性

- 4 个 DMA 通道（通道数量可配置，最多 4 个）
- 支持 Single 和 burst DMA 请求
- 支持 Memory-to-memory, memory-to-peripheral, peripheral-to-memory 传输
- 通道优先级：通道 0 优先级最高，通道 3 优先级最低。如果两个通道的请求同时有效，那么先处理优先级高的
- 支持源/目标地址递增或不变
- 每个通道有 16 bytes 的内部 FIFO
- 支持 8/16/ 32 位宽的传输
- 支持 Big-endian 和 little-endian 的字节存储次序配置
- 支持 DMA 中断功能

8.3. 寄存器描述

DMAC 寄存器基地址：0x40000000

偏置	名称	描述
0x00	DMACIntStatus	中断状态寄存器
0x04	DMACIntTCStatus	传输完成中断寄存器
0x08	DMACIntTCClr	传输完成中断清除寄存器
0x0C	DMACIntErrStatus	传输错误中断寄存器
0x10	DMACIntErrClr	传输错误中断清除寄存器
0x14	DMACRawIntTCStatus	传输完成原始中断寄存器
0x18	DMACRawIntErrStatus	传输错误原始中断寄存器
0x1C	DMACEnChStatus	通道使能状态寄存器
0x30	DMACConfig	DMAC 配置寄存器

0x34	DMACSync	同步寄存器
0x100, 0x120, 0x140, 0x160,	DMACCxSrcAddr	源通道 0/1/2/3 地址寄存器
0x104, 0x124, 0x144, 0x164,	DMACCxDestAddr	目标通道 0/1/2/3 地址寄存器
0x108, 0x128, 0x148, 0x168,	DMACCxLLI	通道 0/1/2/3 链接表寄存器
0x10C, 0x12C, 0x14C, 0x16C,	DMACCxCtrl	通道 0/1/2/3 控制寄存器
0x110, 0x130h, 0x150, 0x170,	DMACCxConfig	通道 0/1/2/3 配置寄存器

8.3.1. 中断状态寄存器DMACIntStatus（偏移：00h）

比特	名称	属性	复位值	描述
31:4	RSV	-	-	保留
3:0	IntStatus	R	0	可被屏蔽的中断状态。 1: 对应的通道产生中断，表示该通道产生了错误或者传输完成 0: 没有产生中断

8.3.2. 传输完成中断寄存器DMACIntTCStatus（偏移：04h）

比特	名称	属性	复位值	描述
31:4	RSV	-	-	保留
3:0	IntTCStatus	R	0	传输完成中断状态 1: 对应的通道传输完成 0: 没有传输完成

8.3.3. 传输完成中断清除寄存器DMACIntTCClr（偏移：08h）

比特	名称	属性	复位值	描述
31:4	RSV	-	-	保留
3:0	IntTCClr	W	0	传输完成中断状态清除 1: 写 1 将对应的 IntTCStatus 位清 0 0: 无动作

8.3.4. 传输错误中断寄存器DMACIntErrStatus（偏移：0Ch）

比特	名称	属性	复位值	描述
31:4	RSV	-	-	保留
3:0	IntErrStatus	R	0	传输错误中断状态 1: 对应的通道传输错误 0: 没有错误

8.3.5. 传输错误中断清除寄存器DMACIntErrClr（偏移：10h）

比特	名称	属性	复位值	描述
31:4	RSV	-	-	保留
3:0	IntErrClr	W	0	传输错误中断状态清除。 1: 写 1 将对应的 IntErrStatus 位清 0 0: 无动作

8.3.6. 传输完成原始中断寄存器DMACRawIntTCStatus（偏移：14h）

比特	名称	属性	复位值	描述
31:4	RSV	-	-	保留
3:0	RawIntTCStatus	R	0	传输完成原始中断状态，不可被屏蔽中断。 1: 对应的通道传输完成 0: 没有传输完成

8.3.7. 传输错误原始中断寄存器DMACRawIntErrStatus（偏移：18h）

比特	名称	属性	复位值	描述
31:4	RSV	-	-	保留
3:0	RawIntErrStatus	R	0	传输错误中断状态，不可被屏蔽中断。

				1: 对应的通道传输错误 0: 没有错误
--	--	--	--	-------------------------

8.3.8. 通道使能状态寄存器DMACEnChStatus（偏移：1Ch）

比特	名称	属性	复位值	描述
31:4	RSV	-	-	保留
3:0	EnChStat	R	0	已被使能的通道号

8.3.9. DMAC配置寄存器DMACConfig（偏移：30h）

比特	名称	属性	复位值	描述
31:3	RSV	-	-	保留
2	M2ENDIAN	R/W	0	Master 2 字节存储次序配置 1: big-endian 模式 0: little-endian 模式
1	M1ENDIAN	R/W	0	Master 1 字节存储次序配置 1: big-endian 模式 0: little-endian 模式
0	EN	R/W	0	Dmac 使能 1: 打开 dmac 0: 关闭 dmac

8.3.10. 同步寄存器DMACSync（偏移：34h）

比特	名称	属性	复位值	描述
31:16	RSV	-	-	保留
15:0	Sync	R/W	0	DMA 同步逻辑控制 1: 同步逻辑不打开 0: 同步逻辑打开

注意：当外设产生请求信号的时钟和 DMAC 的时钟不是同一个时钟时，必须打开同步逻辑。

8.3.11. 源通道地址寄存器DMACCxSrcAddr（偏移：100h, 120h, 140h, 160h）

比特	名称	属性	复位值	描述
31:0	DMACCxSrcAddr	R/W	0	源通道地址寄存器

注意：当通道使能打开时，不允许改变该寄存器的值。

8.3.12. 目标通道地址寄存器DMACCxDestAddr（偏移：104h, 124h, 144h, 164h）

比特	名称	属性	复位值	描述
31:0	DMACCxDestAddr	R/W	0	目标通道地址寄存器

注意：当通道使能打开时，不允许改变该寄存器的值。

8.3.13. 通道链接表寄存器DMACCxLLI（偏移：108h, 128h, 148h, 168h）

比特	名称	属性	复位值	描述
31:2	LLI	R/W	0	通道链接表，表示下一个 32 位的链接表地址，其中[1:0]为 0.
1	RSV	-	-	保留
0	LM	R/W	0	下一个 LLI 的 master 号 1: master 2 0: master 1

8.3.14. 通道控制寄存器DMACCxCtrl（偏移：10Ch, 12Ch, 14Ch, 16Ch）

比特	名称	属性	复位值	描述
31:28	RSV	-	-	保留
27	DI	R/W	0	目标地址递增量，当设置后，每个 transfer 后目标地址将加上一个递增量
26	SI	R/W	0	源地址递增量，当设置后，每个 transfer 后源地址将加上一个递增量
25	D	R/W	0	目标 AHB master 选择

				1: master 1 0: master 0
24	S	R/W	0	源 AHB master 选择 1: master 1 0: master 0
23:21	DWidth	R/W	0	目标传输位宽。DWidth 对应的位宽见下表
20:18	SWidth	R/W	0	源传输位宽。SWidth 对应的位宽见下表
17:15	DBSize	R/W	0	目标 burst size。一个 burst 由 DBSize 个 transfer 组成。用户必须把这个值设置成与目标外设的 burst size 一致；如果目标是 memory，那么必须与 memory 的边界大小一致。Burst size 是目标外设的 DMACxBREQ 为高时传输的数据量。DBSize 对应的 Burst size 见表 8-1
14:12	SBSize	R/W	0	源 burst size。一个 burst 由 SBSize 个 transfer 组成。用户必须把这个值设置成与源外设的 burst size 一致；如果源是 memory，那么必须与 memory 的边界大小一致。Burst size 是源外设的 DMACxBREQ 为高时传输的数据量。SBSize 对应的 Burst size 见表 8-2
11:0	TransferSize	R/W	0	Transfer Size。 这个寄存器从初始值向 0 计数，读该位可反映剩余还未传输的 transfer 个数。DMAC 是流控制器时 Transfer Size 才有意义，当 DMAC 不是流控制器时，需把

				Transfer Size 设为 0
--	--	--	--	--------------------

表 8-1 DBSIZE/SBSIZE 对应的 burst size

DBSIZE/SBSIZE	Burst size
0b000	1
0b001	4
0b010	8
0b011	16
0b100	32
0b101	64
0b110	128
0b111	256

表 8-2 SWidth/DWidth 对应的 width

SWidth/Dwidth	Width
0b000	1 byte
0b001	2 byte
0b010	4 byte
0b011	reserved
0b100	reserved
0b101	reserved
0b110	reserved
0b111	reserved

注意：当通道使能打开时，不允许改变该寄存器的值。在通道使能打开前将各个通道寄存器配置好，打开使能后，配置好的寄存器即生效。

8.3.15. 通道配置寄存器 DMACCxConfig（偏移：110h, 130h, 150h, 170h）

比特	名称	属性	复位值	描述
31:19	RSV	-	-	保留
18	Halt	R/W	0	1: 当 DMA 通道 FIFO 没有 data 时，忽视额外的 DMA 请求。可以与 Active 位配合关掉 DMAC 0: 使能 DMA 请求
17	Active	R	0	1: 通道 FIFO 中有数据 0: 通道 FIFO 中没有数据
16	Lock	R/W	0	1: 打开 lock 功能 0: 无变化
15	ITC	R/W	0	传输完成中断使能 1: 使能该通道的传输完成中断

				0: 屏蔽该通道的传输完成中断
14	IE	R/W	0	传输错误中断使能 1: 使能该通道的传输错误中断 0: 屏蔽该通道的传输错误中断
13:11	FlowCtrl	R/W	0	流控制器和传输类型。对应关系见表 8-3
10	RSV	-	-	保留
9:6	DestPeriph	R/W	0	目标外设的位置。对应关系见表 8-4
5	RSV	-	-	保留
4:1	SrcPeriph	R/W	0	源外设的位置。对应关系见表 8-4
0	EN	R/W	0	通道使能 1: 通道使能 0: 通道关闭 可以读取 DMACEnChStatus 寄存器知道通道使能的情况。

表 8-3 FlowCtrl 对应的传输方向和 controller

FlowCtrl	传输方向	controller
000	memory to memory	DMA
001	memory to peripheral	DMA
010	peripheral to memory	DMA

注：DMA不能访问eflash、ROM、DMA自身寄存器

表 8-4 目标外设和源外设请求号

请求号	请求源	备注
REQ0	保留	
REQ 1	保留	
REQ 2	SDIO 写数据请求	
REQ 3	SDIO 读数据请求	
REQ 4	SPIA 发送请求。	
REQ 5	SPIA 接收请求。	
REQ 6	SPIB 发送请求。	
REQ 7	SPIB 接收请求。	

REQ8~REQ15	保留	
------------	----	--

8.4. 使用说明

8.4.1. DMA优先级

DMA 优先级是固定的，通道 0 最高优先级，通道 3 最低优先级。如果 DMAC 正在为一个低优先级的通道传输数据时，一个高优先级的通道请求产生了，那么 DMAC 将会先把低优先级通道的数据传输完毕，然后再来处理高优先级的通道要求。

8.4.2. 软件注意事项

- 1) 在通道使能后，不能再对通道寄存器有任何写操作；如果一定要写通道寄存器，只能在把使能关闭后再写。
- 2) 源位宽乘以 TransferSize 必须是目标位宽的整数倍。
- 3) 如果软件通过设置 DMACCxConfig 寄存器的 bit 0 位来关闭通道，那么必须在读 EnChStat 对应通道 enable 为 0 后才能再次打开通道。因为关闭通道的过程是有延时的。
- 4) 如果 dmac 是流控制器，而且 TransferSize 为 0，那么打开 enable 之后不会产生任何数据传输。

8.4.3. DMAC使用流程

1. 打开 DMAC 总开关（DMAC 配置寄存器（config）第 0 位）。
2. 配置好 DMAC 的通道，传输方向，DMA master，源地址和目标地址，源位宽，目标位宽，源/目标的 burst size，传输数据的 transfer size 等传输参数。
3. 打开 DMAC 通道 enable（DMAC 通道配置寄存器（DMACCxConfig）第 0 位）。

8.4.4. DMAC链表使用流程

一个链表项（LLI）由四个字组成，这些字按照以下顺序来排列

- (1) REG_DMACCSrcAddr(x)
- (2) REG_DMACCDestAddr(x)
- (3) REG_DMACCLinkList(x)
- (4) REG_DMACCCControl(x)

其中 REG_DMACCLinkList(x)为下一个链表项的地址，最后一个 LLI 的链表项指针需要设置为

0

实现特点：

将第一个先前写入寄存器的链表项写入 DMA 控制器的相关通道。向通道配置寄存器写入通道

配置信息，并置位通道使能位，接下来会启动 DMA 传输，每个数据块传输完毕后，会自动装载下个链表项每个数据块传输完成后都可以产生中断。

保
密

9. USB

9.1. 概述

该控制器支持 USB2.0（高速）和 USB1.1(全速)协议，含有一个控制端点 EP0 和 4 个通用端点 EP1、EP2、EP3、EP4。需要在 RAM 中为每个端点需要分配一个 FIFO，通用端点 FIFO 的起始地址和大小可动态分配。支持 UMS/HID/CCID 协议。

9.2. 主要特性

- 支持 USB2.0 高速（480Mbps）和全速（12Mbps）
- 一个控制端点 EP0 和 4 个通用端点 EP1、EP2、EP3、EP4
- 通用端点的 FIFO 可动态分配，以此提高数据传输效率
- 对于端点某些寄存器（例如控制状态寄存器），各个端点公用一组寄存器偏移地址，通过索引寄存器来配置当前需要访问的端点具体对应的寄存器。
- 支持大数据包（对于全速，大数据包可超过 64bytes，对于高速，大数据包可超过 512bytes）传输，即控制器内部可自动拆分或者合并数据包。

9.3. 寄存器描述

USB 寄存器基地址：0x40030000

偏置	名称	描述
0x00	FADDRR	USB 设备地址寄存器，用来存储 HOST 分配给设备的地址
0x01	UCSR	USB 控制状态寄存器，选择 USB 设备的速度等
0x02	IntrTx	USB 设备发送(IN)中断状态寄存器（与端点有关的中断）
0x04	IntrRx	USB 设备接收(OUT)中断状态寄存器（与端点有关的中断）
0x06	INTRTXE	USB 设备发送中断使能寄存器（与端点有关的中断）
0x08	INTRRXE	USB 设备接收中断使能寄存器（与端点有关的中断）
0x0A	IntrUSB	USB 设备中断状态寄存器（与 USB 整体有关的中断，复位等中断）
0x0B	IntrUSBE	USB 设备中断使能寄存器（与 USB 整体有关的中断，复位等中断）
0x0C	FNUMR	保留的寄存器，未定义
0x0E	Eindex	索引寄存器，某些端点功能寄存器公用同一个偏移地址，通过该寄存器来配置具体要访问的寄存器
0x0F	Testmode	测试模式寄存器
0x10	TXPSZR	设置最大发送数据包，该偏移地址由四个通用端点公用，通过索引寄存器 Index 来配置具体需要访问的端点

0x12	E0CSR TxCSR	EP0 和 EPX (X 取 1、2、3、4) 的发送控制状态寄存器共 16 位, 共用偏移地址 0x12 和 0x13。通过索引寄存器 Eindex 来配置具体需要访问的端点的发送控制状态寄存器
0x14	RXPSZR	设置最大接收数据包, 该偏移地址由四个通用端点公用, 通过索引寄存器 Eindex 来配置具体需要访问的端点
0x16	RxCSR	EPX (X 取 1、2、3、4) 的接收控制状态寄存器共 16 位, 共用偏移地址 0x16 和 0x17。通过索引寄存器 Eindex 来配置具体需要访问的端点的接收控制状态寄存器
0x18	E0COUNTR RXCOUNT	EP0 和 EPX (X 取 1、2、3、4) 的 FIFO 计数寄存器共用偏移地址 0x18。通过索引寄存器 Eindex 来配置具体需要访问的端点的 FIFO 计数寄存器
0x1A	TxFIFO_SIZE	设置发送 FIFO 的最大容量, 通过索引寄存器 Eindex 来配置具体需要访问的端点
0x1B	RxFIFO_SIZE	设置接收 FIFO 的最大容量, 通过索引寄存器 Eindex 来配置具体需要访问的端点
0x1C	TxFIFO_ADDR	设置发送 FIFO 的起始地址, 通过索引寄存器 Eindex 来配置具体需要访问的端点
0x1E	RxFIFO_ADDR	设置接收 FIFO 的起始地址, 通过索引寄存器 Eindex 来配置具体需要访问的端点
0x20	Ep0FIFO	EP0 FIFO 通道
0x24	Ep1FIFO	EP1 FIFO 通道
0x28	Ep2FIFO	EP2 FIFO 通道
0x2C	Ep3FIFO	EP3 FIFO 通道
0x30	Ep4FIFO	EP4 FIFO 通道
0x40	TX_PTR	EPX (X 取 1、2、3、4) 待发送数据的长度, 适用于动态分配 FIFO 起始地址的情况

9.3.1. USB设备地址寄存器 FADDR (偏移: 00h)

比特	名称	属性	复位值	描述
7	AdrFlag	R	0	地址配置状态位, 当写 bit6-bit0 时, 硬件自动将 bit7 置位, 表示有地址待更新, 当新地址生效后, bit7 自动清零。
6:0	Address	R/W	0	USB 设备的地址, 由 HOST 分配

9.3.2. 控制状态寄存器UCSR（偏移：01h）

比特	名称	属性	复位值	描述
7	RSV	-	-	保留
6	Connect	R/W	0	软件控制 USB 的连接与断开，1：连接； 0：断开
5	SpeedSel	R/W	1	USB 速度选择位， 1：高速； 0：全速
4	SpeedState	R	0	USB 复位成功后的速度状态，1：高速； 0：全速
3	ResetState	R	0	USB 总线上的复位状态，1：总线处于复位状态，0：不处于复位状态（检测到 SE0 2.5us 后置位，在高速模式协商成功后或者高速模式协商失败的情况下检测到 SE0 2.1ms 之后会自动清除）
2	Remote_wake up	R/W	0	写 1 将使控制器在总线上产生 resume 信号以实现远程唤醒，需要软件自己清除该位
1	SuspendState	R	0	悬挂状态标志，1：设备正处于悬挂状态，0：非悬挂，读中断状态寄存器 IntrUSB 会清除该位
0	SuspendEn	R/W	0	悬挂使能位，1：使能控制器悬挂，0：不使能悬挂

9.3.3. 发送中断状态寄存器IntrTx（偏移：02h）

比特	名称	属性	复位值	描述
7-5	RSV	-	-	保留
4	EP4	R	0	EP4 产生发送完成中断时，将该标志位置位，读清 0
3	EP3	R	0	EP3 产生发送完成中断时，将该标志位置位，读清 0
2	EP2	R	0	EP2 产生发送完成中断时，将该标志位置位，读清 0
1	EP1	R	0	EP1 产生发送完成中断时，将该标志位置位，读清 0
0	EP0	R	0	EP0 产生中断时，将该标志位置位，读清 0

注意：EP0 产生中断不限于发送数据包完成，EP0 的所有中断均由该位显示。EP0 的中断包括：接收到数据包时，发送数据包完成时，发送了 stall 握手信号时，控制传输异常终止时，状态阶段结束时，EP0 具体的中断来源，需要根据上述各个相关的状态标志综合判定

9.3.4. 接收中断状态寄存器IntrRx（偏移：04h）

比特	名称	属性	复位值	描述
7-5	RSV	-	-	保留
4	EP4	R	0	EP4 产生接收到数据包中断时，将该标志位置位
3	EP3	R	0	EP3 产生接收到数据包中断时，将该标志位置位

2	EP2	R	0	EP2 产生接收到数据包中断时，将该标志位置位
1	EP1	R	0	EP1 产生接收到数据包中断时，将该标志位置位
0	RSV	-	-	保留

注：读该寄存器会自动清除中断。

9.3.5. 发送中断使能寄存器INTRTXE（偏移：06h）

比特	名称	属性	复位值	描述
7-5	RSV	-	-	保留
4	EP4	R/W	1	EP4 发送中断使能位，1：使能；0：不使能
3	EP3	R/W	1	EP3 发送中断使能位，1：使能；0：不使能
2	EP2	R/W	1	EP2 发送中断使能位，1：使能；0：不使能
1	EP1	R/W	1	EP1 发送中断使能位，1：使能；0：不使能
0	EP0	R/W	1	EP0 中断使能位，1：使能；0：不使能

9.3.6. 接收中断使能寄存器INTRRXE（偏移：08h）

比特	名称	属性	复位值	描述
7-5	RSV	-	-	保留
4	EP4	R/W	1	EP4 接收中断使能位，1：使能；0：不使能
3	EP3	R/W	1	EP3 接收中断使能位，1：使能；0：不使能
2	EP2	R/W	1	EP2 接收中断使能位，1：使能；0：不使能
1	EP1	R/W	1	EP1 接收中断使能位，1：使能；0：不使能
0	RSV	-	-	保留

9.3.7. USB中断状态寄存器IntrUSB（偏移：0Ah）

比特	名称	属性	复位值	描述
7-4	RSV	-	-	保留
3	SOFStart	R	0	出现 SOF 包时，该位置位，读清 0
2	Reset	R	0	USB 总线上出现 Reset 信号时，该位置位，读清 0
1	Resume	R	0	USB 总线上出现 Resume 信号时，该位置位，读清 0
0	Suspend	R	0	USB 总线上出现 Suspend 信号时，该位置位，读清 0

9.3.8. USB中断使能寄存器IntrUSBIE（偏移：0Bh）

比特	名称	属性	复位值	描述
7-4	RSV	-	-	保留
3	SOFStart	R/W	0	SOFStart 中断使能位，1：使能；0：不使能
2	Reset	R/W	1	Reset 中断使能位，1：使能；0：不使能
1	Resume	R/W	1	Resume 中断使能位，1：能；0：不使能

0	Suspend	R/W	0	Suspend 中断使能位, 1: 使能; 0: 不使能
---	---------	-----	---	------------------------------

9.3.9. 索引寄存器Eindex (偏移: 0Eh)

比特	名称	属性	复位值	描述
7-4	RSV	-	-	保留
3-0	InDex	R/W	0	选择端点号

注意: 某些端点寄存器共用偏移地址, 为了访问相应的端点寄存器, 需要先在索引寄存器 Eindex 中配置待访问的端点号。例如: 端点 1、2、3、4 的发送控制/状态寄存器地址均为: 0x12, 名称均为 TxCSR。要访问 EP2 的这两个寄存器时, 先向索引寄存器写 2: Eindex = 2; 再访问相应的寄存器。

9.3.10. USB测试模式寄存器Testmode (偏移: 0Fh)

比特	名称	属性	复位值	描述
7	ToggleMask Fifo	R/W	0	该位为 1 时, 当 Out 数据包的 Toggle 出错后, 此 Out 数据包不会写入 FIFO。
6	InPktEaly	R/W	0	该位置 0 时, 设置 rUsbEpxSendCtrlStatusL 寄存器的 TxReady 位为 1 后, 可立即读取 TxReady 位。
5	TestFS	R/W	0	该位置 1 时, 接收到复位信号时, 进入全速模式
4	TestHS	R/W	0	该位置 1 时, 接收到复位信号时, 进入高速模式
3	TestPacket	R/W	0	该位置 1 时, 从端点 0 向总线上发送一个 53 字节的包
2	TestK	R/W	0	该位置 1 时, 向总线上发送 K 态
1	TestJ	R/W	0	该位置 1 时, 向总线上发送 J 态
0	TestNAK	R/W	0	该位置 1 时, 控制器对收到所有的 IN 令牌都回 NAK 包

注意: bit0-bit5 只能同时设置一位。在设置 bit3 之前, 需要先向端点 0 写入 53 字节的数据。

9.3.11. 最大发送数据包设置寄存器TXPSZR (偏移: 10h)

比特	名称	属性	复位值	描述
15:11	PayLoadCnt	R/W	0	一个大数据包中的 PayLoad 数目, 必须设为 0
10:0	PayLoad	R/W	0	USB 总线上单包的最大长度, 最大不超过 512bytes

9.3.12. EP0 控制状态寄存器E0CSR (偏移: 12h)

比特	名称	属性	复位值	描述
7	ClrSetup	R/W	0	写 1 清除 SetupEnd 标志, 硬件会自动清除该位
6	ClrRxReady	R/W	0	写 1 清除 RxReady 标志, 硬件会自动清除该位

5	SendStall	R/W	0	写 1 将使控制器向 host 发送 stall 握手信号
4	SetupEnd	R	0	如果控制传输意外终止，那么该位将置 1
3	DataEnd	R/W	0	在控制传输数据阶段结束时，需要将该位配置为 1，控制器将进入控制传输的状态阶段 硬件会自动清除该位
2	SentStall	R/W	0	该位为 1 时表示发送了 stall 握手信号
1	TxReady	R/W	0	将该位设置为 1 时，表示可以开始发送数据了， 当发送完 FIFO 中的数据后，该位自动清零
0	RxReady	R	0	该位为 1 时表示接收到一个数据包，如果要继续 接收后面的数据，需要先将该位清零（设置 ClrRxReady）

9.3.13. EPx发送控制状态寄存器TxCSR（偏移：12h）

比特	名称	属性	复位值	描述
15	AutoSet	R/W	0	在写 FIFO 模式下，该位置 1 时，当写入发送 FIFO 中的数据数目达到 $PayLoad * (PayLoadCnt + 1)$ 时，TxReady 自动置位
14:12	RSV	-	-	保留
11	FrcDataTog	R/W	0	写 1 将强制 IN DATA 的 toggle 位切换
10:7	RSV	R/W	0	保留
6	ClrToggle	R/W	0	写 1 将使 DATA 包的序号值为 0
5	SentStall	R/W	0	该位为 1 时表示发送了 stall 握手信号
4	SendStall	R	0	写 1 将使控制器向 host 发送 stall 握手信号
3	FlushFifo	R/W	0	写 1 将清除发送 FIFO
2	RSV	-	-	保留
1	FifoNotEmpty	R/W	0	该位为 1 时表示发送 FIFO 中还有一个包的数据
0	TxReady	R/W	0	将该位设置为 1 时，表示可以开始发送数据了， 当控制器发送完 FIFO 中的数据后，该位自动清零

9.3.14. 最大接收数据包设置寄存器RXPSZR（偏移：14h）

比特	名称	属性	复位值	描述
15:11	PayLoadCnt	R/W	0	一个大数据包中的 PayLoad 数目，0 表示 1 个包， 1 表示 2 个包，7 表示 8 个包
10:0	PayLoad	R/W	0	USB 总线上单包的最大长度

注意：该寄存器用来配置大数据包传输时的单包大小和单包的数量，对于软件来说，如果待接收的数据长度为 4096 个 byte，Payload = 512，那么可以将 PayloadCnt 设置为 $4096/512 - 1 = 7$ ；当使能接收时：RxReady = 0；控制器会自动从 USB 总线上接收 4096 长度的数据，然后将接收完成标志置位：RxReady == 1；如果不使用大数据包传输功能，可将 PayloadCnt 配置为 0，之后每次使能接收时：TxReady = 0；控制器只接收 Payload 长度的数据就立即置位：RxReady == 1；

9.3.15. EPx接收控制状态寄存器RxCSR（偏移：16h）

比特	名称	属性	复位值	描述
15	AutoClr	R/W	0	该位置 1 时，如果从 FIFO 中读出的数据达到 Payload* (PayloadCnt+1) 时，RxReady 自动清零
14:13	RSV	-	-	保留
12	DisNYET	R/W	0	该位置 1 时，控制器将不回 NYET 握手包
11:8	RSV	-	-	保留
7	ClrToggle	R/W	0	写 1 将使 DATA 包的序号值为 0
6	SentStall	R/W	0	该位为 1 时表示发送了 stall 握手信号
5	SendStall	R/W	0	写 1 将使控制器向 host 发送 stall 握手信号
4	FlushFifo	R/W	0	写 1 将清除接收 FIFO，也会清掉 bit0 和 fifo counter
3-2	RSV	-	-	保留
1	FifoFull	R	0	该位为 1 时表示接收 FIFO 已满，bit0 可清除该位
0	RxReady	R/W	0	该位为 1 时表示接收到一个数据包，如果要继续接收后面的数据，需要先将该位清零

9.3.16. EP0 FIFO计数寄存器E0COUNTR（偏移：18h）

比特	名称	属性	复位值	描述
7-0	FIFO Count	R	0	记录 EP0 FIFO 中的数据数目（bytes）

9.3.17. EPx FIFO计数寄存器RXCOUNT（偏移：18h）

比特	名称	属性	复位值	描述
15:13	RSV	-	-	保留
12:0	FIFO Count	R	0	记录 EPx FIFO 中的数据数目（bytes）

注意：由索引寄存器决定显示哪个端点的 FIFO 中的数据，且当接收数据准备好标志置位后（RxReady = 1），该组寄存器中的值才有效。

9.3.18. EPx 发送FIFO大小TxFIFO_SIZE（偏移：1Ah）

比特	名称	属性	复位值	描述
7-4	RSV	-	-	保留
3-0	FIFO SIZE	R/W	0	FIFO 大小为： $2^{(SIZE + 3)}$

9.3.19. EPx 接收FIFO大小RxFIFO_SIZE（偏移：1Bh）

比特	名称	属性	复位值	描述
7-4	RSV	-	-	保留
3-0	FIFO SIZE	R/W	0	FIFO 大小为: $2^{(SIZE + 3)}$

9.3.20. EPx 发送FIFO地址TxFIFO_ADDR（偏移：1Ch）

比特	名称	属性	复位值	描述
15:13	RSV	-	-	保留
12:0	FIFO Address	R/W	0	Address[12:0]

注 1: Address[12:0]中配置的值实际 FIFO 地址的[15:3]位, FIFO 基地址需要在 RAM 中 8 字节对齐, 即, 地址的低 3 位总是零, 所以该寄存器中的值应该是实际 FIFO 地址的值右移 3 位的结果。同时, FIFO 占用的空间大小必须是 4 字节的整数倍, 比如发送 5 字节数据, 必须开辟 8 字节的数据空间。

注 2: EP0 端点 FIFO 地址固定指向系统 SRAM 的前 64 bytes 空间, 在 usb 应用中, 开发环境配置的 SRAM 起始地址必须偏置 0x40, 即 0x20000040。

9.3.21. EpX接收FIFO地址RxFIFO_ADDR（偏移：1Eh）

比特	名称	属性	复位值	描述
15:13	RSV	-	-	保留
12:0	FIFO Address	R/W	0	Address[12:0]

注 1: Address[12:0]中配置的值实际 FIFO 地址的[15:3]位, FIFO 需要在 RAM 中 8 字节对齐, 即, 地址的低 3 位总是零, 所以该寄存器中的值应该是实际 FIFO 地址的值右移 3 位的结果。同时, FIFO 占用的空间大小必须是 4 字节的整数倍, 比如接收 5 字节数据, 必须开辟 8 字节的数据空间。

注 2: EP0 端点 FIFO 地址固定指向系统 SRAM 的前 64 bytes 空间, 在 usb 应用中, 开发环境配置的 SRAM 起始地址必须偏置 0x40, 即 0x20000040。

9.3.22. EP0 fifo通道Ep0FIFO（偏移：20h）

比特	名称	属性	复位值	描述
31:8	RSV	-	-	保留
7:0	FIFO	R/W	0	可读该寄存器来获取 fifo 中的数据

9.3.23. EP1 fifo通道Ep1FIFO（偏移：24h）

比特	名称	属性	复位值	描述
31:8	RSV	-	-	保留

7:0	FIFO	R/W	0	可读该寄存器来获取 fifo 中的数据
-----	------	-----	---	---------------------

9.3.24. EP2 fifo通道Ep2FIFO（偏移：28h）

比特	名称	属性	复位值	描述
31:8	RSV	-	-	保留
7:0	FIFO	R/W	0	可读该寄存器来获取 fifo 中的数据

9.3.25. EP3 fifo通道Ep3FIFO（偏移：2Ch）

比特	名称	属性	复位值	描述
31:8	RSV	-	-	保留
7:0	FIFO	R/W	0	可读该寄存器来获取 fifo 中的数据

9.3.26. EP4 fifo通道Ep4FIFO（偏移：30h）

比特	名称	属性	复位值	描述
31:8	RSV	-	-	保留
7:0	FIFO	R/W	0	可读该寄存器来获取 fifo 中的数据

9.3.27. EPx发送数据长度寄存器 TX_PTR（偏移：40h）

比特	名称	属性	复位值	描述
16-13	RSV	-	-	保留
12-0	SendSize	R/W	0	设置待发送数据的长度,动态分配 FIFO 地址时需要用到该功能

9.4. 使用流程

9.4.1. 控制传输

1. EP0 中断源

控制传输分为三个阶段，Setup 包阶段、数据阶段、状态阶段。对于某些控制请求，可能没有数据阶段。USB 控制器在三个阶段均能产生 EP0 中断，所以在处理 EP0 中断时，要分清楚产生中断的源。收到 Setup 包时，控制器会产生 EP0 中断，中断源为 RxReady；在数据阶段接收到主机发送的 DATA 包时，控制器会产生 EP0 中断，中断源为 RxReady；在数据阶段控制器发送完 DATA 包时，控制器会产生 EP0 中断，中断源为 TxReady；在状态阶段，控制器发送完空包，或者接收到空包后产生 EP0 中断

2. 无 DATA 阶段的控制传输

对于没有数据阶段的控制传输：例如 SetAddress、SetConfigure 等，处理完 Setup 包后应该直接进入状态阶段,此时需要同时将 RxReady 清零和 DataEnd 置位,硬件才能识别到应该进入状态阶段,如果先清零 RxReady 而后置为 DataEnd,将有可能导致控制器产生控制传输异常。退出当前控制传输，置位 SetupEnd。

3. SetAddresses 请求的处理

设备地址寄存器 FADDR 可以在收到 SetAddress 请求之后就立刻更新，控制器硬件会在当前（SetAddress 请求）控制传输结束后才会使用更新后的地址，即 SetAddress 请求的状态阶段仍然使用旧地址，等状态阶段完成后下一个请求使用新地址。如果 FADDR 写的太慢，有可能在下一个 setup 包到来时，设备地址没有来得及更新到新的地址，导致控制器接收不到下一个 setup 包。

9.4.2. 大数据包传输（split功能）

1. 通过读写 FIFO 来传递数据

- 接收数据

待接收数据长度为 Len，如果 Len 为 Payload 的整数倍，那么 $\text{PayloadCnt} = (\text{Len}/\text{Payload}) - 1$ ；当 RxReady 置位时，那么 FIFO 中就接收到了 Len 长度的数据。从 FIFO 中直接读出数据即可；如果 Len 不是 Payload 整数倍，那么先设置 $\text{PayloadCnt} = (\text{Len}/\text{Payload}) - 1$ ；等待 RxReady 置位，从 FIFO 中读出 $(\text{PayloadCnt} + 1) * \text{Payload}$ 长度的数据，然后接收最后剩下的数据时， $\text{PayloadCnt} = 0$ ；待 RxReady 置位时，从 FIFO 中读出剩下的数据。

PayloadCnt 的设置要考虑到实际 FIFO 的大小，如果 FIFO 不够大，那么需要减小 PayloadCnt 的值，分多次完成数据接收。

- 发送数据

待发送数据长度为 $\text{Len} \leq 512$ ， $\text{PayloadCnt} = 0$ ， $\text{Payload} = \text{Len}$ ；向 FIFO 中写入 Len 长度的数据，置位 TxReady，当 TxReady 清零时，那么 FIFO 中 Len 长度的数据就已经发送出去了。

2. 通过动态分配 FIFO 首地址来传递数据

- 接收数据

在 RAM 中给接收 FIFO 分配一个首地址，待接收数据长度为 Len，如果 Len 为 Payload 的整数倍，那么 $\text{PayloadCnt} = \text{Len}/\text{Payload} - 1$ ；当 RxReady 置位时，那么 FIFO 中就接收到了 Len 长度的数据。从 FIFO 中直接读出数据即可；如果 Len 不是 Payload 整数倍，那么先设置 $\text{PayloadCnt} = \text{Len}/\text{Payload} - 1$ ；等待 RxReady 置位，从 FIFO 中读出 $(\text{PayloadCnt} + 1) * \text{Payload}$ 长度的数据，然后接收最后剩下的数据时， $\text{PayloadCnt} = 0$ ；待 RxReady 置位时，从 FIFO 中读出剩下的数据。

PayloadCnt 的设置要考虑到实际 FIFO 的大小，如果 FIFO 不够大，那么需要减小 PayloadCnt 的值，分多次完成数据接收。

- 发送数据

在 RAM 中给发送 FIFO 分配一个首地址，将待发送的数据长度填入寄存器 TX_PTR，然后设置 TxPktRdy=1 来启动发送，发送完成后该位会自动清 0。

10. NFM

10.1.概述

NFM 实现将系统 CPU 总线转换成符合外部 NAND Flash 存储器协议的写地址、写命令、写数据、读数据等基本操作。

10.2.主要特性

- 标准的 SDR 接口，符合 ONFi 标准。
- SDR 接口时序可配置，支持多种型号的 NAND Flash 存储器。
- 支持 4 个片选信号
- 大/小端数据可配置。
- NFM 接口支持 8 比特。
- BCH 算法。
- 硬件即时编/译码。
- 每页可纠正 8 个比特的错误（512 字节数据+3 字节信息位+13 字节 ECC 校验位）
- 支持流水线模式。
- 支持自动纠错。

10.3.寄存器描述

NFM 寄存器基地址：0x40040000

偏置	名称	描述
0x00	NFMCTRL	NFM 控制寄存器
0x04	NFMWST	NFM 等待寄存器
0x08	NFMSTATUS	NFM 状态寄存器
0x10	BCH_CONFIG	BCH 配置寄存器
0x14	BCH_CTRL	BCH 控制寄存器
0x18	BCH_STATUS	BCH 状态寄存器
0x1c	BCH_CODE	BCH 校验码寄存器
0x20	BCH_ERRADR	BCH 错误地址寄存器
0x24	BCH_ERRVEC	BCH 错误向量寄存器
0x28	BCH_BASEADDR	BCH SRAM 基地址寄存器
0x2C	BCH_CODEPTR	BCH 校验码指针寄存器

0x30	BCH_ERRADDRPTR	BCH 错误地址指针寄存器
0x34	BCH_ERRVECPTTR	BCH 错误向量指针寄存器
0x38	BCH_PAGENUM	BCH 页计数器。
0x3C	BCH_ADDRLATCH	BCH 地址锁存寄存器
0x40	NFMCMD	NFM 命令通道寄存器
0x44	NFMADDR	NFM 地址通道寄存器
0x48	NFMECCDATA	NFM ECC 数据通道寄存器
0x52	NFMNECCDATA	NFM 非 ECC 数据通道寄存器

10.3.1. NFM控制寄存器NFMCTRL（偏移：00h）

比特	名称	属性	复位值	描述
31: 8	RSV	-	-	保留
7	EDO_EN	RW	0	EDO 功能使能 0: 不使能 EDO 功能 1: 使能 EDO 功能
6	RBNINTEN	RW	0	RBN 管脚上升沿状态中断使能 0: POS_RBN 中断不使能 1: POS_RBN 中断使能
5	ENDIAN	RW	0	NFM 数据大小端模式选择 1: 大端模式 0: 小端模式
4	FWP	RW	0	写保护使能。 1: FWP 是 1, 禁止写保护。 0: FWP 是 0, 使能写保护。
3: 0	FCE	RW	4'b1111	FCE 管脚寄存器。 写该比特位将决定 FCE 管脚的输出数据。

10.3.2. NFM等待寄存器NFMWST(偏移：04h)

比特	名称	属性	复位值	描述
31: 28	RSV	-	-	保留
27: 24	t _{ADL}	RW	0	ALE to data out start.CLK Cycle 0: 1 个 CLK 1: 2 个 CLK 15: 16 个 CLK。
23: 20	t _{RHW}	RW	0	REN High to WEN Low CLK Cycle 0: 1 个 CLK 1: 2 个 CLK

				15: 16 个 CLK。
19: 16	t_{WHR}	RW	0	WEN High to REN Low CLK Cycle 0: 1 个 CLK 1: 2 个 CLK 15: 16 个 CLK。
15: 12	t_{REH}	RW	0	REN High hold time CLK Cycle 0: 1 个 CLK 1: 2 个 CLK 15: 16 个 CLK。
11: 8	t_{RP}	RW	0	REN Pulse width CLK Cycle 0: 1 个 CLK 1: 2 个 CLK 15: 16 个 CLK。
7: 4	t_{WH}	RW	0	WEN High hold time CLK Cycle 0: 1 个 CLK 1: 2 个 CLK 15: 16 个 CLK。
3: 0	t_{WP}	RW	0	WEN pulse width CLK Cycle 0: 1 个 CLK 1: 2 个 CLK 15: 16 个 CLK。

注：1.上述 CLK 个数为系统时钟 HCLK

2.时序上还需要满足：

普通模式： $t_{RP} > t_{REA} + 8ns$ （8ns 为芯片 IO 时序的延迟）

EDO 模式： $t_{RP} + t_{REH} > t_{REA} + 8ns$ （8ns 为芯片 IO 时序的延迟）

10.3.3. NFM状态寄存器NFMSTATUS（偏移：08h）

比特	名称	属性	复位值	描述
31: 1	RSV	-	-	保留
7:4	POS_RBN	RO	0	RBN 管脚上升沿状态。 1: 发生上升沿事件，写 1 清除该标志位。 0: 未发生。
3:0	RBN	RO	0	RBN 管脚状态位，读此位反应 RBN 管脚的状态。

				0 = NAND flash 存储器忙 1 = NAND flash 存储器准备好
--	--	--	--	--

10.3.4. BCH配置寄存器BCH_CONFIG（偏移：10h）

比特	名称	属性	复位值	描述
31:3	RSV	-	-	保留
2: 0	BCH_TYPE	RW	0	BCH 类型选择位。 000 : 512 字节数据+3 字节信息位+13 字节 ECC 校验位，可纠错 8 比特。 其它：保留位。

10.3.5. BCH控制寄存器BCH_CTRL（偏移：14h）

比特	名称	属性	复位值	描述
31: 6	RSV	-	-	保留
5	Auto_Correct	RW	0	自动纠错使能位。当使能自动纠错功能时，将自动启用流水线模式。手动模式时，不会启用流水线模式。 1: 自动纠错模式。 0: 手动纠错模式。
4	decode_int_enable	RW	0	译码中断使能。 1: 使能译码完成中断。 0: 禁止译码完成中断。
3	RESET_CHANNEL	RW	0	复位 ECC 数据通道。 向 RESET_HAND 位写 1，将复位 ECC 数据通道。读该比特总是返回 0。 1: 复位 ECC 数据通道。 0: 无效。
2	RSV	-	-	保留
1	IGALL1	RW	0	忽略全 1 译码使能。IGALL1 比特位决定当数据全是“FF”时，ECC 模块译码的结果。IGALL1 设置为 1 时，译码错误标志位将不会置位。 1: 当数据全是“FF”时，忽略 ECC 模块译码的结果。 0: 当数据全是“FF”时，不忽略模块译码的结果。
0	Mode	RW	0	BCH 模式控制位。 1: BCH 译码。 0: BCH 编码。

10.3.6. BCH状态寄存器BCH_STATUS(偏移：18h)

比特	名称	属性	复位值	描述
31: 24	CEN	RO	0	校验码错误比特数。仅在 DECODE_DONE 标志位置位时，该比特数据有效。
23: 16	DEN	RO	0	数据错误比特数。仅在 DECODE_DONE 标志位置位时，该比特数据有效。

15: 8	ERN	RO	0	错误比特总数。仅在 DECODE_DONE 标志位置位时, 该比特数据有效。
7				保留位
6	DATA_ALL0	RO	0	数据全 0 标志位。译码时读出的数据全 0 时, 该比特位置位。该标志位置位后, 下次译码完成后会自动清零。 1: 数据全 0。 0: 数据不是全 0。
5	DATA_ALL1	RO	0	数据全 1 标志位。译码时读出的数据全 1 时, 该比特位置位。该标志位置位后, 下次译码完成后会自动清零。 1: 数据全 1。 0: 数据不是全 1。
4	BCH_FAIL	RO	0	BCH 错误标志位。在译码阶段, 当发生超过 BCH 纠错的时候, 该比特置位。 1: BCH 错误。写 1 清除该标志位。 0: BCH 没有错误。
3	CORRECT_DONE	RO	0	译码阶段纠错完成。 1: 纠错完成, 写 1 清除该标志位。 0: 纠错没有完成。
2	BM_DONE	RO	0	译码阶段错误多项式求解完成。 1: 错误多项式求解完成。写 1 清除该标志位。 0: 错误多项式求解没有完成。
1	SNYDRONE_DONE	RO	0	译码阶段半随机式求解完成。 1: 半随式求解完成, 写 1 清除该标志位。 0: 半随式求解没有完成。
0	ENCODE_CLR	WC	0	编码器可以即时地对数据进行编码, 输入数据结束后, 编码立即完成, 不需要判断状态。向该位写 1 可以清除校验码寄存器, 写 0 无效。开始新的编码前需要向该位写 1。读该比特总是返回 0。

10.3.7. BCH校验码寄存器BCH_CODE (偏移: 1Ch)

比特	名称	属性	复位值	描述
31: 8	RSV	-	-	保留
7: 0	BCH_CODE	RO	0	BCH 校验码, FIFO 接口。

10.3.8. BCH错误地址寄存器BCH_ERRADR (偏移: 20h)

比特	名称	属性	复位值	描述
31: 8	RSV	-	-	保留
7: 0	BCH_ERRADR	RO	0	BCH 错误地址, FIFO 接口。

10.3.9. BCH错误向量寄存器BCH_ERRVEC (偏移: 24h)

比特	名称	属性	复位值	描述
31: 8	RSV	-	-	保留

7: 0	BCH_ERRVEC	RO	0	BCH 错误向量, FIFO 接口。
------	------------	----	---	--------------------

10.3.10. BCH纠错基地址寄存器BCH_BASEADDR(偏移: 28h)

比特	名称	属性	复位值	描述
31: 0	BCH_BASEADDR0	RW	0	SRAM 基地址 0

10.3.11. BCH校验码指针寄存器BCH_CODEPTR (偏移: 2Ch)

比特	名称	属性	复位值	描述
31: 4	RSV	-	-	保留
3: 0	CODE_PTR	RW	0	BCH 校验位指针寄存器。

10.3.12. BCH错误地址指针寄存器BCH_ERRADDRPTR (偏移: 30h)

比特	名称	属性	复位值	描述
31: 6	RSV	-	-	保留
5: 0	ADDR_PTR	RW	0	BCH 错误地址指针寄存器。

10.3.13. BCH错误向量指针寄存器BCH_ERRVECPTR (偏移: 34h)

比特	名称	属性	复位值	描述
31: 6	RSV	-	-	保留
5: 0	VECTOR_PTR	RW	0	BCH 错误向量指针寄存器。

10.3.14. BCH页记数寄存器BCH_PAGENUM (偏移 38h)

比特	名称	属性	复位值	描述
31: 8	RSV	-	-	保留
7: 0	Page_NUM	RW	0	页记数寄存器。

10.3.15. BCH地址锁存寄存器BCH_ADDRLATCH (偏移: 3Ch)

比特	名称	属性	复位值	描述
7: 0	BCH_ADDRLATCH	RO	0	BCH 纠错地址锁存寄存器。在自动纠错功能打开模式下, 开始计算 SIBM 时锁存住该页的基地址, 当纠错完成后, 自动清零。

10.3.16. NFM命令通道寄存器NFMCMD (偏移: 40h)

比特	名称	属性	复位值	描述
31: 8	RSV	-	-	保留
7: 0	NFMCMD	WO	0	NFM 命令通道寄存器

10.3.17. NFM地址通道寄存器NFMADDR (偏移: 44h)

比特	名称	属性	复位值	描述
31: 8	RSV	-	-	保留
7: 0	NFMADDR	WO	0	NFM 地址通道寄存器

10.3.18. NFM ECC数据通道寄存器NFMECCDATA (偏移: 48h)

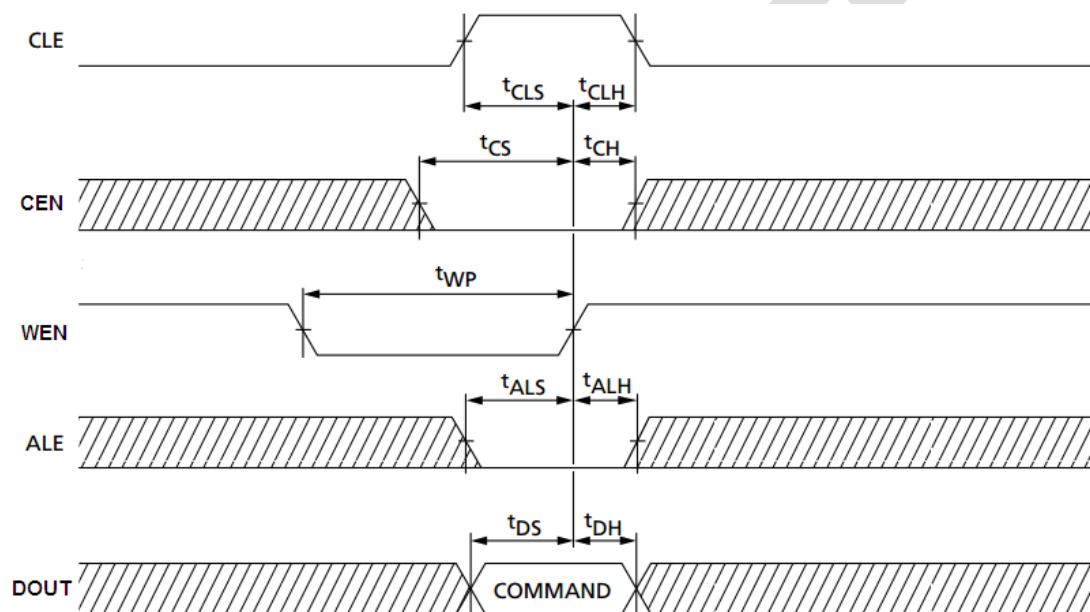
比特	名称	属性	复位值	描述
31: 0	NFMECCDATA	RW	0	NFM ECC 数据通道寄存器 此寄存器可以按照字、半字和字节访问, 具体选择参见表 10-1

10.3.19. NFM非ECC数据通道寄存器NFMNECCDATA（偏移：4Ch）

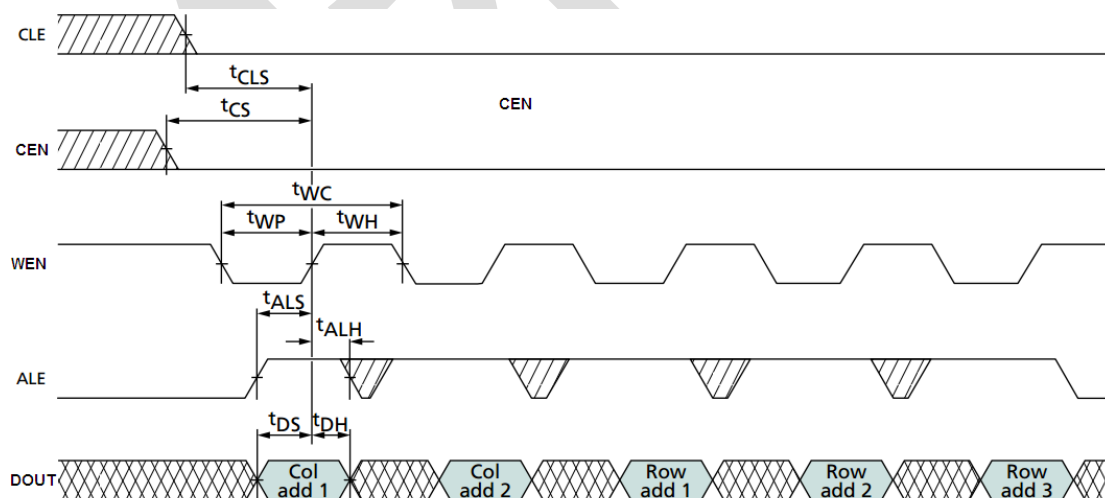
比特	名称	属性	复位值	描述
31:0	NFMNECCDATA	RW	0	NFM 非 ECC 数据通道寄存器 此寄存器可以按照字、半字和字节访问， 具体选择参见表 10-1

10.4.使用流程和注意事项

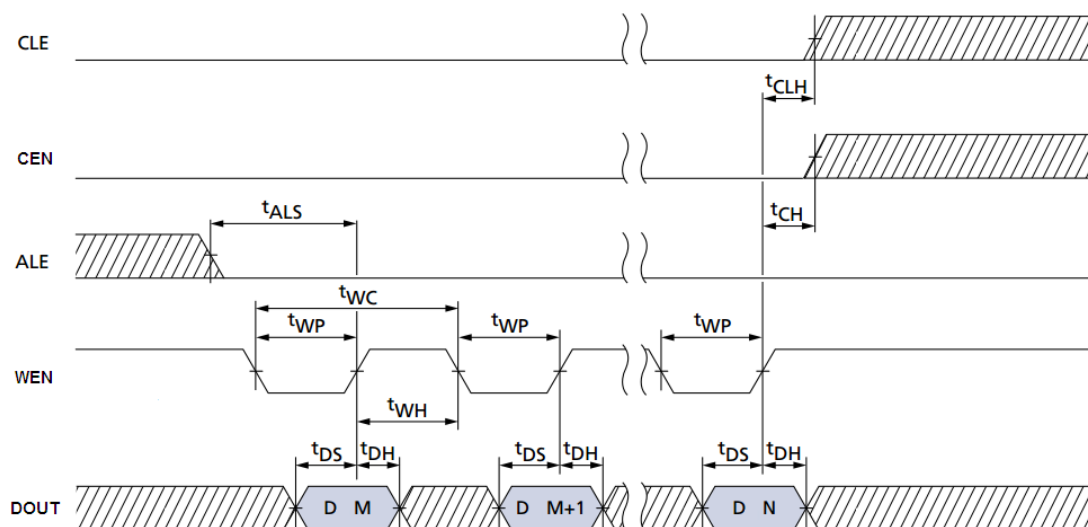
10.4.1. 写命令（COMMAND）时序



10.4.2. 写地址（ADDRESS）时序

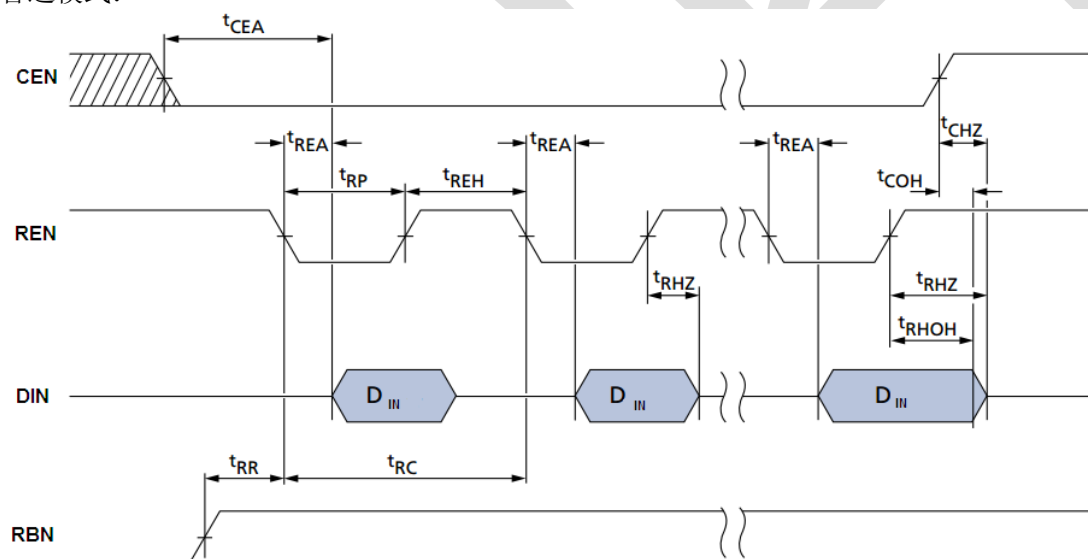


10.4.3. 写数据（DATA）时序

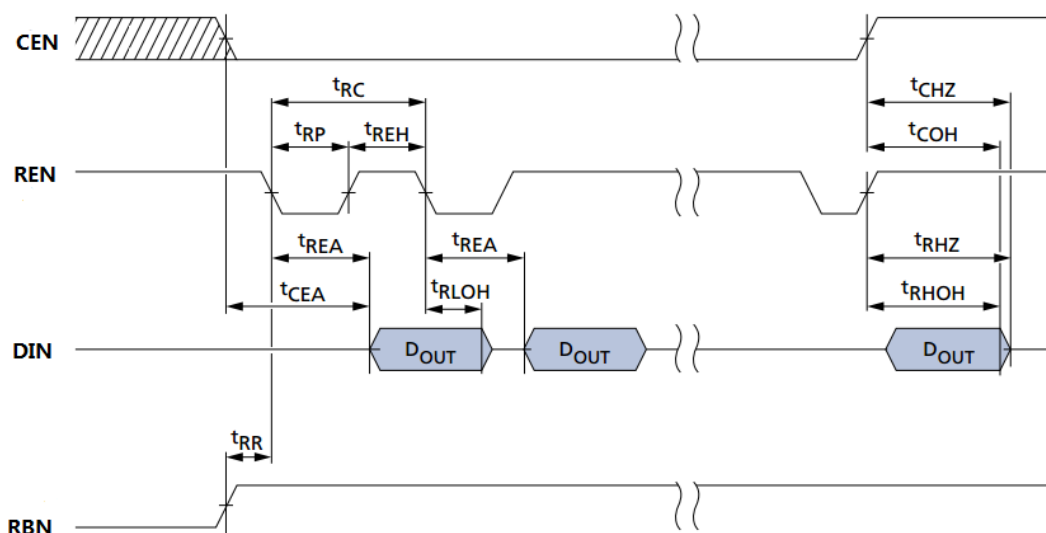


10.4.4. 读数据（DATA）时序

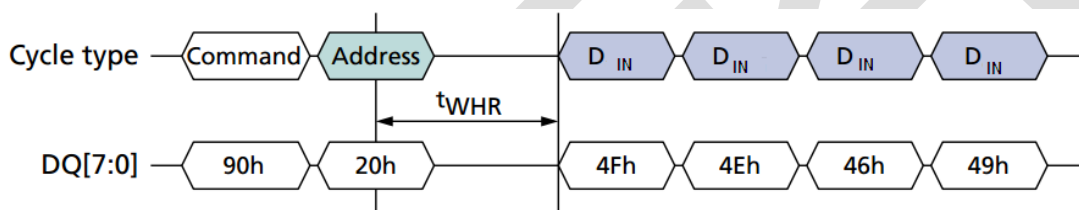
普通模式:



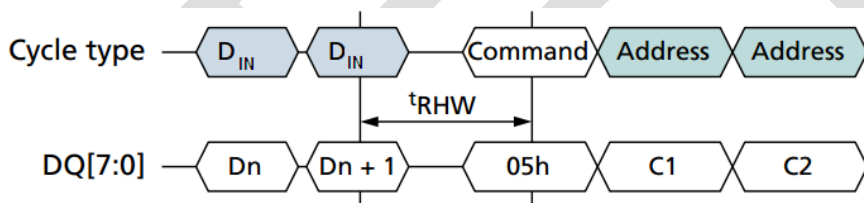
EDO 模式:



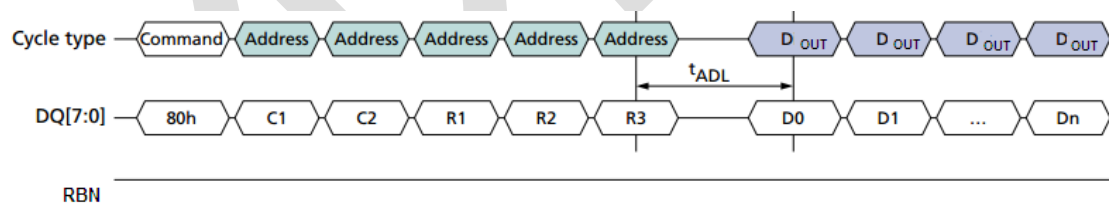
10.4.5. t_{WHR} 时序(WEN High to REN Low)



10.4.6. t_{RHW} 时序(REN High to WEN Low)



10.4.7. t_{ADL} 时序(ALE to data out start)



10.4.8. BCH编码流程

1. 设置 BCH 配置寄存器 BCH_CONFIG
2. 设置 BCH 控制寄存器 BCH_CTRL 中的 Mode 位，将 BCH 设置为编码模式。
3. 设置 BCH 状态寄存器 BCH_STATUS 中的 ENCODE_CLR 位，复位 BCH 编码通道。
4. 通过 ECC 通道写入 BCH 算法规定的的数据。
5. 读 BCH 校验码寄存器。

10.4.9. BCH译码自动纠错模式流程

1. 设置 BCH 配置寄存器 BCH_CONFIG
2. 设置 BCH 控制寄存器 BCH_CTRL 中的 Mode 位，将 BCH 设置为译码模式。
3. 设置 BCH 控制寄存器 BCH_CTRL 中的 Auto_Correct 位，使能自动纠错功能。
4. 设置 BCH 页计数寄存器 BCH_PAGENUM 为 0。
5. 设置 BCH 纠错基地址寄存器 BCH_BASEADDR。
6. 通过 ECC 通道读出 BCH 算法规定的的数据。
7. 等待 SNYDROME_DONE 置位，置位后清零。
8. 重复步骤 5，直到数据全部读出。
9. 根据 BCH_PAGENUM 判断纠错是否完成。

10.4.10. BCH译码手动纠错模式流程

1. 设置 BCH 配置寄存器 BCH_CONFIG
2. 设置 BCH 控制寄存器 BCH_CTRL 中的 Mode 位，将 BCH 设置为译码模式。
3. 设置 BCH 控制寄存器 BCH_CTRL 中的 Auto_Correct 位，使能手动纠错功能。
4. 通过 ECC 通道读出 BCH 算法规定的的数据。
5. 等待 CORRECT_DONE 置位。
6. 读出 BCH 状态寄存器 BCH_STATUS 中的 ERN 位，得出错误的比特数。
7. 将错误地址指针寄存器 BCH_ERRADDRPTR 指向的原数据与错误向量异或得到正确的数据。

10.4.11. 数据通道访问配置

表 10-1 数据通道访问配置

总线类型	ENDIAN	AHBD[31:24]	AHBD[23:16]	AHBD[15:8]	AHBD[7:0]
Byte	0 或 1	-	-	-	FIO[7:0]
Half-Word 拆成两次 Flash 操作	0（小端）	-	-	-	FIO[7:0](1)
		-	-	FIO[7:0](2)	-
	1（大端）	-	-	FIO[7:0](1)	-
		-	-	-	FIO[7:0](2)
Word 拆成四次 Flash 操作	0（小端）	-	-	-	FIO[7:0](1)
		-	-	FIO[7:0](2)	-
		-	FIO[7:0](3)	-	-
		FIO[7:0](4)	-	-	-
	1（大端）	FIO[7:0](1)	-	-	-
		-	FIO[7:0](2)	-	-

		-	-	FIO[7:0](3)	-
		-	-	-	FIO[7:0](4)

注：FIO 指的是 NAND FLASH 端口数据，AHBD 指的是芯片内部总线数据

保 密

11. SDIO

11.1.概述

SDIO controller 作为数据传输接口, 支持 SD 和 eMMC 标准协议, 可作为 SD 卡读卡器和 EMMC 卡读卡器的 master 控制器

11.2.主要特性

- 支持 SD 和 eMMC4.4.1 协议
- 内置 FIFO, 宽度为 32bit, 深度为 16, 支持当 over-run 和 under-run 时停止时钟
- 支持 CRC 产生和校验
- 支持可编程的波特率。支持最多四种时钟分配率以满足在不同波特率下通信的要求
- 支持时钟控制开关
- 支持卡检测
- 支持卡写保护
- 支持 1bit,4bit,8bit 模式的 SDIO 中断
- 支持 block size 从 1 到 65536
- 支持 DMA 传输
- 最高 50M 接口时钟频率

11.3.寄存器描述

SDIO 寄存器基地址: 0x40050000。

偏置	名称	描述
0x00	SDMMC_CTRL	控制寄存器
0x04	SDMMC_POWEREN	电源开关寄存器
0x08	SDMMC_CLKDIV	时钟分频率寄存器
0x0C	SDMMC_CLKSRC	时钟来源寄存器
0x10	SDMMC_CLKENA	时钟使能寄存器
0x14	SDMMC_TIMEOUT	超时寄存器
0x18	SDMMC_WIDTH	位宽寄存器
0x1C	SDMMC_BLKSIZE	Block size寄存器
0x20	SDMMC_BYTCNT	字节个数寄存器
0x24	SDMMC_INTMASK	中断屏蔽寄存器
0x28	SDMMC_CMDARG	命令参数寄存器

0x2C	SDMMC_CMD	命令寄存器
0x30	SDMMC_RESP0	应答0寄存器
0x34	SDMMC_RESP1	应答1寄存器
0x38	SDMMC_RESP2	应答2寄存器
0x3C	SDMMC_RESP3	应答3寄存器
0x40	SDMMC_MINTSTS	可被屏蔽中断状态寄存器
0x44	SDMMC_RINTSTS	原始中断状态寄存器
0x48	SDMMC_STATUS	状态寄存器
0x4C	SDMMC_FIFOTH	FIFO比较寄存器
0x50	SDMMC_CDETECT	卡检测寄存器
0x54	SDMMC_WRTprt	写保护寄存器
0x5C	SDMMC_TCBCNT	TCBCNT寄存器
0x60	SDMMC_TBBCNT	TBBCNT寄存器
0x64	SDMMC_DEBOUNCE	DEBOUNCE寄存器
0x100	SDMMC_DATA	FIFO通道访问寄存器

11.3.1. 控制寄存器SDMMC_CTRL(偏移: 00h)

比特	名称	属性	复位值	描述
31:6	RSV	-	-	保留位。此位必须为 0
24	od_pullup_en	R/W	0	开漏上拉使能 0: 没有开漏上拉 1: 有开漏上拉 当为1时, 输出command在开漏模式下, 输出强0弱1 (输出1时为高阻)。
23:9	RSV	-	-	保留位。此位必须为 0
8	abort_read_data	R/W	0	0: 没有变化 1: 如果在读数据过程中发送了suspend命令后, 软件要轮询每张卡以确定发生suspend的时间。一旦suspend发生, 软件将此位置1以reset数据状态机 (此前数据状态机在等待下一个data block)。数据状态机reset后, 此位自动清0
7	send_irq_resp	R/W	0	0: 没有变化 1: 自动发送IRQ应答 当发送应答后该位自动清0。 为了等待MMC卡中断, host发送CMD40。如果host希望退出等待中断状态, 就可以把该位置1, 这样CMD状态机就会退出等待中断状态, 回到idle状态。
6	read_wait	R/W	0	0: 没有变化 1: 发送read_wait给sdio卡
5	dma_en	R/W	0	0: 关闭dma 1: 使能dma

				如果同时有FIFO push来自CIU, DMA, host, 优先级顺序为CIU, DMA, host。
4	int_en	R/W	0	中断使能 0: 关闭中断 1: 使能中断
3	RSV	-	-	保留位。此位必须为 0
2	dma_rst	R/W	0	dma reset: 0: 无影响 1: dma reset 在dma reset之后, 该位自动清0
1	fifo_rst	R/W	0	fifo reset: 0: 无影响 1: fifo reset 在fifo reset之后, 该位自动清0
0	host_rst	R/W	0	host reset: 0: 无影响 1: hostreset 在host reset之后, 该位自动清0

11.3.2. 电源开关寄存器SDMMC_POWER(偏移: 04h)

比特	名称	属性	复位值	描述
31:1	RSV	-	-	保留位
0	pwr_en	R/W	0	控制卡的电源。一旦打开卡的电源, 固件要等regulator/开关把电压升上去后, 再对卡进行初始化。 0: 电源关闭 1: 电源打开

11.3.3. 时钟分频率寄存器SDMMC_CLKDIV(偏移: 08h)

比特	名称	属性	复位值	描述
31:24	clk_div3	R/W	0	31到24位: clock3的分频率为 2^n 。比如, clk_div3=0, 分频率为 $2^0=1$, 没有分频, 输出card时钟为原频率; clk_div3=1, 分频率为 $2^1=2$, 分频率为2, 输出 card 时钟为原频率的一半, 依次类推。
23:16	clk_div2	R/W	0	23到16位: clock2的分频率为 2^n 。比如, clk_div2=0, 分频率为 $2^0=1$, 没有分频, 输出card时钟为原频率; clk_div2=1, 分频率为 $2^1=2$, 分频率为2, 输出 card 时钟为原频率的一半, 依次类推。
15:8	clk_div1	R/W	0	15到8位: clock1的分频率为 2^n 。比如, clk_div1=0, 分频率为 $2^0=1$, 没有分频, 输出card时钟为原频率; clk_div1=1, 分频率为 $2^1=2$, 分频率为2, 输出 card 时钟为原频率的一半, 依次类推。
7:0	clk_div0	R/W	0	7 到0 位: clock0的分频率为 2^n 。比如, clk_div0=0, 分频率为 $2^0=1$, 没有分频, 输出card时钟为原频率; clk_div0=1, 分频率为 $2^1=2$, 分频率为2, 输出 card 时钟为原频率的一半, 依次类推。

11.3.4. 时钟来源寄存器SDMMC_CLKSRC(偏移: 0Ch)

比特	名称	属性	复位值	描述
31:2	RSV	-	-	保留位
1:0	clk_source0	R/W	0	卡的clock来源 00: clk_div0 01: clk_div1 10: clk_div2 11: clk_div3

11.3.5. 时钟使能寄存器SDMMC_CLKENA(偏移: 10h)

比特	名称	属性	复位值	描述
31:17	RSV	-	-	保留位
16	clk_low_power	R/W	0	低功耗控制 0: 非低功耗模式 1: 低功耗模式。当卡处于idle状态时, 停止host 时钟输出。
15:1	RSV	-	-	保留位
0	clk_en	R/W	0	卡对应的时钟输出使能 0: disable 时钟 1: 使能时钟

11.3.6. 超时寄存器SDMMC_TIMEOUT(偏移: 14h)

比特	名称	属性	复位值	描述
31:8	数据超时	R/W	0xFFFFFFFF	31 到8 位: 读写card时, 读入数据或者写出数据超时时间(时钟的个数)。
7:0	应答超时	R/W	0x40	7 到0 位: 应答的超时时间(时钟的个数)。

11.3.7. 位宽寄存器SDMMC_WIDTH(偏移: 18h)

比特	名称	属性	复位值	描述
31:17	RSV	-	-	保留位
16	width1	R/W	2'b0	0:非8位模式 1:8位模式
15:1	RSV	-	-	保留位
0	width0	R/W	2'b0	0:1位模式 1:4位模式

注意, width1 寄存器比 width0 寄存器优先级高。当同为 1 时, controller 处于 8 位模式。

11.3.8. Block size寄存器SDMMC_BLKSIZE(偏移: 1Ch)

比特	名称	属性	复位值	描述
31:16	RSV	-	-	保留位

15:0	blk_size	R/W	512	15 到0 位：一个block所包含的字节个数
------	----------	-----	-----	-------------------------

11.3.9. 字节个数寄存器SDMMC_BYTCNT(偏移：20h)

比特	名称	属性	复位值	描述
31:0	byte_cnt	R/W	0x200	待传输数据的byte个数。byte_cnt应该是blk_size的整数倍。 byte_cnt为0表示未定义传输数据的数量，这时候应该由host发送stop命令来停止数据传输。

11.3.10. 中断屏蔽寄存器SDMMC_INTMASK(偏移：24h)

比特	名称	属性	复位值	描述
31:17	RSV	-	-	保留位
16	sdio_interrupt	R/W	0	屏蔽SDIO卡中断，屏蔽中断。将相应位清0表示屏蔽中断，置1表示使能中断。
15:0	int_mask	R/W	0	屏蔽各个位的中断，将相应位清0表示屏蔽中断，置1表示使能中断。 15: 最终位错误/无CRC/CRC错误 14: 自动结束命令 13: 起始位错误 12: 改写锁寄存器错误 11: FIFO under-run/over-run 10: 数据缺失超时中断 9: 读数据超时 8: 应答超时 7: 数据CRC错误 6: 应答CRC错误 5: 接受数据FIFO请求 4: 发送数据FIFO请求 3: 数据传输完成 2: 命令结束 1: 应答错误 0: 卡检测

11.3.11. 命令参数寄存器SDMMC_CMDARG(偏移：28h)

比特	名称	属性	复位值	描述
31:0	cmd_arg	R/W	0000	命令参数

11.3.12. 命令寄存器SDMMC_CMD(偏移：2Ch)

比特	名称	属性	复位值	描述
31	start_cmd	R/W	0	Start 命令。当start_cmd置1时，host不应该尝试写任何命令寄存器，如果写了的话，raw interrupt寄存器中的hardware lock error interrupt被置1。当命令start以后，该位被清0。

30:22	RSV	-	-	保留位。此位必须为 0
21	update_clk	R/W	0	0: 正常的命令序列 1: 不发送命令, 仅仅更新时钟相关寄存器的值 时钟相关寄存器包括以下寄存器: CLKDIV, CLRSRC, CLKEN。
20:17	RSV	-	-	保留位。此位必须为 0
16	card_num	R/W	0	选择card number。
15	send_ini_seq	R/W	0	0: 在发送命令之前不发送初始化序列 (80个card 时钟)。 1: 在发送命令之前发送初始化序列 (80个card 时钟)。 在power on之后, 给card发送任何命令之前应该先发送80个时钟。
14	stop_abort_cmd	R/W	0	在发送stop命令 (cmd12) 时, 需要把该位置1; 如果在数据传输过程中要发送reset命令 (CMD0, CMD15, CMD52_reset), 必须把该位置1, 这样就会在命令发送完毕之后停止数据传输。
13	wait_prv_data_finish	R/W	0	0: 立刻发送命令, 即使此前的数据传输尚未完成。 1: 等待此前的数据传输完成后再发送命令。这里有两种情况: 1) 如果此前数据传输已经完成或是字节个数寄存器为0的传输, 那么就马上发送命令; 2) 字节个数寄存器不为0, 那么等待传输完毕后再发送command。
12	send_auto_stop	R/W	0	0: 数据传输完成后不发送stop 命令 1: 数据传输完成后发送stop 命令
11	transfer_mode	R/W	0	0: block数据传输命令 1: stream数据传输命令 当没有数据传输时, 该位无作用。
10	read/write	R/W	0	0: 从卡读数据 1: 向卡写数据
9	data_transfer_expected	R/W	0	0: 不期望与卡数据传输 1: 期望与卡数据传输
8	check_rep_crc	R/W	0	0: 不检查应答 CRC 1: 检查应答 CRC
7	rep_long	R/W	0	0: 期望卡应答为48位 1: 期望卡应答为136位
6	rep_expect	R/W	0	0: 不期望得到应答 1: 期望得到应答
5:0	cmd_index	R/W	0	命令 index

11.3.13. 应答 0 寄存器SDMMC_RESP0(偏移: 30h)

比特	名称	属性	复位值	描述
31:0	应答 0	R	0000	应答的[31:0]位

11.3.14. 应答 1 寄存器SDMMC_RESP1 (偏移: 34h)

比特	名称	属性	复位值	描述
31:0	应答 1	R	0000	应答的[63:32]位

11.3.15. 应答 2 寄存器SDMMC_RESP2 (偏移: 38h)

比特	名称	属性	复位值	描述
31:0	应答 2	R	0000	应答的[95:64]位

11.3.16. 应答 3 寄存器SDMMC_RESP3(偏移: 3Ch)

比特	名称	属性	复位值	描述
31:0	应答 3	R	0000	应答的[127:96]位

11.3.17. 可被屏蔽中断状态寄存器SDMMC_MINTSTS(偏移: 40h)

比特	名称	属性	复位值	描述
31:17	RSV	-	-	保留位。
16	sdio_interrupt	R	0	SDIO卡中断，表示卡产生了中断。只有当对应位没有被中断屏蔽寄存器屏蔽时，中断位才有效。
15:0	int_status	R	00	15: 最终位错误/无 CRC/CRC 错误 14: 自动结束命令 13: 起始位错误 12: 改写锁寄存器错误 11: FIFO under-run/over-run 10: 数据缺失超时中断 9: 读数据超时 8: 应答超时 7: 数据 CRC 错误 6: 应答 CRC 错误 5: 接受数据 FIFO 请求（传输完成后无效） 4: 发送数据 FIFO 请求 3: 数据传输完成 2: 命令结束 1: 应答错误 0: 卡检测

11.3.18. 原始中断状态寄存器SDMMC_RINTSTS(偏移: 44h)

比特	名称	属性	复位值	描述
31:17	RSV	-	-	保留位。
16	sdio_interrupt	R/W	0	SDIO卡中断，表示卡产生了中断。该中断位不受中断屏蔽寄存器影响，始终有效。写1清，写0无效。

15:0	int_status	R/W	0	15: 最终位错误/无 CRC/CRC 错误 14: 自动结束命令 13: 起始位错误 12: 改写锁寄存器错误 11: FIFO under-run/over-run 10: 数据缺失超时中断 9: 读数据超时 8: 应答超时 7: 数据 CRC 错误 6: 应答 CRC 错误 5: 接受数据 FIFO 请求（传输完成后无效） 4: 发送数据 FIFO 请求 3: 数据传输完成 2: 命令结束 1: 应答错误 0: 卡检测 该中断位不受中断屏蔽寄存器影响，始终有效。 写1清，写0无效。
------	------------	-----	---	---

11.3.19. 状态寄存器SDMMC_STATUS(偏移: 48h)

比特	名称	属性	复位值	描述
31	dma_req	R	0	DMA请求信号
30	dma_ack	R	0	DMA应答信号
29:17	fifo_count	R	00	FIFO count
16:11	response_index	R	0	前一个command的response index
10	data_state_machine_busy	R	0	Data状态机处于busy状态，0表示不处于busy状态，1表示处于busy状态
9	data_busy	R	1或者0, 取决于dat[0]的值	Data[0]的取反，卡处于busy状态，0表示不处于busy状态，1表示处于busy状态
8	data_3_status	R	1或者0, 取决于dat[3]的值	可以检查卡是否插入读卡器
7: 4	command_fsm_statuses	R	0	Command FSM status: 0 – Idle 1 – 发送初始化序列 2 – 发送命令的起始位 3 – 发送命令的传输方向位 4 – 发送命令的index和argument 5 – 发送命令的 crc7 6 – 发送命令的停止位 7 – 收到应答的起始位 8 – 收到中断应答 9 – 收到应答的传输方向位 10 – 收到应答的index 11 – 收到应答的数据 12 –收到应答的 crc7 13 – 收到应答的停止位 14 – NCC时间

				15 – 等待；从命令到应答的等待阶段
3	fifo_full	R	0	FIFO已填满
2	fifo_empty	R	1	FIFO已空
1	fifo_tx_level	R	1	FIFO的数据 count小于等于发送的level值 (tx_level)
0	fifo_rx_level	R	0	FIFO的数据 count大于等于接收的level值 (rx_level)

11.3.20. FIFO比较寄存器SDMMC_FIFOTH(偏移：4Ch)

比特	名称	属性	复位值	描述
31	RSV	-	-	保留位。
30:28	multi_transaction_size	R/W	0	Multi-block读写时的burst size。应该与DMA controller的source/dest msize相等。 000: 1 transfers 001: 4 010: 8 011: 16 100: 32 101: 64 110: 128 111: 256
27:16	rx_level	R/W	0x00f	从卡读数据时FIFO的level值。当FIFO的数据count大于等于这个值时，DMA/FIFO 请求信号起来。在整个传输的末端，不管level值的大小，DMA/FIFO 请求信号都会起来使得剩下的数据能被传出去。 在非DMA模式下，当RXDR 中断使能，会产生RXDR中断以代替DMA 请求。在整个传输的末端，当level值比剩下的数据个数大的时候，不会产生中断。当看到数据 Transfer Done中断时，host应该去读剩下的未完成的数据。 在DMA模式下，在整个传输的末端，即使剩下的数据个数比level值小，DMA请求信号也会起来以进行一次单block传输，在数据 Transfer Done中断起来之前传输完所有的数据。
15:12	保留	R	0	保留
11:0	tx_level	R/W	000	向卡写数据时FIFO的level值。当FIFO的数据count小于等于这个值时，DMA/FIFO 请求信号起来。在整个传输的末端，不管level值的大小，DMA/FIFO 请求信号都会起来。 在非DMA模式下，当TXDR 中断使能，会产生TXDR中断以代替DMA 请求。在整个传输的末端，在最后一次中断产生时，host要把要求传输数量中尚未完成的数据填充到FIFO中

				去（不是在FIFO满之前或者传输完成之后，因为FIFO可能并不是空的）。 在DMA模式下，在整个传输的末端，如果最后一个传输少于一个burst size，DMA controller 执行一个single cycle知道要求数量的数据传输完成。
--	--	--	--	---

每当向 FIFO 里搬入数据时，FIFO 的数据 count 加 1；每当从 FIFO 里搬出数据时，FIFO 的数据 count 减 1。

11.3.21. 卡检测寄存器SDMMC_CDETECT(偏移：50h)

比特	名称	属性	复位值	描述
311	RSV	-	-	保留位。
0	card_detect	R/W 或 R	卡检测的输入	0: 成功检测到卡 1: 未能检测到卡

该寄存器写 1 清 0

11.3.22. 写保护寄存器SDMMC_WRTPRT(偏移：54h)

比特	名称	属性	复位值	描述
31:1	RSV	-	-	保留位。
0	write_protect	R	写保护的输入	0: 没有写保护 1: 有写保护

该寄存器写 1 清 0

11.3.23. TCBCNT寄存器SDMMC_TCBCNT(偏移：5Ch)

比特	名称	属性	复位值	描述
31:0	trans_card_byte_count	R	0000	由host发往卡的数据的字节数。在数据传输过程中，该寄存器读到的是0

11.3.24. TBBCNT寄存器SDMMC_TBBCNT(偏移：60h)

比特	名称	属性	复位值	描述
31:0	trans_fifo_byte_count	R	00	在host/DMA内存与FIFO之间传输数据的字节数

11.3.25. DEBOUNCE寄存器SDMMC_DEBOUNCE(偏移：64h)

比特	名称	属性	复位值	描述
31:24	RSV	-	-	保留位。
23:0	debounce_count	R/W	0xffffffff	Debounce时钟周期数，用于卡检测防抖处理。使用系统时钟计数。

11.3.26. FIFO通道寄存器SDMMC_DATA (偏移：100h)

比特	名称	属性	复位值	描述
----	----	----	-----	----

31:0	Data	R/W	0	Fifo 数据访问通道, 读写该寄存器即可以将数据读出或写入内部 fifo
------	------	-----	---	---------------------------------------

11.4. 使用说明

11.4.1. SDIO使用要点

● 软件编程要求

1. 在一个命令发送完成之前, 不要改变跟命令相关的各个寄存器。这些寄存器包括: 命令寄存器, 命令参数寄存器, 字节个数寄存器, block size 寄存器, 时钟分频率寄存器, 时钟使能寄存器, 超时寄存器, 位宽寄存器。
2. 跟数据传输相关各个参数的设置请在数据传输之前设置好。当一次传输开始后, 不要改变各个参数。
3. 当要发送一个命令时, 将 start_cmd (命令寄存器 31 位) 置 1。此时, 如果 update_clk=0 (命令寄存器 21 位), 所有与命令发送相关的参数 (命令寄存器, 命令参数寄存器, 字节个数寄存器, block size 寄存器, 位宽寄存器, 超时寄存器, 时钟使能寄存器, 时钟分频率寄存器) 将被更新。这个更新需要一定的时间 (6 个系统时钟), 在这段时间内不应该再改变命令相关的参数, 否则会发生改写锁寄存器错误。一旦命令被卡接受, 那就可以发送下一个命令, 但是这会有以下几种情况:
 - 1) 如果前一个命令不是数据传输命令, 新的命令将会在前一个命令完成后发送;
 - 2) 如果前一个命令是数据传输命令, 并且 wait_prv_data_finish 置位, 新的命令会在数据完成之后再发送;
 - 3) 如果 wait_prv_data_finish 为 0, 新的命令会在前一个命令完成之后发送。比如, 你要停止数据传输, 或者你要在数据传输过程中查询卡状态。
4. 当数据传输时, 命令不会发送。
5. 一次只能读写一张卡, 也只能向一张卡发送命令。比如, 当你与一张卡传输数据时, 你不能同时向另一种卡发送命令。
6. 对于同一张卡来说, 一次只能发送一个命令。
7. 在一个字节个数寄存器为 0 的写操作中, 如果由于 FIFO 空了而导致时钟停止, 那么应该先把 FIFO 填上数据, 使得时钟开始, 然后才能发送 stop 命令。
8. 如果在数据传输过程中要发送 reset 命令 (CMD0, CMD15, CMD52_reset), 必须把命令寄存器中的 stop_abort_cmd 置位。这样就会在命令发送完毕之后停止数据传输。
9. 如果在读卡时, 由于 FIFO 满了导致时钟停止, 软件应该至少读两个 FIFO 才能使得时钟恢复。
10. 在传输时, 如果卡被暂停或者停止了, 恢复的时候应该把 fifo reset 一下。

● 中断

表 11-1 SDIO 中断源描述

位	中断	描述
15	最终位错误/无 CRC/CRC 错误	在读操作中，数据的最终位不是 1；在写操作中，没有数据 CRC 或者 CRC 错误
14	自动结束命令	当数据传输结束时，由 host 自动发送 stop 命令
13	起始位错误	在读操作中，数据起始位不为 0；在 4bit 模式中，所有的起始位都不为 0
12	改写锁寄存器错误	在命令 start 位起来后 6 个系统时钟内，试图改写锁住的命令相关寄存器
11	FIFO under-run/over-run	FIFO 满了，但是 HOST 要继续往 FIFO 里搬数据；或者 FIFO 空了，但是 HOST 要继续从 FIFO 中取数据
10	数据缺失超时中断	如果 FIFO 满了，还要继续从卡读数据；或者 FIFO 空了，还要继续向卡写数据，那此时就会停掉输出时钟，以避免丢失数据。在超时寄存器个输出时钟时间内还没有解决问题，中断就会产生。
9	读数据超时	读数据超时时此中断产生，同时数据传输完成中断也会产生
8	应答超时	应答超时时此中断产生，同时命令结束中断也会产生
7	数据 CRC 错误	从卡收到的数据 CRC 与内部产生的 CRC 不 match
6	应答 CRC 错误	从卡收到的应答 CRC 与内部产生的 CRC 不 match
5	接受数据 FIFO 请求	从卡读操作中，FIFO count 大于等于 rx_level。
4	发送数据 FIFO 请求	从卡读操作中，FIFO count 小于等于 tx_level
3	数据传输完成	数据传输完成后，即使有起始位错误或者 CRC 错误，中断也会产生。当读数据超时中断产生时，这一位也会置位。（建议，当数据传输完成后，host 应该去把在 FIFO 剩下的数据搬出去；）
2	命令结束	命令发送出去后，收到了卡的应答，即使应答错误或者 CRC 错误，这一位中断也会置位。当应答超时时这一位也会置位
1	应答错误	发生以下情况：1.应答起始位不为 0；2.命令 index 不 match；3.应答终止位不为 1
0	卡检测	当有卡插入或者拔出时。

● 数据传输

在数据写命令的应答接收到 2 个时钟之后，host 就会开始数据传输。即使在应答错误或者应答

CRC 错误，也是这样。但是如果应答超时，那么数据传输就会停止。

- Single block 传输

如果传输模式为 block 传输，而且字节个数寄存器等于 block size 寄存器，那么就会产生 single block 传输。

- Multiple block 传输

如果传输模式为 block 传输，而且字节个数寄存器不等于 block size 寄存器，那么就会产生 multiple block 传输。

如果字节个数为 0，而 block size 不为 0，那么就是一个无终点的传输。Host 将会继续传输数据直到发送 stop 命令。

- 自动停止

当命令寄存器中的 send_auto_stop 置位后，host 会发送一个额外的 block 传输命令把在 FIFO 中剩余的数据传输完毕，然后发送停止命令。

- 时钟控制

时钟的低功耗模式：时钟控制寄存器的低功耗模式位置 1 可以使得输出时钟处于低功耗模式，当卡处于 idle 状态时至少 8 个系统时钟后，会把输出时钟关掉。低功耗模式位会在发送一个新的命令时刷新。另外，当 FIFO over-run 或者 under-run 时，也会把输出时钟关掉。

总结：在以下情况下，输出时钟会被关掉：

clk_en = 1;

低功耗模式下卡处于 idle 状态至少 8 个系统周期；

FIFO 满了，不能再从卡接受数据，因此要关掉时钟以避免 FIFO over-run，导致数据丢失；

FIFO 空了，不能再向卡发送数据，因此要关掉时钟以避免 FIFO under-run。

请注意，要改变输出时钟分频率，请先把时钟使能关闭后再改变。

- 错误检测

应答：

应答超时：在超时寄存器中写入的那么多时钟周期内，没有接收到应答的起始位。

应答 CRC 错误：应答的 CRC7 与内部产生的 CRC7 不匹配。

应答错误：应答的起始位不是 0，或者命令 index 与发送的命令 index 不匹配，或者应答终止位不是 1。

数据发送：

无 CRC 状态：在一次数据写传输时，如果在数据部分的最后一位两个输出时钟周期后，没有收到 CRC 的起始位，产生数据 CRC 错误，并在中断相应位置 1。然后 host 将停止后面的数据传输。

如果 CRC 起始位后的 CRC 状态不是 010，也会产生数据 CRC 错误，并在中断相应位置 1。

FIFO under-run 时，会产生数据传输错误。

数据接收：

数据超时：在一次数据读传输时，如果在超时寄存器中写入的那么多时钟周期内，没有收到数据的起始位，会产生数据超时，并产生 DTO 中断。然后 host 将停止后面的数据传输。

数据起始位错误：在 4bit 或者 8bit 读传输中，如果数据线上没有起始位，那么将产生起始位错误。

数据 CRC 错误：在一次读数据传输中，如果接收到的 CRC16 与内部产生的 CRC16 不匹配，产生 CRC 数据错误并停止后面的数据传输。

数据停止位错误：在一次读传输中，如果停止位不是 1，产生停止位错误并停止后面的数据传输。

FIFO over-run 时，会产生数据传输错误。

11.4.2. SDIO使用流程

在发送一个新的命令之前，需要确保卡不处于 busy 状态。在改变输出时钟频率之前，软件必须保证当前没有数据或者命令传输。

为了避免在输出时钟产生毛刺，改变卡时钟之前应该使用下列步骤：

1. 关闭时钟使能寄存器并用 update_clk 更新。为了保证之前的命令已经完成，先置位以下位：start_cmd, update_clk, wait_prv_data_finish。

2. 写时钟分频率寄存器，置位 start_cmd。

3. 打开时钟使能，然后置位 start_cmd。

- 读操作流程

1. 设置字节个数寄存器

2. 设置 block size 寄存器

3. 初始化命令

3. 设置命令参数寄存器

4. 发送 cmd17/18

- 写操作流程

1. 设置字节个数寄存器

2. 设置 block size 寄存器

3. 初始化命令

3. 设置命令参数寄存器

4. 发送 cmd24/25

12. SPI

12.1. 概述

SPIA 接口模块是满足 AMBA 总线结构的串行通信接口,用于 MCU 与满足 SPI 协议的外设之间进行全/半双工串行通讯;

12.2. 主要特性

- SPIA 模块支持存储映射功能, SPIB 仅支持主从功能, 不支持存储映射。
- 可通过两级分频因子来配置宽范围波特率;
- 批量传输的数据总数可配置;
- 支持 Mode0/Mode2/Mode1/Mode3 四种传输协议;
- 支持 SPI 一线、二线、四线传输;
- 可选择 MSB/LSB 数据在先传输;
- 在主模式下每个字节传输之间的时间间隔可设置;
- SPI 可以在发送和接收传输两个方向上独立的使能和禁止;
- SPIA 模块支持存储映射功能(映射空间为 0x63000000), SPIB 仅支持主从功能。

12.3. 寄存器描述

SPIA 寄存器基地址: 0x40060000

SPIB 寄存器基地址: 0x40070000

偏置	名称	描述
0x00	SPI_TX_DAT	发送数据寄存器
0x00	SPI_RX_DAT	接收数据寄存器
0x04	SPI_BAUD	波特率设置寄存器
0x08	SPI_CTL	控制寄存器
0x0C	SPI_TX_CTL	发送控制寄存器
0x10	SPI_RX_CTL	接收控制寄存器
0x14	SPI_IE	中断控制寄存器
0x18	SPI_STATUS	状态寄存器
0x1C	SPI_TXDelay	发送等待寄存器
0x20	SPI_BATCH	批量数据个数寄存器
0x24	SPI_CS	从设备选择寄存器
0x28	SPI_OUT_EN	管脚输出使能

0x2c	SPI_MEM0_ACC	SPIA 存储映射控制寄存器
0x30	SPI_CMD	SPIA 存储映射命令寄存器
0x34	SPI_PARA	SPIA 存储映射参数寄存器

12.3.1. SPI发送数据寄存器SPI_TX_DAT(偏移 00h)

比特	名称	属性	复位值	描述
31:8				保留位
7:0	TX_DAT	WO	0x00	存储下一次发送的数据。 该寄存器只写，读操作返回 SPI_RX_DAT 的值。

12.3.2. SI接收数据寄存器SPI_RX_DAT(偏移 00h)

比特	名称	属性	复位值	描述
31:8				保留位
7:0	RX_DAT	RO	0x0	存储最近一次传输中接收到的数据。 该寄存器只读，写操作改变 SPI_TX_DAT 的值。

12.3.3. SPI波特率设置寄存器SPI_BAUD(偏移：04h)

比特	名称	属性	复位值	描述
31:16				保留位
15:8	DIV2	RW	0x0	SPI 串行时钟二级分频因子。
7:0	DIV1	RW	0x2	SPI 串行时钟一级分频因子。该分频因子必须是 2 到 254 之间的偶数(包括 2 和 254)。 Bit[0]返回值总是为 0。

SPI 串行时钟: $SPIA_CLK = SYS_CLK / (DIV1 * (DIV2+1))$

$SPIB_CLK = SPIBDIV / (DIV1 * (DIV2+1))$

12.3.4. SPI控制寄存器SPI_CTL(偏移：08h)

比特	名称	属性	复位值	描述
31:16				保留位
15: 8	CS_TIME	RW	0x05	存储映射模式下 CS 高电平持续周期
7				保留
6: 5	X_Mode	RW	0x0	多线模式控制位

				00: 1X 模式。 01: 2X 模式。 10: 4X 模式。 11: 保留
4	LSB_first	RW	0x0	MSB/LSB 在前选择: 1: SPI 总线传输中 LSB 在前。 0: SPI 总线传输中 MSB 在前。
3	CPOL	RW	0x0	时钟极性控制 1: SCLK 低电平有效。空闲状态下为高。 0: SCLK 高电平有效。空闲状态下为低。
2	CPHA	RW	0x0	时钟相位控制 1: 在时钟 SCLK 的偶边沿采样数据。 0: 在时钟 SCLK 的奇边沿采样数据。
1				保留位
0	Mst_mode	RW	0x0	主从模式选择 1: SPI 工作在主模式下 0: SPI 工作在从模式下

12.3.5. SPI发送控制寄存器SPI_TX_CTL (偏移: 0Ch)

比特	名称	属性	复位值	描述
31:17				保留位。
15:8	Dummy	RW	0x0	无效字节。 Dummy 的使用参考功能描述 12.4.2 的 Dummy 字节的使用。
7:4	TX_DMA_Level	0x0		TX DMA 请求 level。当 TX FIFO 中的数据小于等于此值时+1 时, TX DMA 请求有效。
3	TX_Dma_Req_En	RW	0x0	TX DMA 请求使能。 其值为 1, 且 FIFO 中的数据少于等于 TX_DMA_Level+1 时, 发出 DMA 请求。
2				保留
1	TX_FIFO_Reset	WO	0x0	TX_FIFO 复位, 写 1 复位有效, 直到写 0 复位才会撤销。 1: 写 1 复位发送 FIFO 指针。 0: 无影响。

0	TX_EN	RW	0x0	发送使能位。 1: TX 方向使能。 0: TX 方向禁止。
---	-------	----	-----	--------------------------------------

12.3.6. SPI接收控制寄存器SPI_RX_CTL（偏移：10h）

比特	名称	属性	复位值	描述
31:8				保留位。
7: 4	RX_DMA_Level	RW	0x0	RX DMA 请求 level。当 RX FIFO 中的数据大于等于此值时+1 时,RX DMA 请求有效。
3	RX_Dma_Req_En	RW	0x0	RX DMA 请求使能。 其值为 1, 且当 FIFO 中的数据大于等于 RX_DMA_Level+1 时, 发出 DMA 请求。
2				保留
1	RX_FIFO_Reset	WO	0x0	RX_FIFO 复位, 写 1 复位有效, 直到写 0 复位才会撤销。 1: 写 1 复位接收 FIFO 指针。 0: 无影响。
0	RX_EN	RW	0x0	接收使能位。 1: RX 方向使能。 0: RX 方向禁止。

12.3.7. SPI中断控制寄存器SPI_IE（偏移：14h）

比特	名称	属性	复位值	描述
31:14				保留位。
13	CS_POS_EN	RW	0x0	CS 管脚电平上升沿事件中断使能。 1: 中断使能。 0: 中断禁止。
12	RX_FIFO_FULL_OVERFLOW_EN	RW	0x0	接收 FIFO 写入溢出中断使能位。 1: 中断使能。 0: 中断禁止。
11	RX_FIFO_EMPTY_OVERFLOW_EN	RW	0x0	接收 FIFO 读出溢出中断使能位。 1: 中断使能。 0: 中断禁止。
10	TX_FIFO_FULL_O	RW	0x0	发送 FIFO 写入溢出中断使能位。

	VERFLOW_EN			1: 中断使能。 0: 中断禁止。
9	RX_FIFO_HALF_FULL_EN	RW	0x0	接收 FIFO 半满中断使能位。 1: 中断使能。 0: 中断禁止。
8	RX_FIFO_HALF_EMPTY_EN	RW	MP0x0	接收 FIFO 半空中断使能位。 1: 中断使能。 0: 中断禁止。
7	TX_FIFO_HALF_FULL_EN	RW	0x0	发送 FIFO 半满中断使能位。 1: 中断使能。 0: 中断禁止。
6	TX_FIFO_HALF_EMPTY_EN	RW	0x0	发送 FIFO 半空中断使能位。 1: 中断使能。 0: 中断禁止。
5	RX_FIFO_FULL_EN	RW	0x0	接收 FIFO 满中断使能位。 1: 中断使能。 0: 中断禁止。
4	RX_FIFO_EMPTY_EN	RW	0x0	接收 FIFO 空中断使能位。 1: 中断使能。 0: 中断禁止。
3	TX_FIFO_FULL_EN	RW	0x0	发送 FIFO 满中断使能位。 1: 中断使能。 0: 中断禁止。
2	TX_FIFO_EMPTY_EN	RW	0x0	发送 FIFO 空中断使能位。 1: 中断使能。 0: 中断禁止。
1	Batch_DONE_EN	RW	0x0	批量完成中断使能位。 1: 中断使能。 0: 中断禁止。
0	SPI_DONE_EN			字节完成中断使能位。 1: 中断使能。 0: 中断禁止。

12.3.8. SPI状态寄存器SPI_STATUS（偏移：18h）

比特	名称	属性	复位值	描述
31:14				保留位。
13	CS_POS_Flg	RW	0x0	CS 管脚电平上升沿事件。 1:发生管脚电平上升沿事件。写 1 清除该标志位。 0: 未放生事件。
12	RX_FIFO_FULL_OVERFLOW	RO	0x0	接收 FIFO 写入溢出。
11	RX_FIFO_EMPTY_FLOW	RO	0x0	接收 FIFO 读出溢出。
10	TX_FIFO_FULL_OVERFLOW	RO	0x0	发送 FIFO 写入溢出。
9	RX_FIFO_HALF_FULL	RO	0x0	接收 FIFO 中的字节数大于等于 8。
8	RX_FIFO_HALF_EMPTY	RO	0x1	接收 FIFO 的剩余空间大于等于 8。
7	TX_FIFO_HALF3_FULL	RO	0x0	发送 FIFO 中的字节数大于等于 8。
6	TX_FIFO_HALF_EMPTY	RO	0x1	发送 FIFO 的剩余空间大于等于 8。
5	RX_FIFO_FULL	RO	0x0	接收 FIFO 满标志位。
4	RX_FIFO_EMPTY	RO	0x1	接收 FIFO 空标志位。
3	TX_FIFO_FULL	RO	0x0	发送 FIFO 满标志位。
2	TX_FIFO_EMPTY	RO	0x1	发送 FIFO 空标志位。
1	Batch_DONE	RO	0x0	批量传输完成标志位。写 1 则清除该标志位。
0	SPI_DONE	RO	0	字节传输完成位。写 1 或写 FIFO 可以清除该标志位。

注明：SPI 模块作为主机时，批量传输完成后，batch_done 置位，SPI 模块不会再发送/读取数据。

SPI 模块作为从机时，批量传送完成后，batch_done 置位，如主机继续读取数据，SPI 模块会继续发送数据，优先发送 FIFO 中的数据，FIFO 中数据发送为空后，发送 dummybyte。

当主机发送和接收功能同时使能的时候，batch_done 表示批量发送数据完成，接收的数据不产生 batch_done 并不参与批量传输计数。接收数据 batch_done 功能需要关闭发送状态。

不管作为主机还是从机，每发送或接受 1byte 完成，均会产生 spi_done 标志。

12.3.9. SPI发送等待寄存器SPI_TXDelay(偏移: 1Ch)

比特	名称	属性	复位值	描述
31:0	SPI_TDY	RW	0x0	SPI 每发送一个字节需要等待的 SPI 引擎。 该控制位只在主模式下有效。

在发送等待的过程中, CLK 将按照 CPOL 设定值停止。

12.3.10. SPI批量传输数据个数寄存器SPI_BATCH (偏移: 20h)

比特	名称	属性	复位值	描述
31:10				保留位。
9:0	Batch_Number	RW	0x0	该寄存器用来存储 SPI 总线上即将传输的数据字节个数。

12.3.11. SPI从设备选择寄存器SPI_CS(偏移: 24h)

比特	名称	属性	复位值	描述
31:8				保留位。
7:0	SPI_CS	RW	0x0	置 1 将使设备选择信号 SPI_CS[7:0]变低。 主模式下此位应该在配置的最后一步写入, 写此位后数据传输立即开始。如果主模式在传输的不同阶段需要改变其它寄存器的配置, 即么需要重新写此位(不管此位变或不变)来触发下一次传输。 从模式下 SPI_CS[7:1]无效, 写 SPI_CS[0]无效, 读 SPI_CS[0], 反映 CS0 管脚的状态。

12.3.12. SPI管脚输出方向SPI_OUT_EN(偏移: 28h)

比特	名称	属性	复位值	描述
31:4				保留位。
3	SPI_HOLD_DIR	RW	0x0	SPI 管脚输出使能位快速访问存储器功能时硬件自动控制, 其它模式软件配置此位决定收发状态。 1: 输出模式。 0: 输入模式。
2	SPI_WP_DIR	RW	0x0	SPI 管脚输出使能位快速访问存储器功能时硬件自动控制, 其它模式软件配置此位决定收发状态。

				1: 输出模式。 0: 输入模式。
1	SPI_MISO_DIR	RW	0x0	SPI 管脚输出使能位快速访问存储器功能时硬件自动控制, 其它模式软件配置此位决定收发状态。 1: 输出模式。 0: 输入模式。
0	SPI_MOSI_DIR	RW	0x0	SPI 管脚输出使能位快速访问存储器功能时硬件自动控制, 其它模式软件配置此位决定收发状态。 1: 输出模式。 0: 输入模式。

12.3.13. SPIA存储映射控制寄存器SPI_MEMO_ACC(偏移: 2Ch)

比特	名称	属性	复位值	描述
31:19	RSV			保留位。
18: 14	Addr_width	RW	0x10	发送地址的位数等于该寄存器值。 地址从 AHB 地址总线 LSB 向上映射。 最大值为 24, 超过将会按 24 进行后续计算 0x08 : 8 bit 0x10 : 16bit 0x18 : 24bit 其他非 0: 24bit
13: 9	PARA_NO2	RW	0x0	发送参数 2 的位数等于该寄存器值。 参数从 PARA2 的 LSB 向上映射。 0 表示不使用参数 2, 最大值为 16, 超过将会按 16 进行后续计算。 0x08 : 8 bit 0x10 : 16bit 0x00: 不使能 其他非 0: 16bit
8: 5	PARA_NO1	RW	0x0	发送参数 1 的位数等于该寄存器值。 参数从 PARA1 的 LSB 向上映射。 0 表示不使用参数 1, 最大值为 8, 超过将会按 8 进行后续计算。

				0x08: 8 bit 0x00: 不使能 其他非 0: 8bit。
4	RVS			保留
3	Con_Rd_EN	RW	0x0	连读使能位。 1:使能连读。 0:不使能连读。
2	Para_Ord2	RW	0x0	决定在发送地址前后加入参数 2。 1:在地址后发送。 0:在地址前发送。
1	Para_Ord1	RW	0x0	决定在发送地址前后加入参数 1。 1:在地址后发送。 0:在地址前发送。
0	SPI_Acc_EN	RW	0x0	SPI 快速访问存储器功能块使能。 1:使能。 0:不使能, 作为普通 spi 接口。

12.3.14. SPIA存储映射命令寄存器SPI_CMD(偏移: 30h)

比特	名称	属性	复位值	描述
31:16				保留位。
15:8	Wr_Cmd	RW	0x0	存放写 SPI 存储器的指令。
7:0	Rd_Cmd	RW	0x0	存放读 SPI 存储器的指令。

12.3.15. SPIA存储映射参数寄存器SPI_PARA(偏移: 34h)

比特	名称	属性	复位值	描述
31:24				保留位。
23:8	Para2	RW	0x0	存放可能使用的地址 Dummy 或命令参数。
7:0	Para1	RW	0x0	存放可能使用的地址 Dummy 或命令参数。

PS: 存储映射模式多线模式下, 命令、地址、参数, 仍然以一线模式传输。

参数 1 和参数 2 如果需要同时发送, 先发送参数 1, 后发送参数 2。

12.4. 使用流程

12.4.1. SPI主模式发送

1、初始阶段

- 1) 配置 SPI 控制寄存器。X_Mode、LSB_first、CPOL、CPHA、Mst_mode 比特位。
- 2) 配置波特率寄存器。
- 3) 配置发送等待寄存器。
- 4) 配置管脚输出使能寄存器。

2、发送阶段

- 1) 设置 SPI 发送控制——寄存器 SPI_TX_CTRL 中的 TX_EN 比特位。
- 2) 设置 SPI_BATCH 寄存器。
- 3) 通过 SPI 从设备选择寄存器，将 SPI 配置成选择状态。
- 4) 当发送 FIFO 非满时，写入待发送的数据。
- 5) 等待 SPI 状态寄存器 BATCH_DONE 状态位置位。

12.4.2. SPI主模式接收

1、初始阶段

- 1) 配置 SPI 控制寄存器。X_Mode、LSB_first、CPOL、CPHA、Mst_mode 比特位。
- 2) 配置波特率寄存器。
- 3) 配置发送等待寄存器。
- 4) 配置管脚输出使能寄存器。

2、发送阶段

- 1) 设置 SPI 发送控制寄存器 SPI_RX_CTRL 中的 RX_EN 比特位。
- 2) 设置 SPI_BATCH 寄存器。
- 3) 通过 SPI 从设备选择寄存器，将 SPI 配置成选择状态。
- 4) 当接收 FIFO 非空时，读取接收 FIFO 中的数据。
- 5) 等待 SPI 状态寄存器 BATCH_DONE 状态位置位。

12.4.3. SPI主模式接收时Dummy控制位

SPI 工作在主模式，当仅处于接收模式下，MOSI 数据线的状态由 Dummy 控制位决定。SPI 引擎会将 Dummy 数据按比特移出。

12.4.4. SPI从模式发送时Dummy控制位

SPI 工作在从模式，当处于发送模式且发送 FIFO 中没有数据时，MISO 的状态由 Dummy 控制位决定。SPI 引擎会将 Dummy 数据按比特移出。

SPI 发送 FIFO 的指针需要从 CPU 时钟域同步到 SPI 时钟域，同步需要 2 个 SPI 时钟。当主设备不发起传输时，没有 SPI 时钟，FIFO 的状态同步不到 SPI 时钟域，只有当主设备来时钟了，才可

能将 FIFO 的状态同步到 SPI 时钟域。

12.4.5. SPI从模式接收时Dummy控制位

SPI 工作在从模式，当仅处于接收模式下，MISO 数据线的状态由 Dummy 控制位决定。SPI 引擎会将 Dummy 数据按比特移出。

12.4.6. SPIA存储读映射

单次读方式：

该 SPI 接口模块支持对满足 SPI 协议的存储器的快速读取功能，当该功能开启时 MCU 可以如读取普通存储器一般快速读取支持 SPI 协议的 sram 或者 nor_flash。其使用方法及设置步骤如下：

1. 对于不同的 spi 存储器件，如有需要需按照器件的各自要求设置其状态寄存器。关于 spi 存储器的设置要求，请参照其各自 spec。
2. 进行读操作前，需要进行一些基本设定，通过 SPI_BAUD 寄存器设置时钟波特率；通过 SPI_CTL 寄存器的 CPOL 位和 CPHA 位设置 SPI 工作模式；通过设置 SPI_CTL 寄存器的 X_mode 位设置线宽模式；之后在 SPI_CMD 寄存器的 RD_INST 位中写入需要发送的读命令，SPI_PARA 寄存器的 PARA1（2）位设置需要发送的参数 1（2）（如果需要）；再通过 SPI_MOME_ACC 寄存器 Addr_No/ PARA_No1（2）/ PARA_No1（2）设置地址、参数长度及发送次序；最后使能 SPI_ACC_EN 位，开启 SPI_ACC_EN 位后，硬件自动控制 SPI 接口的输入与输出，即完成读取 SPI 存储设备的准备设置。
3. AHB 总线可以直接对存储器地址进行读操作，硬件会自动开启对 SPI 存储器的通信，包括发送命令和地址和接收存储器发回的数据，在通信完成后 SPI 接口将把读取的数值通过 AHB 总线返回 MCU。

连续读方式：

具体设置还是参照单次读方式，但是需要启动连读使能位即 SPI_MEMO_ACC 寄存器的 CON_RD_EN 位之后，当条件满足时，硬件会自动开启连读功能停止发送命令以及地址以减少读操作用时。连读的条件为：

1. 读取地址为连续值，即没有地址跳变。
2. 读取操作没有超出寄存器规定的块。
3. 没有进行过写操作。

除了设置寄存器外，连读模式对用户操作没有影响，当连读模式启动时，用户操作等同于读取普通读取 spi 存储器

12.4.7. SPIA存储写映射

SPI 接口模块支持对满足 SPI 协议的 sram 的写功能，其使用方法如下：

首先需要设置 SPI_CTL 寄存器的 mst_mode 位，将 SPI 接口置于主模式；然后通过 SPI_BAUD 设置时钟波特率，通过 SPI_CTL 的 CPOL 和 CPHA 设置 SPI 工作模式；再后设置 SPI_CTL 的 X_mode

位和 SPI_OUT_EN 寄存器设定通信 IO。以上为 SPI 主模块发送准备设置。

之后通过 SPI_CMD 寄存器的 WR_INST 位设置需要发送的写命令、SPI_PARA 寄存器的 PARA1 (2)位设置需要发送的参数 1(2)(如果需要);再通过 SPI_MOME_ACC 寄存器 Addr_No/ PARA_No1 (2)/ PARA_No1(2)设置地址、参数长度及发送次序,最后使能 SPI_ACC_EN 位,开启 SPI_ACC_EN 位后,硬件自动控制 SPI 接口的输入与输出,即完成写 SPI sram 的准备设置。

最后 AHB 可以直接对 sram 地址进行写操作,硬件会自动开启对 SPI sram 的通信,在通信完成后 SPI 接口会将值写入 sram。

保
密

13. CACHE

13.1. 概述

Cache IP 为 4K 字节大小、4-way set-associate 结构的指令 Cache，用于提高系统的整体性能。此版本的 cache 仅对指令进行 Cache 操作，CPU 从 Cache 取指令仅需要 2 个周期，提高了系统的整体性能。

此 Cache 采用 4 way set-associate 硬件结构，每个 way 含有 64 个 set。每个 line 含有 16 个字节，采用 LRU 算法替换策略。其仅对取指令进行 cache 匹配操作，所有的写操作全部采用写穿的方式。另外，此 Cache 中的 Data Sram 在 Cache 未使能时可以当作系统 Sram 用，偏移地址为 0xf000，支持 word、halfword 和 byte 读写。

此 Cache 在系统中的位置如下图所示：

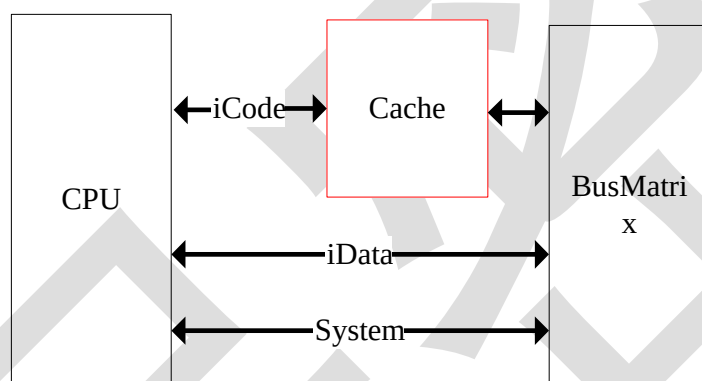


图 13-1 Cache 在系统中位置

13.2. 主要特性

- 与 AMBA AHB 兼容接口；
- Data Space 为 4K 字节大小；
- 4-way set-associate 结构，每个 way 含有 64 个 set；
- 每个 line 含有 16 个字节，对指令操作支持 4 Word 预取功能；
- Data Sram 在 Cache 未使能时可以当系统 Sram 用，支持 word、halfword 和 byte 读写；

13.3. 寄存器描述

CACHE 寄存器基地址：0x40080000

偏置	名称	描述
0xD000	CACHE_CR	控制寄存器
0xF000	CACHE_DATA	Cache 中的数据。此区域可作为系统 Sram 使用。

13.3.1. 控制寄存器CACHE_CR（偏移：d000h）

比特	名称	属性	复位值	描述
31:6	RSV	-	-	保留
5:2	CWLoc	W/R	0	Way Lock 使能信号。每一位对应一个 Way 的替换算法： CWLock[0]：当此位为 1 时，Way0 将被锁定，不参与 Cache 替换； CWLock[1]：当此位为 1 时，Way1 将被锁定，不参与 Cache 替换； CWLock[2]：当此位为 1 时，Way2 将被锁定，不参与 Cache 替换； CWLock[3]：当此位为 1 时，Way3 将被锁定，不参与 Cache 替换；
1	CacheClr	W	0	写 1 清除 Cache 中所有缓存的内容，每次 Cache 使能时，均需要向此位写 1。下一周期自动清零 1：清除 Cache 中的所有缓存数据； 0：不清除 Cache 中缓存的数据；
0	CacheEn	W/R	0	Cache 使能位。 CacheEn = 0，Cache 禁止； CacheEn = 1，Cache 使能。

13.3.2. Cache数据空间CACHE_DATA（偏移：f000h~ffffh）

比特	名称	属性	复位值	描述
31:0	Cache Data Space	R/W	-	Cache 中的数据。此区域可作为系统 Sram 使用

13.4. 使用流程

1. 使能 Cache 时，需要向 CacheEn 和 CacheClr 位写 1；
2. Cache 使用过程中，可以由选择的设置 CWLock 位来提高程序运行的效率。
3. 在 CacheEn 不为 1，即 Cache 未使能时，Cache DataSpace 可以作为系统 Sram 使用。在 CacheEn 为 1 后，读写 CacheDataSpace 将发生数据错误。

14. UART

14.1. 概述

UART 串口模块，带有 16 字节的 FIFO，可小数分频。

14.2. 主要特性

- 16 字节的硬件 FIFO
- 波特率支持整数和小数分频
- UARTA 支持 CTS, RTS 流控制
- 错误起始位侦测
- 帧(从起始位开始到停止位结束)中断检测
- 可编程位宽(5/6/7/8bit)，停止位个数(1/2bit)
- 校验模式为奇偶校验或 0/1 校验

14.3. 寄存器描述

UARTA 寄存器基地址：0x48010000

UARTB 寄存器基地址：0x480D0000

偏置	名称	描述
0x00	UART_DR	数据寄存器
0x04	UART_RSR	接收状态寄存器
0x18	UART_FR	标志寄存器
0x24	UART_IBRD	整数分频因子寄存器
0x28	UART_FBRD	小数分频因子寄存器
0x2C	UART_LCRH	线控制器寄存器
0x30	UART_CR	控制寄存器
0x34	UART_IFLS	FIFO 中断触发值寄存器
0x38	UART_IMSC	中断屏蔽使能/清除寄存器
0x3C	UART_RIS	原始中断状态寄存器
0x40	UART_MIS	屏蔽后的中断状态寄存器
0x44	UART_ICR	中断清除寄存器

14.3.1. 数据寄存器UART_DR(偏移：00h)

比特	名称	属性	复位值	描述
----	----	----	-----	----

15:12	RSV	-	-	保留
11	OE	RO	0	Overrun 错误标志 0: 无错误 1: 有错误
10	BE	RO	0	Break 错误标志 0: 无错误 1: 有错误
9	PE	RO	0	0/1 校验或者奇/偶校验错误标志 0: 无错误 1: 有错误
8	FE	RO	0	帧格式错误标志 0: 无错误 1: 有错误
7:0	DATA	R/W	0	发送或接收的数据

FIFO 方式读时，可从该寄存器中获知每个字节的真实状态。

14.3.2. 接收状态寄存器UART_RSR(偏移: 04h)

比特	名称	属性	复位值	描述
15:4	RSV	-	-	保留
3	OE	R/WC	0	Overrun 错误标志 0: 无错误 1: 有错误
2	BE	R/WC	0	Break 错误标志 0: 无错误 1: 有错误
1	PE	R/WC	0	0/1 校验或者奇/偶校验错误标志 0: 无错误 1: 有错误
0	FE	R/WC	0	帧格式错误标志 0: 无错误 1: 有错误

14.3.3. 标志位寄存器UART_FR(偏移: 18h)

比特	名称	属性	复位值	描述
15:8	RSV	-	-	保留

7	TXFE	RO	1	发送 FIFO/发送保持寄存器空状态位： 0: 发送 FIFO 非空（使能 FIFO）；发送保持寄存器有数据（禁止 FIFO） 1: 发送 FIFO 为空（使能 FIFO）；发送保持寄存器无数据（禁止 FIFO）
6	RXFF	RO	0	接收 FIFO/接收保持寄存器满状态位： 0: 接收 FIFO 非满（使能 FIFO）；接收保持寄存器非满（禁止 FIFO） 1: 接收 FIFO 为满（使能 FIFO）；接收保持寄存器为满（禁止 FIFO）
5	TXFF	RO	0	发送 FIFO/发送保持寄存器满状态位： 0: 发送 FIFO 非满（使能 FIFO）；发送保持寄存器非满（禁止 FIFO） 1: 发送 FIFO 为满（使能 FIFO）；发送保持寄存器为满（禁止 FIFO）
4	RXFE	RO	1	接收 FIFO/接收保持寄存器空状态位： 0: 接收 FIFO 非空（使能 FIFO）；接收保持寄存器非空（禁止 FIFO） 1: 接收 FIFO 为空（使能 FIFO）；接收保持寄存器为空（禁止 FIFO）
3	BUSY	RO	0	发送忙标志 0: 发送 FIFO 为空并且所有位都从移位寄存器中移出 1: 发送 FIFO 有数据
2:1	RSV	-	-	保留
0	CTS	RO	0	CTS 输入管脚状态 0: CTS 输入高电平 1: CTS 输入低电平

14.3.4. 整数分频因子寄存器UART_IBRD(偏移：24h)

比特	名称	属性	复位值	描述
15:0	UART_IBAUD	R/W	0x0000	波特率分频整数因子 $UART_IBAUD = (\text{integer}(F_{sys}/(16*BAUD)))$, integer 为取整操作

14.3.5. 小数分频因子寄存器UART_FBRD (偏移: 28h)

比特	名称	属性	复位值	描述
15:0	RSV	-	-	保留
7:0	UART_FBAUD	R/W	0x00	波特率分频小数因子 $UART_IBAUD = (\text{integer}(\text{badf} * 64 + 0.5))$, integer 为取整操作, badf 为 $F_{\text{sys}}/(16 * \text{BAUD})$ 的小数部分

14.3.6. 线控制器寄存器UART_LCRH (偏移: 2Ch)

比特	名称	属性	复位值	描述
15:8	RSV	-	-	保留
7	SPS	R/W	0	校验模式选择位 0: 奇/偶校验 1: 0/1 校验
6:5	WLEN	R/W	0	字宽选择位 00: 5bits 01: 6bits 10: 7bits 11: 8bits
4	FEN	R/W	0	FIFO 使能位 0: 禁止 FIFO 1: 使能 FIFO
3	STP2	R/W	0	停止位位数选择位 0: 1 位停止位 1: 2 位停止位
2	EPS	R/W	0	0/1 校验或者奇/偶校验选择位(取决于 SPS) 0: 奇/偶校验选择奇校验, 或 0/1 校验选择校验位 强制为 1 1: 奇/偶校验选择偶校验, 或 0/1 校验选择校验位 强制为 0
1	PEN	R/W	0	校验使能位: 0: 禁止奇/偶校验或 0/1 校验 1: 使能奇/偶校验或 0/1 校验
0	BRK	R/W	0	BREAK 发送使能位

				0: 禁止 1: 使能
--	--	--	--	----------------

14.3.7. 控制寄存器UART_CR (偏移: 30h)

比特	名称	属性	复位值	描述
15	CTSEn	R/W	0	CTS 流控制使能位 0: 禁止 1: 使能
14	RTSEn	R/W	0	RTS 流控制使能位 0: 禁止 1: 使能
13:12	RSV	-	-	保留
11	RTS	R/W	0	RTS 输出管脚状态 0: RTS 输出高电平 1: RTS 输出低电平
10	RSV	-	-	保留
9	RXE	R/W	0	接收使能位: 0: 禁止 1: 使能
8	TXE	R/W	0	发送使能位: 0: 禁止 1: 使能
7:1	RSV	-	-	保留
0	UARTEN	R/W	1b'0	UART 功能使能位 0: 禁止 1: 使能

14.3.8. FIFO中断触发寄存器UART_IFLS(偏移: 34h)

比特	名称	属性	复位值	描述
15:6	RSV	-	-	保留
5:3	RXIFLSEL	R/W	010	接收中断的触发点选择位: 000: 1/8 (接收 FIFO 收到 2 个数据)。 001: 1/4 (接收 FIFO 收到 4 个数据)。 010: 1/2 (接收 FIFO 收到 8 个数据)。 011: 3/4 (接收 FIFO 收到 12 个数据)。100: 7/8

				(接收 FIFO 收到 14 个数据)。
2:0	TXIFLSEL	R/W	010	发送中断的触发点选择位: 000: 1/8 (发送到 FIFO 中剩余 2 个数据)。 001: 1/4 (发送到 FIFO 中剩余 4 个数据)。 010: 1/2 (发送到 FIFO 中剩余 8 个数据)。 011: 3/4 (发送到 FIFO 中剩余 12 个数据)。 100: 7/8 (发送到 FIFO 中剩余 14 个数据)。

注：中断产生不是单纯由 FIFO 中的数据数量决定，触发仅产生于数据数量到达中断触发点的瞬间。对于接收 FIFO，仅产生于接收数据时 FIFO 中数据数量到达中断出发点的瞬间，读取时到达触发点不会产生中断；对于发送 FIFO，中断仅产生于发送时 FIFO 中数据数量到达中断出发点的瞬间，往发送 FIFO 中写入数据达到触发点时也不会产生中断。

14.3.9. 中断屏蔽使能/清除寄存器 UART_IMSC (偏移：38h)

比特	名称	属性	复位值	描述
15:11	RSV	-	-	保留
10	OEI	R/W	0	overrun 中断使能位 0: 禁止 1: 使能
9	BEI	R/W	0	break error 中断使能位 0: 禁止 1: 使能
8	PEI	R/W	0	0/1 校验或者奇/偶校验错误中断使能位 0: 禁止 1: 使能
7	FEI	R/W	0	帧格式错误中断使能位 0: 禁止 1: 使能
6	RTI	R/W	0	接收数据读取超时中断使能位 0: 禁止 1: 使能
5	TXI	R/W	0	发送中断使能位 0: 禁止 1: 使能
4	RXI	R/W	0	接收中断使能位

				0: 禁止 1: 使能
3:0	RSV	-	-	保留

14.3.10. 原始中断状态寄存器UART_RIS (偏移: 3Ch)

比特	名称	属性	复位值	描述
15:11	RSV	-	-	保留
10	OEI	R/W	0	overrun 原始中断 0: 无中断 1: 有中断
9	BEI	R/W	0	break error 原始中断 0: 无中断 1: 有中断
8	PEI	R/W	0	奇偶校验错误原始中断 0: 无中断 1: 有中断
7	FEI	R/W	0	帧格式错误原始中断 0: 无中断 1: 有中断
6	RTI	R/W	0	接收数据读取超时原始中断 0: 无中断 1: 有中断
5	TXI	R/W	0	发送原始中断 0: 无中断 1: 有中断
4	RXI	R/W	0	接收原始中断 0: 无中断 1: 有中断
3:0	RSV	-	-	保留

14.3.11. MASK后的中断状态寄存器UART_MIS (偏移: 40h)

比特	名称	属性	复位值	描述
15:11	RSV	-	-	保留
10	OEI	R/W	0	overrunMASK 后的中断

				0: 无中断 1: 有中断
9	BEI	R/W	0	break errorMASK 后的中断 0: 无中断 1: 有中断
8	PEI	R/W	0	奇偶校验错误 MASK 后的中断 0: 无中断 1: 有中断
7	FEI	R/W	0	帧格式错误 MASK 后的中断 0: 无中断 1: 有中断
6	RTI	R/W	0	接收数据读取超时 MASK 后的中断 0: 无中断 1: 有中断
5	TXI	R/W	0	发送 MASK 后的中断 0: 无中断 1: 有中断
4	RXI	R/W	0	接收 MASK 后的中断 0: 无中断 1: 有中断
3:0	RSV	-	-	保留

14.3.12. 中断状态清除UART_ICR (偏移: 44h)

比特	名称	属性	复位值	描述
15:11	RSV	-	-	保留
10	OEI	R/W	0	overrun 中断状态清除位 写 1 清
9	BEI	R/W	0	break error 中断状态清除位 写 1 清
8	PEI	R/W	0	奇偶校验错误中断状态清除位 写 1 清
7	FEI	R/W	0	帧格式错误中断状态清除位 写 1 清
6	RTI	R/W	0	接收数据读取超时中断状态清除位

				写 1 清
5	TXI	R/W	0	发送中断状态清除位 写 1 清
4	RXI	R/W	0	接收中断状态清除位 写 1 清
3:0	RSV	-	-	保留

14.4. 使用流程

14.4.1. 串口的发送和接收

- 配置波特率
- 配置 FIFO(是否使用 FIFO)
- 配置线控制寄存器(奇偶校验位等)
- 配置控制寄存器(是否使用中断)
- 使能 UART

15. TIMER

15.1. 概述

Timer 定时器模块分别有 4 个单独的定时器（timer）单元，这些 timer 可以有多种用途，包括定时计数功能，测量输入信号的脉冲宽度（输入捕获），产生输出波形（PWM）等功能。。

15.2. 主要特性

- 三种计数模式：free-running, cyclic, single
- 计数器可以往上，往下，往上下三种计数方向
- 中断在以下几种情况产生：
 - UEV（Update Event）：counter 往上计数到溢出值，或往下计数到 0
 - 输入捕获
 - PWM 输出
- 可编程预分频因子

15.3. 寄存器描述

Timer 寄存器基地址：0x48030000

偏置	名称	描述
0x00	T0_ARR	Timer0 加载寄存器
0x04	T0_CNT	Timer0 计数寄存器
0x08	T0_CR	Timer0 控制寄存器
0x0C	T0_IF	Timer0 中断状态寄存器
0x10	T0_CIF	Timer0 中断清除寄存器
0x14	T1_ARR	Timer1 加载寄存器
0x18	T1_CNT	Timer1 计数寄存器
0x1C	T1_CR	Timer1 控制寄存器
0x20	T1_IF	Timer1 中断状态寄存器
0x24	T1_CIF	Timer1 中断清除寄存器
0x28	T2_ARR	Timer2 加载寄存器
0x2C	T2_CNT	Timer2 计数寄存器
0x30	T2_CR	Timer2 控制寄存器
0x34	T2_IF	Timer2 中断状态寄存器
0x38	T2_CIF	Timer2 中断清除寄存器
0x3C	T3_ARR	Timer3 加载寄存器

0x40	T3_CNT	Timer3 计数寄存器
0x44	T3_CR	Timer3 控制寄存器
0x48	T3_IF	Timer3 中断状态寄存器
0x4C	T3_CIF	Timer3 中断清除寄存器
0x50	TIM_PSC	定时器预分频寄存器
0x54	ICMODE	工作模式控制寄存器
0x58	CCR	输入捕获模式控制寄存器
0x5C	CCIF	输入捕获模式中断状态寄存器
0x60	C0_CR	Channel0 输入捕获模式 counter 寄存器
0x64	C2_CR	Channe2 输入捕获模式 counter 寄存器
0x68	PCR	PWM 控制寄存器
0x6C	CPIF	PWM 中断状态寄存器
0x70	C0_PR	PWM0 比较寄存器
0x74	C1_PR	PWM1 比较寄存器
0x78	C2_PR	PWM2 比较寄存器
0x7C	C3_PR	PWM3 比较寄存器

15.3.1. TIMER0 加载寄存器(T0_ARR偏移: 00h)

比特	名称	属性	复位值	描述
31:0	LOAD	R/W	0x0	定时器 0 加载值位[31:0]: 定时器0的计数器加载值。在循环模式和单次模式, 写操作立即会把加载值加载到定时器。如果TIMER工作中更改计数模式成循环或者单次, 需要重写加载值, 否则定时器会按照当前计数值继续计数 例如: LOAD 设置为 N, 计数周期为 N+1

15.3.2. TIMER0 计数寄存器(T0_CNT偏移: 04h)

比特	名称	属性	复位值	描述
31:0	T0_CNT	R	0	定时器 0 counter 的当前计数值。

15.3.3. TIMER0 控制寄存器(T0_CR偏移: 08h)

比特	名称	属性	复位值	描述
31:6	RSV	-	-	保留
5	CMS	R/W	0	中心对齐模式选择位 0: 边沿对齐模式。counter 计数方向由 DIR 决定。 1: 中心对齐模式。Counter 往上计数和往下计数交替进行(先往上再往下)。
4	DIR	R/W	0	方向控制位 0: counter 向上计数 1: counter 向下计数
3	INTM	R/W	0	定时器 0 中断屏蔽位 0: 定时器 0 中断使能 1: 定时器 0 中断屏蔽。
2:1	MODE	R/W	0	模式选择位[1:0] 0x: 自由运行模式(free-running), 计数从 0xffffffff 到 0 (向下计数) 或从 0 到 0xffffffff 循环(向上计数) 或先从 0 到 0xffffffff 再从 0xffffffff 到 0 循环(中心对齐模式)。 10: 循环模式(cyclic), 计数从加载值到 0 循环(向下计数) 或从 0 到加载值循环(向上计数) 或先从 0 到加载值再从加载值到 0 循环(中心对齐模式)。 11: 单次模式(single), 计数从 0 到加载值(向上计数) 或从加载值到 0(向下计数), 然后停止。
0	EN	R/W	0	定时器 0 使能位 0: 计数器 0 关闭 1: 计数器 0 使能

注: 每次配置 TIMER 的控制寄存器, enable 总要最后打开; 在 enable 打开的时候, 不要去修改控制寄存器。

15.3.4. TIMER0 中断状态寄存器(T0_IF偏移: 0Ch)

比特	名称	属性	复位值	描述
----	----	----	-----	----

31:1	RSV	-	-	保留
0	INT	R	0	反映定时器 0 的中断状态 1: 中断有效。 0: 中断无效。 请写中断清除寄存器清除中断。

15.3.5. TIMER0 中断清除寄存器(T0_CIF偏移: 10h)

比特	名称	属性	复位值	描述
31:0	T0_CIF	W	0x0000	对该寄存器写任意值，都会清除计数器 0 的中断状态寄存器。

15.3.6. TIMER1 加载寄存器(T1_ARR偏移: 14h)

比特	名称	属性	复位值	描述
31:0	LOAD	R/W	0	定时器 1 加载值位 定时器 1 的计数器加载值。在循环模式和单次模式，写操作立即会把加载值加载到定时器。如果 TIMER 工作中更改计数模式成循环或者单次，需要重写加载值，否则定时器会按照当前计数值继续计数。 例如：LOAD 设置为 N，计数周期为 N+1

15.3.7. TIMER1 计数寄存器(T1_CNT偏移: 18h)

比特	名称	属性	复位值	描述
31:0	T1_CNT	R	0	定时器 1counter 的当前计数值。

15.3.8. TIMER1 控制寄存器(T1_CR偏移: 1Ch)

比特	名称	属性	复位值	描述
31:6	RSV	-	-	保留

5	CMS	R/W	0	中心对齐模式选择位 0: 边沿对齐模式。counter 计数方向由 DIR 决定。 1: 中心对齐模式 1。Counter 往上计数和往下计数交替进行（先往上再往下）。
4	DIR	R/W	0	方向控制位 0: counter 向上计数 1: counter 向下计数
3	INTM	R/W	0	定时器 1 中断屏蔽位 0: 定时器 1 中断使能 1: 定时器 1 中断屏蔽。
2:1	MODE	R/W	0	模式选择位 0x: 自由运行模式(free-running), 计数从 0xffffffff 到 0（向下计数）或从 0 到 0xffffffff 循环（向上计数）或先从 0 到 0xffffffff 再从 0xffffffff 到 0 循环（中心对齐模式）。 10: 循环模式(cyclic), 计数从加载值到 0 循环（向下计数）或从 0 到加载值循环（向上计数）或先从 0 到加载值再从加载值到 0 循环（中心对齐模式）。 11: 单次模式(single), 计数从 0 到加载值（向上计数）或从加载值到 0（向下计数），然后停止。
0	EN	R/W	0	定时器 1 使能位 0: 计数器 1 关闭 1: 计数器 1 使能

注：每次配置 TIMER 的控制寄存器，enable 总要最后打开；在 enable 打开的时候，不要去修改控制寄存器。

15.3.9. TIMER1 中断状态寄存器(T1_IF偏移：20h)

比特	名称	属性	复位值	描述
31:1	RSV	-	-	保留
0	INT	R	0	反映定时器 1 的原始中断状态 1: 中断有效。

				0: 中断无效。 请写中断清除寄存器清除中断。
--	--	--	--	----------------------------

15.3.10. TIMER1 中断清除寄存器(T1_CIF偏移: 24h)

比特	名称	属性	复位值	描述
31:0	T1_CIF	W	0	对该寄存器写任意值，都会清除计数器 1 的中断状态寄存器。

15.3.11. TIMER2 加载寄存器(T2_ARR偏移: 28h)

比特	名称	属性	复位值	描述
31:0	LOAD	R/W	0	定时器 2 加载值位 定时器 2 的计数器加载值。在循环模式和单次模式，写操作立即会把加载值加载到定时器。如果 TIMER 工作中更改计数模式成循环或者单次，需要重写加载值，否则定时器会按照当前计数值继续计数。 例如：LOAD 设置为 N，计数周期为 N+1

15.3.12. TIMER2 计数寄存器(T2_CNT偏移: 2Ch)

比特	名称	属性	复位值	描述
31:0	T2_CNT	R	0	定时器 2counter 的当前计数值。

15.3.13. TIMER2 控制寄存器(T2_CR偏移: 30h)

比特	名称	属性	复位值	描述
31:6	RSV	-	-	保留
5	CMS	R/W	0	中心对齐模式选择位 0: 边沿对齐模式。counter 计数方向由 DIR 决定。 1: 中心对齐模式 1。Counter 往上计数和往下计

				数交替进行（先往上再往下）。
4	DIR	R/W	0	方向控制位 0: counter 向上计数 1: counter 向下计数
3	INTM	R/W	0	定时器 2 中断屏蔽位 0: 定时器 2 中断使能 1: 定时器 2 中断屏蔽。
2:1	MODE	R/W	0	模式选择位 0x: 自由运行模式(free-running), 计数从 0xffffffff 到 0（向下计数）或从 0 到 0xffffffff 循环（向上计数）或先从 0 到 0xffffffff 再从 0xffffffff 到 0 循环（中心对齐模式）。 10: 循环模式(cyclic), 计数从加载值到 0 循环（向下计数）或从 0 到加载值循环（向上计数）或先从 0 到加载值再从加载值到 0 循环（中心对齐模式）。 11: 单次模式(single), 计数从 0 到加载值（向上计数）或从加载值到 0（向下计数），然后停止。
0	EN	R/W	0	定时器 2 使能位 0: 计数器 1 关闭 1: 计数器 1 使能

注：每次配置 TIMER 的控制寄存器，enable 总要最后打开；在 enable 打开的时候，不要去修改控制寄存器。

15.3.14. TIMER2 中断状态寄存器(T2_IF偏移：34h)

比特	名称	属性	复位值	描述
31:1	RSV	-	-	保留
0	INT	R	0	反映定时器 2 的原始中断状态 1: 中断有效。 0: 中断无效。 请写中断清除寄存器清除中断。

15.3.15. TIMER2 中断清除寄存器(T2_CIF偏移：38h)

比特	名称	属性	复位值	描述
31:0	T2_CIF	W	0	对该寄存器写任意值，都会清除计数器 2 的中断状态寄存器。

15.3.16. TIMER3 加载寄存器(T3_ARR偏移：3Ch)

比特	名称	属性	复位值	描述
31:0	LOAD	R/W	0	定时器 3 加载值位 定时器 3 的计数器加载值。在循环模式和单次模式，写操作立即会把加载值加载到定时器。如果 TIMER 工作中更改计数模式成循环或者单次，需要重写加载值，否则定时器会按照当前计数值继续计数。

15.3.17. TIMER3 计数寄存器(T3_CNT偏移：40h)

比特	名称	属性	复位值	描述
31:0	T3_CNT	R	0	定时器 3counter 的当前计数值。

15.3.18. TIMER3 控制寄存器(T3_CR偏移：44h)

比特	名称	属性	复位值	描述
31:6	RSV	-	-	保留
5	CMS	R/W	0	中心对齐模式选择位 0: 边沿对齐模式。counter 计数方向由 DIR 决定。 1: 中心对齐模式 1。Counter 往上计数和往下计数交替进行（先往上再往下）。
4	DIR	R/W	0	方向控制位 0: counter 向上计数 1: counter 向下计数

3	INTM	R/W	0	定时器 3 中断屏蔽位 0: 定时器 3 中断使能 1: 定时器 3 中断屏蔽。
2:1	MODE	R/W	0	模式选择位 0x: 自由运行模式(free-running), 计数从 0xffffffff 到 0 (向下计数) 或从 0 到 0xffffffff 循环 (向上计数) 或先从 0 到 0xffffffff 再从 0xffffffff 到 0 循环 (中心对齐模式)。 10: 循环模式(cyclic), 计数从加载值到 0 循环 (向下计数) 或从 0 到加载值循环 (向上计数) 或先从 0 到加载值再从加载值到 0 循环 (中心对齐模式)。 11: 单次模式(single), 计数从 0 到加载值 (向上计数) 或从加载值到 0 (向下计数), 然后停止。
0	EN	R/W	0	定时器 3 使能位 0: 计数器 1 关闭 1: 计数器 1 使能

注: 每次配置 TIMER 的控制寄存器, enable 总要最后打开; 在 enable 打开的时候, 不要去修改控制寄存器。

15.3.19. TIMER3 中断状态寄存器(T3_IF偏移: 48h)

比特	名称	属性	复位值	描述
31:1	RSV	-	-	保留
0	INT	R	0	反映定时器 3 的原始中断状态 1: 中断有效。 0: 中断无效。 请写中断清除寄存器清除中断。

15.3.20. TIMER3 中断清除寄存器(T3_CIF偏移: 4Ch)

比特	名称	属性	复位值	描述
31:0	T3_CIF	W	0	对该寄存器写任意值, 都会清除计数器 3 的中

				断状态寄存器。
--	--	--	--	---------

15.3.21. 定时器预分频寄存器(TIM_PSC偏移: 50h)

比特	名称	属性	复位值	描述
31:9	RSV	-	-	保留
11:9	DIV3	R/W	0	定时器 3 的预分频位（用各自的定时器使能位来选择相应的计时器分频。） 000: 1 分频 001: 2 分频 010: 4 分频 011: 8 分频 100: 16 分频 101: 32 分频 110: 64 分频 111: 128 分频
8:6	DIV2	R/W	0	定时器 2 的预分频位（用各自的定时器使能位来选择相应的计时器分频。） 000: 1 分频 001: 2 分频 010: 4 分频 011: 8 分频 100: 16 分频 101: 32 分频 110: 64 分频 111: 128 分频
5:3	DIV1	R/W	0	定时器 1 的预分频位（用各自的定时器使能位来选择相应的计时器分频。） 000: 1 分频 001: 2 分频 010: 4 分频 011: 8 分频 100: 16 分频

				101: 32 分频 110: 64 分频 111: 128 分频
2:0	DIV0	R/W	0	定时器 0 预分频位(用各自的定时器使能位来选择相应的计时器分频。) 000: 1 分频 001: 2 分频 010: 4 分频 011: 8 分频 100: 16 分频 101: 32 分频 110: 64 分频 111: 128 分频

注：改变预分配寄存器只能在 **TIMER enable** 关闭时才能进行，在改变预分配寄存器后需要等待至少一个 **TIMER** 时钟（分频后时钟）再打开 **TIMER enable**。

15.3.22. 工作模式控制寄存器ICMODE(偏移：54h)

比特	名称	属性	复位值	描述
31:3	RSV	-	-	保留
2	C2MODE	W/R	0	Channel2 工作模式控制位： 0: 输入捕获模式 1: PWM 模式
1	RSV	-	-	保留
0	C0MODE	W/R	0	Channel0 工作模式控制位： 0: 输入捕获模式 1: PWM 模式

15.3.23. 输入捕获模式控制寄存器CCR(偏移：58h)

比特	名称	属性	复位值	描述
31:6	RSV	-	-	保留
5	TEC2	R/W	0	Channel2 沿触发控制位 0 上升沿触发

				1 下降沿触发
4	INTM2	R/W	0	Channel2 输入捕获模式中断屏蔽位 0 输入捕获模式中断使能 1 输入捕获模式中断屏蔽。
3	EN2	R/W	0	Channel2 输入捕获模式使能位 0: 输入捕获模式关闭 1: 输入捕获模式使能
2	TEC0	R/W	0	Channel0 沿触发控制位 0 上升沿触发 1 下降沿触发
1	INTM0	R/W	0	Channel0 输入捕获模式中断屏蔽位 0 输入捕获模式中断使能 1 输入捕获模式中断屏蔽。
0	EN0	R/W	0	Channel0 输入捕获模式使能位 0: 输入捕获模式关闭 1: 输入捕获模式使能

15.3.24. 输入捕获模式中断状态寄存器CCIF(偏移: 5Ch)

比特	名称	属性	复位值	描述
31:2	RSV	-	-	保留
1	INT2	R	0	Channel2 反映输入捕获模式的中断状态 1: 中断有效。 0: 中断无效。 写 1 清 0
0	INT0	R	0	Channel0 反映输入捕获模式的中断状态 1: 中断有效。 0: 中断无效。 写 1 清 0

15.3.25. Channel0 输入捕获模式counter寄存器C0_CR(偏移: 60h)

比特	名称	属性	复位值	描述
31:0	C0_CR	R	0	输入捕获模式下, 当在 IC0 上出现触发沿的时候, 记录当时的 counter 值。

15.3.26. Channel2 输入捕获模式counter寄存器C2_CR(偏移: 64h)

比特	名称	属性	复位值	描述
31:0	C2_CR	R	0	输入捕获模式下, 当在 IC2 上出现触发沿的时候, 记录当时的 counter 值。

15.3.27. PWM控制寄存器PCR(偏移: 68h)

比特	名称	属性	复位值	描述
31:8	RSV	-	-	保留
7	INTM3	R/W	0	PWM3 中断屏蔽位 0 PWM3 中断使能 1 PWM3 中断屏蔽。
6	EN3	R/W	0	PWM3 使能位 0: PWM3 关闭 1: PWM3 使能
5	INTM2	R/W	0	PWM2 中断屏蔽位 0 PWM2 中断使能 1 PWM2 中断屏蔽。
4	EN2	R/W	0	PWM2 使能位 0: PWM2 关闭 1: PWM2 使能
3	INTM1	R/W	0	PWM1 中断屏蔽位 0 PWM1 中断使能 1 PWM1 中断屏蔽。
2	EN1	R/W	0	PWM1 使能位 0: PWM1 关闭 1: PWM1 使能
1	INTM0	R/W	0	PWM0 中断屏蔽位 0 PWM0 中断使能 1 PWM0 中断屏蔽。
0	EN0	R/W	0	PWM0 使能位 0: PWM0 关闭 1: PWM0 使能

15.3.28. PWM中断状态寄存器CPIF(偏移: 6Ch)

比特	名称	属性	复位值	描述
31:4	RSV	-	-	保留
3	INT3	R	0	反映 PWM3 的中断状态 1: 中断有效。 0: 中断无效。
2	INT2	R	0	反映 PWM2 的中断状态 1: 中断有效。 0: 中断无效。
1	INT1	R	0	反映 PWM1 的中断状态 1: 中断有效。 0: 中断无效。
0	INT0	R	0	反映 PWM0 的中断状态 1: 中断有效。 0: 中断无效。

各个中断状态寄存器写 1 清 0

15.3.29. PWM0 比较寄存器C0_PR(偏移: 70h)

比特	名称	属性	复位值	描述
31:0	P0CR	R/W	0	PWM0 比较寄存器

15.3.30. PWM1 比较寄存器C1_PR(偏移: 74h)

比特	名称	属性	复位值	描述
31:0	P1CR	R/W	0	PWM1 比较寄存器

15.3.31. PWM2 比较寄存器C2_PR(偏移: 78h)

比特	名称	属性	复位值	描述
31:0	P2CR	R/W	0	PWM2 比较寄存器

15.3.32. PWM3 比较寄存器C3_PR(偏移: 7Ch)

比特	名称	属性	复位值	描述
31:0	P3CR	R/W	0	PWM3 比较寄存器

15.4. 使用说明

Timer 的主体部分是四个 32 位的，拥有加载寄存器的计数器。该计数器可以往上、往下、往上/往下计数，也可以处于 free-running, cyclic, single 三种计数模式。计数时钟可以由可编程预分频因子控制。

控制寄存器、加载寄存器、可编程预分频因子都可以用软件读写。

控制这些的寄存器有：

Timer 控制寄存器 (Tx_CR)

可编程预分频因子寄存器 (TIM_PSC)

加载寄存器 (Tx_ARR)

加载寄存器是预先装载的。每次写加载寄存器只是更新了加载寄存器值。只有在 TIMER enable 打开时或者更新事件 (UEV) 发生时加载寄存器值才会真正更新到芯片内部寄存器中。

Counter 的 clock 由可编程预分频因子决定。

另外要注意：

每次配置 Timer 的控制寄存器，enable 总要最后打开；在 enable 打开的时候，不要去修改控制寄存器。

15.4.1. Counter 工作模式

1. 计数方向

- 往上计数

在往上计数模式中，counter 从 0 计数到自动重载值，然后重新到 0 开始计数，并产生中断。而且在此时，UEV 事件发生。

当 UEV 事件发生时，芯片内部加载寄存器才会被更新。

- 往下计数

在往下计数模式中，counter 从自动重载值计数到 0，然后重新到自动重载值开始计数，并产生中断。而且在此时，UEV 事件发生。

当 UEV 事件发生时，芯片内部加载寄存器才会被更新。

- 中心对齐模式（上下计数）

在中心对齐模式中，counter 从 0 计数到自动重载值-1，产生中断；然后又从自动重载值计数到 1，产生中断；然后又从 0 开始计数。当 counter 处于中心对齐模式时，DIR 寄存器无效。

每次向上溢出和向下溢出时，UEV 事件发生。

当 UEV 事件发生时，芯片内部加载寄存器才会被更新。

2. 计数模式

- free-running

在 free-running 模式中, counter 的溢出值是 0xFFFFFFFF, counter 循环计数。

- cyclic

在 cyclic 模式中, counter 的溢出值是可配置的, 配置为自动重载值, 而且只在 UEV 事件发生时才会把加载寄存器中的值加载过来, counter 循环计数。

- single

在 single 模式中, counter 的溢出值是可配置的, 配置为自动重载值, 而且只在 UEV 事件发生时才会把加载寄存器中的值加载过来, counter 只计数一次。

注: PWM 模式和输入捕获模式的计数功能是基于 timer 的定时器功能来实现的。输入捕获的 channel0 采用了 timer0, channel2 采用了 timer2。PWM 的 0/1/2/3 对应于 timer0/1/2/3。

15.4.2. PWM模式

PWM 模式可以产生波形, 其频率取决于 Tx_ARR 寄存器和 TIM_PSC, 而占空比取决于 Cx_PR 寄存器。

使用 PWM 功能时, 用户必须使能对应的 counter 和 PWM; 同时, 如果是使用 IC0 和 IC2, 还需要在 ICMODE 中将其配置为 PWM 输出。

Timer 是产生边沿对齐的 PWM 还是中心对齐的 PWM 取决于 Tx_CR 中的 CMS。

每个 PWM 周期都会产生中断, 产生中断的时间为 $Tx_CNT = Cx_PR$ 。

1. PWM 边沿对齐模式

- 往上计数

在下面这个例子中, 我们考虑 counter 往上计数模式。当 $Tx_CNT < Cx_PR$ 时, ICx 输出高; 其余情况, ICx 输出低。中断只在 $Tx_CNT = Cx_PR$ 时产生。

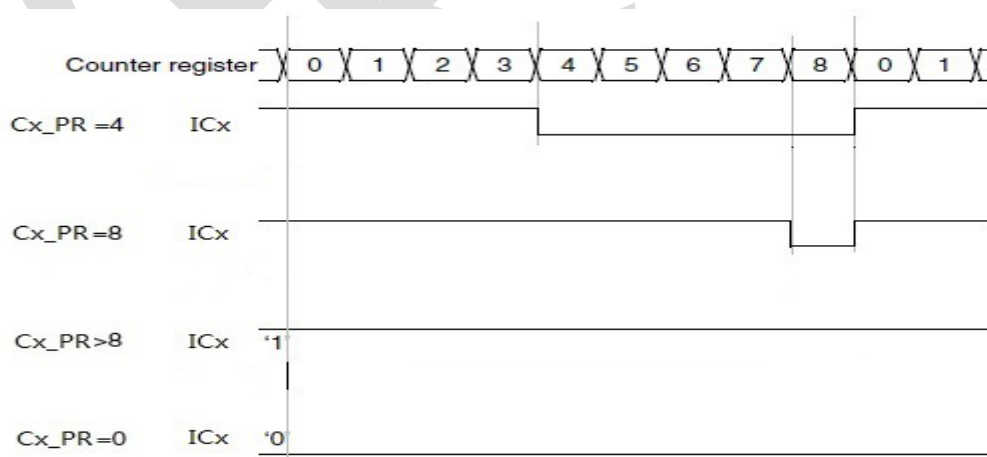


图 15-1 TIMER 向上计数的 PWM 方式

- 往下计数

Counter 处于往下计数模式, 当 $Tx_CNT > Cx_PR$ 时, ICx 输出低; 其余情况, ICx 输出高。

中断只在 $Tx_CNT = Cx_PR$ 时产生。

2. PWM 中心对齐模式

1) 往上计数时, 当 $Tx_CNT < Cx_PR$ 时, ICx 输出高; 其余情况, ICx 输出低。中断只在 $Tx_CNT = Cx_PR$ 时产生。

2) 往下计数时, 当 $Tx_CNT > Cx_PR$ 时, ICx 输出低; 其余情况, ICx 输出高。中断只在 $Tx_CNT = Cx_PR$ 时产生。

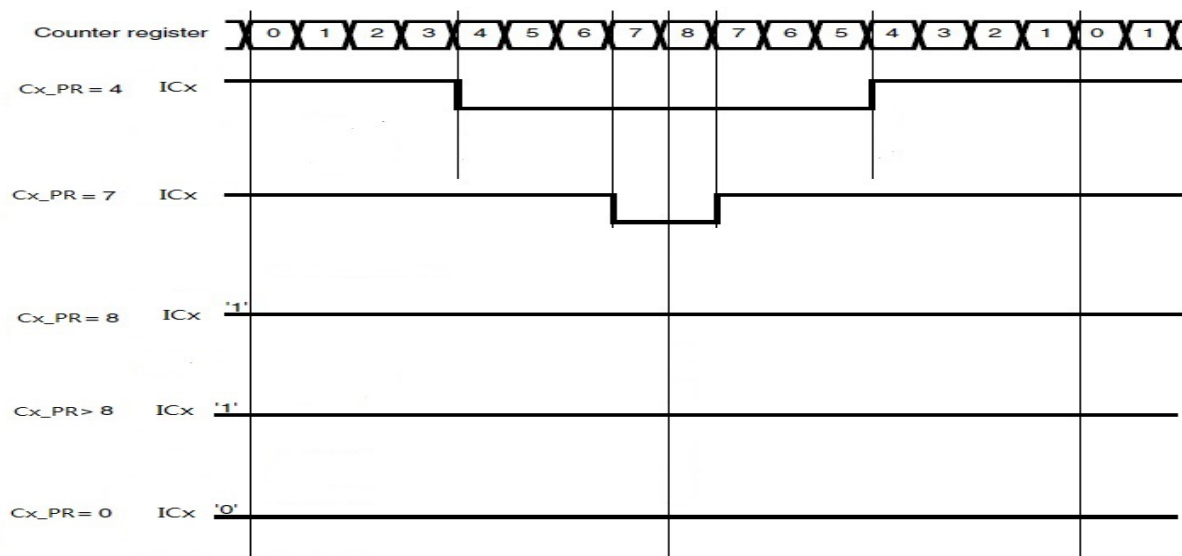


图 15-2 TIMER 向上下计数的 PWM 方式

15.4.3. 输入捕获模式

在输入捕获模式中, 当在相应的 ICx 信号出现触发沿的时候, 捕捉寄存器 (Cx_CR) 会把当时的 counter 值保存下来。当一次捕获发生后, 相应的中断标志被置位, 同时产生一次捕获中断。CCxIF 由软件清 0。触发变化沿可以由寄存器控制是上升沿或下降沿。

注意: 触发信号高电平与低电平持续时间必须大于 5 个内部时钟周期。

16. WDT

16.1. 概述

看门狗定时器是一个可编程预置计数值的向下计数器。在计数到达 0 时,会产生中断或者复位。为了避免周期性的产生中断或者产生复位,软件需要周期性的喂狗。

16.2. 主要特性

- 32 位的向下计数器
- 可编程的时钟预分配
- 可编程的时钟溢出周期
- 可编程的中断时限时间

16.3. 寄存器描述

WDT 寄存器基地址: 0x48040000

偏置	名称	描述
0x00	LOAD	数据加载寄存器
0x04	COUNT	当前计数寄存器
0x08	CONTROL	控制寄存器
0x0C	FEED	喂狗寄存器
0x10	INTCLRT	中断清除时限寄存器
0x14	RAWIS	原始中断状态寄存器

16.3.1. 数据加载寄存器LOAD(偏移: 00h)

比特	名称	属性	复位值	描述
31:0	LOAD	R/W	0xFFFFFFFF	32 位寄存器, 用来存入加载值。

16.3.2. 当前计数值(偏移: 04h)

比特	名称	属性	复位值	描述
31: 0	COUNT	RO	0xFFFFFFFF	32 位寄存器, 看门狗当前计数值。

16.3.3. 控制寄存器(偏移: 08h)

比特	名称	属性	复位值	描述
31:8	RSV	-	-	保留
7	EN	R/W	0	看门狗使能位: 0 禁止, 1 使能。
6	MODE	R/W	0	看门狗模式位: 0 复位, 1 中断。

5	RSV	-	-	保留
4	INTEN	R/W	0	看门狗中断使能位：0 禁止，1 使能。
3	RSV	-	-	保留
2:0	DIVISOR	R/W	011	看门狗时钟预分频： 3'b000: 不分频 3'b001: 2 分频 3'b010: 4 分频 3'b011: 8 分频 3'b100: 16 分频 3'b101: 32 分频 3'b110: 64 分频 3'b111: 128 分频

16.3.4. 喂狗寄存器(偏移：0Ch)

比特	名称	属性	复位值	描述
31:0	FEED	R/W	0x00000000	32 位寄存器，喂狗寄存器，写入特征值 0xAA55A55A 使计数器从装载寄存器加载装载值，加载动作会清除 WDT 中断。

16.3.5. 中断清除时限寄存器(偏移：10h)

比特	名称	属性	复位值	描述
31:16	RSV	-	-	保留
15:0	INTCLRT	R/W	0x0000	中断清除时限寄存器，中断产生后(即使中断未使能)，在这个寄存器定义的时间内没有去喂狗，WDT 会产生复位信号。

16.3.6. 原始中断状态寄存器(偏移：14h)

比特	名称	属性	复位值	描述
31:1	RSV	-	-	保留
0	WDTRIS	RO	0	原始中断状态标志位。

16.4. 使用流程

16.4.1. 定时器溢出产生中断

- 配置加载寄存器；
- 配置中断清除时限寄存器；
- 配置定时器时钟预分频值；

- 配置看门狗工作模式;
- 配置中断使能位;
- 使能看门狗定时器。

注意：如果没有使能中断，则在计数到 0 溢出，经过中断清除时限寄存器值个计数后，将产生一个系统复位。

16.4.2. 定时器溢出产生复位

- 配置加载寄存器;
- 配置中断清除时限寄存器;
- 配置定时器时钟预分频值;
- 配置看门狗工作模式;
- 配置中断使能位;
- 使能看门狗定时器。

注意：计数器计数到 0 溢出，将产生一个系统复位。

17. I2C

17.1. 概述

I2C(芯片间)总线接口连接微控制器和串行 I2C 总线。I2C 模块接收和发送数据，并将数据从串行转换成并行，或并行转换成串行。可以开启或禁止中断。接口通过数据引脚(SDA)和时钟引脚(SCL)连接到 I2C 总线，控制所有 I2C 总线特定的时序、协议。不支持多主机模式，不支持总线仲裁。

17.2. 主要特性

- 并行总线/I2C 总线协议转换器;
- I2C 主设备功能;
- 8bytes FIFO 深度;
- 支持 Standard/Fast/HS 模式
- 支持 7bit 设备地址;

17.3. 寄存器描述

I2C 寄存器基地址: 0x48070000

偏置	名称	描述
0x00	I2C_CSR	控制状态寄存器
0x04	I2C_SLAVE_ADDR	I2C 总线设备地址寄存器
0x08	I2C_CLK_DIV	I2C 的 SCL 速率分频寄存器
0x0C	I2C_FIFO_CTRL	FIFO 控制寄存器
0xffe0	I2C_INT_STAT	中断状态寄存器
0xffe4	I2C_INT_STAT_RAW	原始中断状态寄存器
0xffe8	I2C_INT_EN	中断使能寄存器
0xffec	I2C_INT_SET	中断置位寄存器
0xffff0	I2C_INT_CLR	中断清除寄存器
0x100	I2C_FIFO	FIFO 接口寄存器

17.3.1. 控制状态寄存器I2C_CSR (偏移: 00h)

比特	名称	属性	复位值	描述
31:14	RSV	-	-	保留
13	I2C_ACK_STATUS	RO	0	Slave 返回的 ACK/NACK 状态值 0: 返回 ACK 1: 返回 NACK 当此位为 1 时,此位会保持为 1,直到

				向此位写 1 后,此位清 0.
12	I2C_FORCE_START	WR	0	当 I2C 总线不空闲时, 强制启动操作 1: 强制启动 0: 不强制启动
11	I2C_APPLY_STOP	WR	1	当接收和发送完成时, 硬件是否产生 STOP 信号 1: 硬件自动产生 STOP 信号 0: 不产生 STOP 信号
10	I2C_START_WR	WO	0	启动一个带 7bit 设备地址的写操作 1: 启动写 0: 不启动写
9	I2C_START_RD_WITH_ADDR	WO	0	启动一个带 7bit 设备地址的读操作 1: 启动读 0: 不启动读
8	I2C_START_RD	WO	0	启动一个不带设备地址的读操作 1: 启动读 0: 不启动读
7:0	I2C_BYTE_CNT	RW	0	需要发送或者接收的 byte 数, 当发送或者接收的 byte 数等于该寄存器配置的值时, 硬件会产生发送或者接收完成。如果 I2C_APPLY_STOP 设置为 1 此时硬件自动产生 NACK 和 STOP。

17.3.2. 总线设备地址寄存器 I2C_SLAVE_ADDR (偏移: 04h)

比特	名称	属性	复位值	描述
31:8	RSV	-	-	保留
7: 0	I2C_SLAVE_ADDR	RW	0	7bit I2C 总线设备地址, 在读写启动之前配置该寄存器

17.3.3. SCL速率分频寄存器 I2C_CLK_DIV (偏移: 08h)

比特	名称	属性	复位值	描述
31:8	RSV	-	-	保留

7:0	I2C_CLK_DIV	RW	3	SCL 分频值，通过配置该寄存器设置 I2C 的传输速率。 $F_{scl} = f_{\text{系统时钟}}/16/(\text{DIV 寄存器值})$ 注意：该值必须 ≥ 2
-----	-------------	----	---	--

17.3.4. XFIFO控制寄存器I2C_FIFO_CTRL (偏移：0Ch)

比特	名称	属性	复位值	描述
31:5	RSV	-	-	保留
4	FIFO_ALMOST_EMPTY	RO	1	FIFO 几乎空标志 1: FIFO 中的数据数量小于等于 1bytes 0: FIFO 中的数据数量大于 1bytes
3	FIFO_ALMOST_FULL	RO	0	FIFO 几乎满标志 1: FIFO 中数据数量大于等于 6bytes 0: FIFO 中的数据数量小于 6bytes
2	FIFO_FULL	RO	0	FIFO 是否满标志 1: FIFO 满 0: FIFO 非满
1	FIFO_EMPTY	RO	1	FIFO 是否为空标志 1: FIFO 空 0: FIFO 非空
0	FIFO_CLEAR	WO	0	clear the fifo status 写 1: 清除 FIFO 状态 写 0: 不清除

17.3.5. 中断状态寄存器寄存器I2C_INT_STAT (偏移：0xffe0)

比特	名称	属性	复位值	描述
31:3	RSV	-	-	保留
2	I2C_FIFO_OVERRUNFLOW	RO	0	FIFO 满溢出和空溢出中断状态位，产生该中断后在中断服务程序中需检查 I2C_FIFO_CTRL 寄存器确定具体 FIFO 状态
1	I2C_WRITE_DONE	RO	0	I2c 写完成中断状态位,其值为 I2C_WRITE_DONE_RAW 与 I2C_WRITE_DONE_EN 的逻辑与。
0	I2C_READ_DONE	RO	0	I2c 读完成中断状态位。其值为 I2C_READ_DONE_RAW 与 I2C_READ_DONE_EN 位的逻辑与。

17.3.6. 中断原始状态寄存器 I2C_INT_STAT_RAW (偏移: 0xffe4)

比特	名称	属性	复位值	描述
31:3	RSV	-	-	保留
2	I2C_FIFO_OV ERUNDERFL OW_RAW	RO	0	FIFO 满溢出和空溢出中断状态位, 产生该中断后在中断服务程序中需检查 I2C_FIFO_CTRL 寄存器确定具体 FIFO 状态
1	I2C_WRITE_ DONE_RAW	RO	0	I2c 写完成中断状态位
0	I2C_READ_D ONE_RAW	RO	0	I2c 读完成中断状态位。

17.3.7. 中断使能寄存器 I2C_INT_EN (偏移: 0xffe8)

比特	名称	属性	复位值	描述
31:3	RSV	-	-	保留
2	I2C_FIFO_OV ERUNDERFL OW_EN	WR	0	FIFO 溢出中断使能 1: 使能 FIFO 溢出中断 0: 禁止 FIFO 溢出中断
1	I2C_WRITE_ DONE_EN	WR	0	写完成中断使能 1: 使能写完成中断 0: 禁止写完成中断
0	I2C_READ_D ONE_EN	WR	0	读完成中断使能 1: 使能读完成中断 0: 禁止读完成中断

17.3.8. 中断设置寄存器 I2C_INT_SET (偏移: 0xffec)

比特	名称	属性	复位值	描述
31:3	RSV	-	-	保留
2	I2C_FIFO_OVE RUNDERFLO W	WO	0	FIFO 溢出中断置位, 软件写 1 后会产生 FIFO 溢出中断, 该位可以测试中断产生 1: 测试 FIFO 溢出中断产生 0: 不测试 FIFO 溢出中断产生
1	I2C_WRITE_D ONE	WO	0	写完成中断置位, 软件写 1 后会产生写完成中断, 该位可以测试中断产生 1: 测试写完成中断产生 0: 不测写读完成中断产生

0	I2C_READ_DONE	WO	0	读完成中断置位，软件写 1 后会产生读完成中断，该位可以测试中断产生 1: 测试读完成中断产生 0: 不测试读完成中断产生
---	---------------	----	---	---

17.3.9. 中断清除寄存器 I2C_INT_CLR (偏移: 0xfff0)

比特	名称	属性	复位值	描述
31:3	RSV	-	-	保留
2	I2C_FIFO_OVERUNDERFLOW	WO	0	清除 FIFO 溢出中断 1: 清除 FIFO 溢出中断 0: 不清除 FIFO 溢出中断
1	I2C_WRITE_DONE	WO	0	清除写完成中断 1: 清除写完成中断 0: 不清除写完成中断
0	I2C_READ_DONE	WO	0	清除读完成中断 1: 清除读完成中断 0: 不清除读完成中断

17.3.10. FIFO接口寄存器 I2C_FIFO (偏移: 0x100)

比特	名称	属性	复位值	描述
31:8	RSV	-	-	保留
7:0	DATA	RW	0	发送或者接收的数据

17.4. 使用流程

17.4.1. I2C写

- 1.配置 I2C_CLK_DIV 设置 SCL 的速率，写 FIFO_CLEAR 清除 FIFO 状态；
- 2.配置 I2C_SLAVE_ADDR 表明要对 I2C 总线上哪个 slave 设备进行写操作，写操作时 7bit 地址的 bit0 需配置为 0；
- 3.配置 I2C_BYTE_CNT 表明本次需要写多少个字节，配置 I2C_APPLY_STOP 表明写结束时是否需要产生 STOP 信号；
- 4.配置 I2C_START_WR 为 1，开始写操作，检查 FIFO 满状态，当 FIFO 状态为不满时向 FIFO 装载需要写的数据，检查写完成标志位，当写完成时退出写操作；
- 5.写过程中检查 ACK 标志位，当 ACK 错误标志位置 1 时表明写失败。

17.4.2. I2C读

- 1.配置 I2C_CLK_DIV 设置 SCL 的速率，写 FIFO_CLEAR 清除 FIFO 状态；
- 2.配置 I2C_SLAVE_ADDR 表明要对 I2C 总线上哪个 slave 设备进行读操作，读操作时 7bit 地址的 bit0 需配置为 1；
- 3.配置 I2C_BYTE_CNT 表明本次需要读多少个字节，配置 I2C_APPLY_STOP 表明读结束时是否需要产生 STOP 信号；
- 4.配置 I2C_START_RD_WITH_ADDR 为 1，开始读操作，检查 FIFO 空状态，当 FIFO 状态为不空时读取 FIFO 中的数据，检查读完成标志位，当读完成时退出读操作；
- 5.读过程中检查写设备地址后的 ACK 标志位，当 ACK 错误标志位置 1 时表明读失败；
- 6.对于 I2C 存储器的读过程一般需要先写需要读的内部存储器地址。

18. ISO7816MS

18.1. 概述

7816MS 模块可作为满足 7816-3 标准的卡或读卡器使用，并同时兼容 T = 0 和 T = 1 传输协议。

18.2. 主要特性

- 支持 ISO7816 主从接口；
- 7816 作为卡模式使用时，符合 ISO / IEC 7816-3 和 EMVCo 2011（EMV4.3）标准；
- 支持异步字符（T=0）和异步块（T=1）传输协议；
- 支持字符 MSB 和 LSB；
- 发送时硬件自动生成奇或偶校验位，自动检测错误响应，检测到传输错误响应信号由硬件自动重发字符（自动重发不超过 7 次），是否重发可配；
- 接收时硬件产生奇或偶校验错误响应信号支持使能/禁止；
- 支持硬件 LRC 校验字节，支持使能/清零/读取操作；
- 支持发送时的额外 ETU 功能；
- 支持接收时最大等待时间配置，并提供超时中断；
- FIFO 深度 8 字节

18.3. 寄存器描述

7816MS 寄存器基地址：0x48080000

偏置	名称	描述
0x00	SCISR	中断状态寄存器
0x04	SCIIR	中断使能寄存器
0x08	SCICTRL	控制寄存器
0x0c	SCIMCTRL	主模式控制寄存器
0x10	SCIDR	数据寄存器
0x14	SCIRSTT	复位时间寄存器
0x18	SCIBPR	波特率设定寄存器
0x1c	SCIETU	最小保护间隔和最大等待间隔 ETU 计数寄存器
0x20	SCIEDC	错误校验码寄存器
0x24	SCICCKCNT	CCK 计数寄存器

18.3.1. 状态寄存器SCISR(偏移: 00h)

比特	名称	属性	复位值	描述
31:14	RSV	-	-	保留
13	RX_IDLE	RO	1	接收空闲标志, 硬件置位和清零 RX_IDLE =0: 接收没有完成; RX_IDLE =1: 接收彻底完成。 注: 接收转发送时, 请先查询此位为 1
12	CARD_PRESENT	RO	0	主模式下卡检测寄存器 1: 有卡 0: 无卡
11	CARD_IN	R/W	0	卡插入标志 (写 0 清除, 写 1 无影响) CARD_IN=0: 没有卡插入动作 CARD_IN=1: 有卡插入动作
10	CARD_OUT	R/W	0	卡拔出标志 (写 0 清除, 写 1 无影响) CARD_OUT=0: 没有卡拔出动作 CARD_OUT=1: 有卡拔出动作
9	CCKCNT	R/W	0	CCK 计数溢出标志 (写 0 清除, 写 1 无影响): CCKCNT =0: 没有溢出; CCKCNT =1: 溢出; 当 CCK 计数器不为 0 时此位才有效。
8	ETU_CNT	R/W	0	ETU 定时计数溢出标志 (写 0 清除, 写 1 无影响): ECNTO =0: 没有溢出; ECNTO =1: 溢出; 使能后, 会从使能时刻开始计时 (单位 ETU), 如果一直未收到后续字符, 计数值到达 SCIETU. ECNT_NO 设定的值时, SCISR. ETU_CNT 将置位, 如中断使能信号 SCHIER. ECNT_INTEN 使能将产生中断; 如果在规定时间内接收到字符则计时停止并不产生中断。 ETU_CNT, 只在 IP 处于接收状态下使用。
7	T2R	R/W	0	SCI 快速发送转接收标志, 硬件置位, 软件清 0 (写 0 清除, 写 1 无影响): T2R =0: 发送没有完成 T2R =1: 发送完成,

				此标志较 TXEND 标志快 0.5 个 etu, 可用于快速从发送转为接收, 软件查询到此标志后可立即设置当前工作模式为接收模式以避免长时间等待而错过接收; 此标志位的置位受到寄存器 SCICTRL.PROTOCOL 的控制, 当 T=0 时在发送后第 11.5 个 ETU 时置位, 当 T=1 时在发送后第 10.5 个 ETU 时置位。
6	FIFO_NE	R/W	0	FIFO 非空标志, 实时反应 FIFO 状态 (需读取 FIFO 值以清零): FIFO_NE =0: FIFO 空; FIFO_NE =1: FIFO 非空。
5	FIFO_HF	R/W	0	FIFO 半满标志, 实时反应 FIFO 状态 (需读取 FIFO 值以清零): FIFO_HF =0: FIFO 非半满; FIFO_HF =1: FIFO 半满。
4	FIFO_FU	R/W	0	FIFO 全满标志, 实时反应 FIFO 状态 (需读取 FIFO 值以清零): FIFO_FU =0: FIFO 非全满; FIFO_FU =1: FIFO 全满。
3	FIFO_OV	R/W	0	Rx-FIFO 接收溢出错误, 硬件置位, 软件清 0 (写 0 清除, 写 1 无影响): FIFO_OV =0: 没有接收溢出错误发生; FIFO_OV =1: 发生了接收溢出错误; SCISR.FIFO_OV 被设置为 1, CPU 读此位为 1 后必须从接收 FIFO 读数据并且将 SCISR.FIFO_OV 清零。
2	TXEND	RW	0	SCI 发送完成标志, 硬件置位, 软件清 0 (写 0 清除, 写 1 无影响): TXEND =0: 表示发送没有完成; TXEND =1: 发送完成, 硬件设置, 软件清 0;
1	PARITY	R/W	0	SCI 发送/接收奇偶校验错误标示, 硬件置位, 软件清 0 (写 0 清除, 写 1 无影响): TRE =0: 无奇偶校验错误; TRE =1: 有奇偶校验错误;
0	RETRY_ERR	R/W	0	硬件自动重传 RETRY_TIMES 次仍出错标志, 硬件置

				位，软件清零（写 0 清除,写 1 无影响）： RETRY_ERR =0: 重传未超过 SCICTRL. RETRY_TIMES 规定次数； RETRY_ERR =1: 重传达到 RETRY_TIMES 规定次数 仍有错，硬件停止重传，必须软件清除此位下次奇偶校 验错硬件重传才可正常进行。
--	--	--	--	--

18.3.2. 中断使能寄存器SCIHER(偏移：04h)

比特	名称	属性	复位值	描述
31:12	RSV	-	-	保留
11	CARD_INEN	R/W	0	卡插入中断使能 0: 禁止； 1: 使能；
10	CARD_OUT EN	R/W	0	卡拔出中断使能 0: 禁止； 1: 使能；
9	CCKCNT_IN TEN	R/W	0	CCK 计数溢出中断使能， 0: 禁止； 1: 使能；
8	ECNT_INTE N	R/W	0	字符间隔 ETU TIMER 计数溢出中断使能， 0: 禁止； 1: 使能；
7	T2R_INTEN	R/W	0	TX-快速发送转接收标志使能， 0: 禁止； 1: 使能；
6	FIFO_NE_IN TEN	R/W	0	FIFO 非空中断使能， 0: 禁止； 1: 使能；
5	FIFO_HF_IN TEN	R/W	0	FIFO 半满中断使能， 0: 禁止； 1: 使能；
4	FIFO_FU_IN TEN	R/W	0	FIFO 全满中断使能， 0: 禁止； 1: 使能；

3	FIFO_OV_INTEN	R/W	0	Rx-FIFO 接收溢出中断标志使能， 0: 禁止； 1: 使能；
2	TXEND_INTEN	R/W	0	SCI 发送完成中断使能， 0: 禁止； 1: 使能；
1	PARITY_INTEN	R/W	0	SCI 发送/接收奇偶校验错误中断使能， 0: 禁止； 1: 使能；
0	RETRY_ERR_INTEN	R/W	0	硬件自动重传超过 RETRY_TIMES 次中断使能， 0: 禁止； 1: 使能；

18.3.3. 控制寄存器SCICTRL(偏移：08h)

比特	名称	属性	复位值	描述
31:17	RSV	-	-	保留
16	MS_SEL	RW	0	主从模式选择 1: 选择主模式 0: 选择从模式
15	DATAOE_EN	R/W	0	延长输出使能至 10.125etu，使能后使停止位电平能迅速拉高。 0: 不使能； 1: 使能；
14:12	RETRY_TIMES	R/W	3	重发次数配置： 000: 0 次； 001: 1 次； 010: 2 次； ... 111: 7 次；
11	RXNAK	R/W	1	使能接收数据握手信号（当接收数据奇偶校验错误时下拉数据线） 0: 不使能；（不下拉数据线，接收的数据还是会送入 FIFO）

				1: 使能; (下拉数据线, 接收的数据不送入 FIFO)
10	ECNT_EN	WO	0	SCIETU. ECNT_NO 计时使能 (软件写 1 有效, 高电平持续一个周期, 硬件自动清零) 0: 不使能; 1: 使能; 使能后, 会从使能时刻开始计时 (单位 7816clk), 如果一直未收到后续字符, 计数值到达 SCIETU. ECNT_NO 设定的值时, SCISR. ETU_CNT 将置位, 如中断使能信号 SCHIER. ECNT_INTEN 使能将产生中断; 如果在规定时间内接收到字符则计时停止并不产生中断。
9	CCKCNT_EN	R/W	1	CCK 计数器使能位 0: 不使能; 1: 使能
8:7	RSV	-	-	保留
6	EDC_EN	R/W	0	硬件自动生成错误检测使能位: 0: 软件生成; 1: 硬件自动生成; 发送和接收生成的错误校验码在 SCIEDC 寄存器存储
5	PROTOCOL	R/W	0	传输协议选择 (只影响发送数据的停止位) 0: T=0; (两个停止位) 1: T=1; (一个停止位) 传输协议主要影响字符发送最小间隔; 软件必须保持 SCIETU 设置、该位设置以及 ATR 字节内容三者之间的一致性;
4	FLUSH	R/W	0	接收 FIFO 清除: FLUSH =0: 不清空 FIFO ; FLUSH =1: 清空 FIFO;
3	TRS	R/W	0	SCI 发送/接收模式选择: TRS =0: 选择接收模式; TRS =1: 选择发送模式;
2	RETRY_EN	R/W	0	硬件自动重传使能: RETX_EN =0: 软件负责重发;

				RETX_EN =1: 硬件重发; 发送前设置此位, 如出现奇偶校验错误, 由硬件自动重发 RETRY_TIMES 次;
1	ODD_EN	R/W	0	奇偶校验方式选择: ODD_EN =0: 偶校验; ODD_EN =1: 奇校验;
0	DIS	R/W	0	正反向模式选择 Direct Inverse Mode: DIS =0: 直接模式, 发送数据不改变, LSB 先发; 接收数据不改变, LSB 先收。TS 字节必须为 H'3B; DIS =1: 反转模式, 发送数据发送前 Bit 反转, MSB 先发, 接收数据 MSB 先收, 存储前 Bit 反转。TS 字节必须为 H'3F 推荐采用直接模式;

18.3.4. 主模式控制寄存器SCIMCTRL(偏移: 0Ch)

比特	名称	属性	复位值	描述
31:12	RSV	-	-	保留
11:4	7816CLK_DIV	RW	0	主模式下 7816clk 分频选择寄存器。 从系统时钟分频得到 7816 时钟 分频数等于 7816CLK_DIV 设置值的 2 倍, 即 $2 \times 7816CLK_DIV$ 。 注分频值至少为 1
3	CLK_STOP	RW	0	主模式下时钟停止寄存器。 1: 时钟停止功能开启 0: 时钟停止功能关闭
2	PWR_UP	WO	0	当处于主模式, 即 MS_SEL 为 1 时, 写该寄存器产生一次冷复位流程, 硬件自动控制 7816rst、vccen、7816clk 信号, 一次上电后只能进行一次冷复位操作下电前多次写该位无效。 只写硬件自动清零, 当 MS_SEL 为 0 时, 写无效
1	WARM_RST	WO	0	当处于主模式, 即 MS_SEL 为 1 时, 写该寄存器产生一次热复位流程, 硬件自动控制 7816rst 信号, 上电冷复位前无法进行热复位。 只写硬件自动清零, 当 MS_SEL 为 0 时, 写无效

0	DOWN	WO	0	当处于主模式，即 MS_SEL 为 1 时，写该寄存器产生一次下电流程，硬件自动控制 7816rst、vccen、7816clk 信号。 只写硬件自动清零，当 MS_SEL 为 0 时，写无效
---	------	----	---	---

18.3.5. 数据寄存器SCIDR(偏移：10h)

比特	名称	属性	复位值	描述
31:8	RSV	-	-	保留
7:0	SCIDR	R/W	0	接收发送复用寄存器。 写时，写的的数据放入发送缓冲区； 读时，读到的是数据是接收缓冲区的值。

18.3.6. 复位时间寄存器SCIRSTT(偏移：14h)

比特	名称	属性	复位值	描述
31:16	RSV	-	-	保留
15:0	SCIRSTT	R/W	0x190	复位时间设置寄存器。 冷复位时表示从 7816clk 有效至 RST 信号拉高的时间； 热复位时表示 RST 信号拉低的时间。 单位为从系统时钟分频得到 7816 时钟

18.3.7. 波特率寄存器SCIBPR(偏移：18h)

比特	名称	属性	复位值	描述
31:12	RSV	-	-	保留
11:0	SCIBPR	R/W	0x174	例如：假设接口时钟为 3.57MHz，为获得 9600 波特率，则 $SCIBPR = 3.57 \times 1000000 \div 9600 = 372$ ，即 SCIBRP=0174H。

18.3.8. ETU计数寄存器SCIETU(偏移：1Ch)

比特	名称	属性	复位值	描述
31:16	BWT	R/W	0	BWT 位反映了最大等待 ETU 数，从写 ISO7816MS_CTRL.WT_EN 使计时使能开始计时如果达 BWT 设置值后,将使 ISO7816MS_ISR.ECNT0 置位,如同时有 ISO7816MS_IER .ECNT_INTEN 使能,则产生一个 etu 计时溢出中断。 注：不能作为 CWT 计数器使用

15:0	extra_ETU	R/W	0	extra_ETU 位反映了发送一个字符后需额外等待的 ETU 数，每次发送结束后硬件自动计时至 extra_ETU 后才开始下一次发送操作。
------	-----------	-----	---	---

18.3.9. 错误校验码寄存器SCIEDC(偏移：20h)

比特	名称	属性	复位值	描述
31:12	RSV	-	-	保留
7:0	EDCDR	R/W	0	使能后接收和发送的数据均会按字节做异或校验存储在此字节,通讯时可实现硬件自动计算 TCK/PCK/LRC,但必须在适当的时候读取和清零,否则,计算结果会有误;

18.3.10. CCK计数寄存器SCICCKCNT(偏移：24h)

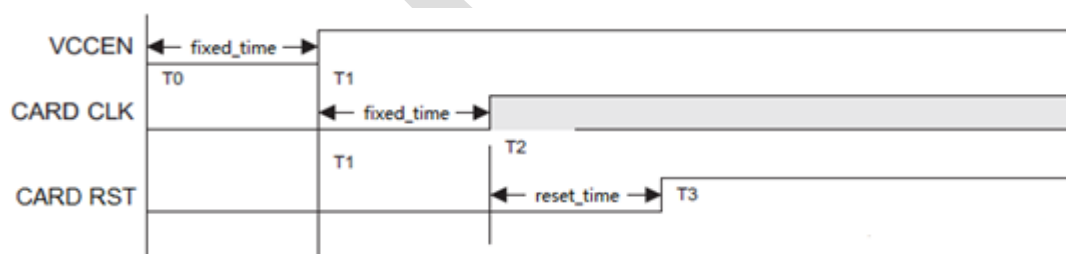
比特	名称	属性	复位值	描述
31:0	CCKCNT	R/W	0x190	CCK 计数器

18.4. 使用流程

18.4.1. 主模式选择及上下电

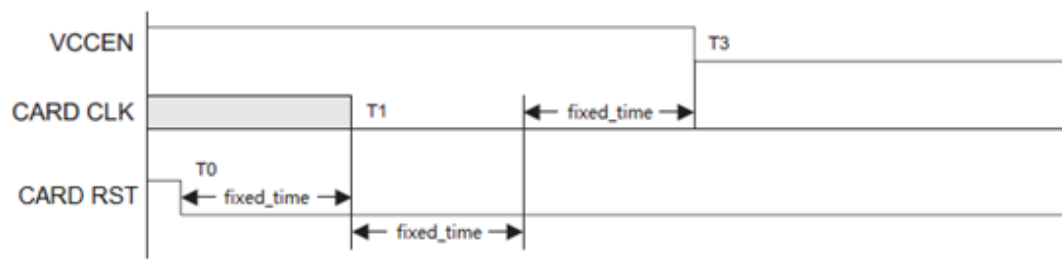
STEP1: 写 ISO7816MS_CTRL.MS_SEL 位为 1, 将 IP 置位主模式工作状态, 如果需要进行上电操作, 转 STEP2; 如果需要下电操作, 转 STEP3; 如果需要热复位, 转 STEP4。

STEP2: 对 ISO7816MS_CTRL.PWR_UP 位写 1, 硬件将会发起一次上电复位(即冷复位)操作, 硬件自动控制信号 Vccen、7816rst 以及 7816Clk, 软件无需参与; 7816IO 信号需要软件参与控制(在冷复位前, 7816IO 需要配置成 GPIO 模式并输出高电平, 等待 RST 状态变高后, 再恢复成 7816IO 模式)。其中图中 T0 到 T1 以及 T1 到 T2 的时间是固定的为 1000 个 7816 主时钟; T2 到 T3 的时间通过复位时间寄存器设置, 为 ISO7816MS_RSTT 个 7816 主时钟。

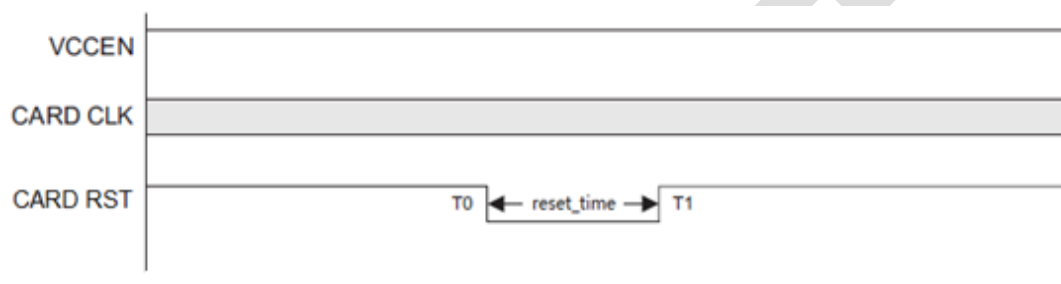


STEP3: 对 ISO7816MS_CTRL.DOWN 位写 1, 硬件将会发起一次下电操作, 硬件自动控制信号 Vccen、7816rst 以及 7816Clk, 软件无需参与; 7816IO 信号需要软件参与控制(配置成 GPIO 模式输出低电平)。

其中图中 T0 到 T1、T1 到 T2 和 T2 到 T3 的时间是固定的为 1000 个 7816 主时钟



STEP4: 对 ISO7816MS_CTRL.WARM_RST 位写 1, 硬件将会发起一次热复位操作, 硬件自动控制信号 Vccen7816rst 以及 7816Clk, 软件无需参与。其中图中 T0 到 T1 的时间通过复位时间寄存器设置, 为 ISO7816MS_RSTT 个 7816 主时钟。



18.4.2. 数据收发初始化

STEP1: 置 SCIHER=0

STEP2: 在接收前, 清空 Receive-FIFO (SCICTRL.FLUSH =1)

STEP3: 清 SCISR 中的状态标志

STEP4: 配置 SCIBPR

STEP5: 设置或清零 SCICTRL.TRS 位, 设置或清零 SCICTRL.PROTOCOL 位 (T=0/T=1)

STEP6: 设置 etu 计数器值包括发送时用的最小字符间隔 (ISO7816MS_ETU.extra_ETU)、接收转发时的最大等待间隔 (ISO7816MS_ETU.BWT) 和接收时的字节最大等待间隔 (ISO7816MS_CCKCNT)

STEP7: 中断使能 (SCIHER 相应位置 1)

18.4.3. 发送字节步骤

STEP 1: 按照上文初始化中描述的步骤初始化模块, 置 SCICTRL.TRS=1, 选择 SCICTRL.PROTOCOL; (连续发送可只在第一次作初始化);

STEP 2: 写入字节到 SCIDR (开始发送字节) (如软件需发送 TCK/PCK 字节, 则可直接从 SCIEDC 寄存器读取并发送);

STEP 3: 查询发送第 1 个字节完成标志 SCISR.TXEND=1 或等待其中断

STEP 4: 如已设置额外 ETU 间隔, 则硬件自动加入额外 ETU 间隔;

STEP 5: 发送错误处理: 读 SCISR.RETRY_EN 标志, 判断是否发生错误, 执行相应的错误处理, 然后清错误标志为 0;

STEP 6: 清 SCISR.TXEND=0 和清 SCISR. ETU_CNT 标志, 如有后续字节, 跳转到 STEP 2; 否则转 STEP 7;

STEP 7: 发送结束;

18.4.4. 接收字节步骤

STEP 1: 按照初始化中描述的步骤初始化模块, 置 ISO7816MS_CTRL.TRS=0, 选择 ISO7816MS_CTRL.PROTOCOL (连续接收可只在第一次作初始化)

STEP 2: 根据需求设置 ISO7816MS_CCKCNT 寄存器相应的值并使能计数器;

STEP 3: 接收错误处理: 读 ISO7816MS_SR. PARITY 和 ISO7816MS_SR. FIFO_OV 标志, 判断是否发生错误, 执行相应的错误处理, 然后清错误标志为 0

STEP 4: 接收超时处理: 读 ISO7816MS_ISR. CCKCNT, 判断是否超时, 执行相应的超时处理; 然后清超时标志;

STEP 5: 状态检测及读接收数据: FIFO 状态中断 ISO7816MS_SR.FIFO_NE / ISO7816MS_SR.FIFO_HF / ISO7816MS_SR.FIFO_FU, 然后从 ISO7816MS_DR 读接收数据; 如果是本轮接收的最后一个 BYTE 且 T=1, 则转 STEP 6, 否则转 STEP 2, 继续接收;

STEP 6: 错误码校验处理: 读 ISO7816MS_EDC 寄存器, 如果非 0 则执行相应的错误码处理, 然后对 ISO7816MS_EDC 清零; 转 STEP 7;

STEP 7: 软件根据上层协议选择的通讯协议对 ISO7816MS_ETU 寄存器设置相应的值并使能计数器, 该步是便于下一步立即转发送过程而设;

STEP 8: 接收结束;

18.4.5. 接收转发送

按照接收步骤收完最后一个字节后执行以下步骤:

STEP 1: 等待 SCISR. ETU_CNT 中断; 或者软件控制做一个合适的延时;

后续步骤继续按发送字节步骤发送数据以支持对方 (IFD 设备) 刚发完数据, 还未处于有效接收状态的情况。

18.4.6. 发送转接收

按发送步骤完最后一个字节后执行以下步骤:

STEP 1: 等待快速发送转接收标志 SCISR.T2R=1;

STEP 2: 清 SCISR.T2R=0;

后续步骤继续按接收字节步骤接收数据以支持对方 (IFD 设备) 太快发送数据而可能错过接收对方的数据的情况, T2R 标志位的置位收受到寄存器 SCICTRL.PROTOCOL 的控制, 当 T=0/T=1 时产生的条件不同。

19. SENSOR

19.1. 概述

SENSOR 模块负责对异常状态的检测和处理，包含频率检测模块（ECLK_FD）、ActiveShield 模块、输入时钟毛刺检测模块(CGD)和总线异常检测。ECLK_FD 模块检测外部输入高频信号，包括 7816 slave 时钟、片外 12M 晶振和特定 GPIO，检测频率范围为 1M—48M，采用 RC32K 时钟或者 USB SOF 为基准。CGD 对输入时钟的毛刺检测。ActiveShield 模块检测主动屏蔽层（8 线）有无被切断。总线异常检测实现对系统总线的 8 位奇校验（四组）。

SENSOR 模块除了包含上述检测模块外，同时实现对模拟检测模块（TD/LD/VD/PGD）的控制和异常判断。

下图为 SENSOR 模块整体框图。

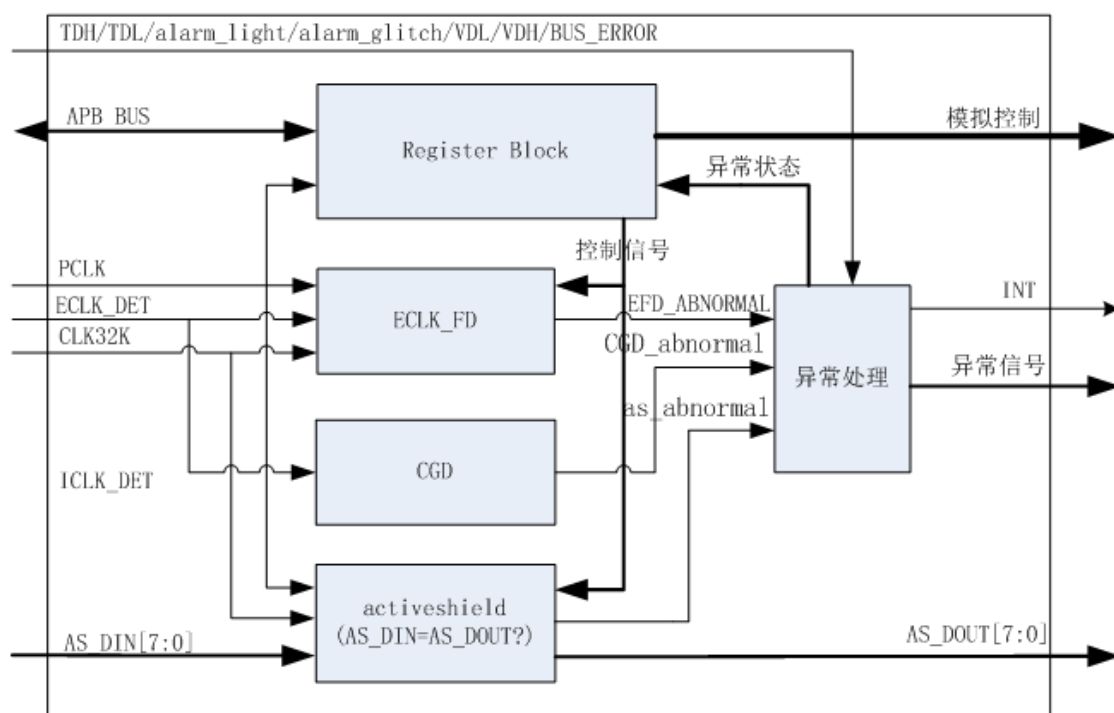


图 19-1 SENSOR 模块整体框图

19.2. 主要特性

- 高频输入时钟检测：1M—48M
- 输入时钟停止检测：低于 104Khz@32Khz（参考时钟频率 x3.25）
- 高低温检测
- 输入电压（VCC）高低检测
- 输入电压（VCC）毛刺检测
- 光检测
- 输入时钟毛刺检测

- 总线异常检测
- 主/被动屏蔽层

19.3. 寄存器描述

SENSOR 寄存器基地址: 0x48090000

偏置	名称	描述
0x00	SECR1	安全检测控制寄存器1
0x04	SECR2	安全检测控制寄存器 2
0x08	EFDTH	外部频率检测阈值寄存器
0x10	SEINTEN	安全检测中断使能寄存器
0x14	SESR	安全检测状态寄存器
0x18	FDCNTR	频率检测计数寄存器

19.3.1. 安全检测控制寄存器 1SECR1(偏移: 00h)

比特	名称	属性	复位值	描述
31:19	RSV	-	-	保留位。
18	CGD_EN	R/W	0	外部时钟毛刺检测使能 0: clock glitch检测禁止 1: clock glitch检测使能
17	ROM_CKEN	RW	0	ROM 校验单独使能 0: ROM 校验不使能 1: ROM 校验使能
16	BUS_ENAB LE	RW	0	BUS_ENABLE, 总线校验使能 0: 总线校验关闭 1: 总线校验使能
15:13	TDL_TRIM	R/W	100	TDL_TRIM, 低温检测TRIM位
12:10	TDH_TRIM	R/W	100	TDH_TRIM, 高温检测TRIM位
9	TD_TRIG	R/W	0	TD_TRIG, TD模块自检有效信号 0: TD模块正常工作 1: TD模块自检, TD模块输出高电平为异常状态
8	TD_ENABLE	R/W	0	TD Enable: 温检模块使能 0: 温度检测禁止 1: 温度检测使能
7:4	RSV	-	-	保留位。
3	ECLK_REF_C LK	R/W	0	外部频率检测REF时钟选择 0: 选择内部RC32K时钟 1: 选择USB SOF信号 REF时钟切换后需要等待3ms
2:1	ECLK_DET_S EL	R/W	00	外部频率检测时钟选择 00: 7816 CCK 01: XCLK (外部晶体) 10: GPIO18 11: RC48M
0	EFD_ENABL E	R/W	0	外部时钟频率检测使能 0: 禁用频率检测 1: 启用频率检测

19.3.2. 安全检测控制寄存器 2 SECR2 (偏移: 04h)

比特	名称	属性	复位值	描述
31:28	VDH_TRIM	RW	0xc	VDH TRIM 值
27:20	AS_Value	R/W	0x5a	AS 静态模式信号值
19	AS_MODE	RW	0	AS 工作模式设置 0: 动态模式, AS 信号来自随机数动态变化 1: 静态模式, AS 信号来自寄存器 AS_Value
18:16	LD_TRIM	R/W	010	LD_TRIM, LD 模块 TRIM 信号
15	LD_TRIG	R/W	0	LD_TRIG 信号, LD 模块自检有效信号 0: LD 模块正常工作 1: LD 模块自检, LD 模块输出高电平为异常状态
14	LD_ENABLE	R/W	0	LD EN, 光检测模块使能 0: LD 模块不使能 1: LD 模块使能
13:12	AS_PULSE_SET	R/W	0	AS Pulse set, 设置 Active shielding 的 Pulse 计数值 00: 100us 01: 200us 10: 300us 11: 400us
11	AS_ENABLE	R/W	0	AS Enable 0: Active Shielding 检测关闭 1: Active Shielding 检测使能
10	RSV	-	-	保留位。
9:6	VDL_TRIM	RW	0xc	VDL TRIM 值
5	VD_TRIG	R/W	0	VD_TRIG 信号, VD 模块自检有效信号 0: VD 模块正常工作 1: VD 模块自检, VD 模块输出高电平为异常状态
4	VD_ENABLE	R/W	0	VD Enable 0: 电压检测关闭.

				1: 电压检测使能.
3:2	PGD_TRIM	R/W	0	PGD_TRIM: 电源毛刺检测阈值修调位
1	PGD_TRIG	R/W	0	PGD_TRIG: PGD 模块自检有效信号 0: PGD 模块正常工作 1: PGD 模块自检, PGD 模块输出高电平为异常状态
0	PGD_ENABLE	R/W	0	PGD Enable: 电源毛刺检测使能/清零信号, 产生 PGD 异常后, 需要软件清零 0: PGD 模块清零 1: PGD 模块使能

19.3.3. 外部频率检测阈值寄存器EFDTH (偏移: 08h)

比特	名称	属性	复位值	描述
31:16	EFD_HIGHTH	R/W	0xAB	EFD_HIGHTH: 外部频率检测高阈值 高频率点除以 32K: $F_{eth}/32K$ (RC32K 基准) 或高频率点除以 1K: $F_{eth}/1K$ (SOF 基准)
15:0	EFD_LOWTH	R/W	0x87	EFD_LOWTH: 外部高频检测低阈值 低频率点除以 32K: $F_{etl}/32K$ (RC32K 基准) 或低频率点除以 1K: $F_{etl}/1K$ (SOF 基准)

注: 以 RC32K 为参考时钟时, 高阈值 EFD_HIGHTH 设置方法如下: $F_{eth}/32K$, 其中 F_{eth} 为外部检测高阈值频率; 低阈值 EFD_LOWTH 设置方法如下: $F_{etl}/32K$, 其中 F_{etl} 为外部检测低阈值频率。例如, 检测时钟标准为 12M, 检测高低阈值频率设为 $F_{eth}=12M*1.05$ 和 $F_{etl}=12M*0.95$; 可以设置 EFD_LOWTH 和 EFD_HIGHTH 分别为 $12000/32*0.95(0x164)$ 和 $12000/32*1.05(0x189)$ 。当 CLK_DET 变为 13.2M 时, 一个 CLK32K 周期计数值为 0x19c, 超过 FD_HIGHTH, 产生 FD_Abnormal 报警; 当 CLK_DET 变为 10.8M 时, 一个 CLK32K 周期计数值为 0x151, 低于 FD_LOWTH, 也会产生报警。以 USB SOF 为参考时钟时, 对应的阈值计算公式改为: $F_{eth}/1K$ 和 $F_{etl}/1K$

19.3.4. 安全检测中断使能寄存器SECINTEN (偏移: 10h)

比特	名称	属性	复位值	描述
31:11	RSV	-	-	保留位。
10	CGD_INT_EN	R/W	0	外部时钟毛刺检测中断使能

				0: 不使能 1: 使能
9	RSV	-	-	保留位。
8	BUS_INT_EN	R/W	0	BUS 检测中断使能 0: 不使能 1: 使能
7	AS_INT_EN	R/W	0	Active shield 检测中断使能 0: 不使能 1: 使能
6	VDH_INT_EN	R/W	0	高压检测中断使能 0: 不使能 1: 使能
5	VDL_INT_EN	R/W	0	低压检测中断使能 0: 不使能 1: 使能
4	PGD_INT_EN	R/W	0	电源毛刺检测中断使能 0: 不使能 1: 使能
3	LD_INT_EN	R/W	0	光检测中断使能 0: 不使能 1: 使能
2	TDL_INT_EN	R/W	0	低温检测中断使能 0: 不使能 1: 使能
1	TDH_INT_EN	R/W	0	高温检测中断使能 0: 不使能 1: 使能
0	EFD_INT_EN	R/W	0	外部频率检测中断使能 0: 不使能 1: 使能

19.3.5. 安全检测状态寄存器SESR(偏移: 14h)

比特	名称	属性	复位值	描述
31:11	RSV	-	-	保留位。

10	CGD_Abnormal	RO	0	CGD 异常状态，外部时钟毛刺检测模块检测到异常状态。此位写 1 清。 0: 外部时钟毛刺频检模块未检测到异常 1: 外部时钟毛刺频检模块检测到异常
9	RSV	-	-	保留位。
8	BUS_ERROR	RO	0	总线校验异常，此位写 1 清。 0: 总线校验未异常 1: 总线校验异常，总线被攻击
7	AS_Abnormal	RO	0	AS 异常状态，，此位写 1 清 0:AS 检测模块未检测到异常 1:AS 检测模块检测到异常
6	VDH_Abnormal	RO	0	VDH 异常状态，此位写 1 清 0:高电压检测模块未检测到异常 1:高电压检测模块检测到异常
5	VDL_Abnormal	RO	0	VDL 异常状态，此位写 1 清 0:低电压检测模块未检测到异常 1:低电压检测模块检测到异常
4	PGD_Abnormal	RO	0	PGD 异常，此位写 1 清。 0: PGD 未异常 1: PGD 异常，检测到注入攻击
3	LD_Abnormal	RO	0	LD 异常状态，此位写 1 清 0:光检模块未检测到异常 1: 光检模块检测到异常
2	TDL_Abnormal	RO	0	TDL 异常状态，此位写 1 清 0:温检模块未检测到低温异常 1: 温检模块检测到低温异常
1	TDH_Abnormal	RO	0	TDH 异常状态，此位写 1 清 0:温检模块未检测到高温异常 1: 温检模块检测到高温异常
0	EFD_Abnormal	RO	0	EFD 异常状态，频率检测模块检测到异常状态。此位写 1 清。 0: 外部频检模块未检测到异常 1: 外部频检模块检测到异常

19.3.6. 频率检测计数寄存器FDCNTR（偏移：18h）

比特	名称	属性	复位值	描述
31:16	RSV	-	-	保留位。
15:0	EFD_Value	RO	0	EFD 计数数值：上一个基准周期内的计数值 (计算数值会比实际的值小 1，所以真实值为：EFD_Value+1)

注：检测时钟停止时，此寄存器不更新

19.4. 使用流程

19.4.1. 模拟自检操作流程

1. 设置模拟模块使能（TD_EN/VD_EN/LD_EN/PGD_EN/AS_EN）
2. 设置对应模块 TRIG 信号（TD_TRIG/VD_TRIG/LD_TRIG/PGD_TRIG），AS 选择静态模式为 AS_MODE=1/AS_Value=0x5a。
3. 等待大约 10us（AS 自检为 200us，更改 AS_Value 为 0xa5,再等待 200us）
4. 检查对应的状态变为 1 正确（TDH/TDL/VDH/VDL/LD/PGD），AS_Abnormal 为 0 正确。
5. 关闭 TRIG 信号和使能信号，清除对应状态

19.4.2. 检测操作流程

1. 设置选择检测模块 TRIM 位（TD/VD/LD/PGD）/检测阈值(EFDTH/IFDTH/AS)
2. 使能检测模块以及对应中断
3. 等待中断状态，读取状态寄存器，处理异常事件。
4. 清除状态寄存器。注意：在异常事情不清除前，状态寄存器不能被清除。

19.4.3. 外部输入时钟频率检测

1. 设置安全检测控制寄存器 SECR1[0]=1（默认关闭），设置 EFD 检测的 REF 时钟（REF 时钟切换后需要等待 3ms）、待检测时钟、高频阈值和低频阈值，启动频率检测模块
2. 设置复位控制寄存器 RCR[5]=1，启动频率检测报警复位功能；或设置 SECINTEN[0]=1，开启频率检测告警中断使能。
3. 系统中断后读取系统配置寄存器 SESR[0]=1，确认为频率检测报警引起的中断然后在中断服务程序处理告警事件。
4. 可以读取 EFD_Value 计算具体的频率值

19.4.4. 温度检测TD

1. 设置安全检测控制寄存器 SECR1[8]=1（默认关闭），设置温度检测的高温 TRIM 和低温 TRIM，启动温度检测检测模块。
2. 设置复位控制寄存器 RCR[3/4]=1，启动 高/低温度检测报警复位功能；或设置 SECINTEN[1]=1（高

温) 或者 SECINTEN[2]=1 (低温), 开启温度检测告警中断使能。

3. 系统中断后读取系统配置寄存器 SESR [1]=1 (高温) 或者 SESR [2]=1 (低温), 确认为温度检测报警引起的中断然后在中断服务程序处理告警事件。

19.4.5. 电压检测VD

1. 设置安全检测控制寄存器 SECR2[5]=1 (默认关闭), 设置电压检测的高压 TRIM 和低压 TRIM, 启动启动检测检测模块。
2. 设置复位控制寄存器 RCR[0/1]=1, 启动 低/高电压检测报警复位功能; 或设置 SECINTEN[5]=1 (低压) 或者 SECINTEN[6]=1 (高压), 开启电压检测告警中断使能。
3. 系统中断后读取系统配置寄存器 SESR [5]=1 (低压) 或者 SESR [6]=1 (高压), 确认为电压检测报警引起的中断然后在中断服务程序处理告警事件。

19.4.6. 电源毛刺检测PGD

1. 设置安全检测控制寄存器 SECR2[0]=1 (默认关闭), 并设置 TRIM 位, 启动电源毛刺检测模块。
2. 设置复位控制寄存器 RCR[7]=1, 启动电源毛刺检测报警复位功能; SECINTEN[4]=1, 开启电源毛刺检测告警中断使能。
3. 系统中断后读取系统配置寄存器 SESR [4]=1, 确认为电源毛刺检测报警引起的中断然后在中断服务程序处理告警事件。

19.4.7. 光检测LD

1. 设置安全检测控制寄存器 SECR2[14]=1 (默认关闭), 并设置 LD_TRIM 位, 启动光检测模块。
2. 设置复位控制寄存器 RCR[2]=1, 启动光检测报警复位功能; 或设置 SECINTEN[3]=1, 开启光检测告警中断使能。
3. 系统中断后读取系统配置寄存器 SESR [3]=1, 确认为光检测报警引起的中断然后在中断服务程序处理告警事件。

19.4.8. 总线异常检测

1. 设置安全检测控制寄存器 SECR1[16]=1 (默认关闭), 启动总线异常检测模块; 并设置 ROM_CKEN 位, 是否启动 ROM 总线异常检测。
2. 设置复位控制寄存器 RCR[8]=1, 启动总线检测报警复位功能; 或设置 SECINTEN[8]=1, 开启总线异常检测告警中断使能。
3. 系统中断后读取系统配置寄存器 SESR [8]=1, 确认为 总线异常检测报警引起的中断然后在中断服务程序处理告警事件。

19.4.9. Active Shielding检测

1. 设置安全检测控制寄存器 SECR2[11]=1 (默认关闭), 并设置检测脉冲宽度 AS_PULSE_SET, 启动 Active Shielding 异常检测模块。
2. 设置 AS 的检测模式: AS_MODE=0 为动态模式; AS_MODE=1 为静态模式。在静态模式需要变

化 AS_Value 值，保证 AS 信号变化。

3. 设置复位控制寄存器 RCR[6]=1，启动 Active Shielding 检测报警复位功能；或设置 SECINTEN[7]=1，开启总线异常检测告警中断使能。
4. 系统中断后读取系统配置寄存器 SESR [7]=1，确认为 Active Shielding 异常检测报警引起的中断然后在中断服务程序处理告警事件。

19.4.10. 时钟毛刺检测CGD

1. 设置安全检测控制寄存器 SECR1[18]=1（默认关闭），启动时钟毛刺检测模块。
2. 设置复位控制寄存器 RCR[11]=1，启动时钟毛刺检测报警复位功能；或设置 SECINTEN[10]=1，开启时钟毛刺检测告警中断使能。
3. 系统中断后读取系统配置寄存器 SESR [10]=1，确认为时钟毛刺检测报警引起的中断然后在中断服务程序处理告警事件。

19.5. 注意事项

如果需要去检测机构进行测试则需要与芯片厂家联系获取安全监测相关配置和使用说明。

20. GPIO

20.1. 概述

GPIO 包含通用数据输入输出接口，这些管脚可以与其他功能管脚共享，这取决于芯片的配置。通过这些数据接口，可以配置任意数目的管脚作为中断信号输入。

20.2. 主要特性

- 所有输入/输出引脚方向都可以通过软件进行配置；
- 每个 GPIO_IN 引脚可配置成边沿或电平方式触发中断；

20.3. 寄存器描述

GPIOA 寄存器基地址：0x480a0000。对应 GPIO 管脚[31:0]

GPIOB 寄存器基地址：0x480b0000。对应 GPIO 管脚[63:32]

偏置	名称	描述
0x00	GPIO_DIR	GPIO 数据方向寄存器
0x08	GPIO_SET	GPIO 输出置位寄存器
0x0C	GPIO_CLR	GPIO 输出清零寄存器
0x10	GPIO_ODATA	GPIO 输出引脚映射寄存器
0x14	GPIO_IDATA	GPIO 输入引脚映射寄存器
0x18	GPIO_IEN	GPIO 中断使能寄存器
0x1C	GPIO_IS	GPIO 中断触发模式寄存器
0x20	GPIO_IBE	GPIO 中断触发模式寄存器
0x24	GPIO_IEV	GPIO 中断触发模式寄存器
0x28	GPIO_IC	GPIO 中断状态清除寄存器
0x2C	GPIO_RIS	GPIO 原始中断状态寄存器
0x30	GPIO_MIS	GPIO 屏蔽后中断状态寄存器

20.3.1. 数据方向寄存器GPIO_DIR(偏移：00h)

比特	名称	属性	复位值	描述
----	----	----	-----	----

31:0	GPIO_DIR	R/W	0x00000000	32 位寄存器，GPIO 输入输出控制寄存器： 0：输入； 1：输出。
------	----------	-----	------------	---

20.3.2. 输出置位寄存器GPIO_SET(偏移：08h)

比特	名称	属性	复位值	描述
31: 0	GPIO_SET	R/W	0x00000000	32 位寄存器，GPIO 输出置位寄存器： 0：无效操作； 1：当 IO 为输出时，IO 置位。

20.3.3. 输出清零寄存器GPIO_CLR(偏移：0Ch)

比特	名称	属性	复位值	描述
31: 0	GPIO_CLR	R/W	0x00000000	32 位寄存器，GPIO 输出清零寄存器： 0：无效操作； 1：当 IO 为输出时，IO 清零。

20.3.4. GPIO输出引脚映射寄存器GPIO_ODATA(偏移：10h)

比特	名称	属性	复位值	描述
31: 0	GPIO_ODATA	R/W	0x00000000	32 位寄存器，GPIO 输出引脚映射寄存器： 当 GPIO 方向为输出有效，写直接写至外部引脚，读获得外部引脚值。

20.3.5. GPIO输入引脚映射寄存器GPIO_IDATA(偏移：14h)

比特	名称	属性	复位值	描述
31: 0	GPIO_IDATA	RO	0x00000000	32 位寄存器，GPIO 输入引脚映射寄存器： 当 GPIO 方向为输入有效，读获得外部引脚值；此寄存器为只读寄存器。

20.3.6. GPIO中断使能寄存器GPIO_IEN(偏移: 18h)

比特	名称	属性	复位值	描述
31: 0	GPIO_IEN	R/W	0x00000000	32 位寄存器, GPIO 中断使能寄存器: 0= 禁止相应引脚中断; 1= 使能相应引脚中断。

20.3.7. GPIO中断触发模式寄存器GPIO_IS(偏移: 1Ch)

比特	名称	属性	复位值	描述
31: 0	GPIO_IS	R/W	0x00000000	32 位寄存器, GPIO 中断模式: 0= 边沿检测; 1= 电平检测。

20.3.8. GPIO中断触发模式寄存器GPIO_IBE(偏移: 20h)

比特	名称	属性	复位值	描述
31: 0	GPIO_IBE	R/W	0x00000000	32 位寄存器, GPIO 中断模式: 0= 单边沿触发; 1= 双边沿触发。

20.3.9. GPIO中断触发模式寄存器GPIO_IEV(偏移: 24h)

比特	名称	属性	复位值	描述
31: 0	GPIO_IEV	R/W	0x00000000	32 位寄存器, GPIO 中断模式: 0= 下降沿/低电平触发; 1= 上升沿/高电平触发。

20.3.10. GPIO中断状态清除寄存器GPIO_IC(偏移: 28h)

比特	名称	属性	复位值	描述
31: 0	GPIO_IC	WO	0x00000000	32 位寄存器, GPIO 中断清除寄存器: 0= 无效操作; 1= 清除对应引脚中断。

20.3.11. GPIO原始中断状态寄存器GPIO_RIS(偏移：2Ch)

比特	名称	属性	复位值	描述
31: 0	GPIO_RIS	R/W	0x00000000	32 位寄存器，GPIO 原始中断寄存器： 0= 对应引脚无中断挂起； 1= 对应引脚有中断挂起。

20.3.12. GPIO屏蔽后中断状态寄存器GPIO_MIS(偏移：30h)

比特	名称	属性	复位值	描述
31: 0	GPIO_MIS	RO	0x00000000	32 位寄存器，GPIO 屏蔽后中断状态寄存器：反映对应引脚屏蔽后的中断状态。

20.4. 使用流程

20.4.1. 输入输出IO

- 配置 GPIO_DIR 寄存器，选择 GPIO 方向
- 可使用 GPIO_SET/GPIO_CLR 或 GPIO_ODATA 来设置输出电平
- 使用 GPIO_IDATA 来获取输入引脚电平

20.4.2. 中断触发模式

中断初始化过程：

1. 设置 GPIO_DIR 为输入
2. 清除 GPIO_IE 以避免异常；
3. 配置寄存器 GPIO_IS，选择是边沿/电平触发类型；
4. 在单边沿触发方式下，配置寄存器 GPIO_IBE，确定是单边触发还是双边触发；
5. 在单边沿触发方式下，配置寄存器 GPIO_IEV，确定是哪种边沿触发类型；
6. 在电平触发方式下，配置寄存器 GPIO_IEV，确定是哪种电平触发类型；
7. 配置寄存器 GPIO_IC 来清除中断；
8. 配置寄存器 GPIO_IE 使能相应位中断。

20.4.3. 清除中断

ISR 写 GPIO_IC 来清除中断状态。如果在清除寄存器的同时有新的边沿触发中断产生，这个新

的中断将会保持有效直到下一次清除。读取中断状态操作应该在清 GPIO_IE 之前进行，清 GPIO_IE 操作将清除相应中断状态。

保
密

21. MIM

21.1. 概述

存储器集成模块(MIM, Memmory Interface Management)负责控制信息的传输,该传输是在内部局部总线 and 外部存储器模块(同步或者异步)之间进行的。同时此模块也可以作为 TFT-LCD 控制器使用,用来支持 8080 接口的 TFT-LCD。

21.2. 主要特性

- 与 AMBA AHB (SLAVE) 兼容接口;
- 支持片外 Sram、Norflash 扩展,支持 8080 TFT-LCD 控制;
- 四个可编程的异步片选信号(低有效),片选信号可以被单独编程,各个独立的 Memory 可以有不同的性能;
- 支持写保护功能;
- 支持外部接口 8/16bit 可配;
- 读写操作等待时间可配。

21.3. 寄存器描述

MIM 寄存器基地址: 0x62000000

偏置	名称	描述
0x00	MIM_CSCR0	Chip Select Control Register 0
0x04	MIM_CSCR1	Chip Select Control Register 1
0x08	MIM_CSCR2	Chip Select Control Register 2
0x0C	MIM_CSCR3	Chip Select Control Register 3
0x10	MIM_TFTS	TFT 片选接口寄存器
0x14	MIM_CMD	TFT 模式下 CMD 通道寄存器
0x18	MIM_DATA	TFT 模式下 DATA 通道寄存器
0x1C	MIM_SEG0	区域划分寄存器 0
0x20	MIM_SEG1	区域划分寄存器 1
0x24	MIM_SEG2	区域划分寄存器 2
0x28	MIM_SEG3	区域划分寄存器 3

21.3.1. 控制寄存器MIM_CSCRx (偏移: 00h/04h/08h/0ch, x=0~3)

比特	名称	属性	复位值	功能描述
31	RSV	-	-	保留
30:28	WATS	W/R	0x7	写信号有效时间选择。定义为 CSn 信号有效到 EBn 信号有效之间的时间间隔。
27	RSV	-	-	保留
26:24	WNTS	RW	0x7	写信号无效时间选择。定义为 EBn 信号无效到 CSn 信号无效之间的时间间隔。
23	RSV	-	-	保留
22:20	RATS	RW	0x7	读信号有效时间选择。定义为 CSn 信号有效到 EBn/OEn 信号有效之间的时间间隔。
19:15	RSV	-	-	保留
14	RO	RW	0	RO 比特位用来限制对相应片选下的地址域进行写访问。如果 RO 比特位为 1, 只有读访问被允许。如果 RO 比特位为 0, 则读写访问都被允许。 当访问一个片选的存储空间时, 片选逻辑将把 RO 比特位同内部 read/write 信号相比较。如果片选逻辑检测到一个违例事件, 访问将被忽视。 1: 只允许读访问; 片选逻辑将忽视写访问; 0: 读写访问都被允许。 此位在 TFT 模式下不起作用。
13	PS	RW	0x0	外部端口比特位宽选择。 0: 16 比特端口; 1: 8 比特端口;
12	RSV	-	-	保留
11:8	WWS	RW	0xf	WWS 字段决定着写等待状态的时钟周期个数。
7:4	RWS	RW	0xf	RWS 字段决定着读等待状态的时钟周期个数。
3:1	RSV	-	-	保留

0	CSEN	RW	0x0	CSEN 比特位用来使能片选逻辑。当片选被禁止时，外部片选信号将会是高电平。 1: 使能片选。 0: 禁止片选。
---	------	----	-----	--

注：MIM_CSCRx中的X可为0~3，每一个寄存器控制一套外部存储器接口。

21.3.2. TFT片选寄存器MIM_TFTS(偏移：10h)

比特	名称	属性	复位值	功能描述
31:2	RSV	-	-	保留
1:0	TFTS	W/R	3	TFT 模式选择。 00: CSn0 控制的外设为 TFT 模式,用于 8080 TFT-LCD 控制; 01: CSn1 控制的外设为 TFT 模式,用于 8080 TFT-LCD 控制; 10: CSn2 控制的外设为 TFT 模式,用于 8080 TFT-LCD 控制; 11: CSn3 控制的外设为 TFT 模式,用于 8080 TFT-LCD 控制;

21.3.3. TFT命令通道寄存器MIM_CMD (偏移：14h)

比特	名称	属性	复位值	功能描述
31:0	CMD	W	0x0	如果外设选择为 TFT 模式，向此寄存器中读写数据，数据将被理解为 CMD 的内容。会发起 TFT CMD 操作。

21.3.4. TFT数据通道寄存器MIM_DATA (偏移：18h)

比特	名称	属性	复位值	功能描述
31:0	DATA	W/R	0x0	如果外设选择为 TFT 模式，向此寄存器中读写数据，数据将被理解为数据的内容。会发起 TFT DATA 操作。

21.3.5. 区域划分寄存器MIM_SEG0（偏移：1ch）

比特	名称	属性	复位值	功能描述
31:28	RSV	-	-	保留
27:15	SIZE	W/R	0x1fff	CSEN0 对应的存储区域（SEG0）的大小设置位。SEG0（容量：byte）=（SIZE + 1）X 1K；
14:0	START	W/R	0	CSEN0 对应的存储区域（SEG0）的起始地址，地址以 1Kbyte 对齐。即此处地址为 1Kbyte 为单位，每一个 SEG 最小单位为 1Kbyte。

21.3.6. 区域划分寄存器MIM_SEG1（偏移：20h）

比特	名称	属性	复位值	功能描述
31:28	RSV	-	-	保留
27:15	SIZE	W/R	0x1fff	CSEN1 对应的存储区域（SEG1）的大小设置位。SEG1（容量：byte）=（SIZE + 1）X 1K；
14:0	START	W/R	0x2000	CSEN1 对应的存储区域（SEG1）的起始地址，地址以 1Kbyte 对齐。即此处地址为 1Kbyte 为单位，每一个 SEG 最小单位为 1Kbyte。

21.3.7. 区域划分寄存器MIM_SEG2（偏移：24h）

比特	名称	属性	复位值	功能描述
31:28	RSV	-	-	保留
27:15	SIZE	W/R	0x1fff	CSEN2 对应的存储区域（SEG2）的大小设置位。SEG2（容量：byte）=（SIZE + 1）X 1K；
14:0	START	W/R	0x4000	CSEN2 对应的存储区域（SEG2）的起始地址，地址以 1Kbyte 对齐。即此处地址为 1Kbyte 为单位，每一个 SEG 最小单位为 1Kbyte。

21.3.8. 区域划分寄存器MIM_SEG3（偏移：28h）

比特	名称	属性	复位值	功能描述
31:28	RSV	-	-	保留
27:15	SIZE	W/R	0x1fff	CSEN3 对应的存储区域（SEG3）的大小设置位。SEG3（容量：byte）=（SIZE + 1）X 1K；
14:0	START	W/R	0x6000	CSEN3 对应的存储区域（SEG3）的起始地址，地址以 1Kbyte 对齐。即此处地址为 1Kbyte 为单位，每一个 SEG 最小单位为 1Kbyte。

21.4. 功能描述

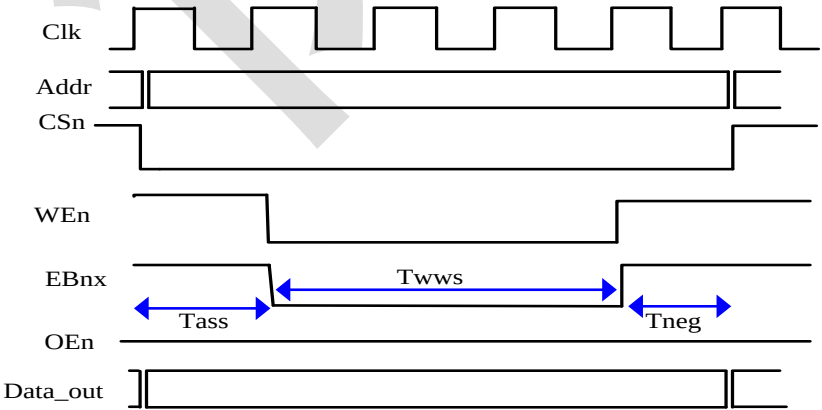
21.4.1. 片选

每个 MIM_CSCRx 寄存器中的 CSEN 信号对应着一个外设的片选使能。设置寄存器内的 CSEN 不意味着外部片选信号 CSnx 为低，只有当寄存器内部 CSEN 为 1，并且对相应的外设地址区域进行读写操作时，外部 CSnx 信号才会有效。

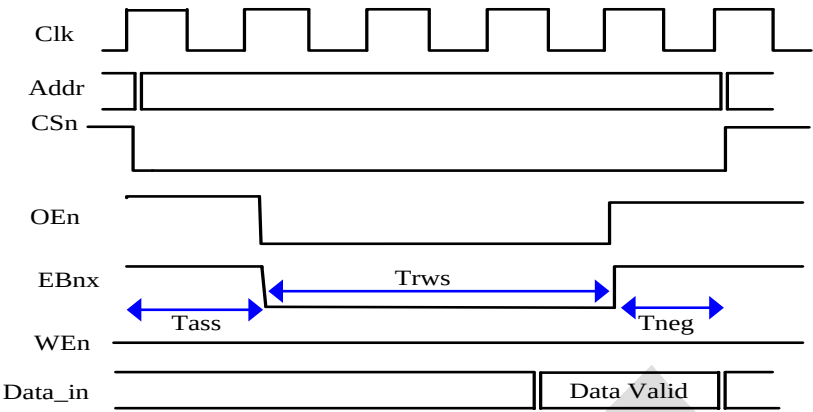
21.4.2. EBn信号时序

对于 CSnx 写操作来说，EBn 信号的建立和取消时间可以灵活地进行配置。然而对于读操作来说，仅仅建立时间被配置。读操作的取消时间总是在上升沿末尾。

写操作时序图如下图所示：



读操作时序图如下图所示：



片外存储器读写时序，对应的时间表格如下所示：

WATS/RATS/AATS	Tass	WNTS	Tneg	WWS/RWS	Twws/Trws
000	0	000	0	0000	1
001	1	001	1	0001	2
010	2	010	2	0010	3
011	3	011	3	0011	4
100	4	100	4	0100	5
101	5	101	5	0101	6
110	6	110	6	0110	7
111	7	111	7	0111	8
-	-	-	-	1000	9
				1001	10
				1010	11
				1011	12
				1100	13
				1101	14
				1110	15
				1111	16

对于 TFT 读情景, Tneg 有效, 即时序中含有 Tneg 时间。

21.4.3. 字节使能管脚

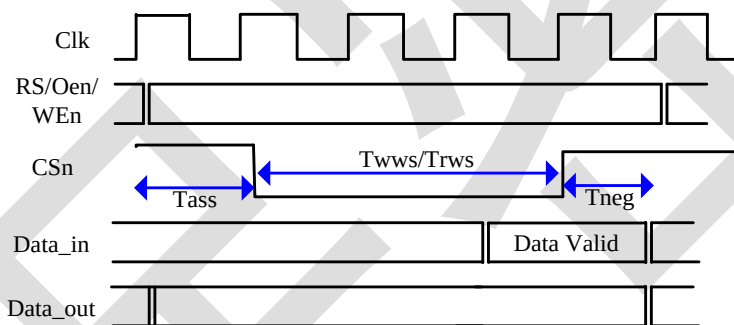
MIM_CSCRx 寄存器中的 PS 根据外部连接的外设的数据位宽来设置。MIM 控制器支持外部数据为 8/16bit 的外设，CPU 可以以高于外设数据的位宽的访问方式进行访问。例如外设数据宽度为 8bit，当 CPU 发起一次 32bit 写命令时，MIM 控制器内部会将 32bit 的命令分成四次独立的写操作，从而达到写 32bit 的目的。当 CPU 发起低于外设最小规定数据宽度的访问时，会发生错误，软件应避免发起这样的操作。

下图为 EBn 与外设数据线之间的对应关系。

外设支持的最小访问数据位宽（单位：bit）	外设的数据位宽（单位：bit）	EBnx	外部数据接口信号
8	16	EBnx[1]	Data_in/Data_out[15:8]
		EBnx[0]	Data_in/Data_out[7:0]
16	16	EBnx[1]	x
		EBnx[0]	Data_in/Data_out[15:0]
8	8	EBnx[1]	x
		EBnx[0]	Data_in/Data_out[7:0]

21.4.4. TFT-LCD控制

MIM 模块支持 8080 TFT-LCD 控制。8080 TFT-LCD 时序接口如下图所示：



MIM 模块中 RS 用来选择是命令操作还是数据传输操作，其根据此次操作的寄存器为 MIM_CMD 或 MIM_DATA 来决定。

注意，对于 TFT 读情景,Tneg 有效,即读时序中含有 Tneg 时间。

21.5. 使用方法

21.5.1. 片外存储器读操作

- 1) 设置 MIM_CSCRx 寄存器的 RATS、AATS、RO、PS、RWS、CDA、CSEN 等参数；
- 2) 设置 MIM_SEG0、MIM_SEG1、MIM_SEG2、MIM_SEG3 寄存器；
- 3) 对相应的合法存储区域发起读操作。

21.5.2. 片外存储器写操作

- 1) 设置 MIM_CSCRx 寄存器的 WATS、WNTS、AATS、RO、PS、WWS、CDA、CSEN 等参数；
- 2) 设置 MIM_SEG0、MIM_SEG1、MIM_SEG2、MIM_SEG3 寄存器；
- 3) 对相应的合法存储区域发起写操作。

21.5.3. TFT-LCD操作

- 1) 设置 MIM_CSCRx 寄存器，设置 TFT-LCD 悬挂位置（TFTS）；
- 2) 向 MIM_CMD 写入数据发起 CMD 传输；
- 3) 向 MIM_DATA 读写数据发起数据传输。

22. CRC16

22.1. 概述

CRC16 是一个以多项式 $G(x) = x^{16} + x^{12} + x^5 + 1$ 为计算式的硬件 16 位 CRC 循环冗余校验计算电路。可以根据用户预设的 CRC 初值，通讯数据计算出合适的 CRC 结果，并且支持设置输入数据与结果的正反向。

22.2. 寄存器描述

CRC 基地址：0x40013000

偏置	名称	描述
0x00	CRC16_DATA	写入需校验的数据与读出 CRC 结果
0x04	CRC16_INIT	写入 16 位 CRC 初值
0x08	CRC16_CTRL	CRC 控制寄存器

22.2.1. 数据寄存器CRC16_DATA（偏移：00H）

比特	名称	属性	复位值	描述
31:16	RSV	-	-	保留
15:8	RSLT2	RO	0x0	读出 16 位 CRC 计算结果的高 8 位
7:0	DATA_RSLT1	RW	0x0	写：写入需要进行 CRC 校验计算的数据，如需要校验的数据不止 8 位需按顺序逐次写入 读：读出 16 位 CRC 计算结果的低 8 位

低 8 位对于需校验数据来说为只写，写入后无法再次读出。

读操作返回 16 位 CRC 计算结果，其中低 8 位为与数据寄存器复用。

读操作将会对 CRC 计算清零，即读操作后将会重新载入初始值，次输入数据时会进行与读之前结果无关的新一轮计算。

22.2.2. 初始值寄存器 CRC16_INIT（偏移：04H）

比特	名称	属性	复位值	描述
31:16	RSV	-	-	保留
15:0	INIT	RW	0x0	写入 16 位 CRC 初始值

22.2.3. 控制寄存器 CRC16_CTRL（偏移：08H）

比特	名称	属性	复位值	描述
----	----	----	-----	----

31:3	RSV	-	-	保留
2	RSLT_REV	RW	0	CRC 计算结果是否进行高低位倒序 1: 倒序 0: 不倒序
1	DATA_REV	RW	0	CRC 计算数据是否进行高低位倒序 1: 倒序 0: 不倒序
0	INITIAL_REV	RW	0	CRC 初始值是否进行高低位倒序 1: 倒序 0: 不倒序

22.3. 操作方法

设置 16 位初始值 CRC16_INIT。

设置 CRC16_CTRL，选择数据是否倒序。

向 CRC16_DATA 中写入 8 位 CRC 计算数据，如没有完成 CRC 数据输入顺次输入之后 8 位数据直至输入完成。

读 CRC16_DATA，将一次返回 CRC 计算结果

注意：读取结果后 CRC 计算模块将结束当前计算并重新载入初始值以备下次计算使用。

23. 算法库

23.1. 数据类型

算法库中的数据类型定义如下：

```
typedef unsigned char UINT8;
```

```
typedef unsigned int  UINT32;
```

即 UINT8 表示 8 位无符号数，UINT32 表示 32 位无符号数。

23.2. HRNG

23.2.1. 主要特性

- 内含可靠噪声振荡器；
- 符合国际 FIPS-140-2 和 NIST SP800-22 测试标准；
- 符合国密局《随机数检测规范》测试标准；

23.2.2. 库文件说明

驱动开发包源文件说明如下：

hrng.lib 实现取随机数操作。

hrng.hrng.lib 对应的头文件

23.2.3. 函数说明

表 28-1 HRNG 驱动函数说明

函数名	功能描述	参数	
hrng_initial	初始化函数用于启动随机数模块	输入：	无
		输出：	无
		返回值：	无
get_hrng8	取 8 位随机数	输入：	无
		输出：	无
		返回值：	8 位随机数
get_hrng32	取 32 位随机数	输入：	无
		输出：	无
		返回值：	32 位随机数
get_hrng_16bytes	取 16 个字节随机数	输入：	无
		输出：	UINT8 *hdata: 存放随机数的起始地址
		返回值：	无
get_hrng	取多个字节随机数	输入：	UINT32 byte_len: 取随机数的字节长度

		输出：	UINT8 *hdata: 存放随机数的起始地址
		返回值：	0: 表示取随机数成功；1: 表示失败

23.2.4. 注意

- 1) 需要确保模块时钟使能寄存器中的 UAC, HRNG,HRNG 慢时钟模块时钟使能;

23.3. DES

23.3.1. 主要特性

- 符合 FIPS 46-3 的加解密标准
- 支持 DES 和 3DES 加解密运算。
- 支持 DES 算法 64（56）位密钥。
- 支持 3DES 算法 128（112）位和 192（168）位密钥。
- 支持 ECB（Electronic Code Book）模式和 CBC（Cipher Block Chaining）模式。
- 数据输入和输出支持 SWAP 模式，即大小端可配置

23.3.2. 库文件说明

des.lib	实现 DES 加解密运算
des.h	DES 相关运算对应的头文件

23.3.3. 函数说明

表 28-2 DES 驱动函数说明

函数名	功能描述	参数	
des_set_key	配置 des 秘钥及对调模式	输入：	UINT8 key_num: des 加密秘钥个数 0x01: 单秘钥 0x02: 双秘钥 0x03: 三秘钥
			UINT32 *keys: 存放秘钥的起始地址
			UINT8 swap_en: 输入输出数据大小端对调使能 0: 输入输出数据大小端对调禁止 1: 输入输出数据大小端对调使能
		输出：	无
		返回值：	无
des_crypt	des 加解密函数	输入：	UINT32 *indata: 存放输入数据的起始地址
			UINT32 block_len: 加解密的块(64 bit)个数

			UINT8 operation:加解密操作选择 0: 加密 1: 解密
			UINT8 mode: 加解密模式选择 0: ECB 模式 1: CBC 模式
			UINT32 *iv: 存放 CBC 模式初始向量的起始地址
			UINT32 security_mode: 安全模式选择 0x12345678: 普通模式 0: 安全模式
		输出:	UINT32 *outdata: 存放输出结果的起始地址
		返回值:	UINT32 加解密运行结果 0: 运行失败 0x5a9ada68: 运行成功

23.3.4. 注意

- 1) 普通模式需要确保模块时钟使能寄存器中的 UAC，EMW，DES 模块时钟使能;
- 2) 安全模式需要确保模块时钟使能寄存器中的 UAC，EMW，DES，HRNG，HRNG 慢时钟模块时钟使能;

23.4. SM1

23.4.1. 主要特性

- 支持国密 SM1 加解密算法
- 支持数据密钥长度 128 比特，系统密钥长度 128 比特
- 支持 128 比特明文和密文长度
- 支持 ECB、CBC 工作模式
- 数据输入和输出支持 SWAP 模式，即大小端可配置

23.4.2. 库文件说明

sm1.lib	实现 SM1 加解密运算
sm1.h	sm1 相关运算对应的头文件

23.4.3. 函数说明

表 28-3 SM1 驱动函数说明

函数名	功能描述	参数	
sm1_set_key	配置 sm1 秘钥 及对调模式	输入:	UINT32 *keyin: 存放秘钥的起始地址
			UINT8 sk: 参数选择 0: 内部参数 1: 外部参数
			UINT8 swap_en: 输入输出数据大小端对调使能 0: 输入输出数据大小端对调禁止 1: 输入输出数据大小端对调使能
			输出: 无
		返回值:	无
sm1_crypt	sm1 加解密函数	输入:	UINT32 *indata: 存放输入数据的起始地址
			UINT32 block_len: 加解密的块(128 bit)个数
			UINT8 operation: 加解密操作选择 0: 解密 1: 加密
			UINT8 mode: 加解密模式选择 0: ECB 模式 1: CBC 模式
			UINT32 *iv: 存放 CBC 模式初始向量的起始地址
			UINT32 security_mode: 安全模式选择 0x12345678: 普通模式 0: 安全模式
		输出:	UINT32 *outdata: 存放输出结果的起始地址
		返回值:	UINT32 加解密运行结果
			0: 运行失败
			0x5aaada6e: 运行成功

23.4.4. 注意

- 1) 普通模式需要确保模块时钟使能寄存器中的 UAC, EMW, SM1 模块时钟使能
- 2) 安全模式需要确保模块时钟使能寄存器中的 UAC, EMW, SM1, HRNG, HRNG 慢时钟模块时钟使能;

23.5. SM4

23.5.1. 主要特性

- 支持国密 SM4 加密和解密运算
- 支持 Electronic Code Book (ECB)模式和 Cipher Block Chaining (CBC)模式
- 数据输入和输出支持 SWAP 模式，即大小端可配置

23.5.2. 库文件说明

sm4.lib 实现 SM4 加解密运算

sm4.h sm4 相关运算对应的头文件

23.5.3. 函数说明

表 28-4 SM4 驱动函数说明

函数名	功能描述	参数	
sm4_set_key	配置 sm4 秘钥及 对调模式	输入:	UINT32 *keyin: 存放秘钥的起始地址
			UINT8 swap_en: 输入输出数据大小端对调使能 0: 输入输出数据大小端对调禁止 1: 输入输出数据大小端对调使能
		输出:	无
		返回值:	无
sm4_crypt	sm4 加解密函数	输入:	UINT32 *indata: 存放输入数据的起始地址
			UINT32 block_len: 加解密的块(128 bit)个数
			UINT8 operation:加解密操作选择 0: 解密 1: 加密
			UINT8 mode: 加解密模式选择 0: ECB 模式 1: CBC 模式
			UINT32 *iv: 存放 CBC 模式初始向量的起始地址
			UINT32 security_mode: 安全模式选择 0x12345678: 普通模式 0: 安全模式
		输出:	UINT32 *outdata: 存放输出结果的起始地址
		返回值:	UINT32 加解密运行结果 0: 运行失败

			0xa59ada68: 运行成功
--	--	--	------------------

23.5.4. 注意

- 1) 普通模式需要确保模块时钟使能寄存器中的 UAC，EMW，SM4 模块时钟使能；
- 2) 安全模式需要确保模块时钟使能寄存器中的 UAC，EMW，SM4，HRNG，HRNG 慢时钟模块时钟使能；

23.6. AES

23.6.1. 主要特性

- 支持 AES 加密和解密运算
- 支持 ECB 模式和 CBC 模式
- 数据输入和输出支持 SWAP 模式，即大小端可配置
- 支持 128/192/256 bit 密钥长度

23.6.2. 库文件说明

aes.lib 实现 AES 加解密运算

aes.h aes.lib 对应的头文件

23.6.3. 函数说明

表 28-5 AES 驱动函数说明

函数名	功能描述	参数	
aes_set_key	配置 aes 密钥及 对调模式	输入:	UINT32 *keyin: 存放密钥的起始地址
			UINT8 key_len: 密钥长度选择 0:128 1:192 2:256
			UINT8 swap_en: 输入输出数据大小端对调使能 0: 输入输出数据大小端对调禁止 1: 输入输出数据大小端对调使能
		输出:	无
		返回值:	无
aes_crypt	aes 加解密函数	输入:	UINT32 *indata: 存放输入数据的起始地址
			UINT32 block_len: 加解密的块(128 bit)个数

			UINT8 operation:加解密操作选择 0: 解密 1: 加密
			UINT8 mode: 加解密模式选择 0: ECB 模式 1: CBC 模式
			UINT32 *iv: 存放 CBC 模式初始向量的起始地址
			UINT32 security_mode: 安全模式选择 0x12345678: 普通模式 0: 安全模式
		输出:	UINT32 *outdata: 存放输出结果的起始地址
		返回值:	UINT32 加解密运行结果 0: 运行失败 0xa59ada68: 运行成功

23.6.4. 注意

- 1) 普通模式需要确保模块时钟使能寄存器中的 UAC, EMW, AES 模块时钟使能;
- 2) 安全模式需要确保模块时钟使能寄存器中的 UAC, EMW, AES, HRNG, HRNG 慢时钟模块时钟使能;

23.7. SSF33

23.7.1. 主要特性

- 支持国密 SSF33 加解密算法
- 支持数据密钥长度 128 比特, 系统密钥长度 128 比特
- 支持 128 比特明文和密文长度
- 支持 ECB、CBC 工作模式
- 支持内部参数和外部参数两种模式

23.7.2. 库文件说明

ssf33.lib 实现 ssf33 相关运算

ssf33.h ssf33 相关运算对应的头文件

23.7.3. 函数说明

表 28-6 SSF33 驱动函数说明

函数名	功能描述	参数
-----	------	----

ssf33_set_key	配置 ssf33 密钥及对调模式	输入:	UINT32 se_sel: 参数选择 0: 内部参数 1: 外部参数
			UINT32 *para: 存放参数的起始地址
			UINT32 *ks: 存放密钥的起始地址
			UINT8 swap_en: 输入输出数据大小端对调使能 0: 输入输出数据大小端对调禁止 1: 输入输出数据大小端对调使能
		输出:	无
		返回值:	无
ssf33_crypt_data	ssf33 加解密函数	输入:	UINT32 *datain: 存放输入数据的起始地址
			UINT32 block_len: 加解密的块(128 bit)个数
			UINT32 enc: 加解密操作选择 0: 解密 1: 加密
			UINT32 data_mode: 加解密模式选择 0: ECB 模式 1: CBC 模式
			UINT32 *iv: 存放 CBC 模式初始向量的起始地址
			UINT32 security_mode: 安全模式选择 0x5a3489a5: 普通模式 0: 安全模式
		输出:	UINT32 *dataout: 存放输出结果的起始地址
		返回值:	UINT32 加解密运行结果 0: 运行失败 0xabef5a58: 运行成功

23.7.4. 注意

- 3) 普通模式需要确保模块时钟使能寄存器中的 UAC, EMW, SSF33 模块时钟使能;
- 4) 安全模式需要确保模块时钟使能寄存器中的 UAC, EMW, SSF33, HRNG, HRNG 慢时钟模块时钟使能;

23.8. RSA

23.8.1. 主要特性

- 支持 64 到 4096 位的模乘、模平方、模加减、模幂和模逆运算操作;

- 支持 64 到 2048 位 CRT 密钥生成和加解密，其中密钥生成的步长是 64 位；

23.8.2. 库文件说明

rsa_keygen.lib 实现 rsa 及除法运算
 rsa_keygen.h 支持 rsa 各种运算的头文件
 divider.h 支持除法运算的头文件

23.8.3. 函数说明

表 28-7 RSA 驱动函数说明

函数名	功能描述	参数	
rsa_mul_me	大数模乘模幂运算	输入：	UINT32 *a_data: 模幂底数或者模乘被乘数的起始地址
			UINT8 a_length: a_data 的字长度
			UINT32 *b_data: 模幂指数者模乘乘数的起始地址
			UINT8 b_length: b_data 的字长度
			UINT32 *n_data: 模数据的起始地址
			UINT8 n_length: n_data 的字长度
			UINT8 mode: 模式功能选择, 0x01: 模乘; 0x30: 模幂
		输出：	UINT32 *out_data: 模幂或者模乘结果的起始地址
			UINT8* out_length: out_data 的字长度
		返回值：	0: 运算成功; 1: 运算失败
rsa_decrypt_CRT	CRT 方式的模幂运算	输入：	UINT32 *in_data: 模幂底数的起始地址
			UINT8 in_length: in_data 的字长度
			UINT32 *p_data: p 的起始地址
			UINT8 p_length: p 的字长度
			UINT32 *q_data: q 的起始地址
			UINT8 q_length: q 的字长度
			UINT32 *dp_data: dp 的起始地址
			UINT8 dp_length: dp 的字长度
			UINT32 *dq_data: dq 的起始地址
			UINT8 dq_length: dq 的字长度
			UINT32 *qInv_data: qInv 的起始地址

			UINT8 qInv_length: qInv 的字长度
			RSA_G_STR *p_rsa_str: RSA_G_STR 结构指针
			MATH_G_STR *p_math_st : MATH_G_STR 结构指针
			UINT32 *e_data: 公钥 e 的起始地址
			UINT8 e_length: 公钥 e 的字长度
			UINT32 mode: 安全模式选择 0x57D525A0: 普通模式 其他数据: 安全模式
		输出:	UINT32 *out_data: 模幂结果的起始地址
RSA_keygen	RSA 密钥生成		UINT8* out_length: out_data 的字长度
		返回值:	0: 运算成功; 1: 运算失败
		输入:	RSA_KEYGEN_G_STR*p_rsa_keygen_str: RSA_KEYGEN_G_STR 结构体指针
			MATH_G_STR *p_math_st : MATH_G_STR 结构指针
			UINT8 nDigits: 密钥生成的字长度
RSA_keygen_CRT	RSA CRT 密钥生成	输出:	输出数据存放在 RSA_KEYGEN_G_STR 结构体指针对应的空间
		返回值:	0: 运算成功; 1: 运算失败
		输入:	RSA_KEYGEN_G_STR*p_rsa_keygen_str: RSA_KEYGEN_G_STR 结构体指针
			MATH_G_STR *p_math_st : MATH_G_STR 结构指针
			UINT8 nDigits: 密钥生成的字长度
		输出:	输出数据存放在 RSA_KEYGEN_G_STR 结构体指针对应的空间
		返回值:	0: 运算成功; 1: 运算失败

23.8.4. 注意

- 1) 需要确保模块时钟使能寄存器中的 UAC, PKI, HRNG, HRNG 慢时钟模块时钟使能;

- 2) 单独使用 PKI 模块只需确保模块时钟使能寄存器中的 UAC，PKI 模块时钟使能
- 3) 通过时钟分频寄存器配置好系统时钟和算法时钟；
- 4) 使用私钥运算请调用 rsa_decrypt_CRT（）函数的安全模式；
- 5) 使用公钥运算可以调用 rsa_mul_me()函数；
- 6) RSA 的所有数据存放方式如下：

低 32 位字数据在数组的低位，每个字以大端方式存放。

若：A = 0x11223344556677889900；

则：A[0] = 0x77889900；

A[1] = 0x33445566；

A[2] = 0x00001122；

7) RSA_keygen 和 RSA_keygen_CRT 函数需要给结构体指针 p_rsa_keygen_str 中的变量按照密钥长度进行分配空间，具体可见 demo 工程中的 lib_variable_initial 函数。不需要给结构体指针 math_glb_str 分配空间。rsa_decrypt_CRT 函数也不需要给结构体指针 math_glb_str 和 rsa_glb_str 分配空间。
- 23.9. SM2_SM3
- 23.9.1. 主要特性
- 支持国密 256 位的 SM2 密钥生成、加/解密、签名验证以及密钥交换
 - 支持国密 SM3 杂凑算法
- 23.9.2. 库文件说明
- sm2sm3.lib 实现 SM2 加解密，签名验证，密钥交换运算和 SM3 运算

sm2.h 实现 SM2 相关算法的头文件

sm3.h 实现 SM3 相关算法的头文件
- 23.9.3. 函数说明
- 表 28-8 SM2_SM3 驱动函数说明
- | 函数名 | 功能描述 | 参数 | |
|----------|----------|-----|---|
| sm2_sign | SM2 签名运算 | 输入： | ECC_G_STR *p_ecc_para: ECC_G_STR 结构指针 |
| | | | MATH_G_STR *p_math_st : MATH_G_STR 结构指针 |
| | | | UINT8 *hashdata: 摘要的起始地址 |
| | | | UINT8*PrivateKey: 私钥的起始地址 |
- Copyright © 2008~2017 上海爱信诺航芯电子科技有限公司
- Page221

			UINT8 mode: 模式功能选择, 0xF27E4B40: 普通模式 其他数据: 安全模式
		输出:	UINT8 *Signature0: 签名结果 r 的起始地址
			UINT8 *Signature1: 签名结果 s 的起始地址
		返回值:	0: 运算成功; 其他: 运算失败
sm2_verify	SM2 验签运算	输入:	ECC_G_STR *p_ecc_para: ECC_G_STR 结构指针
			MATH_G_STR *p_math_st : MATH_G_STR 结构指针
			UINT8 *hashdata: 摘要的起始地址
			UINT8* PublicKey: 公钥的起始地址
			UINT8 *Signature0: 签名 s 的起始地址
			UINT8 *Signature1: 签名 r 的起始地址
		输出:	无
		返回值:	0: 验签成功; 其他: 验签失败
sm2_encrypt	SM2 加密运算	输入:	ECC_G_STR *p_ecc_para: ECC_G_STR 结构指针
			SM2_CRYPT_CTX* sm2_context : SM2_CRYPT_CTX 结构指针
			SM3_CTX * sm3_context: SM3_CTX 结构指针
			UINT8 *plain: 加密数据的起始地址
			UINT32 byte_len: 加密数据的字节长度
			UINT8 pubkey[64]: 公钥的起始地址
		输出:	UINT8 C1[64]: C1 的起始地址
			UINT8 *C2: C2 的起始地址
			UINT8 C3[32]: C3 的起始地址
		返回值:	0: 运算成功; 其他: 运算失败
sm2_decrypt	SM2 解密运算	输入:	ECC_G_STR *p_ecc_para: ECC_G_STR 结构指针

			SM2_CRYPT_CTX* sm2_context :
			SM2_CRYPT_CTX 结构指针
			SM3_CTX * sm3_context: SM3_CTX 结构指针
			UINT8 prikey [64]: 私钥的起始地址
			UINT8 C1[64]: C1 的起始地址
			UINT8 *C2: C2 的起始地址
			UINT8 C3[32]: C3 的起始地址
			UINT32 byte_len: 密文数据的字节长度
			UINT8 mode: 模式功能选择, 0xF27E4B40: 普通模式 其他数据: 安全模式
		输出:	UINT8 *plain: 解密数据的起始地址
		返回值:	0: 解密成功; 其他: 解密失败
sm2_Exchange_Key	SM2 密钥交换运算	输入:	ECC_G_STR *p_ecc_para: ECC_G_STR 结构指针
			UINT8 role: 密钥交换角色 1: 发起者 0: 接受者
			UINT8 *Prikey: 自身私钥的起始地址
			UINT8 *PubkeyB: 对方公钥的起始地址
			UINT8 *Prikey_temp: 自身临时私钥的起始地址
			UINT8 *Pubkey_temp: 自身临时公钥的起始地址
			UINT8 *Pubkey_tempB: 对方临时公钥的起始地址
			UINT8 *ZA: 自身的 Z 值
			UINT8 *ZB: 对方的 Z 值
			UINT32 klen: 交换密钥的位长度
		输出:	UINT8 *Ex_K: 交换密钥的起始地址
			UINT8 *S1: S1 的起始地址
			UINT8 *SA: SA 的起始地址

		返回值:	0: 交换成功; 其他: 交换失败
sm3_hash	SM3 运算	输入:	UINT8 *pDataIn: 待压缩数据的起始地址
			UINT32 DataLen: 待压缩数据的字节长度
		输出:	UINT8 *pDigest: 摘要值的起始地址
		返回值:	无

23.9.4. 注意

- 1) 需要确保模块时钟使能寄存器中的 UAC, PKI, HRNG, HRNG 慢时钟, HASH 模块时钟使能;
- 2) 单独使用 SM3/SHA1/SHA256 模块则只需要确保模块时钟使能寄存器中的 UAC, HASH 模块时钟使能;
- 3) 通过时钟分频寄存器配置好系统时钟和算法时钟;
- 4) 在调用签名函数 (SM2_sign) 时, 要保证输入参数 *hashdata* 的字长度为 CurveLength, 输出参数 *Signature0*, *Signature1* 的字长度为 CurveLength, 并初始化为 0;
- 5) 函数注释中的 big endian 表示高位字节数据存放在数组的低位, 其中涉及到 SM3 运算的数据格式也是 big endian;

若: A = 0x11223344556677889900;

则: A[0] = 0x11;

A[1] = 0x22;

A[2] = 0x33;

A[3] = 0x44;

A[4] = 0x55;

A[5] = 0x66;

A[6] = 0x77;

A[7] = 0x88;

A[9] = 0x99;

A[10] = 0x00;

23.10. HASH

23.10.1. 主要特性

- 支持 SHA1/SHA256/SHA384/SHA512

23.10.2. 库文件说明

sha1.lib	实现 sha1 运算
sha1.h	sha1.lib 对应的头文件

sha256.lib	实现 sha256 运算
sha256.h	sha256.lib 对应的头文件
sha384_512.lib	实现 sha384,sha512 运算
sha384.h	sha384_512.lib 对应的头文件

23.10.3. 函数说明

表 28-8 HASH 驱动函数说明

SHA1_hash	SHA1 运算	输入:	UINT8 *pDataIn: 待压缩数据的起始地址
			UINT32 DataLen: 待压缩数据的字节长度
		输出:	UINT8 *pDigest: 摘要值的起始地址
		返回值:	无
SHA256_hash	SHA256 运算	输入:	UINT8 *pDataIn: 待压缩数据的起始地址
			UINT32 DataLen: 待压缩数据的字节长度
		输出:	UINT8 *pDigest: 摘要值的起始地址
		返回值:	无
SHA384_hash	SHA384 运算	输入:	UINT8 *pDataIn: 待压缩数据的起始地址
			UINT32 DataLen: 待压缩数据的字节长度
		输出:	UINT8 *pDigest: 摘要值的起始地址
		返回值:	无
SHA512_hash	SHA512 运算	输入:	UINT8 *pDataIn: 待压缩数据的起始地址
			UINT32 DataLen: 待压缩数据的字节长度
		输出:	UINT8 *pDigest: 摘要值的起始地址
		返回值:	无

23.10.4. 注意

- 1) 需要确保模块时钟使能寄存器中的 UAC，HASH 模块时钟使能；
- 2) 通过时钟分频寄存器配置好系统时钟和算法时钟；
- 3) 涉及到 HASH 运算的数据格式都是 big endian；

若：A = 0x11223344556677889900；

则：A[0] = 0x11；

A[1] = 0x22；

A[2] = 0x33；

A[3] = 0x44；

A[4] = 0x55；

```
A[5] = 0x66;
A[6] = 0x77;
A[7] = 0x88;
A[9] = 0x99;
A[10] = 0x00;
```

23.11. ECDSA

23.11.1. 主要特性

- 支持 160~521 比特的任意素域上的 ECDSA 密钥生成，签名验证算法;

23.11.2. 库文件说明

- ecdsa.lib
- 实现 ECDSA 密钥生成以及签名验签运算
- ecdsa.h
- 实现 ECDSA 相关算法的头文件

23.11.3. 函数说明

表 28-9 ECDSA 驱动函数说明

函数名	功能描述	参数	
ECDSA_sign	ECDSA 签名运算	输入:	ECC_G_STR *p_ecc_para: ECC_G_STR 结构指针
			MATH_G_STR *p_math_st : MATH_G_STR 结构指针
			UINT8 *hashdata: 摘要的起始地址
			UINT8*PrivateKey: 私钥的起始地址
		输出:	UINT8 *Signature0: 签名结果 r 的起始地址
			UINT8 *Signature1: 签名结果 s 的起始地址
		返回值:	0: 运算成功; 1: 运算失败
ECDSA_verify	ECDSA 验签运算	输入:	ECC_G_STR *p_ecc_para: ECC_G_STR 结构指针
			MATH_G_STR *p_math_st : MATH_G_STR 结构指针
			UINT8 *hashdata: 摘要的起始地址
			UINT8* PublicKey: 公钥的起始地址
			UINT8 *Signature0: 签名 s 的起始地址

			UINT8 *Signature1: 签名 r 的起始地址
		输出:	无
		返回值:	0: 验签成功; 1: 验签失败
ECDSA_keypair	密钥生成函数	输入:	ECC_G_STR *p_ecc_para: ECC_G_STR 结构指针
		输出:	UINT32 PrivateKey[]: 私钥的起始地址
			UINT32 PublicKeyX[]: 公钥 X 的起始地址
			UINT32 PublicKeyY[]: 公钥 Y 的起始地址
		返回值:	无

23.11.4. 注意

- 1) 需要确保模块时钟使能寄存器中的 UAC, PKI, HRNG, HRNG 慢时钟模块时钟使能;
- 2) 通过时钟分频寄存器配置好系统时钟和算法时钟;
- 3) 在调用签名函数 (ECDSA_sign) 时, 要保证输入参数 **hashdata** 的字长度为 CurveLength, 输出参数 **Signature0**, **Signature1** 的字长度为 CurveLength, 并初始化为 0;
- 4) ECDSA 的所有数据存放方式如下:
低 32 位字数据在数组的低位, 每个字以大端方式存放。

若: A = 0x11223344556677889900;

则: A[0] = 0x77889900;

A[1] = 0x33445566;

A[2] = 0x00001122;

23.12. 注意事项

1、如果需要将算法 sram 作为普通 sram 使用, 请先调用 alg2nor_sram() 函数, 具体见 release/pki/demo 工程。

2、release 中提供的算法库是通用库, 如果需要去检测机构进行测试, 则需要与芯片厂家联系获取在该检测机构备案的算法库。

24. 电气参数

24.1. 绝对最大额定值

在实际操作时不要超过这些参数，否则将永久地损坏芯片。

表 24-1 绝对最大额定值

符号	描述	Min	Max	Unit
TS	存储温度	-55	150	°C
VCC	电源电压	-0.5	6.0	V
VDDIO	IO电源	-0.5	4.6	V
VESD	最大ESD电压（HBM）	-	4K	V

24.2. 典型操作条件

表 24-2 典型操作条件

Symbol	Parameter	Conditions	Min	TYP	Max	Unit
FCLK	系统时钟		-	-	110	MHz
VCC	供电电压		1.68	5	5.5	V
VDDIO	IO电源		1.68	3.3	3.63	V
TA	工作温度		-40	-	85	°C
T _{start}	芯片启动时间	上电/PowerOff			2	ms

24.3. DC参数

DC 特性包括每一个引脚地输入门限以及输出驱动电压及电流。这些参数能够决定最大的 DC 负载，并决定给定负载的条件下的最大的传送时间。下表显示了高低电压输入、输出以及 IO 引脚情况下的 DC 操作条件，所有的 DC 参数值在整个温度范围内有效。

表 24-3DC 参数

3.3V IO 电气参数

Symbol	Parameter	Conditions	Min	Type	Max
--------	-----------	------------	-----	------	-----

VDDIO			2.97V	3.3V	3.63V
VIH	I/O输入高电压	-	2.0V		3.63V
VIL	I/O输入低电压	-	-0.3V		0.8V
VHYS	输入施密特窗口			0.35V	
IL	输入漏电流				±10uA
VOH	I/O输出高电压		2.4V		
VOL	I/O输出低电压				0.4V
RPu	上拉电阻		28K Ω	41 K Ω	65 K Ω

1.8V IO 电气参数

Symbol	Parameter	Conditions	Min	Type	Max
VDDIO			1.68V	1.8V	1.98V
VIH	I/O输入高电压	-	0.65V		1.98V
VIL	I/O输入低电压	-	-0.3V		0.35V
VHYS	输入施密特窗口			0.32V	
IL	输入漏电流				±10uA
VOH	I/O输出高电压		1.17V		
VOL	I/O输出低电压				0.45V
RPu	上拉电阻		58K Ω	96 K Ω	165 K Ω

LDO 输出电流

Symbol	Parameter	Conditions	Min	Type	Max
IO33	LDO33输出驱动能力	VCC=3.6v	120 mA		200mA
IREF	VREF 1.8V输出驱动能力	VCC=1.98v	10 mA		

Sensor 参数

Symbol	Parameter	Conditions	Min	Type	Max
VDH	高电压报警阈值			6.0v	
VDL	低电压报警阈值			2.5v	
TDH	高温报警阈值			85℃	
TDL	低温报警阈值			-40℃	

24.4. 时钟参数

Symbol	Parameter	Min	TYP	Max	Unit
RC48M	内部 RC48M 振荡器	45	-	48	MHz
RC32K	内部 RC32K 振荡器	20	-	45	KHz

24.5. 功耗参数

应用场景	工作电流
USB应用系统@110Mhz	25mA （RSA运算增加12mA）
指纹应用系统@110Mhz	20mA
STANDBY系统@110Mhz	300 uA(usb 应用) 160 uA (非 usb 应用)
PowerOff（打开LDO18）	14uA
PowerOff（关闭LDO18）	3uA